Automating the Derivation of Advanced Algorithms in Numerical Linear Algebra using Associative Algebras with Involution

Ran Gutin

Department of Computing

Imperial College London

A thesis submitted for the degree of

Doctor of Philosophy (PhD)

Submitted September 2025

Revised following examination, October 2026

Abstract

Standard algorithms of numerical linear algebra use their scalars through seven operations: addition, subtraction, multiplication, division, the adjoint, the $*$-square root and (i, j) indexing. This thesis overloads these operations, as automatic differentiation overloads arithmetic. The scalars may then lie in a $*$-*algebra*, an associative algebra with an involution. Matrix decompositions, and several algorithms that compute them, are thereby unified. Most of what follows reduces to the Hermitian eigendecomposition over $*$-algebras.

The decompositions are organised by *doubly-signed quivers*. These are directed graphs whose vertices are labelled by whether their scalars are real, complex or quaternionic, and by how the involution acts there. We prove an analogue of Gabriel's theorem. Every finite-dimensional real $*$-algebra is a matrix algebra, with sizes and signs, over a quotient of the path $*$-algebra of its doubly-signed quiver. Each quiver encodes a classical problem as unitary similarity of Hermitian matrices. The spectral theorems, the singular value decomposition, the Takagi factorisation, Sylvester's law of inertia and the real Jordan form arise in this way.

Division and square roots take place in a completed localisation, which inverts the arrows. Generic algorithms then become classical ones: for the singular value decomposition, Householder tridiagonalisation becomes Golub–Kahan bidiagonalisation. Over the split-complex numbers (whose Hermitian eigendecomposition is the real Jordan form), the QR algorithm becomes Rutishauser's LR algorithm, two steps at a time. The Cholesky factorisation becomes Doolittle's LU factorisation. These need suitable $*$-square roots and hold whenever the steps are defined.

Infinitesimals make limits of algorithms exact computations. Thus the Gram–Schmidt process yields the Bruhat decomposition, and Gaussian elimination, with pivots chosen by orders of magnitude, yields Dijkstra's shortest-path algorithm in leading terms. A Python library implements the seven operations and these algorithms. An appendix visualises the QR algorithm on 2×2 real matrices.

Declarations

Originality

I declare that the work in this thesis is my own, and that the work of others is acknowledged and referenced where it is used. I had substantial help from AI assistants. Gemini, made by Google, helped me to understand large amounts of background material. Claude, made by Anthropic, and ChatGPT, made by OpenAI, helped me to carry out experiments, to write and test software, and to edit and revise the text much faster than I could have done alone. I take full responsibility for the content of this thesis.

Copyright

The copyright of this thesis rests with the author. Unless otherwise indicated, its contents are licensed under a Creative Commons Attribution 4.0 International Licence (CC BY 4.0). The licence is at https://creativecommons.org/licenses/by/4.0/.

Statement of Provenance

Parts of this thesis build on papers and preprints written by the author during his doctoral studies. Their relationship to the thesis is as follows.

- Theorem 4.12 and Corollaries 4.13 and 4.17 in Chapter 4 generalise, from the dual numbers to formal power series, results of [39]: its spectral theorems for symmetric and for Hermitian matrices, and its T-SVD $M = U\Sigma V^T$ with $U^TU = V^TV = I$. Its $*$-SVD is not treated in this thesis.

- Section 1.3.2 and Proposition 6.6, on the split-complex numbers ${}^2\mathbb{R}$, develop the preprint [38], which observed, under the name of double numbers, that over this algebra the LDL^* decomposition yields the LU decomposition of a real matrix and an algorithm for the singular value decomposition becomes the LR algorithm. Proposition 6.6 makes these statements precise, with the choice of $*$-square roots and the conditions under which the steps are defined.

- The parity-equivariant spectral theorem of Section 4.9 makes precise a unifying observation made in [44] about canonical forms over Clifford algebras.

- Appendix A reproduces, with editorial changes, the preprint [41]; its warm-up visualisation for positive semidefinite matrices first appeared in [40]. The appendix is self-contained and is not required for the remainder of the thesis.

Further related work by the author is [37, 42, 43]. Results that are not new are attributed to their sources at the point where they are stated.

Acknowledgements

I am grateful to my supervisor, Professor Abbas Edalat, for his guidance and support throughout this work. I thank my examiners, Lihua Feng and Herbert Wiklicky, for their careful reading of the thesis and for detailed comments that improved it. I also thank Gregory Gutin for helpful discussions.

Contents

Abstract **2**

Declarations **4**

Statement of Provenance **5**

Acknowledgements **6**

List of Notation **12**

1 Introduction **14**

1.1 Seven Overloaded Operations 16

1.2 Two Outcomes: Unified Decompositions and Unified Algorithms 20

1.3 Two Examples 24

1.3.1 Example: From QR to LU by an Algebraic Limit 24

1.3.2 Example: Similarity Transformations via the $*$-Algebra ${}^{2}\mathbb{R}$ 27

1.4 Limitations of Existing Unifying Frameworks 28

1.5 Related Work and the State of the Art 30

1.6 Contributions and Outline 33

2 $*$-Algebras, Modules and Quivers **38**

2.1 Notation and Conventions 38

2.2 $*$-Algebras 39

2.3 The Structure of Finite-Dimensional Algebras 41

2.3.1 Radicals, Semisimple Algebras and Artinian Rings 41

2.3.2 Basic Algebras and Idempotents . . . 43
2.3.3 Tensor Products . . . 45
2.4 Modules and Hermitian Modules . . . 45
2.5 The Krull–Schmidt Theorem and Its Hermitian Analogue . . . 48
2.6 Quivers and Path Algebras . . . 52

3 The Seven Operations over $*$-Algebras 55
3.1 Matrix Indexing in Finite-Dimensional $*$-Algebras . . . 55
3.2 Division and Zero Divisors . . . 58
3.3 $*$-Square Roots . . . 62
3.4 A First Unification: The SVD as a Spectral Theorem . . . 66

4 Generalising Decompositions Using Infinitesimals and Constructive Transfer Principles 72
4.1 Motivation and Overview . . . 72
4.2 Real Closed Fields . . . 74
4.3 First-Order Logic and Transfer Principles . . . 75
4.3.1 First-Order Languages . . . 75
4.3.2 The Tarski–Seidenberg Transfer Principle . . . 76
4.3.3 Robinson's Nonstandard Analysis . . . 77
4.3.4 Comparison of the Two Approaches . . . 77
4.4 Puiseux Series . . . 78
4.5 Infinitesimals and the Standard Part . . . 79
4.5.1 Finite Elements, Infinitesimals and Asymptotic Notation . . . 79
4.5.2 Limits of Algorithms as Standard Parts . . . 80
4.5.3 Nilpotent Infinitesimals as Quotients . . . 81
4.6 Pythagorean Fields and Laurent Series . . . 82
4.7 The Bruhat Decomposition as an Auto-Limit . . . 83
4.8 Matrix Decompositions over Formal Power Series . . . 86
4.8.1 The Spectral Theorem . . . 86

4.8.2 The Singular Value Decomposition . . . 88
4.9 A Parity-Equivariant Spectral Theorem . . . 89
4.10 Shortest Paths as an Auto-Limit . . . 92
4.11 A Chain of Limits . . . 96

5 Doubly-Signed Quivers and the Structure of Real $*$-Algebras 99

5.1 Quivers . . . 100
5.1.1 Representation Type and the Tits Form . . . 101
5.2 Symmetric Quivers . . . 103
5.3 Signed Quivers . . . 104
5.3.1 $*$-Representations . . . 106
5.4 Doubly-Signed Quivers . . . 107
5.5 Real $*$-Algebras with Radical Square Zero . . . 110
5.6 $*$-Simple Real $*$-Algebras and Matrices with Signs . . . 121
5.7 Path $*$-Algebras . . . 124
5.8 The Structure Theorem: An Analogue of Gabriel's Theorem . . . 129
5.8.1 Examples . . . 132
5.8.2 Proof of the Structure Theorem . . . 134
5.8.3 Uniqueness of the Relations . . . 138

6 Doubly-Signed Quivers Encode Matrix Decompositions 140

6.1 The Encoding Theorem . . . 141
6.2 A Catalogue of Decompositions and Their Classical Analogues . . . 144
6.2.1 The Symmetric Eigendecomposition . . . 145
6.2.2 The Normal Form of Skew-Symmetric Matrices . . . 145
6.2.3 The Real Singular Value Decomposition . . . 146
6.2.4 The Complex Singular Value Decomposition . . . 146
6.2.5 The Quaternionic Singular Value Decomposition . . . 146
6.2.6 Hermitian Eigendecompositions over $\mathbb{C}$ and $\mathbb{H}$ . . . 147
6.2.7 The Takagi Factorisation . . . 147

6.2.8 Congruence of Symmetric Matrices and the LDL^T Decomposition . 148
6.2.9 Similarity of Real Matrices 149
6.2.10 From a Matrix Problem to a Quiver 150
6.3 Generic Algorithms on Encoded Matrices 152
6.4 Beyond Degree One: Perturbation Theory 159
6.5 Representation Type and the Behaviour of Generalised Algorithms 161

7 Localisation (Adjoining Inverse Arrows) and Completion of Path Algebras 164

7.1 The Division Problem 165
7.2 Localisation at the Arrows 166
7.3 Completion in the Path Degree 169
7.4 Inverses in the Completed Localisation 170
7.5 $*$-Square Roots in the Completed Localisation 172
7.6 Reducing Back to the Original Algebra 174
7.7 Beyond the Completed Localisation: Simple Artinian Rings 176

8 Implementation 181

8.1 Overview 182
8.2 Doubly-Signed Quivers 183
8.3 The Algebra and Its Operations 184
8.4 Algorithms Written Once 185
8.5 Experiments 185
8.6 Limitations 186

9 Conclusion and Future Work 189

9.1 Summary of Contributions 189
9.2 Finite-Dimensional versus Operator-Algebraic Settings 194
9.3 Limitations and Open Problems 196
9.4 Future Research Directions 197
9.5 The Role of the Visualisation Appendix 198

Bibliography **200**

A Visualising the Dynamics of the QR Algorithm on 2×2 Real Matrices **211**

A.1 Overview . . . 211

A.2 Visualisation v1 (Warm-Up) . . . 213

A.3 Visualisation v1.5 (Warm-Up) . . . 214

A.4 Visualisation v2 . . . 215

A.4.1 Group Viewpoints and the Boundary Circle . . . 215

A.4.2 Oriented Cycles and the Lie Quadric . . . 216

A.4.3 The Envelope as a Tangent Pair on L . . . 217

A.4.4 The Two Circles as Level Sets . . . 218

A.4.5 The Negative Determinant Case ($\Delta < 0$) . . . 221

A.4.6 Practical Recipe . . . 222

A.5 How to Read the Pictures of the QR Algorithm . . . 222

A.6 The QR Algorithm for Matrices with Non-Negative Determinant . . . 223

A.6.1 Behaviour near the Upper Triangular Limit . . . 224

A.6.2 Behaviour near the Identity Matrix and near Singular Matrices . . . 225

A.6.3 The Role of Shifts . . . 226

A.6.4 The Complex Eigenvalue Case . . . 226

A.7 The QR Algorithm for Matrices with Negative Determinant . . . 227

B Source Code of the Library **228**

B.1 The Package . . . 229

List of Notation

The main symbols, with the place where they are defined. Where a letter has a second meaning that occurs in several places, both are listed. Letters are also used locally, within a proof or an example, with meanings that are introduced there.

Symbol	Meaning	Defined in
Algebras and involutions		
x^*, X^*	the adjoint of x, given by the involution of a $*$-algebra; for matrices over it, the conjugate transpose	Def. 2.2, Conv. 2.1
X^T, X^H	transpose; conjugate transpose of a real or complex matrix	Conv. 2.1
$\mathrm{Herm}_n(A)$, $\mathrm{U}_n(A)$	Hermitian and unitary $n \times n$ matrices over A	Conv. 2.1
${}^2\mathbb{R}$	the split-complex numbers $\mathbb{R} \oplus \mathbb{R}$, with $(x,y)^* = (y,x)$	Conv. 2.1, Sec. 1.3.2
X^ω, J	the symplectic involution $J^{-1}X^TJ$, with $J = \left(\begin{smallmatrix} 0 & I_n \\ -I_n & 0 \end{smallmatrix}\right)$; in Section 4.9, J is $\mathrm{diag}(I_p, -I_q)$	Ex. 2.4, Sec. 3.3
ε	a positive infinitesimal; the generator of the dual numbers $\mathbb{R}[\varepsilon]/(\varepsilon^2)$; the sign of an ε-Hermitian form	Ch. 4, Ex. 2.4, Def. 2.17
$\mathrm{rad}(A)$	the Jacobson radical	Sec. 2.3.1
S, R	a $*$-stable semisimple subalgebra and the radical, in $A = S \oplus R$ with $R = \mathrm{rad}(A)$; in a factorisation $H = QR$, R is the triangular factor	Thm. 2.12, Lemma 5.8
e_v, 1_c	the idempotent of a vertex v; of an orbit c of vertices	Def. 2.25, Def. 5.15
c_k	a position: the k-th index carrying the idempotent 1_c	Sec. 3.1, Cor. 5.18
Quivers		
$Q = (Q_0, Q_1, s, t)$	a quiver; in a factorisation $H = QR$, Q is the unitary factor	Def. 2.24
kQ, R_Q	the path algebra and its arrow ideal	Def. 2.25
r	the cycle rank of a quiver	Sec. 5.1.1

Symbol	Meaning	Defined in
$\dagger$; $\varepsilon(v)$, $\varepsilon(\alpha)$	the involution and the signs of a signed quiver	Def. 5.4
$v^\top$, $\alpha^\top$; $\bar{v}$, $\bar{\alpha}$	the transpose and the conjugation of a doubly-signed quiver	Def. 5.7
$\varepsilon_{\mathrm{div}}$, $\varepsilon_{\mathrm{form}}$	the two signs of a vertex of a doubly-signed quiver	Def. 5.7, Table 5.1
$(D, \varepsilon_{\mathrm{form}})$, $(\mathbb{C}, \mathrm{u})$	labels of the ten vertex types	Sec. 5.4
G_v, C_v; κ_α, μ_α	the matrices and the signs of the block model	Table 5.2, (5.3)
$\mathbb{R}Q$, S_Q, M_Q, B_Q	the path $*$-algebra, its parts of degree 0 and 1, and $\mathbb{R}Q/\mathbb{R}Q_{\geq 2}$	Def. 5.13
$\mathcal{O}(Q)$	the orbits of the vertices under the transpose and the conjugation	Sec. 5.6
$M_{n,\eta}(B)$; η_c, n_c	matrices with signs; the form and the number of positions at an orbit c	Sec. 5.6, Def. 5.15
E, E_0, E_1	the matrix algebra of the encoding theorem and its parts of degree 0 and 1	Thm. 6.1
Localisation and completion		
kQ_{loc}	the localisation of kQ at its arrows	Def. 7.1
Λ	the completed localisation; in Section 6.2.1, a diagonal matrix of eigenvalues	Def. 7.4
$v(x)$, x°	the valuation and the lowest-degree part of an element of Λ	Def. 7.4
$k\{X\}$, D_r	the free field; the coefficient skew field of the embedding theorem	Sec. 7.7
Infinitesimals		
$\mathbb{R}\langle\varepsilon\rangle$	the real closed field of algebraic Puiseux series	Sec. 4.4
$k((t))$, $k[[t]]$	formal Laurent series and formal power series	Sec. 4.4
$\mathrm{st}(x)$	the standard part of a finite element	Sec. 4.5
${}^*\mathbb{R}$	the hyperreal numbers of nonstandard analysis	Sec. 4.3.3
Groups		
$\mathrm{O}(n)$, $\mathrm{U}(n)$, $\mathrm{Sp}(n)$, $\mathrm{Sp}(2n,\mathbb{R})$, $\mathrm{O}^*(2n)$, $\mathrm{O}(n,\mathbb{C})$, $\mathrm{Sp}(2n,\mathbb{C})$, $\mathrm{GL}_n(D)$	the unitary groups of the ten vertex types	Table 5.1

Chapter 1

Introduction

The standard algorithms of numerical linear algebra, such as Householder reduction, the QR algorithm and the Cholesky and LU factorisations, use their scalars through seven operations: addition, subtraction, multiplication, division, the adjoint $x \mapsto x^*$, the $*$-square root and (i, j) indexing. This thesis overloads these seven operations, as forward-mode automatic differentiation overloads arithmetic [36], so that the scalars may come from an associative algebra with an involution, called a $*$-*algebra* (Definition 2.3), instead of from $\mathbb{R}$ or $\mathbb{C}$; the involution is the adjoint.

The outcome is that matrix decompositions are unified, and so are several of the algorithms that compute them. The spectral theorems, the singular value decompositions over $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$, the Takagi factorisation, Sylvester's law of inertia and the real Jordan form are instances of one problem, unitary similarity of Hermitian matrices, over different $*$-algebras (Theorem A). And an algorithm written once becomes different classical algorithms over different $*$-algebras: Householder tridiagonalisation becomes the bidiagonalisation of Golub and Kahan, the QR algorithm becomes forms of Rutishauser's LR algorithm, and the Cholesky factorisation becomes Doolittle's LU factorisation (Theorem B).

Most of the thesis therefore reduces to one problem and to the algorithms that solve it: the eigendecomposition of a Hermitian matrix, computed by Householder tridiagonalisation and the QR algorithm. Doubly-signed quivers encode matrix decompositions in exactly this way: the decompositions of Theorem A are Hermitian eigendecompositions

over $*$-algebras. Here a Hermitian eigendecomposition is a unitary similarity to a direct sum of blocks that are not unitarily similar to direct sums of smaller blocks. Over $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$ the blocks are 1×1, and the result is diagonal; over other $*$-algebras they can be larger, as for the real Jordan form over ${}^2\mathbb{R}$, whose blocks can be taken tridiagonal (Section 1.3.2). The spectral theorems over formal power series in Chapter 4 are Hermitian eigendecompositions too. The classical algorithms of Theorem B(1)–(3) are Householder tridiagonalisation, the QR algorithm and the Cholesky factorisation, run over $*$-algebras. The other factorisations follow by limits (Section 4.11). The QR factorisation, as distinct from the QR algorithm, is a limit of the singular value decomposition, itself a Hermitian eigendecomposition (Proposition 4.19). The Bruhat decomposition of Theorem B(4) and the LU factorisation are auto-limits of the QR factorisation (Theorem 4.11 and Proposition 1.1). Dijkstra's shortest-path algorithm is the leading-order part of Gaussian elimination, with the pivots chosen by orders of magnitude (Proposition 4.18). What does not itself reduce to the Hermitian eigendecomposition is the machinery that sets up the reduction: the structure theorem for real $*$-algebras (Chapter 5), division and square roots in the completed localisation (Chapter 7) and the transfer principle (Chapter 4). The reductions through $*$-algebras are not the only ones: many of the decompositions are also, classically, the Hermitian eigendecomposition of a larger block matrix, such as $\left(\begin{smallmatrix} 0 & W^* \\ W & 0 \end{smallmatrix}\right)$ for the singular value decomposition of W, and the thesis points these out where they arise.

Addition, subtraction, multiplication and the adjoint carry over to every $*$-algebra; the algorithms use the adjoint to form conjugate transposes. The other three operations do not carry over directly, and giving them a meaning is the technical content of the thesis. Indexing must say what the entries of a matrix over a $*$-algebra are; for finite-dimensional real $*$-algebras, an analogue of Gabriel's theorem for *doubly-signed quivers* answers this (Chapter 5). Division and the $*$-square root can fail in the algebra itself, because the unified algorithms divide by zero divisors and normalise vectors whose norm vanishes; we carry them out in a *completed localisation* of a path algebra, in which the arrows of the quiver are inverted (Chapter 7). The simplest instance is the field of Laurent

series, which is the completed localisation of the quiver with one loop. Its variable ε is an infinitesimal, and running an algorithm over a field with such an infinitesimal, then taking standard parts of its finite outputs, gives their limits as a real parameter ε tends to zero (Chapter 4).

Section 1.1 lists the seven operations and what each needs over a $*$-algebra, and Section 1.2 states the two outcomes as Theorems A and B. Section 1.3 describes the two examples from which the approach grew. Section 1.4 explains why existing unifying frameworks do not provide transformations of programs, and Section 1.5 surveys related work. Section 1.6 describes the contributions chapter by chapter.

1.1 Seven Overloaded Operations

An algorithm of numerical linear algebra, written for real or complex matrices, uses its scalars through the following operations:

(O1) addition, $x + y$;

(O2) subtraction, $x - y$;

(O3) multiplication, xy;

(O4) division, $x/y = xy^{-1}$;

(O5) the *adjoint*, $x \mapsto x^*$;

(O6) the $*$-*square root*: for a self-adjoint s, some x with $x^*x = s$;

(O7) (i, j) indexing: access to the entry X_{ij} of a matrix X.

The adjoint is complex conjugation over $\mathbb{C}$ and the identity over $\mathbb{R}$, which is why it is rarely counted as an operation; the algorithms use it to form conjugate transposes X^* and inner products $x^*y = \sum_i x_i^* y_i$. Over $\mathbb{R}$ and $\mathbb{C}$, the $*$-square roots of $s \geq 0$ are the numbers $\sqrt{s}$ multiplied by a sign or a phase. An algorithm may also compare the sizes of scalars, where it chooses a sign or a pivot. Listing 1.1 shows Householder tridiagonalisation written against these operations.

To *overload* the operations is to give each of them a meaning for scalars from a $*$-algebra A, so that the same program runs on matrices with entries in A. In forward-mode automatic differentiation, A is the algebra of dual numbers $\mathbb{R}[\varepsilon]/(\varepsilon^2)$ with the trivial

```
def householder(H, sqrt):
    """Tridiagonalise a Hermitian H: return T, U with H = U T U*."""
    n = H.shape[0]
    T, U = H.copy(), identity(n)
    for k in range(n - 2):
        x = T[k+1:, k]                    # (i, j) indexing
        gamma = sqrt(x.H @ x)             # *-square root: gamma* gamma = x* x
        if norm(x[0] + gamma) > norm(x[0] - gamma):
            gamma = -gamma                # comparison: avoid cancellation
        w = x.copy()
        w[0] = x[0] - gamma               # subtraction
        s = w.H @ x                       # adjoint, multiplication and addition
        P = identity(n)
        P[k+1:, k+1:] -= (w / s) @ w.H    # division, in the corner of column k
        T = P @ T @ P.H
        U = U @ P.H
    return T, U
```

Listing 1.1: Householder tridiagonalisation written against the seven operations, in Python with NumPy-style slicing; `X.H` is the conjugate transpose, formed with the adjoint. It simplifies the function `householder` of Appendix B, which also reports breakdowns and skips columns that are already reduced.

involution, and the program computes a derivative along with each value. Suppose that every step of Listing 1.1 is defined in the sense of Proposition 3.8: each root γ lies in the corner of the next position, and each s is invertible in the corner of its column. (Corners and positions refine the entries of a matrix by self-adjoint idempotents; they are explained in the paragraph on indexing below.) Then the listing computes a tridiagonal T and a unitary U over A with $H = UTU^*$, because the proof that the reflectors $P = I - ws^{-1}w^*$ are unitary uses only identities that hold in every $*$-algebra. Table 1.1 summarises what each operation means over A, what makes it difficult, and where the difficulty is addressed.

Addition, subtraction and multiplication. These are the operations of A. To compute with them we need a representation of the elements of A. Any basis with its structure constants would serve for these three operations; we use the one provided by the analogue of Gabriel's theorem, because it also serves indexing and division: the elements are matrices over a quotient of the path $*$-algebra of a quiver, that is, over linear combinations of paths (Theorem 5.16). After complexification, the coefficient of each path is a block of size at most two (Proposition 5.14).

Table 1.1: The seven operations over a $*$-algebra A: what they mean, what makes them difficult, and where the difficulty is addressed.

Operation	**Meaning over A**	**Difficulty**	**Addressed by**
$+$, $-$, $\times$	the operations of A	representing the elements of A	paths with coefficient blocks (Theorem 5.16, Proposition 5.14)
$/$	$x/y = xy^{-1}$, with y invertible between idempotents (Definition 3.5)	zero divisors: no non-zero element of the radical is invertible	the completed localisation Λ (Chapter 7)
adjoint	$x \mapsto x^*$, the involution of A	it is not the identity: over ${}^2\mathbb{R}$ it exchanges the components, and in a path algebra it reverses words	the block model (5.2); on matrices, the conjugate transpose (5.4)
$*$-square root	for self-adjoint s, some x with $x^*x = s$	not unique, and need not exist	any root in the corner of the next position with w^*x invertible (Proposition 3.8); roots in Λ (Section 7.5)
(i, j) indexing	entries $X_{c_k,d_l} \in 1_c A 1_d$ at positions c_k, d_l	which idempotents, and in which order	the orbits of the vertices of a quiver (Corollary 5.18); a cyclic order (Proposition 3.9)

The adjoint. The adjoint $x \mapsto x^*$ is the involution of A: it reverses products, $(xy)^* = y^*x^*$, and its square is the identity. Over $\mathbb{R}$ it is the identity, but over ${}^2\mathbb{R}$ it exchanges the two components, and in a path $*$-algebra it reverses each path and transposes its arrows, with signs and with the matrices G_v of the block model (5.2). On matrices, the adjoints of the entries, with the indices exchanged, give the conjugate transpose, which algorithms for Hermitian matrices use at every step.

Indexing. Over A, the entries of a matrix are refined into elements of corners $1_c A 1_d$ cut out by self-adjoint idempotents, and a matrix is indexed by *positions*, pairs c_k of an idempotent 1_c and a counter k (Section 3.1). Because the idempotents are self-adjoint, the adjoint acts on matrices as the conjugate transpose, $(X^*)_{ij} = (X_{ji})^*$, possibly with signs; this is what an algorithm written for Hermitian matrices needs. Which idempotents to use is answered by the analogue of Gabriel's theorem: every finite-dimensional real $*$-

algebra is a matrix algebra over a quotient of the path $*$-algebra of a doubly-signed quiver, and the positions are grouped by the orbits of its vertices under the transpose and the conjugation (Corollary 5.18). The positions are listed in a cyclic order through the orbits. The order matters: for the quiver of the singular value decomposition, it is what turns tridiagonalisation into bidiagonalisation (Proposition 3.9).

Division. Most $*$-algebras contain zero divisors, and the generalised algorithms divide by them, even when division is taken between idempotents (Definition 3.5). For example, over the four-dimensional $*$-algebra of the singular value decomposition (Section 3.4), whose radical squares to zero, the norms x^*x of the columns that Householder's algorithm reduces vanish. In the path algebra of its quiver they are multiples cba or cab of paths of length two, which have the $*$-square roots $\pm\sqrt{c}\,a$ or $\pm\sqrt{c}\,b$ but no inverses. We therefore run the algorithms in the *completed localisation* Λ of the path algebra, obtained by inverting every arrow between its end points and completing in the path degree (Chapter 7). In Λ every path is invertible between its end points. When the underlying graph of every component of the quiver has at most one cycle, and that cycle has non-zero degree, as for all the classical examples, each corner of Λ is, after complexification, a space of matrices over a field, and elimination inverts every invertible element (Proposition 7.6). Results return to the original algebra whenever the quantities that certify them contain no inverted arrows (Theorem 7.8).

The $*$-square root and comparisons. A $*$-square root is not unique: if a root x is invertible from f to g, between self-adjoint idempotents, then every other root x' with $x'f = x'$ has the form ux with $u^*u = g$ (Lemma 3.7). Householder reflectors work with any $*$-square root that lies in the corner of the next position and for which a certain denominator is invertible (Proposition 3.8), so the phase condition of the classical algorithm is not needed, and in Λ the roots are computed degree by degree (Section 7.5). Comparisons, such as the choice of sign in Listing 1.1, need a measure of the size of a scalar. Over a real closed field with an infinitesimal, they are comparisons in an ordered field, and such fields also supply square roots (Chapter 4); in Λ, the implementation compares

norms of coefficients and, to choose pivots, degrees and leading coefficients (Section 7.4).

1.2 Two Outcomes: Unified Decompositions and Unified Algorithms

We now state the two outcomes, with references to the precise results. A quiver is a directed graph. A *doubly-signed quiver* Q (Definition 5.7) is a quiver with two involutions, a *transpose* that reverses arrows and a *conjugation* that preserves them, together with signs; at each vertex they record whether the scalars are real, complex or quaternionic, and how the involution acts there. Its path $*$-algebra $\mathbb{R}Q$ is graded by the length of paths: the vertices have degree 0 and the arrows degree 1.

Theorem A (Unified decompositions)**.**

1. *Every finite-dimensional real $*$-algebra is a matrix algebra, with sizes and signs, over a quotient $\mathbb{R}Q/I$ of the path $*$-algebra of a doubly-signed quiver Q, and Q, the sizes and the signatures are invariants of the algebra (Theorem 5.16).*

2. *Let Q be a doubly-signed quiver, let $I \subseteq \mathbb{R}Q$ be an ideal spanned by homogeneous elements of degree at least 2 and stable under the involution, and consider matrices over $\mathbb{R}Q/I$ indexed by positions. Two Hermitian matrices of degree 1, that is, with entries that are linear combinations of arrows, are unitarily similar if and only if they are related by a unitary matrix of degree 0. The unitary matrices of degree 0 form a product of classical groups, one for each orbit of vertices, and the Hermitian matrices of degree 1 form a direct sum of classical matrix spaces, one for each orbit of arrows (Theorem 6.1).*

3. *For suitable Q, the problem of (2) is the one solved by the spectral theorem for real symmetric, real skew-symmetric, complex Hermitian or quaternionic Hermitian matrices; by the singular value decomposition over $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$; by the Takagi factorisation or the Youla normal form; by Sylvester's law of inertia; or by the real Jordan form (Section 6.2).*

Part (1) says how matrices over a $*$-algebra are indexed: by positions, grouped by the orbits of the vertices of Q. Part (2) says which classical problem Q encodes, and part (3) that the classical decompositions are among these problems. In this sense a doubly-signed quiver indexes a decomposition, and the decompositions of part (3) are instances of one problem over different $*$-algebras; Section 6.2.10 reads part (2) backwards, as a recipe for encoding a new problem. Theorem A does not say that a normal form exists; that depends on the representation type of the quiver (Section 6.5). Part (1) rests on Theorem 5.11; parts (2) and (3) use only the explicit construction of the path $*$-algebra in Section 5.7.

The second outcome concerns the following programs, each written once against the seven operations: Householder tridiagonalisation (Listing 1.1); the Gram–Schmidt process, and the QR algorithm, whose step $H = QR \mapsto RQ$ computes Q and R by it; and the Cholesky factorisation $H = R^*R$. They are run over the completed localisation Λ, with the graded $*$-square roots of Section 7.5.

Theorem B (Unified algorithms)**.**

1. *Over the doubly-signed quiver of the real singular value decomposition, with the positions alternating between its two vertices, $u_1, v_1, u_2, v_2, \ldots$, Householder tridiagonalisation of the matrix that encodes a real $m \times n$ matrix W, $m \geq n$, is defined at every step, and it is the bidiagonalisation of W by Golub and Kahan, up to signs (Proposition 3.9). More generally, if every component of the complexified quiver is an oriented cycle or a linearly oriented path, then Householder tridiagonalisation, whenever its steps are defined, transforms a Hermitian matrix of degree* 1*, with all signs at the positions positive, only by the classical groups of Theorem A(2), and so computes a condensed form for the encoded problem (Proposition 6.4).*

2. *Over the quiver of the singular value decomposition over $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$, whose involution fixes the idempotents of its two orbits of vertices, with the positions alternating between its two orbits of vertices and with principal graded square roots, let H encode an invertible square matrix W. Two steps of the QR algorithm on H are one step of the unshifted QR algorithm on each of the Gram matrices W^*W and*

WW^*, and a single step is one step of the Cholesky LR algorithm on each of them (Proposition 6.5).

3. *The split-complex numbers ${}^2\mathbb{R}$ are the path $*$-algebra of a doubly-signed quiver without arrows whose two vertices are real and exchanged by the transpose, a single vertex orbit of type $(\mathbb{R}, 0)$; their Hermitian matrices (X, X^T), of degree 0, encode similarity of real matrices X (Section 1.3.2). Let X be invertible, and use the $*$-square roots $(c, 1)$ of the self-adjoint elements (c, c). Then the Cholesky factorisation of (X, X^T) is Doolittle's LU factorisation of X, and one step of the QR algorithm is two steps of Rutishauser's LR algorithm on X, whenever these are defined (Proposition 6.6). The QR step is the same over the similarity quiver, which encodes this problem in degree 1 (Section 6.2.9).*

4. *Over the field $\mathbb{R}((\varepsilon))$ of Laurent series, which is the completed localisation of the quiver with one loop, the Gram–Schmidt process with the inner product $\sum_i \varepsilon^{i-1} x_i y_i$, followed by taking standard parts, computes a Bruhat decomposition of every invertible real matrix, and the orders of the norms determine its permutation (Theorem 4.11).*

Parts (2) and (3) concern different $*$-algebras, and different relations between the QR and LR algorithms. Both algebras are $\mathbb{R} \times \mathbb{R}$ in degree 0, but for the real singular value decomposition the involution fixes the two vertex idempotents, so that the unitary group is $\mathrm{O}(n) \times \mathrm{O}(m)$ and the matrix W sits on the arrows, whereas in ${}^2\mathbb{R}$ it exchanges them, so that the unitary group is $\mathrm{GL}_n(\mathbb{R})$ and X sits in degree 0. In (2) the LR steps are Cholesky steps on positive definite Gram matrices, two of which make one QR step, as for every positive definite matrix [91]; in (3) they are steps of the LR algorithm on an invertible real matrix, which need not be symmetric. It is (3) that unifies the QR algorithm with Rutishauser's LR algorithm.

In each case the classical algorithm arises from the overloaded operations alone: the programs contain no code for bidiagonalisation, for the LR algorithm or for elimination. This is a statement about how the algorithms are related, not about their cost: the

Table 1.2: One program, many algorithms: what programs written once against the seven operations compute over different scalars. For a quiver, the scalars are the completed localisation of its path algebra, and the matrices are the Hermitian matrices of degree 1 that it encodes. The statements, with their hypotheses, are those of Theorem B and the results that it cites; the row for Householder tridiagonalisation over ${}^2\mathbb{R}$ holds whenever every step is defined (Section 1.3.2). Table 6.2 gives further cases.

Program, written once	**Scalars**	**Classical algorithm obtained**
Householder tridiagonalisation	$\mathbb{R}$, $\mathbb{C}$, $\mathbb{H}$	Householder tridiagonalisation
	the real SVD quiver	Golub–Kahan bidiagonalisation
	${}^2\mathbb{R}$	tridiagonalisation by similarity
QR step, by the Gram–Schmidt process	$\mathbb{R}$, $\mathbb{C}$, $\mathbb{H}$	QR step
	the SVD quivers	two steps: a QR step on W^*W and on WW^*; one step: a Cholesky LR step on each
	the Takagi quiver	two steps: a QR step on WW^*; one step: a Cholesky LR step on WW^*, by a unitary congruence of W
	${}^2\mathbb{R}$	two steps of Rutishauser's LR algorithm
Cholesky factorisation	${}^2\mathbb{R}$	Doolittle LU factorisation
Gram–Schmidt process, weighted by ε^{i-1}	$\mathbb{R}((\varepsilon))$	Bruhat decomposition

specialised implementations, such as those in LAPACK, are faster and better studied, and the generic programs are not proposed as replacements for them. Example 3.10 follows Householder's algorithm through a 3×2 example by hand, and Table 1.2 collects these results by program.

The generic algorithms can also break down where the encoded problem has a solution. Over the congruence quiver, which encodes Sylvester's law of inertia, every column whose entries are linear combinations of arrows is isotropic, $x^*x = 0$, so Householder reduction stops at the first non-zero column; the generic algorithm there is the LDL^* factorisation, which needs no norms (Section 6.2.8). Over ${}^2\mathbb{R}$, the QR step on an invertible X breaks down exactly when a leading principal minor of X^2 vanishes (Proposition 6.6). And when the quiver is wild, as for a vertex with two loops, there is no canonical form of the familiar

kind, and a Jacobi-type method needs quotients that do not exist in Λ but do exist in a non-commutative skew field, such as Cohn's free field (Section 6.5).

1.3 Two Examples

The approach grew out of two examples. The first shows that the limit of an algorithm can be computed by overloading its operations; the second, that overloading can turn one classical algorithm into another, and where it can fail.

1.3.1 Example: From QR to LU by an Algebraic Limit

The QR and LU decompositions are among the most important factorisations in numerical analysis [34]. Compare the Gram–Schmidt process, which computes a QR factorisation, with Gaussian elimination, which computes an LU factorisation [34]:

$$u_k = a_k - \sum_{j=1}^{k-1} \frac{\langle u_j, a_k \rangle}{\langle u_j, u_j \rangle}\, u_j \qquad \text{(Gram–Schmidt for QR)}, \tag{1.1}$$

$$u_k^T = a_k^T - \sum_{j=1}^{k-1} l_{kj}\, u_j^T \qquad \text{(Gaussian elimination for LU)}. \tag{1.2}$$

Here a_k denotes the k-th column of a square matrix A in (1.1) and the k-th row of A in (1.2). The two recurrences have the same shape: each step subtracts from the current vector suitable multiples of the vectors already produced. They differ only in how the multipliers are chosen.

This resemblance can be made precise. Fix $\varepsilon > 0$ and replace the Euclidean inner product by the weighted inner product

$$\langle x, y \rangle_\varepsilon \;=\; \sum_{i=1}^{n} \varepsilon^{\,i-1} x_i y_i, \qquad \text{with quadratic form} \qquad Q_\varepsilon(x) = \sum_{i=1}^{n} \varepsilon^{\,i-1} x_i^2. \tag{1.3}$$

For small ε, the first coordinate dominates every inner product, then the second, and so on. Running the (unnormalised) Gram–Schmidt process (1.1) with respect to $\langle \cdot, \cdot \rangle_\varepsilon$ and letting $\varepsilon \to 0^+$ produces Gaussian elimination.

Proposition 1.1. *Let $A \in M_n(\mathbb{R})$ have non-zero leading principal minors. Apply the recurrence (1.1), with the inner product (1.3), to the columns of A, and denote the resulting vectors by $u_1(\varepsilon), \dots, u_n(\varepsilon)$. As $\varepsilon \to 0^+$, the matrix $[u_1(\varepsilon) \cdots u_n(\varepsilon)]$ converges to the lower triangular factor $\hat{L}$ of the Crout factorisation $A = \hat{L}R$, in which R is unit upper triangular, and the Gram–Schmidt coefficients converge to the entries of R.*

Sketch of proof. One shows by induction on k that the i-th entry of $u_k(\varepsilon)$ is $O(\varepsilon^{k-i})$ for $i < k$. Because u_j is $\langle \cdot, \cdot \rangle_\varepsilon$-orthogonal to $u_1, \dots, u_{j-1}$, the coefficient $\langle u_j, a_k \rangle_\varepsilon / \langle u_j, u_j \rangle_\varepsilon$ equals $\langle u_j, a_k^{(j)} \rangle_\varepsilon / \langle u_j, u_j \rangle_\varepsilon$, where $a_k^{(j)}$ is a_k with its components along $u_1, \dots, u_{j-1}$ removed. Both inner products are dominated by their j-th terms, which are of order ε^{j-1}, so the coefficient tends to the ratio of the j-th entries of $a_k^{(j)}$ and u_j. Subtracting this multiple of u_j annihilates the j-th entry, which is exactly the elimination step of Crout's method. (Crout's method eliminates by columns; applied to A^T it is the row elimination (1.2).) $\square$

Theorem 4.11 in Chapter 4 removes the assumption on the minors: for every invertible A, the limit is a Bruhat decomposition, whose permutation is read off from the orders of magnitude of the norms. The limiting algorithm is only *almost* Gaussian elimination. If the vectors are normalised, as in the usual QR factorisation, the normalising factors $Q_\varepsilon(u_k)^{-1/2}$ grow like $\varepsilon^{-(k-1)/2}$ and the normalised algorithm has no limit. Moreover, classical Gram–Schmidt is rarely used to compute QR factorisations in practice, because it can lose orthogonality for ill-conditioned matrices [34, 48]. If instead we use Householder reflections [34], the limit is a genuine elimination step. Let $W_\varepsilon = \operatorname{diag}(1, \varepsilon, \dots, \varepsilon^{n-1})$ and let a_1 be the first column of A, with $a_{11} \neq 0$. The W_ε-orthogonal reflection that maps a_1 to a multiple of e_1 is

$$H_\varepsilon \;=\; I - \frac{2\, v v^T W_\varepsilon}{v^T W_\varepsilon v}, \qquad v = a_1 + \operatorname{sgn}(a_{11})\, Q_\varepsilon(a_1)^{1/2}\, e_1.$$

As $\varepsilon \to 0^+$ we have $Q_\varepsilon(a_1)^{1/2} \to |a_{11}|$, $W_\varepsilon v \to 2a_{11}e_1$ and $v^T W_\varepsilon v \to 4a_{11}^2$, hence

$$H_\varepsilon \;\longrightarrow\; G \;=\; \operatorname{diag}(-1, 1, \dots, 1)\,\bigl(I - \ell\, e_1^T\bigr), \qquad \ell = \Bigl(0, \tfrac{a_{21}}{a_{11}}, \dots, \tfrac{a_{n1}}{a_{11}}\Bigr)^T. \tag{1.4}$$

The matrix $I - \ell e_1^T$ is the Gauss transformation that eliminates the first column of A

below the diagonal, and G differs from it only by the sign of the first row; like each H_ε, it is an involution. Applying the same reasoning to the trailing submatrix, on which the weights are again graded, shows that Householder QR with respect to $\langle \cdot, \cdot \rangle_\varepsilon$ tends to Gaussian elimination without pivoting, up to signs. Limits of this kind, of Householder QR applied to a problem with heavily weighted rows, also appear in the analysis of the weighting method for least squares problems with equality constraints, where they yield elimination methods [18].

From limits to infinitesimals. Computing such limits by hand is laborious, and it has to be repeated for every algorithm. The key observation is that the limit can be computed *algebraically*, by running the algorithm once over a larger field, that is, by overloading its operations. Every step of Householder QR is a sequence of field operations and square roots applied to quantities depending on ε. Suppose we execute the algorithm with ε a *formal positive infinitesimal*: an element of an ordered field $\mathbb{K} \supset \mathbb{R}$ satisfying $0 < \varepsilon < 1/m$ for every positive integer m. A suitable field is the field of real algebraic Puiseux series (Section 4.4), whose elements are convergent series $\sum_q c_q \varepsilon^q$ with rational exponents. All intermediate quantities are then exact elements of $\mathbb{K}$, and at the end we keep only the constant terms (the *standard parts*) of the outputs. Proposition 4.7 shows that, whenever the outputs are finite, these standard parts are exactly the limits as $\varepsilon \to 0^+$ of the outputs of the same algorithm executed in real arithmetic. Thus the analytic operation "take the limit of the algorithm" is replaced by the algebraic operation "run the algorithm over a larger field and truncate", just as automatic differentiation replaces symbolic differentiation by arithmetic in the dual numbers.

Two properties of $\mathbb{K}$ are needed for this to work. First, square roots of positive elements must exist in $\mathbb{K}$, since the algorithm computes norms; this holds because $\mathbb{K}$ is a *real closed field* (Definition 4.1). Second, the facts used to prove the algorithm correct over $\mathbb{R}$ (for example, that H_ε is orthogonal for $\langle \cdot, \cdot \rangle_\varepsilon$, or that norms of non-zero vectors are positive) must remain true over $\mathbb{K}$. This is guaranteed by the *transfer principle* of Tarski (Theorem 4.2). Chapter 4 develops both points. In the present example the infinitesimal ε lives in a commutative field, so the limit can also be computed directly, as in the proof

of Proposition 1.1. The algebraic viewpoint is more convenient once ε is combined with the more general algebras of the next example and of later chapters, but there too the limit can be found by asymptotic analysis, by computer algebra with ε as a symbol or, less reliably, by numerical evaluation at small ε.

1.3.2 Example: Similarity Transformations via the $*$-Algebra ${}^2\mathbb{R}$

Not every relationship between algorithms is a limit. Consider the problem of reducing a real square matrix to tridiagonal form by a similarity transformation. Every real square matrix is similar to a tridiagonal matrix. For symmetric matrices, orthogonal reduction to tridiagonal form precedes the computation of eigenvalues; general matrices are usually reduced to Hessenberg form instead, because reduction to tridiagonal form by similarity can break down [34]. Over $\mathbb{R}$, the natural canonical form under similarity is the real Jordan form, obtained from the primary decomposition of the matrix.

Let ${}^2\mathbb{R} := \mathbb{R} \oplus \mathbb{R}$ with componentwise operations and with the involution that exchanges the two components, $(x, y)^* = (y, x)$. This is the algebra of split-complex numbers $p + qj$, with $j^2 = 1$ and $j^* = -j$, written in the basis of its idempotents $\frac{1}{2}(1 \pm j)$. A matrix over ${}^2\mathbb{R}$ is a pair (X, Y) of real matrices, and the involution of ${}^2\mathbb{R}$ induces the conjugate transpose $(X, Y)^* = (Y^T, X^T)$. One checks directly that

- the Hermitian matrices, $(X, Y)^* = (X, Y)$, are exactly the pairs (X, X^T) with $X \in M_n(\mathbb{R})$ arbitrary;
- the unitary matrices, $U^*U = I$, are exactly the pairs $U = (P, P^{-T})$ with $P \in \mathrm{GL}_n(\mathbb{R})$;
- for such U and $H = (X, X^T)$, one has $UHU^* = (PXP^{-1}, (PXP^{-1})^T)$.

Thus *unitary similarity of Hermitian matrices over* ${}^2\mathbb{R}$ *is exactly similarity of arbitrary real matrices.* If we take an algorithm for the unitary tridiagonalisation of Hermitian matrices written against the seven operations, such as Listing 1.1, and execute it with these operations overloaded for ${}^2\mathbb{R}$, the result is an algorithm that reduces arbitrary real matrices to tridiagonal form by similarity. Square roots always exist here: a self-adjoint

element (c, c) has the $*$-square root $z = (c, 1)$, since $z^*z = (1, c)(c, 1) = (c, c)$, whatever the sign of c. In exact arithmetic the generalised algorithm can, however, break down: it may encounter a non-zero vector $v = (x, y)$ with $v^*v = (y^Tx, y^Tx) = 0$, by which it would have to divide. This mirrors the "serious breakdown" of the unsymmetric Lanczos process [34]. Run with the QR algorithm instead, the computation becomes Rutishauser's LR algorithm, two steps at a time, and the Cholesky factorisation becomes Doolittle's LU factorisation (Proposition 6.6). The analogue of the spectral theorem in its diagonal form fails here: a matrix with non-real eigenvalues, or one that is not diagonalisable, is not similar to a real diagonal matrix. What remains is the Hermitian eigendecomposition in the sense used throughout this thesis: a unitary similarity to a direct sum of blocks, none of which is unitarily similar to a direct sum of smaller blocks (compare Theorem 2.23), and each of which can be taken tridiagonal. Over ${}^2\mathbb{R}$ these blocks are the Hermitian matrices (J, J^T) for the real Jordan blocks J, so the real Jordan form is the Hermitian eigendecomposition over ${}^2\mathbb{R}$.

This example shows the two themes of the thesis in miniature: a classical algorithm transported to a new $*$-algebra solves a different classical problem, and the points where it fails (division by elements that are not invertible) are precisely where further theory is required.

1.4 Limitations of Existing Unifying Frameworks

Several theories relate matrix decompositions to each other, but they do not, as they stand, provide transformations of programs.

Lie groups and symmetric spaces. A Lie group is a group that is also a smooth manifold, such as $\mathrm{GL}_n(\mathbb{R})$, the orthogonal group $\mathrm{O}(n)$ or the symplectic group $\mathrm{Sp}(2n, \mathbb{R})$. Many matrix decompositions are instances of structural decompositions of Lie groups: the polar decomposition and the singular value decomposition (SVD) are instances of the Cartan and KAK decompositions, and the QR decomposition is an instance of the Iwasawa decomposition [27, 47]. A symmetric space is a quotient G/K of a Lie group by the fixed points of an involutive automorphism; algebraic treatments exist [65]. These theo-

ries describe the *sets* and *groups* that appear in a decomposition, and they are expressed in terms of Lie algebras and root systems. Numerical code, however, is expressed in terms of arithmetic on scalars. The theories therefore explain why decompositions exist, but in general they do not tell us how to turn one program into another. An exception is the family of Jacobi-type methods, which have been formulated on compact Lie algebras so that one algorithm covers several eigenvalue problems [56]; see Section 1.5.

Mismatch of objectives. Decompositions are used differently in representation theory and in numerical computation. The *Bruhat decomposition* of GL_n writes an invertible matrix as $M = LPU$, with L lower triangular, U upper triangular and P a permutation matrix. This resembles LU factorisation with pivoting, but it is qualitatively different: the permutation P is uniquely determined by M (it records the ranks of the leading, upper-left, submatrices of M) and it depends discontinuously on M. In computable analysis every computable function on real inputs is continuous [89], so P cannot be computed from arbitrarily accurate approximations of M; it is therefore unclear what it would mean to compute the Bruhat decomposition numerically. From exact input it can be computed directly, by Gaussian elimination with a suitable choice of pivots (Section 4.7), and also by running the Gram–Schmidt process over a field of Laurent series (Theorem 4.11).[1] By contrast, LU factorisation with partial pivoting chooses P by comparisons of approximate magnitudes, and this choice is not required to be unique.[2]

More generally, the purpose of a decomposition in representation theory is usually to *classify* objects up to isomorphism, typically via the Krull–Schmidt theorem (Theorem 2.19) and its analogues for forms (Theorem 2.23). The purpose of a decomposition in numerical analysis is weaker: to find a basis in which certain computations become easy. Despite this difference, there is a clear pattern relating the computationally useful

[1] Care is needed for decompositions whose factors are not unique, such as the SVD, where the singular vectors are determined only up to symmetries. There the appropriate notion is that of a computable multi-valued function [89].

[2] The Jordan normal form suffers from the same difficulty as the Bruhat decomposition: its block structure is uniquely determined by the matrix and varies discontinuously with it. In numerical practice one replaces it by decompositions with less rigidly determined factors, such as the Schur decomposition. Our framework covers both kinds: the real Jordan form is among the decompositions of Theorem A(3), and Theorem B(4) computes the Bruhat decomposition from exact input, while the algorithms of Theorem B(1)–(3) compute decompositions of the latter kind, such as condensed forms.

decompositions, and it is this pattern that the thesis aims to exploit.

1.5 Related Work and the State of the Art

We now situate the thesis within existing work. No previous framework, to our knowledge, overloads the operations of numerical algorithms for finite-dimensional $*$-algebras, including non-semisimple ones, and uses the representation theory of quivers to identify the problem that the transported algorithm solves.

Structured numerical linear algebra. Structure-preserving algorithms for matrices that are symmetric, skew-symmetric, Hamiltonian, symplectic, or self-adjoint with respect to an indefinite form are surveyed in [12]. Mackey, Mackey and Tisseur [67] unify many structured factorisations by working in a *scalar product space*, that is, a real or complex vector space with a non-degenerate bilinear or sesquilinear form; the matrices that are self-adjoint, skew-adjoint or unitary with respect to the form give rise to Jordan algebras, Lie algebras and automorphism groups. This is the closest existing framework to ours. Its scalars remain $\mathbb{R}$ or $\mathbb{C}$, however, and the scalar product is an extra datum. In our setting the form is absorbed into the involution of a matrix $*$-algebra, and the scalars may themselves be quaternions, dual numbers, or elements of any finite-dimensional $*$-algebra. Conversely, matrices over many of these algebras can be represented by larger real or complex matrices on which the involution acts as the adjoint for a form: a quaternionic matrix by a complex matrix of twice the size, with the conjugate transpose [96], and a matrix (X, Y) over ${}^2\mathbb{R}$ by $\operatorname{diag}(X, Y)$, with the adjoint for the indefinite form $\left(\begin{smallmatrix} 0 & I \\ I & 0 \end{smallmatrix}\right)$. Quaternionic matrices and their SVD are treated in [96]; Hamiltonian square roots of skew-Hamiltonian matrices are studied in [28], and we use them in Section 3.3.

Classical relations between algorithms. Some of the relations that Theorem B establishes are classical in special cases. For a square upper bidiagonal matrix W, a perfect shuffle of the rows and columns of $\left(\begin{smallmatrix} 0 & W^T \\ W & 0 \end{smallmatrix}\right)$ gives a symmetric tridiagonal matrix with zero diagonal, whose eigenvalues are plus and minus the singular values of W [20, 33];

this is the encoding of Section 3.4 with the labels of the arrows removed. Two steps of the Cholesky LR algorithm on a positive definite matrix are one step of the QR algorithm [91]. The framework explains these relations by the algebra of the scalars, and extends them, under the hypotheses of Theorem B, to several of the other algebras of Chapter 6.

Lie-theoretic unification. Edelman and Jeong [27] derive fifty-three matrix factorisations systematically from the Cartan (KAK) decompositions of the classical symmetric spaces. Kleinsteuber, Helmke and Hüper [56] formulate Jacobi-type algorithms on compact Lie algebras, so that a single algorithm covers the symmetric, Hermitian, skew-symmetric and other eigenvalue problems. These works unify decompositions through the groups involved. Our approach is complementary: we unify through the *scalars*, so that the same program, with overloaded arithmetic, implements the different cases. The two viewpoints meet in Chapter 6, where the unitary groups of matrix $*$-algebras turn out to be exactly the classical groups.

Operator algebras in numerical analysis. Hagen, Roch and Silbermann [45] use C^*-algebras to study approximation methods for operators on infinite-dimensional Hilbert spaces. A typical result (see also [10]) is that the finite section method for a Toeplitz operator is stable if and only if a certain element of a quotient C^*-algebra of matrix sequences is invertible. There, C^*-algebras serve as a tool for proving stability and convergence of discretisations. In this thesis, $*$-algebras play a different role: they are the scalars over which a finite-dimensional algorithm is run. Moreover, our algebras are generally not C^*-algebras, since a finite-dimensional C^*-algebra is semisimple (Remark 2.5), whereas the non-semisimple algebras, such as the dual numbers, are essential to our applications. Section 9.2 discusses the possibility of extending our framework to operator algebras.

Automatic differentiation and perturbation theory. Forward-mode automatic differentiation evaluates a program over the dual numbers [36]. For matrix functions, the Fréchet derivative of f at A in direction E can be read off from f applied to the block matrix $\left(\begin{smallmatrix} A & E \\ 0 & A \end{smallmatrix}\right)$ [49], which is the same construction viewed through a matrix representa-

tion of the dual numbers. Analytic perturbation theory describes how eigenvalues and eigenvectors of a symmetric matrix depend on a parameter; by Rellich's theorem they can be chosen analytically [55], and analytic singular value decompositions can be computed numerically [11]. Decompositions of matrices over the dual numbers were studied by the author in [39]. Chapter 4 recovers and extends its spectral theorems and its T-SVD by a single algebraic argument.

Representation theory of quivers and of forms. A quiver is a directed graph, and the representations of quivers describe modules over finite-dimensional algebras [3, 22, 77]. Gabriel's theorem [31] and Kac's theorem [54] relate representation type to Dynkin diagrams and root systems, and Drozd's theorem [25] shows that every finite-dimensional algebra over an algebraically closed field is tame or wild. Species [24] extend quivers to non-algebraically closed fields. Representations with additional orthogonal or symplectic structure are described by symmetric quivers [21] and by Shmelkin's signed quivers [80]. Sergeichuk reduced the classification of systems of forms and linear maps to the classification of representations of quivers together with Hermitian forms over division algebras [79]; see also [51]. The decomposition of Hermitian forms into orthogonally indecomposable summands was studied in additive categories by Quebbemann, Scharlau and Schulte [71], and in the context of linear algebra with duality by Lorand [66]. Our doubly-signed quivers combine the signed quivers with a second signing that records real division algebras, and the novelty lies in using this combinatorial data to transform algorithms.

Real algebraic geometry and infinitesimals. Tarski's theorem [84] shows that the theory of real closed fields admits quantifier elimination, so that first-order statements transfer between real closed fields [9, 69]. Computational real algebraic geometry routinely uses formal infinitesimals to perturb inputs into general position [7]. Robinson's nonstandard analysis [74] provides infinitesimals with a transfer principle for a much richer language, but for the single extension $\mathbb{R} \subset {}^*\mathbb{R}$ and at the cost of constructivity. We use the former and compare it with the latter in Chapter 4.

Localisation of path algebras. The localisation of a path algebra at its arrows is a universal localisation in the sense of Cohn [16], and it is the algebra of the free groupoid of the quiver [32]. The free field of Cohn [16, 17] is the universal skew field of fractions of a free algebra, and Schofield [78] studied homomorphisms from rings to skew fields and to simple Artinian rings. Leavitt path algebras [1] and graph C^*-algebras [72] adjoin partial inverses instead. Each edge e acquires a "ghost" edge e^*, with e^*e the idempotent of its range, but ee^* is the idempotent of its source only when no other edge leaves that source; in general an edge is a partial isometry, not an invertible element. For a cycle the two constructions agree, $L_k(C_n) \cong M_n(k[x, x^{-1}])$, but for a vertex with two loops the Leavitt path algebra is the Leavitt algebra $L(1, 2)$, in which the loops have left inverses and are not invertible, whereas the localisation is the group algebra of the free group. The ghost edges are also new generators, whereas the involution of a doubly-signed quiver maps arrows to arrows of the same quiver.

Position of this thesis. The thesis combines these strands. From structured linear algebra and Lie theory it takes the catalogue of decompositions to be unified; from representation theory it takes the classification of the relevant $*$-algebras by combinatorial data; from model theory it takes the transfer principle that justifies running algorithms over fields with infinitesimals; and from the localisation of rings it takes the means of dividing by elements that are zero divisors in the original algebra, while the theory of skew fields shows how far such means can go.

1.6 Contributions and Outline

The main contributions of the thesis are the seven overloaded operations and the two outcomes of Section 1.2. We describe them, and the theory that supports them, chapter by chapter.

Chapter 2 fixes notation and collects the algebraic background: $*$-algebras, which supply the scalars; radicals, semisimple and Artinian algebras, the Wedderburn theorems, basic algebras and idempotents, and tensor products; the modules and Hermitian modules

on which the decompositions act; and quivers and path algebras. It states the Krull–Schmidt theorem and its analogue for Hermitian modules, and it determines exactly when the uniqueness part of the latter holds for quadratic forms.

Chapter 3 gives the seven operations their meaning over a $*$-algebra: the adjoint, which on matrices is the conjugate transpose; indexing by positions with self-adjoint idempotents, in a cyclic order through their orbits; division between idempotents; and $*$-square roots, which are not unique (Lemma 3.7). Householder reflectors work with any $*$-square root in the corner of the next position for which a certain denominator is invertible (Proposition 3.8). Over the completed localisation of the quiver of the singular value decomposition, in which its arrows are inverted, Householder tridiagonalisation becomes the bidiagonalisation of Golub and Kahan, and the cyclic order of the positions is what makes it so (Proposition 3.9 and Example 3.10); this is the first part of Theorem B.

Chapter 4 shows how Tarski's transfer principle for real closed fields, combined with Puiseux series, turns the computation of limits of algorithms into exact computation over a field with infinitesimals (an "auto-limit"), and compares this approach with Robinson's nonstandard analysis, which is not needed. Run over a field of Laurent series, which is the completed localisation of the quiver with one loop, the Gram–Schmidt process computes the Bruhat decomposition of every invertible matrix, with its permutation read off from the orders of the norms (Theorem 4.11); this is part (4) of Theorem B. The chapter also proves spectral theorems and singular value decompositions over formal power series rings, including a parity-equivariant version. Finally, it shows that Dijkstra's shortest-path algorithm is the leading-order part of Gaussian elimination over a field of Puiseux series, which relates the min-plus semiring to the operations of this thesis (Proposition 4.18). It ends with the chain of limits that leads from the Hermitian eigendecomposition, through the singular value decomposition and the QR factorisation, to the Bruhat and LU factorisations (Section 4.11).

Chapter 5 develops the graphical language of the thesis: quivers, symmetric quivers, signed quivers and doubly-signed quivers. Doubly-signed quivers are based on Shmelkin's signed quivers [80], adapted to the real numbers. In the signed-quiver formalism, signs

distinguish the possible involutions on basic algebras over algebraically closed fields and the sign $\varepsilon = \pm 1$ of ε-Hermitian forms; signing the vertices a second time lets the same formalism distinguish the real division algebras $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$ as well. Doubly-signed quivers classify the basic real $*$-algebras whose radical squares to zero (Theorem 5.11). The path $*$-algebra of a doubly-signed quiver is constructed explicitly, by paths with coefficient blocks of size at most two, in which quaternions and symplectic forms enter only through two matrices attached to each vertex (Definition 5.13 and Proposition 5.14); this is the form in which addition, subtraction and multiplication are computed. The main result of the chapter, an analogue of Gabriel's theorem, is Theorem A(1): every finite-dimensional real $*$-algebra is a matrix algebra, with sizes and signs at the vertices, over a quotient of the path $*$-algebra of its doubly-signed quiver, and the quiver, the sizes and the signatures are invariants (Theorem 5.16). The matrix part supplies the positions of the indexing (Corollary 5.18). The chapter also explains the role of the Tits form in determining representation type.

Chapter 6 proves the encoding theorem (Theorem 6.1), which is Theorem A(2) and makes precise the sense in which a quiver indexes a decomposition. A catalogue of doubly-signed quivers recovers the spectral theorems, the singular value decompositions over $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$, the Takagi factorisation, Sylvester's law of inertia and the real Jordan form, and a recipe shows how to encode a new problem (Section 6.2.10). The chapter then proves the rest of part (1) of Theorem B, and parts (2) and (3) (Propositions 6.4, 6.5 and 6.6); most of these proofs follow the generic program step by step and identify its intermediate quantities with those of the classical algorithm. Finally, it relates the success of generalised algorithms to the representation type of the quiver. When the quiver is wild, the number of continuous parameters needed to describe its representations grows quadratically with their dimension, so there is no canonical form of the familiar kind; for a vertex with two loops, the quotients needed by a Jacobi-type method do not exist in the completed localisation, although they exist in a non-commutative skew field.

Chapter 7 constructs the algebra in which the algorithms run, and in which division and the $*$-square root are carried out. Unifying algorithms sometimes requires dividing

one zero divisor by another, which is meaningless in the original algebra, and taking norms that vanish there. We lift to the path algebra, invert every arrow between its end points and complete in the path degree. In the resulting *completed localisation*, which keeps the involution, inverses are computed by series and by elimination, and the elimination is complete when the underlying graph of every component of the quiver has at most one cycle, and that cycle has non-zero degree (Proposition 7.6). $*$-square roots are computed degree by degree, and results return to the original algebra whenever the quantities that certify them contain no inverse letters (Theorem 7.8). Beyond the completed localisation, the path algebra of every finite quiver embeds, constructively, in a full matrix ring over a skew field, in which every non-zero element between two vertices is invertible (Theorem 7.11); the skew field can be taken commutative, a field of rational functions, if and only if no vertex lies on two distinct simple oriented cycles. After complexification, the localisation also embeds, compatibly with the involution and the conjugation, in a finite product of simple Artinian rings that they permute.

Chapter 8 describes a Python library that implements the seven operations, and the matrix algorithms written against them, in the completed localisation of the path $*$-algebra of any doubly-signed quiver. It explains how the parts of the library correspond to the theory, and it reports experiments in which the generic algorithms, run over the quivers of the catalogue and over ${}^{2}\mathbb{R}$, reproduce classical algorithms, such as the bidiagonalisation of Golub and Kahan, the LR algorithm of Rutishauser and the Doolittle LU decomposition, to within rounding errors.

Chapter 9 summarises the results, with the status of each, discusses the extent to which the framework is inherently finite-dimensional, and lists limitations and open problems.

Appendix A, which is adapted from the preprint [41], visualises the dynamics of the QR algorithm on 2×2 real matrices, in a way inspired by Yaglom's accounts of the geometry of low-dimensional algebras [92, 93]. It uses the geometry of the projective lines over the two-dimensional $*$-algebras of this thesis, Möbius geometry for the complex numbers, Laguerre geometry for the dual numbers and Minkowski geometry (also called hyperbolic Laguerre geometry) for ${}^{2}\mathbb{R}$, and two-dimensional Lie sphere geometry, which contains all three. The

group of Lie sphere geometry has identity component $\mathrm{PSp}(4,\mathbb{R})$, which is the projective unitary group of the 2×2 matrices over $M_2(\mathbb{R})$ with its symplectic involution. In the resulting picture an arbitrary 2×2 matrix with non-negative determinant is represented, up to a non-zero scalar factor, by a pair of mutually inverse oriented circles.

Appendix B contains the source code of the library of Chapter 8; its test suite is part of the source distribution submitted with the thesis.

Reading guide. Figure 1.1 shows how the chapters depend on each other. Chapter 2 collects background and can be consulted as needed. The main line runs through Chapters 3, 5, 6 and 7. Sections 6.3 and 6.5 use results of Chapter 7; Section 6.3 begins with a summary of what it uses, so Chapter 7 can also be read after Chapter 6. Chapter 4 is a side branch: it is needed for the limits of algorithms in Section 1.3.1, for the transfer principle with which Proposition 3.6 passes from $\mathbb{R}$ to real closed fields, for Theorem B(4), and for the power-series results to which the catalogue of Chapter 6 refers. Chapter 8 describes the implementation of Chapters 3 to 7, and Appendix A is independent of the rest.

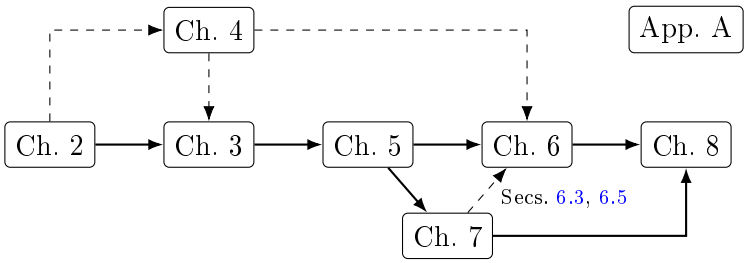


Figure 1.1: Dependencies between the chapters. Solid arrows show the main line. Dashed arrows show the side branch of Chapter 4, and the use of Chapter 7 in Sections 6.3 and 6.5. Appendix A is independent of the rest.

Chapter 2

$*$-Algebras, Modules and Quivers

This chapter fixes notation and recalls the algebraic objects on which the thesis is built: $*$-algebras, which supply the scalars for the seven operations of Section 1.1; the structure of finite-dimensional algebras (radicals, semisimple and Artinian rings, the theorems of Wedderburn, basic algebras and idempotents, tensor products); modules over them, and modules equipped with Hermitian forms; and quivers and path algebras, which describe finite-dimensional algebras combinatorially. Most of the material is standard; the references are [3, 57, 58, 60, 76]. The main point of the chapter is Section 2.5, which states the Krull–Schmidt theorem and its analogue for Hermitian modules, and determines precisely when the uniqueness part of the latter can hold for quadratic forms.

2.1 Notation and Conventions

Convention 2.1. Throughout the thesis the following conventions are in force.

1. Rings are associative and have an identity element; ring homomorphisms preserve the identity. Unless stated otherwise, k denotes a field of characteristic different from 2, and algebras are finite-dimensional over k.

2. The involution of a $*$-ring or $*$-algebra is always written $x \mapsto x^*$, using the (centred) mathematical asterisk, and x^* is called the *adjoint* of x. Complex conjugation of a scalar is written $z \mapsto \bar{z}$, and the transpose of a matrix X is written X^T. For

matrices of real or complex numbers we write $X^H = \bar{X}^T$.

3. If A is a $*$-algebra, the *conjugate transpose* of $X = (x_{ij}) \in M_n(A)$ is the matrix X^* with entries $(X^*)_{ij} = (x_{ji})^*$. With this involution $M_n(A)$ is again a $*$-algebra. We write $\mathrm{Herm}_n(A) = \{X \in M_n(A) : X^* = X\}$ and $\mathrm{U}_n(A) = \{U \in M_n(A) : U^*U = I\}$.

4. The symbols $\dagger$, $(\cdot)^\top$ and $\overline{(\cdot)}$ are reserved for involutions of quivers (Chapter 5); they are never used for the involution of an algebra. When two different involutions of the same algebra have to be compared, the second one receives its own symbol, such as X^ω for the symplectic involution (Section 3.3) or $x^\natural$ for the involution induced on a quotient (Lemma 5.8).

5. ${}^2\mathbb{R}$ denotes the $*$-algebra $\mathbb{R} \oplus \mathbb{R}$ with involution $(x, y)^* = (y, x)$.

2.2 $*$-Algebras

Many algebras that occur in analysis, geometry and physics carry a natural "adjoint" or "conjugation" operation. Examples are complex conjugation, the transpose and conjugate transpose of matrices, and the adjoint of an operator on a Hilbert space. All of these reverse the order of products and square to the identity, and abstracting these two properties leads to the notion of an involution.

Definition 2.2. A *$*$-ring* (or *involutive ring*) is a ring A together with a map $A \to A$, $x \mapsto x^*$, called an *involution*, such that for all $x, y \in A$:

1. $(x^*)^* = x$;

2. $(x + y)^* = x^* + y^*$ and $(xy)^* = y^*x^*$.

It follows from the axioms that $1^* = 1$, since $1^* = 1^*(1^*)^* = (1^*\,1)^* = (1^*)^* = 1$. Thus the involution is an anti-automorphism of A that is its own inverse.

Definition 2.3 ($*$-algebra)**.** Let R be a commutative $*$-ring with involution $r \mapsto r'$. A *$*$-algebra over R* is an associative R-algebra A which is also a $*$-ring, such that

$$(rx)^* = r'x^* \qquad \text{for all } r \in R,\ x \in A.$$

A $*$-*homomorphism* $f : A \to B$ between $*$-algebras is an algebra homomorphism such that $f(a^*) = f(a)^*$ for all $a \in A$.

If $R = \mathbb{C}$ with complex conjugation, the involution of A is *conjugate-linear*: $(\lambda x)^* = \bar{\lambda} x^*$. If the involution of R is trivial, as for $R = \mathbb{R}$, the involution of A is R-linear. Every $*$-ring is a $*$-algebra over $\mathbb{Z}$ with the trivial involution. Unless said otherwise, the $*$-algebras in this thesis are finite-dimensional over $k = \mathbb{R}$ (or over a real closed field), with k-linear involution.

Example 2.4. The following $*$-algebras will be used repeatedly.

1. $\mathbb{C}$ with $z^* = \bar{z}$, viewed as a two-dimensional $\mathbb{R}$-algebra.
2. $M_n(\mathbb{R})$ with $X^* = X^T$; $M_n(\mathbb{C})$ with $X^* = X^H$.
3. The quaternions $\mathbb{H}$ with quaternion conjugation $(a+bi+cj+dk)^* = a-bi-cj-dk$.
4. The *dual numbers* $\mathbb{R}[\varepsilon]/(\varepsilon^2)$, with either the trivial involution $\varepsilon^* = \varepsilon$ or the involution $\varepsilon^* = -\varepsilon$.
5. The algebra ${}^2\mathbb{R}$ of Convention 2.1, whose involution exchanges the two summands.
6. $M_{2n}(k)$ with the *symplectic involution* $X^* = J^{-1}X^TJ$, where $J = \left(\begin{smallmatrix} 0 & I_n \\ -I_n & 0 \end{smallmatrix}\right)$.
7. For any $*$-algebra A, the matrix algebra $M_n(A)$ with the conjugate transpose of Convention 2.1.

The algebras in (2), (3) and (6) are *semisimple*, that is, they have no non-zero nilpotent ideals (Definition 2.6); the dual numbers are not, since $\mathbb{R}\varepsilon$ is a nilpotent ideal, and ${}^2\mathbb{R}$ is semisimple but not simple.

Remark 2.5 (C^*-algebras). A C^*-*algebra* is a complex Banach algebra with a conjugate-linear involution whose norm satisfies $\|x^*x\| = \|x\|^2$ for all x. Every finite-dimensional C^*-algebra is $*$-isomorphic to a finite direct sum $\bigoplus_i M_{n_i}(\mathbb{C})$ with the conjugate-transpose involution [19, Theorem III.1.1]; in particular it is semisimple. The C^*-identity implies that $x^*x = 0$ forces $x = 0$, and that a self-adjoint nilpotent element h is zero, since

$\|h\|^{2^m} = \|h^{2^m}\|$ for all m. The dual numbers violate this in either of their involutions: $\varepsilon^*\varepsilon = \pm\varepsilon^2 = 0$ although $\varepsilon \neq 0$. More generally, every finite-dimensional $*$-algebra A with a non-zero radical (the largest nilpotent ideal of A; Section 2.3.1) admits no C^*-norm: the radical is stable under the involution (Lemma 2.11), so for $0 \neq x \in \mathrm{rad}(A)$ the element $h = x^*x$ is self-adjoint and nilpotent, hence $h = 0$ and then $x = 0$. Since non-semisimple algebras are central to this thesis, the theory of C^*-algebras does not apply directly to our setting; we return to this point in Section 9.2.

2.3 The Structure of Finite-Dimensional Algebras

2.3.1 Radicals, Semisimple Algebras and Artinian Rings

Let A be a ring. The *(Jacobson) radical* $\mathrm{rad}(A)$ is the intersection of all maximal left ideals of A; it is a two-sided ideal, and it also equals the intersection of all maximal right ideals [60, §4]. If A is a finite-dimensional algebra over a field, then $\mathrm{rad}(A)$ is the largest nilpotent two-sided ideal of A: there is an N with $\mathrm{rad}(A)^N = 0$, and every nilpotent two-sided ideal is contained in $\mathrm{rad}(A)$. Intuitively, the radical consists of the "infinitesimal" elements of A, which are invisible to the simple modules.

Definition 2.6. A finite-dimensional algebra A is *semisimple* if $\mathrm{rad}(A) = 0$. It is *simple* if its only two-sided ideals are 0 and A.

Definition 2.7. A ring R is *left Artinian* if every descending chain $I_1 \supseteq I_2 \supseteq \cdots$ of left ideals of R is eventually constant. A *simple Artinian ring* is a non-zero left Artinian ring whose only two-sided ideals are 0 and R.

Every finite-dimensional algebra is Artinian, since a strictly descending chain of subspaces has length at most the dimension. The ring $\mathbb{Z}$ is not Artinian, because $2\mathbb{Z} \supsetneq 4\mathbb{Z} \supsetneq 8\mathbb{Z} \supsetneq \cdots$. The following theorem, which is the theorem of Wedderburn used in Chapter 7, describes semisimple and simple Artinian rings completely.

Theorem 2.8 (Wedderburn–Artin)**.** *A ring R is left Artinian with* $\mathrm{rad}(R) = 0$ *if and*

only if

$$R \cong M_{n_1}(D_1) \times \cdots \times M_{n_r}(D_r)$$

for some integers $n_i \geq 1$ and division rings (skew fields) D_i. The pairs (n_i, D_i) are uniquely determined up to isomorphism of the D_i and permutation. In particular, R is simple Artinian if and only if $R \cong M_n(D)$ for a single n and a single division ring D.

See [60, Theorems 3.5 and 3.10]. Over $k = \mathbb{R}$ the finite-dimensional division algebras are $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$ (Frobenius' theorem), so every finite-dimensional semisimple real algebra is a product of matrix algebras over $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$. We shall also need the following classical result about automorphisms of simple algebras. An algebra over k is *central simple* if it is simple and its centre is k; examples are $M_n(k)$ and, for $k = \mathbb{R}$, $M_n(\mathbb{H})$.

Theorem 2.9 (Skolem–Noether)**.** *Let A be a finite-dimensional central simple algebra over a field k. Every k-algebra automorphism φ of A is inner: there is an invertible $g \in A$ with $\varphi(x) = gxg^{-1}$ for all $x \in A$.*

See [53, 58]. Applied to the composite of an anti-automorphism with the transpose, it shows that every anti-automorphism of $M_n(k)$ has the form $X \mapsto GX^TG^{-1}$; this is how the types of involutions are determined in Chapter 5.

Example 2.10.

1. The dual numbers $\mathbb{R}[\varepsilon]/(\varepsilon^2)$ have radical $\mathbb{R}\varepsilon$ and $A/\operatorname{rad}(A) \cong \mathbb{R}$.
2. In the algebra $T_n(k)$ of lower triangular $n \times n$ matrices, the radical consists of the strictly lower triangular matrices, and $T_n(k)/\operatorname{rad}(T_n(k)) \cong k^n$, the diagonal.
3. $M_n(\mathbb{R})$, $\mathbb{H}$, $\mathbb{C}$ and ${}^2\mathbb{R}$ are semisimple; $M_n(\mathbb{R})$ and $\mathbb{H}$ are simple, whereas ${}^2\mathbb{R}$ is not.

Lemma 2.11. *If A is a finite-dimensional $*$-algebra, then* $\operatorname{rad}(A)^* = \operatorname{rad}(A)$.

Proof. The involution is an anti-automorphism, so $\operatorname{rad}(A)^*$ is a two-sided ideal, and it is nilpotent because $(\operatorname{rad}(A)^*)^N = (\operatorname{rad}(A)^N)^* = 0$. Hence $\operatorname{rad}(A)^* \subseteq \operatorname{rad}(A)$. Applying the involution once more gives the reverse inclusion. □

The semisimple quotient $A/\operatorname{rad}(A)$ can be realised inside A. The following result is the associative analogue of the Levi decomposition of a Lie algebra.

Theorem 2.12 (Wedderburn–Malcev)**.** *Let A be a finite-dimensional algebra over a field k of characteristic 0 (more generally, assume that $A/\operatorname{rad}(A)$ is separable over k). Then there is a subalgebra $S \subseteq A$ such that $A = S \oplus \operatorname{rad}(A)$ as vector spaces; in particular $S \cong A/\operatorname{rad}(A)$. Any two such subalgebras are conjugate by an element of the form $1 + r$ with $r \in \operatorname{rad}(A)$. If A is a $*$-algebra, S can be chosen with $S^* = S$.*

See [70, §11.6] for the first two statements and [81] for the last. We call $A = S \oplus \operatorname{rad}(A)$ a *Wedderburn–Malcev decomposition* of A. When $\operatorname{rad}(A)^2 = 0$, the existence of a $*$-stable S has an elementary proof, which we give in Lemma 5.8.

2.3.2 Basic Algebras and Idempotents

In ordinary linear algebra the entries of a matrix are scalars, and a matrix algebra $M_n(k)$ is built from the one-dimensional algebra k. For a general finite-dimensional algebra, the role of the scalars is played by its *basic algebra*: the smallest algebra from which it can be rebuilt by forming matrices. We first give the definition, which uses only the semisimple quotient, and then explain how idempotents make the idea precise.

Definition 2.13. A finite-dimensional algebra A is *basic* if $A/\operatorname{rad}(A)$ is a product of division algebras, that is, if all the integers n_i in Theorem 2.8 for $A/\operatorname{rad}(A)$ equal 1.

For example, $\mathbb{R}$, $\mathbb{C}$, $\mathbb{H}$, the dual numbers, ${}^2\mathbb{R}$ and $T_n(k)$ are basic, whereas $M_n(k)$ for $n \geq 2$ is not. Its basic algebra is k, from which it is rebuilt as $M_n(k)$.

An element $e \in A$ is an *idempotent* if $e^2 = e$. Idempotents e, f are *orthogonal* if $ef = fe = 0$, and a non-zero idempotent is *primitive* if it is not the sum of two non-zero orthogonal idempotents. A *complete set of primitive orthogonal idempotents* is a set $\{e_1, \ldots, e_n\}$ of pairwise orthogonal primitive idempotents with $e_1 + \cdots + e_n = 1$. In $M_n(k)$ the diagonal matrix units $E_{11}, \ldots, E_{nn}$ form such a set. Given such a set, every $a \in A$ decomposes as

$$a = \sum_{i,j} e_i a e_j, \qquad e_i a e_j \in e_i A e_j, \tag{2.1}$$

the *Peirce decomposition*, which generalises the decomposition of a matrix into its entries. Two idempotents e, f are *equivalent* if the left modules Ae and Af are isomorphic; equiv-

alently, there are $u \in fAe$ and $v \in eAf$ with $vu = e$ and $uv = f$. A finite-dimensional algebra is basic if and only if, in a complete set of primitive orthogonal idempotents, no two distinct idempotents are equivalent [3, §I.6]. If e is the sum of one primitive idempotent from each equivalence class, then $AeA = A$ and the algebra eAe is basic. The algebras A and eAe are then *Morita equivalent*: their categories of modules are equivalent [3, 4]. In this sense every finite-dimensional algebra behaves like a "matrix algebra over its basic algebra".

For $*$-algebras we need a variant that takes the involution into account, since a corner fAf is a $*$-algebra only when f is self-adjoint. The involution maps idempotents to idempotents, but it need not fix any primitive idempotent. For instance, in $M_2(\mathbb{R})$ with the symplectic involution $X^* = J^{-1}X^TJ$, the only self-adjoint elements are the scalar matrices, and the involution exchanges E_{11} and E_{22}; this $*$-algebra cannot be cut down to a smaller one. We therefore count only self-adjoint idempotents.

Definition 2.14. A finite-dimensional $*$-algebra A is *basic as a $*$-algebra* if every self-adjoint idempotent of $A/\operatorname{rad}(A)$ is central. If $f \in A$ is a self-adjoint idempotent with $AfA = A$ and fAf is basic as a $*$-algebra, then fAf is called a *basic $*$-algebra of A*.

Without the word "self-adjoint", the condition would say that $A/\operatorname{rad}(A)$ is a product of division algebras, which is Definition 2.13, because $M_n(D)$ has a non-central idempotent exactly when $n \geq 2$. Thus $M_n(\mathbb{R})$ with the transpose has the basic $*$-algebra $E_{11}M_n(\mathbb{R})E_{11} \cong \mathbb{R}$, whereas $M_2(\mathbb{R})$ with the symplectic involution is itself basic as a $*$-algebra, although it is not basic as an algebra. The difference between the two is exactly the difference between the vertex signs $+1$ and -1 of a signed quiver (Chapter 5). It would not suffice to ask that the involution permute a complete set of primitive idempotents: in $M_2(\mathbb{R})$ with the involution $X \mapsto \eta X^T \eta$, $\eta = \operatorname{diag}(1, -1)$, the involution exchanges the primitive idempotents $\frac{1}{2}\left(\begin{smallmatrix} 1 & \pm 1 \\ \pm 1 & 1 \end{smallmatrix}\right)$, but E_{11} is a non-central self-adjoint idempotent, and the basic $*$-algebra is $E_{11}M_2(\mathbb{R})E_{11} \cong \mathbb{R}$. Over $\mathbb{R}$, a $*$-algebra A is basic as a $*$-algebra exactly when $A/\operatorname{rad}(A)$ is a product of copies of the ten $*$-algebras of Table 5.1, and every finite-dimensional real $*$-algebra has a basic $*$-algebra (Lemma 5.12 and Theorem 5.16).

2.3.3 Tensor Products

Definition 2.15. Let A and B be algebras over k. Their *tensor product* $A \otimes_k B$ is the vector space tensor product with multiplication determined by $(a \otimes b)(a' \otimes b') = aa' \otimes bb'$. If A and B are $*$-algebras, $A \otimes_k B$ is a $*$-algebra with $(a \otimes b)^* = a^* \otimes b^*$.

The tensor product of matrix algebras is the *Kronecker product*: the map $X \otimes Y \mapsto X \otimes_{\mathrm{Kr}} Y$, where $X \otimes_{\mathrm{Kr}} Y$ is the block matrix $(x_{ij}Y)_{ij}$, is an isomorphism $M_m(k) \otimes_k M_n(k) \cong M_{mn}(k)$. More generally $M_n(A) \cong M_n(k) \otimes_k A$, and if A is a $*$-algebra this is a $*$-isomorphism when $M_n(k)$ carries the transpose and $M_n(A)$ the conjugate transpose. Two further examples are used in Chapter 5: the complexification $B \otimes_{\mathbb{R}} \mathbb{C}$ of a real algebra B, for instance $\mathbb{C} \otimes_{\mathbb{R}} \mathbb{C} \cong \mathbb{C} \times \mathbb{C}$ and $\mathbb{H} \otimes_{\mathbb{R}} \mathbb{C} \cong M_2(\mathbb{C})$.

2.4 Modules and Hermitian Modules

Linear algebra is built on vector spaces over fields. Modules are the analogue of vector spaces when the scalars form a ring [26, 60].

Definition 2.16 (Module)**.** Let R be a ring. A *left R-module* is an abelian group $(M, +)$ together with a map $R \times M \to M$, $(r, m) \mapsto rm$, such that for all $r, s \in R$ and $m, n \in M$:

$$r(m+n) = rm + rn, \qquad (r+s)m = rm + sm, \qquad (rs)m = r(sm), \qquad 1m = m.$$

A *right R-module* is defined analogously with a map $M \times R \to M$, $(m, r) \mapsto mr$, satisfying in particular $m(rs) = (mr)s$.

A module is *indecomposable* if it is non-zero and is not the direct sum of two non-zero submodules. It is *Noetherian* (respectively *Artinian*) if every ascending (respectively descending) chain of submodules is eventually constant, and it has *finite length* if it is both. Every finitely generated module over a finite-dimensional algebra has finite length.

Let A be a $*$-algebra and M a left A-module. The *dual module* $M^\vee = \mathrm{Hom}_A(M, A)$ consists of the left A-linear maps $f : M \to A$. It becomes a left A-module through the involution, by setting $(a \cdot f)(m) = f(m)\, a^*$.

We now generalise the Hermitian inner product on $\mathbb{C}^n$ to modules over $*$-algebras. The key notion is that of a sesquilinear form, "sesqui" (Latin for "one and a half") referring to linearity in one argument and conjugate-linearity in the other.

Definition 2.17 (Sesquilinear and ε-Hermitian forms)**.** Let A be a $*$-algebra and M a left A-module. A *sesquilinear form* on M is a biadditive map $h : M \times M \to A$ such that for all $m, n \in M$ and $a \in A$:

$$h(am, n) = a\, h(m, n), \qquad h(m, an) = h(m, n)\, a^*.$$

Let $\varepsilon \in \{1, -1\}$. The form is *$\varepsilon$-Hermitian* if $h(n, m) = \varepsilon\, h(m, n)^*$ for all $m, n \in M$; it is *Hermitian* if $\varepsilon = 1$ and *skew-Hermitian* if $\varepsilon = -1$. It is *non-degenerate* if M is finitely generated and projective and the map $M \to M^\vee$, $n \mapsto h(\cdot, n)$, is an isomorphism. A pair (M, h) consisting of a module and a non-degenerate ε-Hermitian form is called an *ε-Hermitian module*, or simply a *Hermitian module* when $\varepsilon = 1$.

For $A = \mathbb{C}$ with complex conjugation this is the usual notion of a Hermitian form on $\mathbb{C}^n$, except that positivity is not required. If (M, h) is Hermitian then $h(m, m) = h(m, m)^*$ is self-adjoint for every m, generalising the fact that $\langle v, v \rangle$ is real for a complex inner product.

Example 2.18.

1. If $A = \mathbb{R}$ with the trivial involution, Hermitian modules are real vector spaces with a non-degenerate symmetric bilinear form, and skew-Hermitian modules are symplectic vector spaces.

2. If $A = \mathbb{C}$ with complex conjugation, a Hermitian form on $\mathbb{C}^n$ is $h(x, y) = y^H S x$ for a Hermitian matrix S; skew-Hermitian forms are i times Hermitian forms.

3. Let $A = {}^2\mathbb{R}$ and let $e_1 = (1, 0)$ and $e_2 = (0, 1)$, so that $e_1^* = e_2$. An A-module is a pair $M = M_1 \oplus M_2$ of real vector spaces, with $M_i = e_i M$. Since $h(e_1 m, e_1 n) = e_1 h(m, n) e_1^* = e_1 e_2\, h(m, n) = 0$, a sesquilinear form pairs M_1 only with M_2, and an ε-Hermitian form is determined by a bilinear pairing $M_1 \times M_2 \to \mathbb{R}$. Non-degeneracy says that this pairing identifies M_2 with the dual space $M_1^\vee$. The isometries are

the maps (P, P^{-T}) with $P \in \mathrm{GL}(M_1)$. This is the structure that turned unitary similarity into ordinary similarity in Section 1.3.2.

A Hermitian form brings geometric notions into the purely algebraic setting of modules. Let (M, h) be an ε-Hermitian module.

- Two elements $m, n \in M$ are *orthogonal*, written $m \perp n$, if $h(m, n) = 0$. Because h is ε-Hermitian, $m \perp n$ if and only if $n \perp m$.
- The *orthogonal complement* of a submodule $W \subseteq M$ is $W^{\perp} = \{m \in M : h(m, w) = 0 \text{ for all } w \in W\}$.
- If $M = M_1 \oplus M_2$ with $M_1 \subseteq M_2^{\perp}$, the decomposition is an *orthogonal sum*, written $(M, h) = (M_1, h_1) \perp (M_2, h_2)$, where h_i is the restriction of h.
- An *isometry* $(M, h) \to (M', h')$ is an isomorphism $g : M \to M'$ with $h'(gm, gn) = h(m, n)$. The isometries of (M, h) to itself form the *unitary group* $\mathrm{U}(M, h)$.
- The *adjoint* of $T \in \mathrm{End}_A(M)$ is the unique $T^{\sharp} \in \mathrm{End}_A(M)$ with $h(Tm, n) = h(m, T^{\sharp}n)$; it exists by non-degeneracy. T is *self-adjoint* if $T^{\sharp} = T$. For $M = A^n$ and the standard form $h(x, y) = \sum_i x_i y_i^*$, the adjoint is the conjugate transpose and $\mathrm{U}(M, h) = \mathrm{U}_n(A)$.

The decompositions of numerical linear algebra are, in this language, orthogonal decompositions of a Hermitian module into submodules invariant under a self-adjoint endomorphism. For example, the spectral theorem for a Hermitian matrix $S \in M_n(\mathbb{C})$ asserts that $(\mathbb{C}^n, h)$, with h the standard form, is the orthogonal sum of the eigenspaces of S.

A particularly simple class of ε-Hermitian modules is formed by the hyperbolic modules. If P is a finitely generated projective A-module, the *hyperbolic module* $\mathbb{H}_{\varepsilon}(P)$ is $P \oplus P^{\vee}$ with the form

$$h\big((p, f), (q, g)\big) = g(p) + \varepsilon\, f(q)^*.$$

One checks directly that h is sesquilinear and ε-Hermitian, and it is non-degenerate. The hyperbolic module depends only on P and not on any form on P. For example, for

$A = P = \mathbb{R}$ and $\varepsilon = 1$, the hyperbolic plane $\mathbb{H}_1(\mathbb{R})$ has the Gram matrix $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, which is congruent to $\left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$ via the change of basis $\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, up to scaling.

2.5 The Krull–Schmidt Theorem and Its Hermitian Analogue

None of the results in this section is new; they are recalled because the classification problems addressed later are of this kind, and because the precise statements clarify what can and cannot be expected of numerical decompositions.

Theorem 2.19 (Krull–Schmidt)**.** *Let R be a ring and M a non-zero R-module of finite length. Then*

1. *$M \cong M_1 \oplus \cdots \oplus M_r$ for some indecomposable modules $M_1, \ldots, M_r$;*
2. *if also $M \cong N_1 \oplus \cdots \oplus N_s$ with every N_j indecomposable, then $r = s$ and there is a permutation π of $\{1, \ldots, r\}$ such that $M_i \cong N_{\pi(i)}$ for all i.*

For a proof see [60, §19] or [3]. Existence follows by induction on the length. Uniqueness rests on *Fitting's lemma*: every endomorphism of an indecomposable module of finite length is either invertible or nilpotent. It follows that the endomorphism ring of such a module is a local ring.

Remark 2.20. The uniqueness in Theorem 2.19 is uniqueness *up to isomorphism*: the summands themselves, as submodules, are generally not unique. For example, $k^2 = ke_1 \oplus ke_2 = k(e_1 + e_2) \oplus ke_2$. This is the essential difference between classification in representation theory and decomposition in numerical analysis, where one wants an actual basis or an actual collection of submodules.

We now ask for an analogue of the Krull–Schmidt theorem for Hermitian modules. A non-degenerate ε-Hermitian module (M, h) is *orthogonally indecomposable* if $M \neq 0$ and (M, h) is not an orthogonal sum of two non-zero ε-Hermitian modules. Two questions arise: does every (M, h) decompose as an orthogonal sum of orthogonally indecomposable

modules, and is such a decomposition unique up to isometry of the summands? The answers are, respectively, "yes" and "it depends on the field" [66, 71, 79].

Proposition 2.21. *Let A be a finite-dimensional $*$-algebra. Every non-degenerate ε-Hermitian A-module is an orthogonal sum of finitely many orthogonally indecomposable ε-Hermitian modules.*

Proof. We argue by induction on the length of M. If (M, h) is orthogonally indecomposable there is nothing to prove. Otherwise $(M, h) = (M_1, h_1) \perp (M_2, h_2)$ with $M_1, M_2 \neq 0$. The summands are again non-degenerate, because $M^\vee = M_1^\vee \oplus M_2^\vee$ and the map $n \mapsto h(\cdot, n)$ respects the decomposition; and both have smaller length than M. The induction hypothesis applies to each summand. □

Uniqueness is more delicate. It fails already in the simplest case $A = k$ with the trivial involution, where Hermitian modules are symmetric bilinear spaces. In characteristic different from 2 every such space has an orthogonal basis, so the orthogonally indecomposable modules are the one-dimensional forms $\langle a \rangle$, $a \in k^\times$, and $\langle a \rangle \cong \langle b \rangle$ if and only if $b \in a\, k^{\times 2}$. Uniqueness of orthogonal decompositions therefore means uniqueness of diagonalisations up to squares. Recall that a field is *quadratically closed* if every element is a square, and *Euclidean* if it is ordered and every positive element is a square (for instance $\mathbb{R}$, or any real closed field).

Proposition 2.22. *Let k be a field of characteristic different from 2. The following are equivalent.*

1. *Every non-degenerate symmetric bilinear space over k has a unique orthogonal decomposition into one-dimensional summands, up to isometry and permutation of the summands.*

2. *k is quadratically closed or Euclidean.*

Proof. (2)⇒(1). If k is quadratically closed, every one-dimensional form is isometric to $\langle 1 \rangle$, and the number of summands is the dimension. If k is Euclidean, every one-dimensional form is isometric to $\langle 1 \rangle$ or $\langle -1 \rangle$, and the numbers of each are determined by

Sylvester's law of inertia, whose usual proof works over any ordered field in which positive elements are squares [61].

(1)⇒(2). We use the elementary fact that if $c \in k^\times$ is represented by the form $\langle a, b\rangle$, then $\langle a, b\rangle \cong \langle c, abc\rangle$ [61]. Suppose first that -1 is a square. Then $\langle 1, 1\rangle \cong \langle 1, -1\rangle$ is a hyperbolic plane, which represents every $c \in k^\times$, so $\langle 1, 1\rangle \cong \langle c, c\rangle$; by (1), c is a square. Hence k is quadratically closed. Suppose next that -1 is not a square. The hyperbolic plane $\langle 1, -1\rangle$ again represents every c, so $\langle 1, -1\rangle \cong \langle c, -c\rangle$, and (1) forces $c \in k^{\times 2}$ or $c \in -k^{\times 2}$. Thus $k^\times = k^{\times 2} \cup (-k^{\times 2})$. Moreover, if $c = x^2 + y^2 \neq 0$ then $\langle 1, 1\rangle \cong \langle c, c\rangle$, and (1) forces $c \in k^{\times 2}$, so sums of squares are squares; in particular -1 is not a sum of squares, so k is formally real. Declaring the non-zero squares to be the positive elements then defines an ordering in which every positive element is a square, so k is Euclidean. □

Proposition 2.22 shows that uniqueness fails over most fields of characteristic different from 2, for example over $\mathbb{Q}$ (where $\langle 1, 1\rangle \cong \langle 2, 2\rangle$ but $\langle 1\rangle \ncong \langle 2\rangle$), over finite fields of odd characteristic (where $\langle 1, 1\rangle \cong \langle u, u\rangle$ for a non-square u), and over $\mathbb{R}(t)$. In characteristic 2 the theory is different again: symmetric bilinear forms need not be diagonalisable (alternating forms are not), and quadratic forms are no longer determined by their bilinear forms; we exclude this case throughout. For the fields used in this thesis, uniqueness holds.

Theorem 2.23 (Hermitian Krull–Schmidt theorem)**.** *Let k be an algebraically closed field of characteristic different from 2, or a real closed field (for example $\mathbb{R}$). Let A be a finite-dimensional $*$-algebra over k and $\varepsilon \in \{1, -1\}$. Then every non-degenerate ε-Hermitian A-module is an orthogonal sum of orthogonally indecomposable ε-Hermitian modules, and the summands are unique up to isometry and permutation.*

Existence is Proposition 2.21. For uniqueness we refer to [71, 79] (see also [57, 66, 87]) and only outline the argument, which is a chain of reductions.

1. *Reduction to the semisimple case.* Let $J = \mathrm{rad}(A)$ be the radical of A, its largest nilpotent ideal (Section 2.3.1), which is stable under the involution (Lemma 2.11). The form h induces a non-degenerate form on M/JM over A/J. Orthogonal decom-

positions of M/JM lift to orthogonal decompositions of M, uniquely up to isometry, by lifting idempotents (elements e with $e^2 = e$; Section 2.3.2) of $\mathrm{End}_A(M)$ modulo its radical compatibly with the adjoint involution [71].

2. *Reduction to simple $*$-stable factors.* A semisimple $*$-algebra is a product of simple algebras that are either stable under the involution or exchanged in pairs $B \times B^{\mathrm{op}}$. Over an exchanged pair, every ε-Hermitian module is hyperbolic, $\mathbb{H}_\varepsilon(P)$ for a B-module P, and its decompositions are governed by the ordinary Krull–Schmidt theorem for P.

3. *Reduction to division algebras.* A simple $*$-stable algebra is $M_n(D)$ for a division algebra D (Theorem 2.8). Hermitian Morita theory [57, Ch. I, §9] identifies ε-Hermitian modules over $M_n(D)$ with ε'-Hermitian forms over D, with respect to a suitable involution of D and some $\varepsilon' = \pm 1$. Concretely, for a rank-one idempotent $e \in M_n(D)$, a module M over $M_n(D)$ corresponds to the D-vector space eM, and the form on M corresponds to an ε'-Hermitian form on eM. For example, for $M_{2m}(\mathbb{R})$ with the symplectic involution, Hermitian modules correspond to skew-symmetric bilinear forms over $\mathbb{R}$.

4. *Division algebras.* Over a division algebra with involution, non-alternating forms are diagonalisable and alternating ones are sums of hyperbolic planes. Uniqueness thus reduces to the uniqueness of diagonalisations. When k is real closed, D is k, $k(\sqrt{-1})$ or the quaternions over k, and uniqueness follows from Sylvester's law of inertia and its Hermitian and quaternionic analogues; when k is algebraically closed, $D = k$ and the forms are classified by their dimension.

By Proposition 2.22, the last step, and with it the uniqueness statement of Theorem 2.23, fails for $A = k$ unless k is quadratically closed or Euclidean. This is why the thesis works over $\mathbb{R}$, over real closed fields, and over algebraically closed fields.

2.6 Quivers and Path Algebras

Quivers provide the combinatorial language in which we describe finite-dimensional algebras. We introduce them here and develop the variants with involution in Chapter 5.

Definition 2.24 (Quiver)**.** A *quiver* $Q = (Q_0, Q_1, s, t)$ consists of a finite set Q_0 of *vertices*, a finite set Q_1 of *arrows*, and two maps $s, t : Q_1 \to Q_0$ assigning to each arrow α its *source* $s(\alpha)$ and *target* $t(\alpha)$. We write $\alpha : i \to j$ if $s(\alpha) = i$ and $t(\alpha) = j$. Loops ($i = j$) and multiple arrows are allowed.

In other words, a quiver is a finite directed multigraph. A *path* of length $\ell \geq 1$ is a sequence of arrows $p = \alpha_\ell \cdots \alpha_2 \alpha_1$ with $t(\alpha_i) = s(\alpha_{i+1})$; it starts at $s(p) = s(\alpha_1)$ and ends at $t(p) = t(\alpha_\ell)$. We compose from right to left, as for functions. For every vertex v there is a path e_v of length 0 with $s(e_v) = t(e_v) = v$.

Definition 2.25 (Path algebra)**.** The *path algebra* kQ of a quiver Q over a field k is the vector space with basis the set of all paths in Q, including the paths e_v of length zero. The product of two paths p and q is their concatenation pq if $t(q) = s(p)$, and 0 otherwise; this is extended bilinearly to all of kQ.

The elements e_v form a complete set of primitive orthogonal idempotents, and every path satisfies $p = e_{t(p)}\, p\, e_{s(p)}$. Thus $e_j kQ e_i$ is spanned by the paths from i to j, and the Peirce decomposition (2.1) of an element of kQ is its decomposition according to start and end points. The path algebra is finite-dimensional if and only if Q has no oriented cycles. The *arrow ideal* R_Q is the two-sided ideal spanned by the paths of length at least one. If Q has no oriented cycles then $R_Q = \operatorname{rad}(kQ)$. For every quiver, the quotient

$$kQ/R_Q^2$$

is finite-dimensional, with basis the vertices and the arrows; in it, the product of any two arrows is zero. It is a basic algebra whose radical R_Q/R_Q^2 squares to zero. Conversely, by a theorem of Gabriel, every basic finite-dimensional algebra over an algebraically closed field is a quotient kQ/I of the path algebra of a uniquely determined quiver Q by an ideal

with $R_Q^m \subseteq I \subseteq R_Q^2$ for some m [3, Theorem II.3.7]. In particular, the basic algebras with radical square zero are exactly the algebras kQ/R_Q^2.

Example 2.26. 1. For the linearly oriented quiver $1 \xrightarrow{\alpha_1} 2 \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_{n-1}} n$, the map $e_i \mapsto E_{ii}$, $\alpha_i \mapsto E_{i+1,i}$ extends to an isomorphism of kQ with the algebra $T_n(k)$ of lower triangular matrices; the path from i to $j > i$ maps to E_{ji}. (With the opposite convention for composing paths one obtains upper triangular matrices.)

2. For the quiver with one vertex and one loop x, the path algebra is the polynomial ring $k[x]$, and $kQ/R_Q^2 = k[x]/(x^2)$ is the algebra of dual numbers.

3. For the quiver with two vertices u, v and two arrows $a : u \to v$ and $b : v \to u$, the algebra kQ/R_Q^2 is four-dimensional, with basis e_u, e_v, a, b and $ab = ba = 0$. It reappears in Section 3.4.

Definition 2.27 (Representation)**.** A *representation* V of Q over k assigns a finite-dimensional vector space V_i to each vertex i and a linear map $V_\alpha : V_{s(\alpha)} \to V_{t(\alpha)}$ to each arrow α. Its *dimension vector* is $(\dim V_i)_{i \in Q_0}$. A morphism $V \to W$ is a family of linear maps $f_i : V_i \to W_i$ with $f_{t(\alpha)} V_\alpha = W_\alpha f_{s(\alpha)}$ for all α.

Representations of Q are the same as kQ-modules, the vertex spaces being $V_i = e_i V$ [3, 77]. Representations of dimension vector $\mathbf{d}$, up to isomorphism, are the orbits of the group $\prod_i \mathrm{GL}(d_i)$ acting on the space $\bigoplus_\alpha \mathrm{Hom}(k^{d_{s(\alpha)}}, k^{d_{t(\alpha)}})$ by change of basis at every vertex. For the loop quiver of Example 2.26(2), a representation is a square matrix up to similarity, whose classification is the Jordan normal form. For the quiver with two vertices and a single arrow $u \to v$, a representation is a rectangular matrix up to the action $X \mapsto PXQ^{-1}$, whose classification is given by the rank. Requiring the base changes to be orthogonal instead of arbitrary turns the latter problem into the singular value decomposition; this is the role of the involutions introduced in Chapter 5.

A preview of signed and doubly-signed quivers. A *signed quiver* (Definition 5.4) is a quiver equipped with an involution that reverses arrows, together with signs ± 1 on the vertices and arrows that the involution fixes; it describes a basic $*$-algebra with radical

square zero over an algebraically closed field. A *doubly-signed quiver* (Definition 5.7) carries in addition a second involution, which preserves arrows, together with a second set of signs on its fixed vertices; it describes $*$-algebras over the real numbers, and the second signs record whether the scalars at a vertex are real, complex or quaternionic. Figures 5.1 and 5.2 show examples.

Chapter 3

The Seven Operations over $*$-Algebras

This chapter gives the seven operations of Section 1.1 their meaning over a $*$-algebra. Addition, subtraction, multiplication and the adjoint need little discussion: they are the operations of the algebra, and on matrices the adjoint gives the conjugate transpose (Convention 2.1). The other three do: (i, j) indexing, which we base on self-adjoint idempotents and *positions* (Section 3.1); division, which we define between idempotents (Section 3.2); and the $*$-square root, which is not unique, although Householder reflectors work with any $*$-square root in the corner of the next position for which a certain denominator is invertible (Section 3.3). Section 3.4 then shows, in the simplest non-trivial example, how the singular value decomposition arises as a spectral theorem over a non-semisimple $*$-algebra, why the generalised algorithms must leave that algebra, and how, once they do, Householder tridiagonalisation becomes the Golub–Kahan bidiagonalisation; this is the first case of Theorem B. The algebraic background that it uses, from radicals and idempotents to quivers and path algebras, is collected in Chapter 2.

3.1 Matrix Indexing in Finite-Dimensional $*$-Algebras

One of the most basic operations on a matrix is to access its entry M_{ij} using two indices. This operation generalises to every finite-dimensional $*$-algebra, and it still needs only two indices. The entries, however, are no longer scalars: they are elements of the corners eAf cut out by idempotents e, f. This section explains which idempotents to use. The

choice matters for algorithms: Section 3.4 shows that the order of the indices decides which classical algorithm a generic algorithm becomes. We first treat the case in which A really is a matrix algebra over a smaller algebra.

Proposition 3.1. *Let A be a finite-dimensional algebra with a complete set of orthogonal idempotents $e_1, \dots, e_n$ that are pairwise equivalent. Choose $u_i \in e_i A e_1$ and $v_i \in e_1 A e_i$ with $v_i u_i = e_1$ and $u_i v_i = e_i$ (with $u_1 = v_1 = e_1$). Then*

$$\varphi : A \to M_n(e_1 A e_1), \qquad \varphi(a)_{ij} = v_i \, a \, u_j,$$

is an algebra isomorphism. If A is a $*$-algebra, the e_i are self-adjoint and $v_i = u_i^*$, then φ is a $*$-isomorphism, where $M_n(e_1 A e_1)$ carries the conjugate transpose.

Proof. The elements $E_{ij} := u_i v_j$ satisfy $E_{ij} E_{kl} = u_i (v_j u_k) v_l = \delta_{jk} u_i e_1 v_l = \delta_{jk} E_{il}$, because $v_j u_k \in e_1 A e_j e_k A e_1$ vanishes for $j \neq k$ and equals e_1 for $j = k$; moreover $\sum_i E_{ii} = \sum_i e_i = 1$. Using $1 = \sum_k u_k v_k$ we get

$$\varphi(ab)_{ij} = v_i a b \, u_j = \sum_k (v_i a u_k)(v_k b u_j) = \sum_k \varphi(a)_{ik} \varphi(b)_{kj},$$

so φ is multiplicative, and $(b_{ij}) \mapsto \sum_{i,j} u_i b_{ij} v_j$ is its inverse. In the $*$-case, $\varphi(a^*)_{ij} = u_i^* a^* u_j = (u_j^* a u_i)^* = (\varphi(a)_{ji})^*$. □

Proposition 3.1 applies, for example, to $M_n(\mathbb{R}[\varepsilon]/(\varepsilon^2))$, the matrix algebras over the dual numbers, and it identifies an algebra with a single equivalence class of primitive idempotents as a matrix algebra over its basic algebra $e_1 A e_1$.

Peirce indexing. For a general finite-dimensional $*$-algebra A we can use the Peirce decomposition (2.1) directly. Let $E = \{e_1, \dots, e_n\}$ be a complete set of primitive orthogonal idempotents that is closed under the involution, and let σ be the permutation with $e_i^* = e_{\sigma(i)}$. For $a \in A$ put $a_{ij} := e_i a e_j \in e_i A e_j$. Then

$$(ab)_{ij} = \sum_k a_{ik} b_{kj}, \qquad (a^*)_{ij} = e_i a^* e_j = (e_{\sigma(j)} \, a \, e_{\sigma(i)})^* = \big(a_{\sigma(j)\sigma(i)}\big)^*. \tag{3.1}$$

Thus every element of A is a generalised matrix (a_{ij}) whose (i,j) entry lies in e_iAe_j; multiplication is matrix multiplication, and the involution is "transpose, then re-index by σ, then apply the involution entrywise". For $M_2(\mathbb{R})$ with the symplectic involution, $E = \{E_{11}, E_{22}\}$ and σ exchanges the two indices; formula (3.1) reproduces $X^* = J^{-1}X^TJ = \left(\begin{smallmatrix} x_{22} & -x_{12} \\ -x_{21} & x_{11} \end{smallmatrix}\right)$, the signs coming from the involution on the corners. For ${}^2\mathbb{R}$, $E = \{(1,0),(0,1)\}$, the off-diagonal corners vanish, and (3.1) reduces to the exchange of the two components.

Indexing by self-adjoint idempotents. If the indexing idempotents are self-adjoint, then σ is the identity, and (3.1) becomes

$$(a^*)_{ij} = (a_{ji})^* :$$

the involution acts on generalised matrices exactly as the conjugate transpose acts on ordinary ones. This is the property that an algorithm written for matrices with the conjugate transpose needs in order to run without change. A set E as above can be coarsened to self-adjoint idempotents by replacing e_i and $e_{\sigma(i)}$, when they differ, by their sum, which is an idempotent because e_i and $e_{\sigma(i)}$ are orthogonal, and which is self-adjoint. We call the resulting idempotents 1_c, one for each orbit c of σ, the *orbit idempotents*. The price of the coarsening is that the corners 1_cA1_d are larger than the corners e_iAe_j. For a basic real $*$-algebra, the orbits correspond to the orbits of the vertices of its doubly-signed quiver under the transpose and the conjugation (Chapter 5); for a matrix algebra over it, the positions introduced next account for the matrix part.

Positions. Matrices over A are indexed in the same way. Let 1_c ($c \in \mathcal{O}$) be orthogonal self-adjoint idempotents of A with $\sum_c 1_c = 1$. In $M_n(A)$ the elements $E_{kk}1_c$, for $1 \le k \le n$ and $c \in \mathcal{O}$, are again orthogonal self-adjoint idempotents with sum 1. We call the pairs (c,k) *positions* and write c_k for (c,k). A matrix $X = (x_{kl}) \in M_n(A)$ becomes the

generalised matrix with entries

$$X_{c_k,d_l} = 1_c \, x_{kl} \, 1_d \in 1_c A 1_d,$$

it is multiplied as a matrix, and its adjoint is its conjugate transpose. Different orbits may carry different numbers n_c of positions. The matrices are then the elements of a corner $eM_N(A)e$, as in the rectangular problems of Chapter 6, and they form the $*$-algebra $\mathrm{End}(V,\eta) \otimes_{\mathbb{R}\mathcal{O}} A$ of Definition 5.15 with $\dim V_c = n_c$ and all signs $\eta_i = +1$. By Theorem 5.16, every finite-dimensional real $*$-algebra has the form $\mathrm{End}(V,\eta) \otimes_{\mathbb{R}\mathcal{O}} \mathbb{R}Q/I$, with signs $\eta_i = \pm 1$ that can be taken to be $+1$ except at orbits of signature type, and a matrix algebra over it has the same form (Corollary 5.18). Position indexing is therefore the data layout of every algorithm in this thesis; signs $\eta_i = -1$ change the adjoint to (5.4) (Remark 6.2).

The positions are listed in a *cyclic order*: we fix an order $c^1, \ldots, c^m$ of the orbits and take $c^1_1, c^2_1, \ldots, c^m_1, c^1_2, c^2_2, \ldots$, skipping an orbit once its positions are used up. For example, with 3 positions for an orbit u and 5 for an orbit v, the order is $u_1, v_1, u_2, v_2, u_3, v_3, v_4, v_5$. For the singular value decomposition, Householder's algorithm requires the positions to alternate between the two orbits, and this turns tridiagonalisation into bidiagonalisation (Proposition 3.9 and the remarks after it).

An element of A, or a matrix over A, is therefore stored as an array indexed by pairs of positions, with entries in the corner spaces. How the entries themselves are represented, as series over the paths of a quiver, is the subject of Chapters 7 and 8.

3.2 Division and Zero Divisors

Division is the operation most likely to fail when an algorithm is transported to a new algebra. Let A be a finite-dimensional algebra, and let $A = S \oplus \mathrm{rad}(A)$ be a Wedderburn–Malcev decomposition (Theorem 2.12). Since $\mathrm{rad}(A)$ is nilpotent, its elements are "infinitesimal", and invertibility is decided by the semisimple part.

Proposition 3.2. *For $x \in A$ the following are equivalent:*

1. *x is invertible in A;*

2. *the image of x in $A/\operatorname{rad}(A)$ is invertible;*

3. *the left multiplication map $L_x : A \to A$, $y \mapsto xy$, has non-zero determinant.*

Proof. (1)⇒(2) is clear. For (2)⇒(1), let $y \in A$ be such that the images of xy and yx in $A/\operatorname{rad}(A)$ equal 1. Then $xy = 1 - r$ with $r \in \operatorname{rad}(A)$. Since $r^N = 0$ for some N, $1 - r$ is invertible with inverse $1 + r + \cdots + r^{N-1}$, so x has the right inverse $y(1-r)^{-1}$. Similarly x has a left inverse, and therefore x is invertible. For (1)⇔(3): if x is invertible then L_x is invertible with inverse $L_{x^{-1}}$. Conversely, if L_x is bijective there is y with $xy = 1$; then $L_x L_y = \mathrm{id}$, hence $L_y L_x = \mathrm{id}$ by finite-dimensionality, and evaluating at 1 gives $yx = 1$. □

Proposition 3.2 gives an explicit formula for inverses. Let $n = \dim_k A$ and let $\chi_x(t) = \det(tI - L_x) = t^n + c_{n-1}t^{n-1} + \cdots + c_1 t + c_0$ be the characteristic polynomial of L_x, so that $c_0 = (-1)^n \det(L_x)$. Write $\chi_x(t) = t\,p_x(t) + c_0$.

Proposition 3.3. *If $\det(L_x) \neq 0$, then*

$$x^{-1} = \frac{(-1)^{n+1}}{\det(L_x)}\, p_x(x).$$

Proof. The map $x \mapsto L_x$ is an injective algebra homomorphism $A \to \operatorname{End}_k(A)$, since $L_{xy} = L_x L_y$ and $L_x(1) = x$. By the Cayley–Hamilton theorem $\chi_x(L_x) = 0$, hence $\chi_x(x) = 0$ in A, that is, $x\,p_x(x) = -c_0 = (-1)^{n+1}\det(L_x)$. Since $p_x(x)$ commutes with x, this gives a two-sided inverse. □

The formula requires only ring operations and one division by a scalar, so it can be implemented for every finite-dimensional algebra once a basis and its structure constants are known. (The sign $(-1)^{n+1}$ equals -1 when $\dim_k A$ is even and $+1$ when it is odd.)

Example 3.4 (Division of dual numbers)**.** Let $A = \mathbb{R}[\varepsilon]/(\varepsilon^2)$ and $x = a + b\varepsilon$. In the basis

$(1, \varepsilon)$ we have $x \cdot 1 = a + b\varepsilon$ and $x \cdot \varepsilon = a\varepsilon$, so

$$L_x = \begin{pmatrix} a & 0 \\ b & a \end{pmatrix}, \qquad \chi_x(t) = (t-a)^2 = t^2 - 2at + a^2, \qquad \det(L_x) = a^2, \qquad p_x(t) = t - 2a.$$

Here $n = 2$, and Proposition 3.3 gives, for $a \neq 0$,

$$(a + b\varepsilon)^{-1} = -\frac{x - 2a}{a^2} = \frac{a - b\varepsilon}{a^2} = \frac{1}{a} - \frac{b}{a^2}\,\varepsilon.$$

Consequently, for $c + d\varepsilon \in A$,

$$\frac{c + d\varepsilon}{a + b\varepsilon} = (c + d\varepsilon)\,\frac{a - b\varepsilon}{a^2} = \frac{c}{a} + \frac{ad - bc}{a^2}\,\varepsilon.$$

One checks directly that $(a + b\varepsilon)\big(\frac{c}{a} + \frac{ad-bc}{a^2}\varepsilon\big) = c + \frac{ad-bc}{a}\varepsilon + \frac{bc}{a}\varepsilon = c + d\varepsilon$. If $c + d\varepsilon = f(t) + f'(t)\varepsilon$ and $a + b\varepsilon = g(t) + g'(t)\varepsilon$, the ε-coefficient of the quotient is $(f'g - fg')/g^2$, which is the quotient rule; this is how automatic differentiation differentiates quotients.

If $a = 0$, then $x = b\varepsilon$ is a zero divisor and has no inverse. Division of one zero divisor by another is not well defined: the equation $\varepsilon y = \varepsilon$ is solved by $y = 1 + t\varepsilon$ for every $t \in \mathbb{R}$. Nevertheless, in many computations there is an obvious "intended" value of such a quotient. For instance, the quotient $(\varepsilon b)/(\varepsilon c)$ should be b/c.

The last observation is not a curiosity. Consider the Jacobi eigenvalue algorithm [34] applied to the symmetric matrix $H = \left(\begin{smallmatrix} 1+\varepsilon a & \varepsilon b \\ \varepsilon b & 1+\varepsilon c \end{smallmatrix}\right)$ over the dual numbers. The rotation angle θ that diagonalises H is determined by $\tan 2\theta = 2h_{12}/(h_{11} - h_{22}) = 2\varepsilon b/(\varepsilon(a - c))$, a quotient of two zero divisors. The intended value is $\tan 2\theta = 2b/(a - c)$: with this choice the rotation diagonalises the first-order part $\left(\begin{smallmatrix} a & b \\ b & c \end{smallmatrix}\right)$ and hence H itself. First-order perturbation theory finds this value directly: it diagonalises the perturbation within each eigenspace of the unperturbed matrix [55], as Proposition 6.7 does over the dual numbers. It needs to know which eigenvalues coincide, and an iterative algorithm does not know this in advance. Inside such an algorithm, the quotient is the simplest instance of the *division problem*, which is solved in Chapter 7. There, ε is lifted from the dual numbers

$k[x]/(x^2)$ to the path algebra $k[x]$ of the loop quiver, where x is no longer nilpotent; x is then made invertible, and the computation is carried out in the field $k((x))$ of Laurent series, where $(xb)/(xc) = b/c$; finally, the result is reduced modulo x^2.

Division between idempotents. In an algebra that is not a division algebra, the useful notion of invertibility is invertibility *between* two idempotents.

Definition 3.5. Let A be a ring and $f, g \in A$ idempotents. An element $p \in gAf$ is *invertible from f to g* if there is $q \in fAg$ with $qp = f$ and $pq = g$. The element q is then unique, and we write $q = p^{-1}$.

Uniqueness holds because $q = qg = q(pq') = (qp)q' = fq' = q'$ for any two such elements q, q'. For example, in $M_2(k)$ the matrix unit E_{21} is invertible from E_{11} to E_{22}, with inverse E_{12}, although it is nilpotent as an element of $M_2(k)$. On the other hand, a non-zero element p of $\mathrm{rad}(A)$ is never invertible between idempotents, because $f = qp$ would be an idempotent in the ideal $\mathrm{rad}(A)$, hence nilpotent and equal to 0, and then $p = pf = 0$. Radical elements can therefore only become invertible in a larger ring; Chapter 7 constructs one.

The division used by the algorithms of this thesis is defined relative to a fixed complete set of orthogonal idempotents $\{e_i\}$ that is permuted by the involution, for example the vertices of a quiver. For $y \in A$ let f_y be the sum of the e_j with $ye_j \neq 0$ and g_y the sum of the e_i with $e_i y \neq 0$, so that $y \in g_y A f_y$. We write y^{-1} for the inverse of y from f_y to g_y if it exists, and $x/y := xy^{-1}$. For $y = 0$ we have $f_y = g_y = 0$, and the definition gives $0^{-1} = 0$. For an invertible element y of A, y^{-1} is the ordinary inverse, since then $ye_j \neq 0$ for all j. Since the involution permutes the e_i, we have $f_{y^*} = g_y^*$ and $g_{y^*} = f_y^*$, and applying the involution to $y^{-1}y = f_y$ and $yy^{-1} = g_y$ shows that $(y^*)^{-1} = (y^{-1})^*$.

The quotient satisfies $(x/y)\, y = x f_y$, so it solves $zy = x$ exactly when $x f_y = x$, that is, when x vanishes outside the corner on which y is invertible. For zero divisors this can fail. In ${}^2\mathbb{R}$ with $e_1 = (1,0)$ and $e_2 = (0,1)$, the element $(c,0)$ with $c \neq 0$ has $f = g = e_1$ and the inverse $(c^{-1}, 0)$, but $(p,q)/(c,0) = (p/c, 0)$ does not solve $z\,(c,0) = (p,q)$ when $q \neq 0$; relative to the single self-adjoint idempotent $1 = e_1 + e_2$, the element $(c,0)$ has no

inverse at all. The implementation of Chapter 8 therefore checks $(x/y)\,y = x$ for every quotient and reports a breakdown when it fails. Householder reflectors need more: by Proposition 3.8, their denominator must be invertible in the corner fAf of its column, where f is the idempotent of the column's position. In the Gram–Schmidt process, on the other hand, $0^{-1} = 0$ lets a column that depends on the previous ones, and whose residual is therefore zero, produce a zero column of Q; Chapter 8 shows that Q is then a partial isometry.

3.3 $*$-Square Roots

In standard linear algebra the length of a vector is computed as

$$\|v\| = \Big(\sum_{i=1}^{n} v_i^* v_i\Big)^{1/2} = (v^* v)^{1/2}.$$

The map $x \mapsto x^*x$ is the *Hermitian square*, and computing $\|v\|$ requires the inverse operation, a *$*$-square root*: given a self-adjoint s, find x with $x^*x = s$. The root x need not be self-adjoint. Norms are needed by Householder reflections, Givens rotations and Gram–Schmidt orthonormalisation, so the existence of $*$-square roots is an important question for our programme. The answer depends strongly on the involution.

- For $\mathbb{C}$, or $M_n(\mathbb{C})$ with the conjugate transpose, Hermitian squares are exactly the non-negative real numbers, respectively the positive semidefinite matrices; $*$-square roots of other self-adjoint elements do not exist.

- For ${}^2\mathbb{R}$, the self-adjoint elements are (c, c) with $c \in \mathbb{R}$, and every such element is a Hermitian square: $(c, c) = z^*z$ for $z = (1, c)$, whatever the sign of c.

- For the dual numbers with $\varepsilon^* = \varepsilon$, every element is self-adjoint and $(a+b\varepsilon)^*(a+b\varepsilon) = a^2 + 2ab\varepsilon$; the Hermitian squares are 0 and the elements $c + d\varepsilon$ with $c > 0$. With $\varepsilon^* = -\varepsilon$, the self-adjoint elements are the real numbers, and the Hermitian squares are the non-negative reals.

The symplectic involution behaves like ${}^2\mathbb{R}$: every self-adjoint element is a Hermitian square.

Proposition 3.6. *Let R be a real closed field (for example $\mathbb{R}$), let $J = \left(\begin{smallmatrix} 0 & I_m \\ -I_m & 0 \end{smallmatrix}\right)$, and equip $M_{2m}(R)$ with the symplectic involution $X^\omega = J^{-1}X^TJ$.*

1. *For every $M \in M_m(R)$ the matrix $N = \left(\begin{smallmatrix} 0 & M \\ -I & 0 \end{smallmatrix}\right)$, which in Kronecker product notation is $N = \left(\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right) \otimes M - \left(\begin{smallmatrix} 0 & 0 \\ 1 & 0 \end{smallmatrix}\right) \otimes I$, satisfies*

$$N^\omega N = \begin{pmatrix} M^T & 0 \\ 0 & M \end{pmatrix}.$$

2. *Every $H \in M_{2m}(R)$ with $H^\omega = H$ is a Hermitian square: $H = Y^\omega Y$ for some $Y \in M_{2m}(R)$.*

Proof. (1) We have $N^T = \left(\begin{smallmatrix} 0 & -I \\ M^T & 0 \end{smallmatrix}\right)$, hence $N^TJ = \left(\begin{smallmatrix} I & 0 \\ 0 & M^T \end{smallmatrix}\right)$ and $N^\omega = J^{-1}N^TJ = \left(\begin{smallmatrix} 0 & -M^T \\ I & 0 \end{smallmatrix}\right)$. Multiplying, $N^\omega N = \left(\begin{smallmatrix} M^T & 0 \\ 0 & M \end{smallmatrix}\right)$.

(2) The condition $H^\omega = H$ says that JH is skew-symmetric, that is, H is *skew-Hamiltonian*. Every real skew-Hamiltonian matrix is similar, via a symplectic matrix S (one with $S^\omega S = I$), to a block-diagonal matrix $D = \operatorname{diag}(M, M^T)$ [28, 85]. By (1), applied to M^T in place of M, there is N' with $N'^\omega N' = D$. Then $H = SDS^{-1} = SN'^\omega N'S^\omega = (N'S^\omega)^\omega(N'S^\omega)$, since $(S^\omega)^\omega = S$. This proves (2) for $R = \mathbb{R}$. For fixed m, statement (2) is a first-order sentence in the language of ordered fields, so it holds over every real closed field by Tarski's transfer principle (Theorem 4.2). □

Roots are not unique. If x is a $*$-square root of s and u satisfies $u^*u = 1$, then ux is another. Conversely, if x is invertible from f to g, every other root $x' \in Af$ has the form ux with $u^*u = g$, and u is unitary from g to g' when x' is invertible from f to g'.

Lemma 3.7. *Let f, g be self-adjoint idempotents of a $*$-algebra A, let $x \in gAf$ be invertible from f to g, and let $x' \in Af$ satisfy $x'^*x' = x^*x$. Then $u = x'x^{-1}$ satisfies $u^*u = g$ and $x' = ux$. If, moreover, x' is invertible from f to a self-adjoint idempotent g', then u is unitary from g to g': $u^*u = g$ and $uu^* = g'$.*

Proof. We have $ux = x'x^{-1}x = x'f = x'$ and

$$u^*u = (x^{-1})^*\, x'^* x'\, x^{-1} = (x^{-1})^* x^* x x^{-1} = (xx^{-1})^*(xx^{-1}) = g.$$

If x' is invertible from f to g', then $u \in g'Ag$ is invertible from g to g', with inverse xx'^{-1}. Multiplying $u^*u = g$ on the right by u^{-1} gives $u^* = u^*g' = u^{-1}$, hence $uu^* = g'$. □

A $*$-square root is therefore a choice, and an algorithm should depend on it as little as possible. Classically, a Householder reflector maps a column x to a multiple of the first unit column $\hat{e}_1$ with a prescribed phase, $\gamma = -e^{i\theta}\|x\|$ where $e^{i\theta}$ is the phase of x_1. Over a $*$-algebra no phase condition is needed: any $*$-square root of x^*x will do, provided that a certain denominator is invertible. We state this in the generality of position indexing, with self-adjoint idempotents $E_1, \ldots, E_m$ and the identity $I = \mathrm{diag}(E_1, \ldots, E_m)$.

Proposition 3.8. *Let A be a $*$-algebra, let $E_1, \ldots, E_m$ and f be self-adjoint idempotents of A, and let $x = (x_1, \ldots, x_m)^T$ be a column with $x_i \in E_iAf$. Let $\gamma \in E_1Af$ satisfy $\gamma^*\gamma = x^*x$, put $w = x - \hat{e}_1\gamma$ and $s = w^*x \in fAf$, and suppose that s is invertible from f to f. Then $P = I - ws^{-1}w^*$ satisfies*

$$P^*P = PP^* = I \qquad \text{and} \qquad Px = \hat{e}_1\gamma.$$

Proof. Since $\gamma^*\gamma = x^*x$,

$$w^*w = x^*x - x_1^*\gamma - \gamma^*x_1 + \gamma^*\gamma = (x^*x - \gamma^*x_1) + (x^*x - x_1^*\gamma) = s + s^*.$$

The inverse s^{-1} lies in fAf, and $(s^*)^{-1} = (s^{-1})^*$. Hence $(s^*)^{-1}(w^*w)s^{-1} = (s^*)^{-1} + s^{-1}$, and, since $Iw = w$,

$$P^*P = I - ws^{-1}w^* - w(s^*)^{-1}w^* + w\big((s^*)^{-1} + s^{-1}\big)w^* = I.$$

The same computation with the factors in the other order, using $s^{-1}(w^*w)(s^*)^{-1} = (s^*)^{-1} + s^{-1}$, gives $PP^* = I$. Finally, $Px = x - ws^{-1}(w^*x) = x - wf = x - w = \hat{e}_1\gamma$. □

Over $\mathbb{C}$, the classical choice of γ makes s real and P Hermitian; by the proposition it is needed only for P to be Hermitian, not for P to be unitary. What remains of the classical choice is the condition that s be invertible from f to f. Over $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$ with their standard involutions, it holds for every $x \neq 0$ if the sign of γ is chosen to avoid cancellation in $x_1 - \gamma$, as in the classical algorithm; this choice also keeps a nearly reduced column accurate in floating-point arithmetic. In corners with zero divisors, the choice of the root itself can matter. In ${}^2\mathbb{R}$, with $(p,q)^* = (q,p)$, the column $x = \big((1,-1),(\sqrt{2},\sqrt{2})\big)^T$ has $x^*x = 1$; the roots $\gamma = \pm(1,1)$ give the zero divisors $s = (0,2)$ and $s = (2,0)$, so they yield no reflector, whereas the root $\gamma = (2,\frac{1}{2})$ gives the invertible $s = (\frac{1}{2},3)$.

Norms in the radical. If B is a $*$-algebra whose radical R satisfies $R^2 = 0$, then every column x with entries in R has

$$x^*x = \sum_i x_i^* x_i \in R^2 = 0,$$

because $R^* = R$ (Lemma 2.11). Such columns have zero norm. A Householder reflector or a Gram–Schmidt step for them would need an element of the radical, such as $s = w^*x$ when γ lies in the radical, to be invertible from f to f, which is impossible by Section 3.2. Section 3.4 shows that this happens for every column of the matrices that encode the singular value decomposition, and how lifting to a path algebra removes the obstruction.

Proposition 3.6 gives $*$-square roots in one class of semisimple algebras. For the algebras of this thesis, square roots are computed directly in a path $*$-algebra and its completed localisation, degree by degree (Section 7.5). In the algebra K of a single vertex (Table 5.1), -1 is a Hermitian square exactly when K is not of signature type: if it is not, $-1 = uu^*$ for some u by Lemma 5.12(1), and if K is $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$ with its standard involution, then $y^*y = |y|^2 \geq 0$ for every y.

3.4 A First Unification: The SVD as a Spectral Theorem

We now show, in the simplest non-trivial case, how a classical decomposition arises as the analogue of the Hermitian eigendecomposition over a non-semisimple $*$-algebra, and how the generic algorithm for the latter becomes the classical algorithm for the former. Let $B = \mathbb{R}Q/R_Q^2$ be the four-dimensional algebra of Example 2.26(3), with basis e_u, e_v, a, b, where $a = e_v a e_u$ is an arrow from u to v, $b = e_u b e_v$ is an arrow from v to u, and $ab = ba = 0$. Define an involution on B by $e_u^* = e_u$, $e_v^* = e_v$ and $a^* = b$. In the language of Chapter 5, B is the $*$-algebra of the doubly-signed quiver of Figure 5.2(a); here we work with it directly.

A matrix $H \in M_n(B)$ can be written uniquely as $H = e_u X + e_v Y + aW + bZ$ with $X, Y, W, Z \in M_n(\mathbb{R})$, where $e_u X$ denotes the matrix with entries $x_{ij} e_u$. Since $(aW)^*$ has entries $(w_{ji} a)^* = w_{ji} b$, we have $H^* = e_u X^T + e_v Y^T + bW^T + aZ^T$. Hence

$$H \in \mathrm{Herm}_n(B) \iff X = X^T, \quad Y = Y^T, \quad Z = W^T.$$

Similarly, a direct computation using $e_u a = 0$, $a e_u = a$, $e_v a = a$, $a e_v = 0$ (and the analogous relations for b) shows that $U = e_u P + e_v R + aS + bT$ is unitary if and only if

$$P^T P = I, \qquad R^T R = I, \qquad T = -P S^T R.$$

Let $U_0 = e_u P + e_v R$ denote the semisimple part of U. If all entries of H lie in the radical, that is, $H = aW + bW^T$, then every term of UHU^* that involves S or T is a product of two radical elements and vanishes, and

$$UHU^* = U_0 H U_0^* = a\,(RWP^T) + b\,(RWP^T)^T. \tag{3.2}$$

Thus *unitary similarity of Hermitian matrices over B with entries in the radical is the same as the action $W \mapsto RWP^T$ of $\mathrm{O}(n) \times \mathrm{O}(n)$ on real matrices.* A unitary diagonal-

isation of H, with diagonal entries $a\sigma_i + b\sigma_i$, is precisely a singular value decomposition $W = R^T \operatorname{diag}(\sigma_i) P$. Theorem 6.1 generalises this computation to every doubly-signed quiver.

When X and Y are non-zero the situation changes. Taking $P = R = I$ and $T = -S^T$, one finds that the a-component of UHU^* is $W + SX - YS$. If $X = \operatorname{diag}(x_j)$ and $Y = \operatorname{diag}(y_i)$, then the (i, j) entry of $SX - YS$ is $s_{ij}(x_j - y_i)$, so every entry w_{ij} with $x_j \neq y_i$ can be removed. In general, therefore, the canonical form consists of eigendecompositions of X and Y together with singular value decompositions of the blocks of W that connect equal eigenvalues of X and Y. The SVD appears in pure form exactly when $X = Y = cI$ for some $c \in \mathbb{R}$, that is, when all entries of $H - c \cdot 1$ lie in the radical; the case $c = 0$ is the case $H = aW + bW^T$ above.

This has an algorithmic consequence. An algorithm such as the Jacobi method, transported to $M_n(B)$, computes rotation angles from quotients of entries of H. When the semisimple parts of the relevant entries coincide, as they do in the pure SVD case, these quotients are quotients of zero divisors, such as $(aw_{ij})/(aw_{jj})$, whose intended value is w_{ij}/w_{jj}. As in Example 3.4, such quotients are meaningless in B.

Householder's algorithm. The obstruction is even more basic for Householder's algorithm, because the norms vanish. We index $H = aW + bW^T$ by positions (Section 3.1), using the self-adjoint idempotents e_u and e_v, and we allow $W \in M_{m \times n}(\mathbb{R})$ to be rectangular: there are n positions $u_1, \ldots, u_n$ and m positions $v_1, \ldots, v_m$, and

$$H_{v_i, u_j} = w_{ij}\, a, \qquad H_{u_j, v_i} = w_{ij}\, b,$$

all other entries being zero. The part of the column of u_1 below its diagonal is a column x with entries $w_{i1}a$ at the positions v_i, and $x^*x = \sum_i w_{i1}^2\, ba$, which is 0 in B. In the path algebra $\mathbb{R}Q$, however, $ba \neq 0$, and $x^*x = c\, ba$ with $c = \sum_i w_{i1}^2$ has the $*$-square root $\gamma = \sqrt{c}\, a$, since $a^*a = ba$. Householder's algorithm therefore runs over $\mathbb{R}Q$ once the elements $s = w^*x$, which are multiples of ba or ab, can be inverted. To this end we adjoin inverses of the arrows: elements $a^{-1} \in e_u \Lambda_0 e_v$ and $b^{-1} \in e_v \Lambda_0 e_u$ with $a^{-1}a = e_u$,

$aa^{-1} = e_v$, $b^{-1}b = e_v$ and $bb^{-1} = e_u$, and we extend the involution by $(a^{-1})^* = b^{-1}$. Chapter 7 constructs the resulting algebra Λ_0 for every quiver (Section 7.2).

Proposition 3.9. *Let $W \in M_{m\times n}(\mathbb{R})$ with $m \geq n$, and let $H = aW + bW^T$ be indexed by the positions $u_1, v_1, u_2, v_2, \ldots$ in cyclic order. Run Householder tridiagonalisation over Λ_0, taking for the $*$-square root of $x^*x = cba$ (respectively cab) the element $\gamma = \pm\sqrt{c}\,a$ (respectively $\pm\sqrt{c}\,b$), with the sign that avoids cancellation, and inverting each denominator s in the corner of its column. Then:*

1. *every step is defined, and every reflector has the form $P = e_uP_u + e_vP_v$, that is, P acts on the positions of u by a real orthogonal matrix P_u and on those of v by P_v, and has no other entries;*

2. *the reflectors are, in order, the left and right Householder reflectors of the Golub–Kahan bidiagonalisation of W [33], with the same choice of signs, except where the part of a column or row beyond its first entry vanishes but the first entry does not: there the generic step multiplies one position by -1, whereas the classical algorithm, as implemented in LAPACK, applies no reflector;*

3. *the result is $T = U^*HU = aW' + bW'^T$ with $U = e_uU_u + e_vU_v$ and $W' = U_v^TWU_u$ upper bidiagonal, and W' is the Golub–Kahan bidiagonal matrix of W up to the signs of its entries.*

Proof. Suppose that, before some step, the current matrix is $a\tilde{W} + b\tilde{W}^T$ for a real matrix $\tilde{W} = (\tilde{w}_{ij})$ whose first $k-1$ columns and first $k-1$ rows are already in bidiagonal form, and that the step concerns the column of u_k. Since Q has no loops, the entries of this column at positions of u vanish, so the part x below the diagonal has the entries $\tilde{w}_{ik}a$ at the positions v_i with $i \geq k$ and zero entries elsewhere. Hence $x^*x = cba$ with $c = \sum_{i\geq k}\tilde{w}_{ik}^2$, and $\gamma = \pm\sqrt{c}\,a$ lies in $e_v\Lambda_0e_u$, which is the corner of the next position, v_k, as Proposition 3.8 requires. If $c = 0$, then $w = 0$ and $P = I$. Otherwise $w = x - \hat{e}_{v_k}\gamma$, where $\hat{e}_{v_k}$ is the unit column of the position v_k, has the entries $\hat{w}_i a$ at the positions v_i, where $\hat{w} = (\tilde{w}_{kk} \mp \sqrt{c},\, \tilde{w}_{k+1,k}, \ldots, \tilde{w}_{mk})^T$ is the classical Householder vector of the column $(\tilde{w}_{kk}, \ldots, \tilde{w}_{mk})^T$, and $s = w^*x = c'\,ba$ with $c' = \hat{w}^T(\tilde{w}_{kk}, \ldots, \tilde{w}_{mk})^T = c + \sqrt{c}\,|\tilde{w}_{kk}| > 0$

for the sign that avoids cancellation. In Λ_0 we have $s^{-1} = c'^{-1}a^{-1}b^{-1}$, and the entry of $ws^{-1}w^*$ at the positions (v_i, v_j) is

$$\hat{w}_i a \; c'^{-1}a^{-1}b^{-1} \; \hat{w}_j b = c'^{-1}\hat{w}_i\hat{w}_j \, e_v,$$

while all its other entries vanish. Thus P is the identity at the positions of u and the classical reflector $I - \hat{w}\hat{w}^T/c'$ at the positions $v_k, \ldots, v_m$. Conjugation by P replaces $\tilde{W}$ by $P_v\tilde{W}$, which is the left Householder step of the Golub–Kahan bidiagonalisation on column k. A step at the column of v_k is the same computation with a and b, u and v, and rows and columns exchanged: its reflector acts on the positions $u_{k+1}, \ldots, u_n$ and replaces $\tilde{W}$ by $\tilde{W}P_u$, which is the right Householder step on row k. The positions alternate between u and v, so the steps come in the order of the Golub–Kahan algorithm, and tridiagonalisation, which treats the columns of the first $m+n-2$ positions, includes all its steps because $m \geq n$. When the entries $\tilde{w}_{k+1,k}, \ldots, \tilde{w}_{mk}$ vanish but $\tilde{w}_{kk}$ does not, the sign that avoids cancellation gives $\gamma = -\tilde{w}_{kk}a$, and P multiplies the position v_k by -1, whereas the classical algorithm applies no reflector; this proves (1) and (2). All reflectors have the form $e_uP_u + e_vP_v$, hence so does U, and $U^*(aW)U = a\,U_v^TWU_u$, which gives (3). □

The proposition shows several things at once. The generic tridiagonalisation, written once for Hermitian matrices, becomes the Golub–Kahan bidiagonalisation without any code for it. The unitary U has the form $e_uU_u + e_vU_v$ with $U_u \in \mathrm{O}(n)$ and $U_v \in \mathrm{O}(m)$, so it lies in the group $\mathrm{O}(n) \times \mathrm{O}(m)$ that acts on such matrices by Theorem 6.1, and the result T has entries in the radical, so U and T lie in B even though the intermediate quantities s^{-1} do not; Theorem 7.8 makes this precise. And the order of the positions matters. If the part x of the column of a position of u below the diagonal is non-zero, then $x^*x = c\,ba$ with $c > 0$, and no $*$-square root of it lies in the corner $e_u\Lambda_0e_u$: that corner is the ring $\mathbb{R}[g, g^{-1}]$ with $g = ba = g^*$, in which $\gamma^*\gamma = \gamma^2$, and $c\,g$ is not a square. By Proposition 3.8, the next position must therefore belong to v, and symmetrically for columns of v. For generic W, for which these parts are non-zero at every step, the orders that Householder's

algorithm accepts are therefore exactly the two cyclic orders, starting with u or with v. With the plain indexing of $M_n(B)$, in which every position carries the idempotent 1, the corner condition is empty, but the result is then only a tridiagonal matrix. In the experiments of Chapter 8, the case $m < n$ and the complex and quaternionic singular value decompositions behave in the same way.

With the labels a and b removed, H is the matrix $\left(\begin{smallmatrix} 0 & W^T \\ W & 0 \end{smallmatrix}\right)$, listed in the cyclic order $u_1, v_1, u_2, v_2, \ldots$ instead of $u_1, \ldots, u_n, v_1, \ldots, v_m$, that is, after a perfect shuffle of its rows and columns. That this permuted matrix is tridiagonal, with zero diagonal, when W is upper bidiagonal, and that its non-zero eigenvalues are $\pm$ the non-zero singular values of W, is classical; it relates the singular values of bidiagonal matrices to symmetric tridiagonal eigenproblems [20, 33]. What the algebra adds is that the labels force this order, through the corner condition of Proposition 3.8, and that the same program runs unchanged over the other algebras of Chapter 6.

Example 3.10 (A 3×2 matrix)**.** Let

$$W = \begin{pmatrix} 3 & 1 \\ 4 & 2 \\ 0 & 0.3 \end{pmatrix}, \qquad H = \begin{pmatrix} 0 & 3b & 0 & 4b & 0 \\ 3a & 0 & a & 0 & 0 \\ 0 & b & 0 & 2b & 0.3b \\ 4a & 0 & 2a & 0 & 0 \\ 0 & 0 & 0.3a & 0 & 0 \end{pmatrix},$$

with the rows and columns of H indexed by the positions u_1, v_1, u_2, v_2, v_3 in this cyclic order. Householder tridiagonalisation treats the columns of the first three positions.

Column of u_1. Below the diagonal it is $x = (3a, 0, 4a, 0)^T$, at the positions v_1, u_2, v_2, v_3, and $x^*x = 25\,ba$. The graded root is $5a$, and the sign that avoids cancellation gives $\gamma = -5a$, which lies in the corner of the next position, v_1. Then $w = (8a, 0, 4a, 0)^T$, $s = w^*x = 40\,ba$ and $s^{-1} = \frac{1}{40}\,a^{-1}b^{-1}$. In $ws^{-1}w^*$ the inverse letters cancel: its entry at the positions (v_i, v_j) is $\frac{1}{40}\hat{w}_i\hat{w}_j\,e_v$ with $\hat{w} = (8, 4, 0)^T$, and its other entries vanish. So P is

the identity at the positions of u and the classical reflector

$$P_v = I - \tfrac{1}{40}\hat{w}\hat{w}^T = \begin{pmatrix} -0.6 & -0.8 & 0 \\ -0.8 & 0.6 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

at those of v, and the new matrix PHP^* encodes $P_vW = \left(\begin{smallmatrix} -5 & -2.2 \\ 0 & 0.4 \\ 0 & 0.3 \end{smallmatrix}\right)$.

Column of v_1. Below the diagonal it is $(-2.2b, 0, 0)^T$, at the positions u_2, v_2, v_3. Only its first entry is non-zero, which is the exceptional case of Proposition 3.9(2). Here $\gamma = 2.2\,b$, $s = 9.68\,ab$, and P multiplies the position u_2 by -1, so that the encoded matrix becomes $\left(\begin{smallmatrix} -5 & 2.2 \\ 0 & -0.4 \\ 0 & -0.3 \end{smallmatrix}\right)$.

Column of u_2. Below the diagonal it is $(-0.4a, -0.3a)^T$, at the positions v_2, v_3, with $x^*x = 0.25\,ba$ and $\gamma = 0.5\,a$. Then $s = 0.45\,ba$, and P acts on v_2, v_3 by the classical reflector $\left(\begin{smallmatrix} -0.8 & -0.6 \\ -0.6 & 0.8 \end{smallmatrix}\right)$.

The result is $T = U^*HU$ with $U = e_uU_u + e_vU_v$, and T encodes the upper bidiagonal matrix

$$W' = U_v^T W U_u = \begin{pmatrix} -5 & 2.2 \\ 0 & 0.5 \\ 0 & 0 \end{pmatrix},$$

whose singular values $5.4663\ldots$ and $0.4573\ldots$ are those of W. The bidiagonalisation of Golub and Kahan, with the sign conventions of LAPACK, gives the entries -5, -2.2 and -0.5; the two results differ by the sign of the second column, as Proposition 3.9 predicts. Neither U nor T contains an inverse letter, so $H = UTU^*$ holds over the four-dimensional algebra B itself (Theorem 7.8). Listing 8.1 runs this example with the library of Chapter 8.

Chapter 4

Generalising Decompositions Using Infinitesimals and Constructive Transfer Principles

This chapter treats ordered fields with infinitesimals as scalars. Over a real closed field, the seven operations of Section 1.1 and the comparisons satisfy the same first-order statements as over $\mathbb{R}$ (Theorem 4.2), and running an algorithm once over the field of algebraic Puiseux series in an infinitesimal ε, then taking standard parts of its finite outputs, gives their limits as a real parameter ε tends to zero (Proposition 4.7). The Bruhat decomposition of Theorem B(4) is computed in this way (Theorem 4.11).

4.1 Motivation and Overview

Infinitesimals enter this thesis in three ways.

1. *Automatic limits.* Section 1.3.1 showed that Gaussian elimination is the limit, as a weight ε tends to 0, of Householder QR with respect to a weighted inner product. Running an algorithm once over a field containing a formal infinitesimal ε, and then discarding infinitesimal parts, computes such limits mechanically (Proposition 4.7). We call this an *auto-limit*, by analogy with automatic differentiation. As with derivatives, there are other ways to find these limits: analysis by hand, as in the

proof of Proposition 1.1; computer algebra, with ε kept as a symbol; and, less reliably, running the algorithm numerically at a small value of ε.

2. *Perturbation theory.* A matrix whose entries are power series in ε describes a family of matrices depending on a parameter. Decompositions over power series rings describe how decompositions vary with the parameter, and truncating modulo ε^2 recovers decompositions over the dual numbers [39].

3. *Division.* The radical of a finite-dimensional algebra consists of nilpotent elements, which behave like infinitesimals but cannot be inverted. Chapter 7 replaces them by *invertible* ones, by inverting the arrows of a path algebra and completing. For the quiver with one loop, this gives the field of Laurent series (Section 4.6), in which the auto-limits of this chapter can be computed.

To use infinitesimals rigorously we need two things. First, an ordered field $\mathbb{K} \supsetneq \mathbb{R}$ containing infinitesimals in which the operations required by numerical algorithms, in particular square roots of positive elements, are available. Second, a *transfer principle*, guaranteeing that the theorems which justify an algorithm over $\mathbb{R}$ (for instance, the existence of an orthonormal eigenbasis for a symmetric matrix) remain true over $\mathbb{K}$. Without a transfer principle every such theorem would have to be re-proved for each new field.

There are two well-known ways of obtaining these. *Robinson's nonstandard analysis* [74] produces a field ${}^*\mathbb{R}$ with infinitesimals to which *every* first-order statement about $\mathbb{R}$ transfers, but ${}^*\mathbb{R}$ is obtained non-constructively and cannot be computed with. *Tarski's theorem* on real closed fields [84] gives a transfer principle only for statements about polynomial equations and inequalities, but it applies to explicit, computable fields such as the field of real algebraic Puiseux series. All the statements we need are of this restricted kind, so the results of this thesis rely only on Tarski's theorem. We nevertheless describe Robinson's approach (Section 4.3.3), both because it is the best-known theory of infinitesimals and because it clarifies the relationship between nilpotent infinitesimals, such as those of the dual numbers, and invertible ones (Section 4.5). In short, *nonstandard analysis in Robinson's sense is not needed for the results of this thesis*; what we use instead is the

constructive transfer principle of Theorem 4.2. The statements that we transfer, such as the spectral theorem, could also be proved directly over an arbitrary real closed field.

The chapter is organised as follows. Section 4.2 recalls real closed fields. Section 4.3 states the two transfer principles precisely. Section 4.4 introduces Puiseux series, the fields with infinitesimals that we use. Section 4.5 defines the standard part map, proves that it computes limits of algorithms, and explains how nilpotent infinitesimals arise as quotients. Section 4.6 shows that fields of Laurent series, which are the completed localisations of the quiver with one loop, are Pythagorean, and Section 4.7 uses them to compute the Bruhat decomposition as an auto-limit of the Gram–Schmidt process. Sections 4.8 and 4.9 apply these tools to prove spectral theorems and singular value decompositions over formal power series rings. Section 4.10 shows that Dijkstra's shortest-path algorithm is the leading-order part of Gaussian elimination over Puiseux series, and Section 4.11 assembles the chain of limits that leads from the Hermitian eigendecomposition, through the singular value decomposition and the QR factorisation, to the Bruhat and LU factorisations and to Dijkstra's algorithm.

4.2 Real Closed Fields

Definition 4.1. A field R is *real closed* if it is an ordered field in which every positive element has a square root and every polynomial of odd degree has a root.

By the Artin–Schreier theory [9, §1.2], a field R is real closed if and only if any of the following equivalent conditions holds:

- R is not algebraically closed, but $R(\sqrt{-1})$ is;
- R admits an ordering, and no proper algebraic extension of R admits an ordering extending it;
- R admits an ordering, and every polynomial over R factors into linear and quadratic factors.

A real closed field has a unique ordering, in which the positive elements are exactly the non-zero squares. Examples are $\mathbb{R}$, the field of real algebraic numbers, and the field of real Puiseux series (Section 4.4). The theorem of Frobenius extends to real closed fields: the finite-dimensional division algebras over a real closed field R are R, $R(\sqrt{-1})$ and the quaternions over R.

Two weaker notions will also be useful. An ordered field is *Euclidean* if every positive element is a square, and a field is *Pythagorean* if every sum of squares is a square. Real closed fields are Euclidean, and Euclidean fields are Pythagorean.

The properties of real closed fields used in practice are these: square roots of positive elements exist, which is needed for orthogonalisation, polar decompositions and SVDs; every polynomial splits into linear and quadratic factors, which is needed for eigenvalue problems; and the transfer principle of the next section holds.

4.3 First-Order Logic and Transfer Principles

4.3.1 First-Order Languages

We recall only what is needed; see [69] for details. A *signature* is a set of constant, function and relation symbols. The signature of *ordered rings* is $\mathcal{L}_{\mathrm{or}} = \{0, 1, +, -, \cdot, <\}$. *Terms* are built from variables and constants using the function symbols; in $\mathcal{L}_{\mathrm{or}}$ they are polynomials with integer coefficients. *Formulas* are built from equations and relations between terms using $\neg, \wedge, \vee, \Rightarrow$ and the quantifiers $\forall x$ and $\exists x$, where x ranges over *elements* of the structure. A *sentence* is a formula without free variables. A structure in which a sentence is true is called a *model* of it.

The restriction that quantifiers range over elements, and not over subsets, functions or natural numbers, is what "first-order" means. Statements about matrices of a *fixed* size are first-order, because a matrix is a tuple of elements. For example, the spectral theorem

for symmetric 2×2 matrices over an ordered field is the sentence

$$\forall a\, \forall b\, \forall c\, \exists p\, \exists q\, \exists \lambda\, \exists \mu\, \big(p^2 + q^2 = 1 \ \wedge\ a = p^2\lambda + q^2\mu \\ \wedge\ b = pq(\lambda - \mu) \ \wedge\ c = q^2\lambda + p^2\mu\big),$$

which says that $\left(\begin{smallmatrix} a & b \\ b & c \end{smallmatrix}\right) = R\,\mathrm{diag}(\lambda, \mu)\,R^T$ for the rotation $R = \left(\begin{smallmatrix} p & -q \\ q & p \end{smallmatrix}\right)$. For each n there is a similar sentence φ_n expressing the spectral theorem for $n \times n$ symmetric matrices. The statement "φ_n holds for all n" is not first-order, so transfer principles are applied to each φ_n separately.

4.3.2 The Tarski–Seidenberg Transfer Principle

Theorem 4.2 (Tarski)**.** *The theory of real closed fields admits quantifier elimination in the signature $\mathcal{L}_{\mathrm{or}}$: every formula is equivalent, in all real closed fields, to a quantifier-free formula with the same free variables. Consequently:*

1. (Transfer) *A sentence of $\mathcal{L}_{\mathrm{or}}$ holds in one real closed field if and only if it holds in every real closed field.*

2. (Transfer with parameters) *If $R \subseteq R'$ are real closed fields and $\varphi(x_1, \dots, x_m)$ is a formula of $\mathcal{L}_{\mathrm{or}}$, then for all $a_1, \dots, a_m \in R$, $\varphi(a_1, \dots, a_m)$ holds in R if and only if it holds in R'.*

See [84], [9, Proposition 5.2.3] and [69, Corollary 3.3.16]. Quantifier elimination is effective: there are algorithms that compute the quantifier-free equivalent of a given formula [7]. In this sense the transfer principle is constructive. We use it as follows: to show that a theorem about $n \times n$ matrices, true over $\mathbb{R}$, also holds over a real closed field R containing infinitesimals, it suffices to write the theorem as a first-order sentence in $\mathcal{L}_{\mathrm{or}}$.

4.3.3 Robinson's Nonstandard Analysis

Robinson's nonstandard analysis rests on the compactness theorem of first-order logic, which together with Gödel's completeness theorem is one of the two fundamental theorems of the subject.

Theorem 4.3 (Compactness)**.** *A set of first-order sentences has a model if and only if each of its finite subsets has a model.*

Consider the signature Σ that contains a function symbol for every function $\mathbb{R}^m \to \mathbb{R}^n$ (including constants, as functions $\mathbb{R}^0 \to \mathbb{R}$) and a relation symbol for every subset of $\mathbb{R}^n$, for all m, n. This signature is uncountable, which is allowed. Let T be the set of all Σ-sentences that are true in $\mathbb{R}$. Add a new constant symbol δ and the sentences $0 < \delta < 1/m$ for every positive integer m. Every finite subset of the enlarged set of sentences has a model, namely $\mathbb{R}$ itself with δ interpreted as a sufficiently small positive real number. By compactness, the whole set has a model ${}^*\mathbb{R}$. In ${}^*\mathbb{R}$ the element δ is positive but smaller than every $1/m$, so it is a non-zero infinitesimal, and ${}^*\mathbb{R} \neq \mathbb{R}$.

Theorem 4.4 (Transfer principle of nonstandard analysis)**.** *There is an ordered field ${}^*\mathbb{R} \supsetneq \mathbb{R}$ in which every function and relation on $\mathbb{R}$ has a natural extension, such that a Σ-sentence with parameters from $\mathbb{R}$ holds in $\mathbb{R}$ if and only if it holds in ${}^*\mathbb{R}$.*

Its language is much richer than that of Theorem 4.2: it applies to all functions (exponentials, trigonometric functions, arbitrary maps), not only to polynomials. On the other hand, it concerns the single extension $\mathbb{R} \subset {}^*\mathbb{R}$, whereas Theorem 4.2 applies to every pair of real closed fields, including the fields of Puiseux series used below. Its main use in mathematics is to replace arguments involving many nested quantifiers, the "epsilon management" of analysis, by arguments with infinitesimals [83]. The construction is, however, non-constructive: it requires the compactness theorem (or an ultrafilter), and there is no explicit, computable description of the elements of ${}^*\mathbb{R}$.

4.3.4 Comparison of the Two Approaches

Each approach is strong in one respect and weak in another.

- Robinson's approach preserves *every* first-order property of the real numbers, but it does not provide a constructive or computable field of infinitesimals.

- The approach based on Tarski's theorem transfers only properties expressible with polynomial equations and inequalities. In exchange it applies to concrete, computable fields with infinitesimals, such as the real algebraic Puiseux series. Moreover, real algebraic Puiseux series converge, so they can be interpreted as functions of a real parameter; this suggests borrowing ideas from complex analysis, such as the substitution of a root of unity in the proof of Theorem 4.12, although that proof is purely algebraic (Section 4.8).

For extending linear algebra to the perturbative setting, the second approach provides exactly the right balance, and we use it throughout. All arithmetic operations of numerical linear algebra, as well as square roots, are semialgebraic, so the restriction to polynomial statements is harmless for our purposes.

4.4 Puiseux Series

Let k be a field of characteristic 0 and t an indeterminate.

- The field of *formal Laurent series* is $k((t))$, whose elements are series $\sum_{j \geq j_0} c_j t^j$ with $j_0 \in \mathbb{Z}$; the ring of *formal power series* $k[[t]]$ consists of the series with $j_0 \geq 0$.

- The field of *formal Puiseux series* is $k\{\{t\}\} = \bigcup_{N \geq 1} k((t^{1/N}))$, whose elements are Laurent series in a root $t^{1/N}$ of t.

- If $k = \mathbb{R}$ or $k = \mathbb{C}$, a Puiseux series is *convergent* if it converges on some punctured disc $0 < |t| < \rho$; the convergent Puiseux series form a subfield.

If k is real closed, $k\{\{t\}\}$ is ordered by the sign of the leading coefficient; in this ordering t is a positive infinitesimal.

Theorem 4.5 (Newton–Puiseux)**.** *If k is algebraically closed of characteristic 0, then $k\{\{t\}\}$ is algebraically closed. If k is real closed, then $k\{\{t\}\}$ is real closed. The same holds for convergent Puiseux series over $\mathbb{C}$ and $\mathbb{R}$ respectively.*

See [7, §2.6] and [9]. The real closure of the rational function field $\mathbb{R}(t)$, ordered so that t is a positive infinitesimal, is the field $\mathbb{R}\langle t\rangle$ of *algebraic* Puiseux series, consisting of the Puiseux series that are algebraic over $\mathbb{R}(t)$. These series converge, and $\mathbb{R}\langle t\rangle$ is isomorphic to the field of germs at 0^+ of continuous semialgebraic functions on intervals $(0, \rho)$ [7, §2.6 and Chapter 3]; its elements can be represented exactly and manipulated algorithmically. More general real closed fields with infinitesimals are the Hahn fields $k((t^{\mathbb{Q}}))$ of series with well-ordered support in $\mathbb{Q}$ [73], which contain the Puiseux series.

Formal Puiseux series supply a clean algebraic, ordered and real closed base field; convergent Puiseux series provide the analytic counterpart, in which each element is a function of the real parameter t.

4.5 Infinitesimals and the Standard Part

4.5.1 Finite Elements, Infinitesimals and Asymptotic Notation

Let F be an ordered field containing $\mathbb{R}$. An element $x \in F$ is *finite* if $|x| < m$ for some integer m, and *infinitesimal* if $|x| < 1/m$ for every positive integer m. We write F_{fin} for the set of finite elements and $\mathfrak{m}$ for the set of infinitesimals.

Proposition 4.6. *F_{fin} is a subring of F, $\mathfrak{m}$ is its unique maximal ideal, and every $x \in F_{\text{fin}}$ is infinitely close to a unique real number* $\text{st}(x)$*, the* standard part *of x. The map* $\text{st} : F_{\text{fin}} \to \mathbb{R}$ *is a surjective ring homomorphism with kernel $\mathfrak{m}$.*

Proof. Closure of F_{fin} and $\mathfrak{m}$ under the ring operations follows from the corresponding facts for bounds. If $x \in F_{\text{fin}} \setminus \mathfrak{m}$ then $1/m < |x| < m'$ for some integers, so $1/x$ is finite; hence every element outside $\mathfrak{m}$ is a unit and $\mathfrak{m}$ is the unique maximal ideal. For $x \in F_{\text{fin}}$, the set $\{r \in \mathbb{R} : r < x\}$ is non-empty and bounded above, so it has a supremum $s \in \mathbb{R}$ by completeness of $\mathbb{R}$; for every m, $s - 1/m < x \leq s + 1/m$, so $x - s \in \mathfrak{m}$. Uniqueness holds because two distinct reals are not infinitely close. That st is a homomorphism follows since $\mathfrak{m}$ is an ideal. □

Asymptotic notation has a precise meaning in this setting: for $x \in F$ we set

$$O(x) = x\,F_{\mathrm{fin}}, \qquad o(x) = x\,\mathfrak{m},$$

so that $O(1) = F_{\mathrm{fin}}$ and $o(1) = \mathfrak{m}$. These are F_{fin}-submodules of F, and the usual rules such as $O(x)O(y) = O(xy)$ and $O(x) + o(x) = O(x)$ hold.

4.5.2 Limits of Algorithms as Standard Parts

We now justify the "auto-limit" of Section 1.3.1. By an *algorithm over ordered fields* we mean a finite sequence of instructions, each of which either applies one of the operations $+, -, \times$, division by a non-zero element, or the square root of a non-negative element to previously computed quantities, or branches according to the sign of a previously computed quantity. The algorithm may loop, provided that each run on the inputs under consideration executes finitely many instructions.

Proposition 4.7 (Auto-limit)**.** *Let $\mathcal{A}$ be an algorithm over ordered fields and let its inputs $x_1(\varepsilon), \dots, x_m(\varepsilon)$ be elements of the field $\mathbb{R}\langle\varepsilon\rangle$ of algebraic Puiseux series, for example polynomials in ε. Suppose that $\mathcal{A}$, executed over $\mathbb{R}\langle\varepsilon\rangle$, terminates without error and produces outputs $y_1, \dots, y_r \in \mathbb{R}\langle\varepsilon\rangle$. Then there is $\varepsilon_0 > 0$ such that for every real $\varepsilon' \in (0, \varepsilon_0)$, executing $\mathcal{A}$ in real arithmetic on the inputs $x_1(\varepsilon'), \dots, x_m(\varepsilon')$ follows the same branches and produces the outputs $y_1(\varepsilon'), \dots, y_r(\varepsilon')$. In particular, if y_j is finite, then*

$$\lim_{\varepsilon' \to 0^+} y_j(\varepsilon') = \mathrm{st}(y_j).$$

Proof. Identify $\mathbb{R}\langle\varepsilon\rangle$ with the field of germs at 0^+ of continuous semialgebraic functions (Section 4.4). The field operations and square roots of germs are computed pointwise, and a non-zero germ has constant sign on some interval $(0, \rho)$. Each of the finitely many executed instructions therefore agrees with its pointwise counterpart on some interval $(0, \rho_i)$, and each branch is decided in the same way there; take $\varepsilon_0 = \min_i \rho_i$. Finally, a finite element $y = \sum_{q \geq 0} c_q \varepsilon^q$ of $\mathbb{R}\langle\varepsilon\rangle$ is a convergent series without negative exponents, so $y(\varepsilon') \to c_0 = \mathrm{st}(y)$. $\square$

The proposition explains the transition from limits to infinitesimals in Section 1.3.1: instead of analysing the limit of an algorithm as a parameter tends to zero, we execute the algorithm once, exactly, over $\mathbb{R}\langle\varepsilon\rangle$ and take standard parts. Two remarks are in order. First, the limit of the outputs of an algorithm need not equal the output of the algorithm applied to the limiting input; the latter may not even be defined, as when Householder QR with respect to $\langle\cdot,\cdot\rangle_\varepsilon$ is applied at $\varepsilon = 0$, where the "inner product" is degenerate. Second, Theorem 4.2 guarantees that the algorithm is correct over $\mathbb{R}\langle\varepsilon\rangle$ whenever its correctness over $\mathbb{R}$ is a first-order statement, for each fixed matrix size.

4.5.3 Nilpotent Infinitesimals as Quotients

Theories of *nilpotent* infinitesimals, such as the dual numbers, make it easy to automate the computation of derivatives by operator overloading [36], and they support intuitive geometric derivations of formulas for areas, volumes and lengths. They are also the basis of Synthetic Differential Geometry [59], which develops calculus with nilpotent infinitesimals using intuitionistic logic. The following proposition shows that, for the purposes of this thesis, nilpotent infinitesimals can be obtained from invertible ones by passing to a quotient, so that no separate logical framework is required.

Proposition 4.8. *Let $F \supseteq \mathbb{R}$ be an ordered field and $\delta \in F$ a positive infinitesimal. Then $A_\delta := \mathbb{R}+O(\delta)$ is a subring of F_{fin}, $o(\delta)$ is an ideal of A_δ, and the map $x+a\delta \mapsto x+\mathrm{st}(a)\,\varepsilon$ ($x \in \mathbb{R}$, $a \in F_{\mathrm{fin}}$) induces an isomorphism*

$$A_\delta/o(\delta) \;\cong\; \mathbb{R}[\varepsilon]/(\varepsilon^2).$$

Proof. A_δ is closed under multiplication because $(x+a\delta)(x'+a'\delta) = xx'+(xa'+x'a+aa'\delta)\delta$. The map is well defined: if $x + a\delta = x' + a'\delta$ then $x - x' \in O(\delta) \cap \mathbb{R} = \{0\}$ and $a = a'$. It is a ring homomorphism because $\delta^2 = \delta \cdot \delta \in o(\delta)$ maps to $\varepsilon^2 = 0$; it is surjective; and its kernel consists of the elements $a\delta$ with $\mathrm{st}(a) = 0$, that is, of $o(\delta)$. □

If $f : \mathbb{R} \to \mathbb{R}$ is differentiable at $x \in \mathbb{R}$ and f extends to F (for instance, f semialgebraic and F real closed, or $F = {}^*\mathbb{R}$), then $f(x+\delta) = f(x)+f'(x)\delta+o(\delta)$ for every infinitesimal

$\delta \neq 0$, so $f(x+\delta) \in A_\delta$ and its image in the dual numbers is $f(x) + f'(x)\varepsilon$. This is precisely the value computed by forward-mode automatic differentiation. In this sense the dual numbers, and similarly the algebras $\mathbb{R}[\varepsilon]/(\varepsilon^m)$ with $A_\delta = \mathbb{R} + \mathbb{R}\delta + \cdots + O(\delta^{m-1})$, are absorbed into the theory of ordered fields with infinitesimals. The same idea, reversed, underlies the completed localisation of Chapter 7: a nilpotent element is lifted to an invertible infinitesimal, the computation is carried out in a ring in which it is invertible (for the quiver with one loop, a field of Laurent series), and the result is reduced again.

Remark 4.9 (Historical note). Infinitesimals were used informally by Leibniz, Euler and others. In the nineteenth century Cauchy, Bolzano and above all Weierstrass based analysis on the notion of limit, formulated with inequalities and nested quantifiers. Robinson showed in the 1960s that a consistent theory of infinitesimals can be obtained from model theory [74]. Fields such as $\mathbb{R}(\varepsilon)$, which contain infinitesimals but support little analysis, had long been known; the point of the transfer principles above is that they allow statements to pass between $\mathbb{R}$ and its extension in both directions.

4.6 Pythagorean Fields and Laurent Series

The field of Laurent series is not real closed (t has no square root in $k((t))$), but it has the property needed for Gram–Schmidt orthogonalisation.

Proposition 4.10. *Let k be a real closed field. Then the field $k((t))$ of formal Laurent series, ordered by the sign of the leading coefficient, is Pythagorean. A sum of squares $s = \sum_i x_i^2$ of elements of $k((t))$ has even valuation, and s is zero only if all x_i are. If $x \in k((t))^n$ satisfies $\sum_i x_i^2 = 1$, then $x \in k[[t]]^n$. The same statements hold for convergent Laurent series over $\mathbb{R}$.*

Proof. Let v be the smallest valuation among the non-zero x_i, and let c_i be the coefficient of t^v in x_i. The coefficient of t^{2v} in s is $\sum_i c_i^2 > 0$, and all lower coefficients vanish. Hence $s = c\,t^{2v}(1+h)$ with $c > 0$ and $h \in t\,k[[t]]$. Then $s = (\sqrt{c}\,t^v(1+h)^{1/2})^2$, where $(1+h)^{1/2}$ is given by the binomial series, which converges t-adically, and converges analytically if h

does. If $\sum_i x_i^2 = 1$ then $2v = 0$ for the smallest valuation v, so no x_i has a negative power of t. □

Thus over $k((t))$ the norm $\|x\| = (\sum_i x_i^2)^{1/2}$ of a vector is defined, it vanishes only for $x = 0$, and normalised vectors have entries in $k[[t]]$. Consequently the Gram–Schmidt process can be applied without restriction over $k((t))$.

The field $\mathbb{R}((t))$ is the completed localisation (Definition 7.4, with $k = \mathbb{R}$) of the path algebra $\mathbb{R}[t]$ of the quiver with one loop t: the localisation adjoins t^{-1}, and the completion admits series in t. With $t^* = t$, it is the real part of the completed localisation of the doubly-signed quiver with one vertex of type $(\mathbb{R}, +1)$ and one loop fixed by the transpose with sign $+1$ (Section 7.3). So the computations of this chapter in $\mathbb{R}((t))$ are computations in Λ, and the implementation of Chapter 8 performs them there. Since $\mathbb{R}((t))$ is Pythagorean but not real closed, an algorithm that needs square roots of elements of odd order, such as t itself, must be run over a larger field, for example the Puiseux series; in the algorithms below this is avoided by writing the parameter as $\varepsilon = t^2$.

4.7 The Bruhat Decomposition as an Auto-Limit

Proposition 1.1 showed that the Gram–Schmidt process, with respect to the weighted inner product (1.3), tends to Gaussian elimination when the leading principal minors of the matrix are non-zero. Without this assumption the limit is a Bruhat decomposition. Every $M \in \mathrm{GL}_n(\mathbb{R})$ can be written as $M = LwU$, with L lower triangular, U upper triangular and w a permutation matrix, and w is determined by M; we write $we_j = e_{w(j)}$. This is the Bruhat decomposition $\mathrm{GL}_n = \bigsqcup_w B_- wB$, where B and B_- are the upper and lower triangular groups, which follows from the decomposition $\mathrm{GL}_n = \bigsqcup_w BwB$ [52] applied to $w_0 M$, where w_0 reverses the order of the coordinates and $w_0 B_- = B w_0$. It can be computed directly by Gaussian elimination: working through the columns from left to right, take as pivot in each column its first non-zero entry in a row not yet used as a pivot row, and use it to eliminate the entries below it; the pivot of column j then lies in row $w(j)$. We now obtain the decomposition in another way, as an auto-limit

(Proposition 4.7), in fields of Laurent series.

Theorem 4.11. *Let $M \in \mathrm{GL}_n(\mathbb{R})$, and let $M = LwU$ be a Bruhat decomposition. Apply the Gram–Schmidt recurrence (1.1), with the inner product $\langle x, y\rangle_\varepsilon = \sum_i \varepsilon^{i-1} x_i y_i$, to the columns of M over the field $\mathbb{R}((\varepsilon))$, in which ε is a positive infinitesimal. This gives vectors $u_1, \dots, u_n$ and a unit upper triangular matrix $R = (r_{kj})$ of coefficients with $M = [u_1 \cdots u_n]\, R$.*

1. *$\langle u_j, u_j\rangle_\varepsilon = \varepsilon^{w(j)-1}\big(c_j + O(\varepsilon)\big)$ with $c_j = (L_{w(j)w(j)} U_{jj})^2 > 0$. In particular, the orders of the norms determine w, and c_j depends only on M.*

2. *The entries of the u_j and of R are finite. Put $P' = \mathrm{st}[u_1 \cdots u_n]$ and $U' = \mathrm{st}(R)$. Then $M = P'U'$, U' is unit upper triangular, and $L' = P'w^{-1}$ is lower triangular and invertible. Thus $M = L'wU'$ is a Bruhat decomposition.*

3. *Let $t = \sqrt{\varepsilon}$, and let $\mathrm{diag}(1, t, \dots, t^{n-1})\, M = \hat{Q}\hat{R}$ be the QR factorisation over $\mathbb{R}((t))$, with positive diagonal in $\hat{R}$. Then $\hat{Q}$ has finite entries, and $\mathrm{st}(\hat{Q})$ is the signed permutation matrix whose column j is $\mathrm{sgn}(L'_{w(j)w(j)})\, e_{w(j)}$.*

By Proposition 4.7, applied in the field of algebraic Puiseux series, which contains all the quantities computed here, these standard parts are the limits as $\varepsilon \to 0^+$ of the same computations in real arithmetic.

Proof. For sets S, T of indices, write $M_{S,T}$ for the corresponding submatrix of M, and put $[j] = \{1, \dots, j\}$, $W_j = \{w(1), \dots, w(j)\}$ and $\sigma_j = \sum_{k \le j} (w(k) - 1)$. Since U is upper triangular, the first j columns of M are the columns $Le_{w(1)}, \dots, Le_{w(j)}$ of L, multiplied on the right by the leading $j \times j$ block $U_{[j]}$ of U. So for $|S| = j$,

$$\det M_{S,[j]} = \pm \det L_{S,W_j} \det U_{[j]}.$$

If $s_1 < \cdots < s_j$ and $q_1 < \cdots < q_j$ are the elements of S and W_j, then $\det L_{S,W_j} = 0$ unless $s_k \ge q_k$ for all k: otherwise, if $s_m < q_m$, the rows $s_1, \dots, s_m$ of L_{S,W_j} vanish in the columns $q_m, \dots, q_j$, because L is lower triangular, and a $j \times j$ matrix with a zero block of size $m \times (j - m + 1)$ is singular. Moreover $\det L_{W_j,W_j} = \prod_{k \le j} L_{w(k)w(k)} \neq 0$. Let

$G = M^T D_\varepsilon M$ be the Gram matrix of the columns of M, where $D_\varepsilon = \operatorname{diag}(1, \varepsilon, \dots, \varepsilon^{n-1})$, and let G_j be its leading $j \times j$ submatrix. By the Cauchy–Binet formula,

$$\det G_j = \sum_{|S|=j} \varepsilon^{\sum_{s\in S}(s-1)} \big(\det M_{S,[j]}\big)^2.$$

Every set S that contributes satisfies $s_k \geq q_k$ for all k, so its exponent is at least σ_j, with equality only for $S = W_j$. Hence $\det G_j = \varepsilon^{\sigma_j}(\gamma_j + O(\varepsilon))$ with $\gamma_j = \prod_{k\leq j}(L_{w(k)w(k)}U_{kk})^2 > 0$.

(1) The vectors u_j are orthogonal for $\langle\cdot,\cdot\rangle_\varepsilon$, so $G = R^T\Delta^2 R$ with $\Delta^2 = \operatorname{diag}(\langle u_j, u_j\rangle_\varepsilon)$. Hence $\langle u_j, u_j\rangle_\varepsilon = \det G_j / \det G_{j-1} = \varepsilon^{w(j)-1}(c_j + O(\varepsilon))$, with $c_j = \gamma_j/\gamma_{j-1}$.

(2) Since $R^T\Delta^2$ is lower triangular, the first k rows of G are $(R^T\Delta^2)_{[k],[k]}R_{[k],[n]}$, so for $k < j$ we get $\det G_{[k],[k-1]\cup\{j\}} = \det G_k \cdot \det R_{[k],[k-1]\cup\{j\}} = \det G_k \cdot r_{kj}$. By the Cauchy–Binet formula, $\det G_{[k],[k-1]\cup\{j\}} = \sum_{|S|=k} \varepsilon^{\sum_{s\in S}(s-1)} \det M_{S,[k]} \det M_{S,[k-1]\cup\{j\}}$, which is $O(\varepsilon^{\sigma_k})$ by the argument above, whereas $\det G_k = \varepsilon^{\sigma_k}(\gamma_k + O(\varepsilon))$. So r_{kj} is finite for all $k < j$, R is finite, and so is $[u_1 \cdots u_n] = MR^{-1}$. Taking standard parts in $M = [u_1 \cdots u_n]R$ gives $M = P'U'$, where U' is unit upper triangular and $P' = MU'^{-1}$ is invertible. By (1), $\varepsilon^{i-1}u_{j,i}^2 \leq \langle u_j, u_j\rangle_\varepsilon = O(\varepsilon^{w(j)-1})$, so $\operatorname{st}(u_{j,i}) = 0$ for $i < w(j)$: column j of P' vanishes above row $w(j)$. This column is column $w(j)$ of $L' = P'w^{-1}$, so L' is lower triangular, and it is invertible.

(3) The factorisation exists because $\mathbb{R}((t))$ is Pythagorean (Proposition 4.10). Put $T = \operatorname{diag}(1, t, \dots, t^{n-1})$ and $\Delta = \operatorname{diag}(\langle u_j, u_j\rangle_\varepsilon^{1/2})$. Then

$$TM = \big(T[u_1 \cdots u_n]\Delta^{-1}\big)(\Delta R),$$

and the first factor has orthonormal columns, so it is $\hat{Q}$. By Proposition 4.10, the entries of $\hat{Q}$ are finite, and $\operatorname{st}(\hat{Q})$ is orthogonal because st is a ring homomorphism. By (1), $\hat{Q}_{ij} = t^{i-1}u_{j,i}/\Delta_{jj}$ with $\Delta_{jj} = t^{w(j)-1}(\sqrt{c_j} + O(t^2))$. For $i > w(j)$ this is infinitesimal, since $u_{j,i}$ is finite, and for $i = w(j)$ its standard part is $\operatorname{st}(u_{j,w(j)})/\sqrt{c_j}$, which has the sign of $L'_{w(j)w(j)}$. So column j of $\operatorname{st}(\hat{Q})$ vanishes below row $w(j)$. By induction on $w(j)$, the column with $w(j) = m$ is orthogonal to the columns $\pm e_1, \dots, \pm e_{m-1}$ found before, so it

is $\pm e_m$. $\square$

For $w = 1$ the theorem is Proposition 1.1. Two further remarks. First, parts (1) and (2) use only field operations, in the field $\mathbb{R}((\varepsilon))$, which is the completed localisation of the quiver with one loop ε; part (3) also needs the square roots t^{i-1} of the weights, and $\mathbb{R}((t))$ is the completed localisation of the quiver with one loop t. So the whole computation takes place in completed localisations, and the implementation of Chapter 8 carries it out there; for $n = 4$ it recovers w, L' and U' in all 24 Bruhat cells (Table 8.2). Second, the computation needs M exactly: the orders of the norms, and hence w, change under arbitrarily small perturbations of M, in agreement with the discontinuity of the Bruhat decomposition discussed in Section 1.4.

4.8 Matrix Decompositions over Formal Power Series

4.8.1 The Spectral Theorem

The following theorem generalises the result of [39], which concerns matrices over the dual numbers $k[\varepsilon]/(\varepsilon^2)$; that result is recovered by reducing modulo ε^2.

Theorem 4.12 (Spectral theorem over formal power series)**.** *Let k be a real closed field and let $S(\varepsilon)$ be a symmetric matrix with entries in $k[[\varepsilon]]$. Then there are an orthogonal matrix $U(\varepsilon)$ and a diagonal matrix $\Lambda(\varepsilon)$, both with entries in $k[[\varepsilon]]$, such that*

$$S(\varepsilon) = U(\varepsilon)\,\Lambda(\varepsilon)\,U(\varepsilon)^T.$$

If $k = \mathbb{R}$ and the entries of $S(\varepsilon)$ are convergent power series, then $U(\varepsilon)$ and $\Lambda(\varepsilon)$ can be chosen with convergent entries.

For $k = \mathbb{R}$ and convergent S, Theorem 4.12 is a form of Rellich's theorem from analytic perturbation theory [55, Chapter II]. The proof below is purely algebraic; it applies to formal power series over any real closed field, and it illustrates the use of the transfer principle.

Proof. Let $K = k\{\{\varepsilon\}\}$ be the field of formal Puiseux series, which is real closed (Theorem 4.5) and contains $k[[\varepsilon]]$.

Step 1: the eigenvalues lie in K. For each n, the spectral theorem for $n \times n$ symmetric matrices is a first-order sentence (Section 4.3.1), true over $\mathbb{R}$. By Theorem 4.2 it holds over K. Hence $S(\varepsilon)$ is orthogonally diagonalisable over K, and all roots of its characteristic polynomial lie in K.

Step 2: the eigenvalues are finite. Over $\mathbb{R}$, every eigenvalue λ of a symmetric matrix satisfies $\lambda^2 \leq \sum_{i,j} s_{ij}^2$; this inequality is first-order, so it transfers to K. The right-hand side lies in $k[[\varepsilon]]$, so it is smaller than $c = 1 + \sum_{i,j} s_{ij}(0)^2 \in k$, and since $c \geq 1$ we get $|\lambda| < c$. An element of K that is bounded in absolute value by an element of k has no term with a negative exponent, since such a term would dominate. Hence no eigenvalue has a term with a negative exponent.

Step 3: the eigenvalues have no fractional exponents. Let λ be an eigenvalue, let $d \geq 1$ be minimal with $\lambda \in k((\varepsilon^{1/d}))$, put $s = \varepsilon^{1/d}$ and write $\lambda = f(s) = \sum_j f_j s^j$. Suppose, for a contradiction, that $d \geq 2$. Let $C = k(\sqrt{-1})$, which is algebraically closed, and let $\omega \in C$ be a primitive $2d$-th root of unity, so that $\omega^d = -1$. The substitution $s \mapsto \omega s$ defines a ring automorphism of $C((s))$ that maps $\varepsilon = s^d$ to $-\varepsilon$. Applying it to the identity $\det(S(\varepsilon) - f(s)I) = 0$ yields

$$\det\big(S(-\varepsilon) - f(\omega s)\, I\big) = 0.$$

So $f(\omega s) = \sum_j f_j \omega^j \varepsilon^{j/d}$ is an eigenvalue of the symmetric matrix $S(-\varepsilon)$, whose entries again lie in $k[[\varepsilon]]$. By Step 1 applied to $S(-\varepsilon)$, this eigenvalue lies in K, that is, all its coefficients $f_j \omega^j$ lie in k. Since $\omega^j \in k$ only when $\omega^j = \pm 1$, that is, when d divides j, we conclude that $f_j = 0$ unless $d \mid j$. Then $\lambda \in k((\varepsilon))$, contradicting the minimality of d. Together with Step 2, every eigenvalue lies in $k[[\varepsilon]]$.

Step 4: eigenvectors. For each eigenvalue $\lambda \in k[[\varepsilon]]$, the matrix $S(\varepsilon) - \lambda I$ has entries in the field $k((\varepsilon))$, so Gaussian elimination over $k((\varepsilon))$ yields a basis of its kernel with entries in $k((\varepsilon))$.

Step 5: orthonormalisation. By Proposition 4.10, $k((\varepsilon))$ is an ordered Pythagorean

field, so the Gram–Schmidt process applies within each eigenspace and produces an orthonormal basis of it with entries in $k((\varepsilon))$.

Step 6: orthogonality between eigenspaces. Eigenvectors of a symmetric matrix for distinct eigenvalues are orthogonal over $\mathbb{R}$; this is first-order and transfers to K.

Step 7: integrality. The resulting eigenvectors are unit vectors over $k((\varepsilon))$, so by Proposition 4.10 their entries lie in $k[[\varepsilon]]$. Assembling them into the columns of $U(\varepsilon)$ gives the required decomposition.

For the convergent case, replace formal Puiseux and Laurent series by convergent ones throughout. Convergent Puiseux series form a real closed field (Theorem 4.5), convergent Laurent series form a Pythagorean field (Proposition 4.10), and the substitution in Step 3 preserves convergence. □

4.8.2 The Singular Value Decomposition

Corollary 4.13 (SVD over formal power series)**.** *Let k be a real closed field and $B \in M_{p\times q}(k[[\varepsilon]])$. Then there are orthogonal matrices $Q \in \mathrm{O}_p(k[[\varepsilon]])$ and $P \in \mathrm{O}_q(k[[\varepsilon]])$ and a diagonal matrix $\Sigma \in M_{p\times q}(k[[\varepsilon]])$, whose non-zero diagonal entries have positive leading coefficients, such that $B = Q\Sigma P^T$. If $k = \mathbb{R}$ and B is convergent, the factors can be chosen convergent.*

Proof. Apply Theorem 4.12 to $B^TB = P\Lambda P^T$, with $\Lambda = \mathrm{diag}(\mu_1, \ldots, \mu_q)$ and columns p_i of P. Then $\mu_i = \|Bp_i\|^2$ is a sum of squares in $k[[\varepsilon]]$, so by Proposition 4.10 it has a square root $\sigma_i \in k[[\varepsilon]]$ with non-negative leading coefficient. Order the columns of P so that $\sigma_1, \ldots, \sigma_r$ are the non-zero ones; then $Bp_i = 0$ for $i > r$, since $\|Bp_i\|^2 = \mu_i = 0$. For $i \le r$ put $q_i = Bp_i/\sigma_i$. These are orthonormal vectors over $k((\varepsilon))$, since $q_i^Tq_j = p_i^TB^TBp_j/(\sigma_i\sigma_j) = \delta_{ij}$, and by Proposition 4.10 their entries lie in $k[[\varepsilon]]$. Extend $q_1, \ldots, q_r$ to an orthonormal basis $q_1, \ldots, q_p$ of $k((\varepsilon))^p$ by the Gram–Schmidt process; again all entries lie in $k[[\varepsilon]]$. With $Q = [q_1 \cdots q_p]$ and Σ the $p \times q$ diagonal matrix with entries $\sigma_1, \ldots, \sigma_r$ followed by zeros, we have $BP = Q\Sigma$. □

For $k = \mathbb{R}$ and convergent B, this is the existence of an analytic singular value decomposition, which can also be computed numerically [11]; reducing modulo ε^2 recovers

the SVD of dual-number matrices of [39].

4.9 A Parity-Equivariant Spectral Theorem

In [44] we observed that many important decompositions of numerical linear algebra can be viewed as variants of the spectral theorem over various $*$-algebras, equivariant with respect to a group of symmetries. In this section we make this observation precise for the most important case, in which the symmetry is the parity automorphism $\varepsilon \mapsto -\varepsilon$ of the power series ring.

Throughout this section, k is a real closed field, $\mathcal{R} = k[[\varepsilon]]$, and $\gamma : \mathcal{R} \to \mathcal{R}$ is the automorphism $\gamma(f)(\varepsilon) = f(-\varepsilon)$, extended entrywise to matrices. The fixed ring is $\mathcal{F} = \mathcal{R}^{\gamma} = k[[\varepsilon^2]]$. An element or matrix X is *even* if $\gamma(X) = X$ and *odd* if $\gamma(X) = -X$; every X is uniquely the sum of an even and an odd part. Fix $p, q \geq 0$, let $J = \operatorname{diag}(I_p, -I_q)$, and let $\tau = J\gamma$ be the semilinear involution $v \mapsto J\gamma(v)$ of $\mathcal{R}^{p+q}$. A vector v is *τ-homogeneous* of parity ± 1 if $\tau v = \pm v$.

Step 3 of the proof below uses Galois descent [58, §18] for a quadratic extension, in the following elementary form.

Lemma 4.14. *Let L be a field of characteristic different from 2, θ an automorphism of L of order 2, and $K = L^{\theta}$ its fixed field. Let θ act on L^n coordinatewise. Then every θ-stable L-subspace $W \subseteq L^n$ has an L-basis consisting of θ-fixed vectors.*

Proof. Choose $a \in L \setminus K$ and put $c = a - \theta(a) \neq 0$, so that $\theta(c) = -c$. For $w \in W$ we have

$$w = \tfrac{1}{2}\,(w + \theta w) + \tfrac{1}{2}\,c^{-1}\big(c\,(w - \theta w)\big),$$

and both $w + \theta w$ and $c(w - \theta w)$ are θ-fixed elements of W. Hence the θ-fixed vectors span W over L, and a basis can be extracted from them. □

Theorem 4.15 (Parity-equivariant spectral theorem)**.** *Let S be a symmetric $(p+q) \times (p+q)$ matrix over $\mathcal{R}$ satisfying the equivariance condition $J\gamma(S)J = S$; equivalently, the diagonal blocks of S (of sizes p and q) are even and the off-diagonal blocks are odd. Then*

there exist an orthogonal matrix U over $\mathcal{R}$ and a diagonal sign matrix J' with the same number of entries $+1$ as J, such that $J\gamma(U) = UJ'$ and U^TSU is block diagonal, each block being either

- *a 1×1 block (α) with $\alpha \in \mathcal{F}$, or*
- *a 2×2 block $\begin{pmatrix} \alpha & \beta \\ \beta & \alpha \end{pmatrix}$ with $\alpha \in \mathcal{F}$ and $\beta \in \varepsilon\mathcal{F}$ odd, on which J' restricts to* $\mathrm{diag}(1,-1)$.

Proof. By Theorem 4.12, $S = \sum_\lambda \lambda P_\lambda$, where λ runs over the distinct eigenvalues in $\mathcal{R}$ and P_λ is the orthogonal projector onto the λ-eigenspace; the P_λ have entries in $\mathcal{R}$, since $P_\lambda = \sum_i u_i u_i^T$ for orthonormal eigenvectors u_i over $\mathcal{R}$. Note that $\langle \tau u, \tau v\rangle = \gamma\langle u, v\rangle$ for the standard bilinear form, because $J^TJ = I$.

Step 1: τ permutes the eigenspaces. The equivariance condition says that $\tau S = S\tau$. If $Sv = \lambda v$ then $S\tau v = \tau(Sv) = \tau(\lambda v) = \gamma(\lambda)\tau v$. Hence τ maps the λ-eigenspace onto the $\gamma(\lambda)$-eigenspace, and the eigenvalues fall into orbits $\{\lambda, \gamma(\lambda)\}$ of size one (when λ is even) or two.

Step 2: orbits of size two. Let $\lambda \neq \gamma(\lambda)$ and write $\lambda = \alpha + \beta$ with α even and $\beta \neq 0$ odd. Let $v_1, \ldots, v_m$ be an orthonormal basis over $\mathcal{R}$ of the λ-eigenspace. Then $\tau v_1, \ldots, \tau v_m$ is an orthonormal basis of the $\gamma(\lambda)$-eigenspace, and $\langle v_i, \tau v_j\rangle = 0$ because eigenvectors for distinct eigenvalues are orthogonal. Put

$$u_i^{\pm} = \tfrac{1}{\sqrt{2}}\,(v_i \pm \tau v_i).$$

Then $\tau u_i^{\pm} = \pm u_i^{\pm}$, the vectors $u_i^{\pm}$ are orthonormal (using $\langle \tau v_i, \tau v_j\rangle = \gamma(\delta_{ij}) = \delta_{ij}$), and

$$Su_i^{+} = \tfrac{1}{\sqrt{2}}\big(\lambda v_i + \gamma(\lambda)\tau v_i\big) = \alpha u_i^{+} + \beta u_i^{-}, \qquad Su_i^{-} = \alpha u_i^{-} + \beta u_i^{+}.$$

So on $\mathrm{span}(u_i^{+}, u_i^{-})$ the matrix of S is $\left(\begin{smallmatrix} \alpha & \beta \\ \beta & \alpha \end{smallmatrix}\right)$, and τ acts as $\mathrm{diag}(1,-1)$.

Step 3: orbits of size one. Let $\lambda = \gamma(\lambda)$, so $\lambda \in \mathcal{F}$, and let W be the λ-eigenspace over the fraction field $k((\varepsilon))$. By Step 1, W is τ-stable, and τ acts on $k((\varepsilon))^{p+q}$ as J composed

with the coordinatewise automorphism γ. Applying Lemma 4.14 after the change of coordinates $v \mapsto (v_1, \dots, v_p, \varepsilon v_{p+1}, \dots, \varepsilon v_{p+q})$, which turns τ into the coordinatewise action of γ, shows that W has a basis of τ-fixed vectors. We orthonormalise it by the Gram–Schmidt process and claim that homogeneity is preserved. If u, w are homogeneous of parities π_u, π_w, then $\gamma\langle u, w\rangle = \langle \tau u, \tau w\rangle = \pi_u\pi_w\langle u, w\rangle$, so $\langle u, w\rangle$ is homogeneous of parity $\pi_u\pi_w$, and $\langle u, u\rangle$ is even. Hence $w - \frac{\langle u,w\rangle}{\langle u,u\rangle}u$ is homogeneous of parity π_w. Finally, $\langle w, w\rangle \in k((\varepsilon^2))$ is positive, so by the argument of Proposition 4.10 its square root is $\sqrt{c}\,\varepsilon^m(1+h)^{1/2}$ with h even, which is homogeneous of parity $(-1)^m$; dividing by it preserves homogeneity. We obtain an orthonormal basis of W consisting of homogeneous unit vectors, whose entries lie in $\mathcal{R}$ by Proposition 4.10. On these vectors S acts as the scalar $\lambda \in \mathcal{F}$.

Step 4: assembly. Let U be the orthogonal matrix whose columns are the vectors constructed in Steps 2 and 3, grouped by blocks, and let J' be the diagonal matrix of their parities. Then $\tau U = UJ'$, that is, $J\gamma(U) = UJ'$, and U^TSU has the stated block form. Evaluating $J\gamma(U) = UJ'$ at $\varepsilon = 0$ gives $J' = U(0)^TJU(0)$, so $\operatorname{tr} J' = \operatorname{tr} J$, and J' has the same number of entries $+1$ as J. □

The 2×2 parity blocks play the role of the 2×2 blocks in the real normal forms of normal and skew-symmetric matrices: they occur exactly when the eigenvalues are exchanged by the symmetry. Two consequences are worth recording.

Corollary 4.16 (SVD of odd matrices)**.** *Let $B \in M_{p\times q}(\mathcal{R})$ be odd. Then $B = Q\Sigma P^T$ with $P \in \mathrm{O}_q(\mathcal{F})$ and $Q \in \mathrm{O}_p(\mathcal{F})$ even and orthogonal, and Σ diagonal with odd entries in $\varepsilon\mathcal{F}$.*

Proof. Since B is odd, $B = \varepsilon C$ with C having entries in $\mathcal{F} = k[[\varepsilon^2]]$, which is again a formal power series ring over k, in the variable ε^2. Corollary 4.13 applied to C over $\mathcal{F}$ gives $C = Q\Sigma_0P^T$ with P, Q, Σ_0 over $\mathcal{F}$, and then $B = Q(\varepsilon\Sigma_0)P^T$. □

Corollary 4.16 is the decomposition behind the self-adjoint matrix $\left(\begin{smallmatrix} 0 & B^T \\ B & 0 \end{smallmatrix}\right)$, which satisfies the hypothesis of Theorem 4.15 with $J = \operatorname{diag}(I_q, -I_p)$ precisely when B is odd; its eigenvalues $\pm\sigma_i$ are exchanged by γ, and Theorem 4.15 produces the parity blocks $\left(\begin{smallmatrix} 0 & \sigma_i \\ \sigma_i & 0 \end{smallmatrix}\right)$.

Corollary 4.17 (Hermitian matrices over $(\mathcal{R}, \varepsilon^* = -\varepsilon)$)**.** *Equip $\mathcal{R}$ with the involution $f^* = \gamma(f)$, so that $\varepsilon^* = -\varepsilon$, and let $H \in M_n(\mathcal{R})$ satisfy $H^* = H$, that is, $\gamma(H)^T = H$. Then there is $U \in M_n(\mathcal{R})$ with $U^*U = I$ such that U^*HU is block diagonal with 1×1 blocks (α), $\alpha \in \mathcal{F}$, and 2×2 blocks $\left(\begin{smallmatrix} \alpha & \beta \\ -\beta & \alpha \end{smallmatrix}\right)$ with $\alpha \in \mathcal{F}$ and $\beta \in \varepsilon\mathcal{F}$.*

Sketch of proof. Let $C = k(\sqrt{-1})$ and $i = \sqrt{-1}$. The substitution $\varepsilon = i\delta$ embeds $\mathcal{R}$ into $C[[\delta]]$, and under this embedding the involution γ becomes complex conjugation of the coefficients, $\sigma(\sum c_j \delta^j) = \sum \bar{c}_j \delta^j$. Hence H becomes a complex Hermitian matrix over $C[[\delta]]$. Applying Theorem 4.12 to its realification $\left(\begin{smallmatrix} \operatorname{Re} H & -\operatorname{Im} H \\ \operatorname{Im} H & \operatorname{Re} H \end{smallmatrix}\right)$ shows that it is unitarily diagonalisable over $C[[\delta]]$ with eigenvalues in $k[[\delta]]$. The image of $\mathcal{R}$ is the fixed ring of the automorphism $\theta = \sigma \circ (\delta \mapsto -\delta)$ of $C[[\delta]]$, and θ permutes the eigenvalues as $\lambda(\delta) \mapsto \lambda(-\delta)$. The argument of Theorem 4.15, with τ replaced by θ and with the anti-fixed unit i used to convert anti-fixed vectors into fixed ones, produces a θ-fixed unitary matrix, that is, a unitary matrix over $\mathcal{R}$. An orbit $\{\lambda(\delta), \lambda(-\delta)\}$ of size two with $\lambda = \alpha + \beta'$ (α even, β' odd in δ) yields the block $\left(\begin{smallmatrix} \alpha & i\beta' \\ -i\beta' & \alpha \end{smallmatrix}\right)$, and rewriting $\delta = -i\varepsilon$ shows that $\alpha \in \mathcal{F}$ and $\beta := i\beta' \in \varepsilon\mathcal{F}$. □

For example, $H = \left(\begin{smallmatrix} 0 & \varepsilon \\ -\varepsilon & 0 \end{smallmatrix}\right)$ is Hermitian for $\varepsilon^* = -\varepsilon$ but cannot be diagonalised: its eigenvalues, $\pm\sqrt{-1}\,\varepsilon$, do not lie in $\mathcal{R}$. It is already a parity block. Modulo ε^2, Corollary 4.17 describes Hermitian matrices over the dual numbers with the involution $\varepsilon^* = -\varepsilon$; Chapter 6 shows that the corresponding classical problem is the normal form of real skew-symmetric matrices under orthogonal similarity.

4.10 Shortest Paths as an Auto-Limit

Auto-limits also apply to algorithms whose inputs carry orders of magnitude. Let G be a directed graph with vertex set $V = \{1, \dots, n\}$, without loops and with at most one arrow from i to j, carrying a positive rational weight d_{ij} on each arrow $i \to j$; put $d_{ij} = \infty$ if there is no such arrow. (Dijkstra's algorithm also allows the weight 0, which we exclude.) The *length* $\ell(\gamma)$ of a walk γ is the sum of the weights of its arrows. Write $\operatorname{dist}(i, j)$ for the least length of a path from i to j, with $\operatorname{dist}(i, i) = 0$ and $\operatorname{dist}(i, j) = \infty$ if there is no

path, and $N(i,j)$ for the number of paths of that length. Encoding each weight d by the quantity ε^d, of order of magnitude d, put

$$M = I - A, \qquad A = \left(\varepsilon^{d_{ij}}\right)_{i,j\in V}, \qquad \text{that is,} \quad m_{ij} = \delta_{ij} - \varepsilon^{d_{ij}},$$

over the field $\mathbb{R}\{\{\varepsilon\}\}$ of real Puiseux series (Section 4.4), with $\varepsilon^\infty = 0$.

A non-zero $x \in \mathbb{R}\{\{\varepsilon\}\}$ has a *leading term* $c\,\varepsilon^q$, where q is the least exponent of x, its *order*, and $c \neq 0$. For series with non-negative coefficients nothing cancels: the order of a sum is the least order of the summands, whose leading coefficients are added if several have this order, and orders add under multiplication. Taking leading terms therefore turns sums into minima of exponents and products into sums, and maps such series to the min-plus, or tropical, semiring [68]. That shortest paths can be found by Gaussian elimination over this semiring is classical [14, 35]. The semiring has no subtraction, and elimination there replaces the inverse of $1-a$ by the closure $a^* = 1 \oplus a \oplus a^2 \oplus \cdots$. Over $\mathbb{R}\{\{\varepsilon\}\}$, subtraction and division are available, like the other operations of Section 1.1, and the elimination becomes ordinary Gaussian elimination, followed by taking leading terms.

For $S \subseteq V$ and $i, j \in V$, let $w_S(i,j)$ be the sum of $\varepsilon^{\ell(\gamma)}$ over the walks γ from i to j that have at least one arrow and whose interior vertices, all but the first and the last, lie in S. If the weights have common denominator r, a walk with k arrows has length at least $k \min d_{ij}$, so only finitely many walks have length below any bound, and $w_S(i,j) \in \mathbb{R}((\varepsilon^{1/r}))$. Its coefficients are non-negative integers. If $w_S(i,j) \neq 0$, its order is the least length $\mathrm{dist}_S(i,j)$ of such a walk, and its leading coefficient is the number $N_S(i,j)$ of these walks of that length; for $i \neq j$ they are paths, because removing a closed sub-walk shortens a walk.

Proposition 4.18. *In this notation:*

1. *M is invertible, and $M^{-1} = I + \left(w_V(i,j)\right)_{i,j\in V}$. If $\mathrm{dist}(i,j) < \infty$, the leading term of $(M^{-1})_{ij}$ is $N(i,j)\,\varepsilon^{\mathrm{dist}(i,j)}$; otherwise $(M^{-1})_{ij} = 0$.*

2. *For every ordering of the vertices, Gaussian elimination without pivoting applies to*

M. Once the vertices of a set S have been eliminated, the Schur complement is $\big(\delta_{ij} - w_S(i,j)\big)_{i,j\notin S}$, and every pivot has leading term 1.

3. *Let $s \in V$, and solve $M^T z = e_s$ by Gaussian elimination, taking as the next pivot a remaining vertex u whose current right-hand side entry b_u has the least order. Then b_u and z_u both have the leading term $N(s,u)\,\varepsilon^{\mathrm{dist}(s,u)}$ if $\mathrm{dist}(s,u) < \infty$, and both vanish otherwise. Eliminating u gives each remaining b_j the leading term of $b_j + \varepsilon^{d_{uj}} b_u$.*

Proof. (1) The (i,j) entry of A^k is the sum of $\varepsilon^{\ell(\gamma)}$ over the walks from i to j with k arrows, and its order is at least $k \min d_{ij}$. So $\sum_{k\geq 0} A^k$ converges, it is the inverse of $I - A$, and its (i,j) entry is $\delta_{ij} + w_V(i,j)$. The rest follows from the description of $w_V(i,j)$ above.

(2) Let $T = V \setminus S$. By (1) applied to the graph on S, $M_{S,S}$ is invertible, and $(M_{S,S}^{-1})_{pq}$ is δ_{pq} plus the sum over the walks from p to q inside S. A walk from $i \in T$ to $j \in T$ with interior in S is either an arrow $i \to j$, or an arrow $i \to p$ into S, followed by a walk from p to q inside S, possibly of length zero, and an arrow $q \to j$. So the (i,j) entry of the Schur complement $M_{T,T} - M_{T,S} M_{S,S}^{-1} M_{S,T}$, which is $\delta_{ij} - \varepsilon^{d_{ij}} - \sum_{p,q\in S} \varepsilon^{d_{ip}} (M_{S,S}^{-1})_{pq}\, \varepsilon^{d_{qj}}$, is $\delta_{ij} - w_S(i,j)$. Its diagonal entries $1 - w_S(i,i)$ are the pivots, and $w_S(i,i)$ has positive order.

(3) Initially $b = e_s$, so s is eliminated first, and b_s and $z_s = 1 + w_V(s,s)$ have leading term 1. By the same computation as in (2), once a set $S \ni s$ has been eliminated, $b_j = w_S(s,j)$ for $j \notin S$. Let u be the next pivot and $S' = S \cup \{u\}$. We show, by induction on $|S|$, that every shortest path from s to u has its interior in S and that $\mathrm{dist}(s,p) \leq \mathrm{dist}(s,u)$ for $p \in S$. If $\mathrm{dist}(s,u) = \infty$, then $b_u = 0$ and $z_u = 0$. Otherwise, let γ be a shortest path from s to u and v its first vertex outside S. The part of γ up to v has its interior in S, so $\mathrm{dist}_S(s,v) \leq \mathrm{dist}(s,u)$, with strict inequality if $v \neq u$. Since b_u has the least order, $\mathrm{dist}_S(s,u) \leq \mathrm{dist}_S(s,v)$. Hence $v = u$, $\mathrm{dist}_S(s,u) = \mathrm{dist}(s,u)$ and $N_S(s,u) = N(s,u)$, so $b_u = w_S(s,u)$ has the leading term $N(s,u)\,\varepsilon^{\mathrm{dist}(s,u)}$, and so has $z_u = (M^{-1})_{su}$, by (1).

After the elimination of u, $b_j = w_{S'}(s,j)$. The walks counted in $w_S(s,j) + \varepsilon^{d_{uj}} w_S(s,u)$ are those that avoid u, and those that visit u once and then take the arrow $u \to j$. Every

other walk counted in $w_{S'}(s,j)$ is longer than one of these. If it visits u more than once, removing the closed walk between its first and last visits to u gives a shorter walk that visits u once. If it visits u once and then takes an arrow $u \to p$ with $p \in S$, replace its part up to p, of length at least $\mathrm{dist}(s,u) + d_{up}$, by a shortest path from s to p; by induction, this path has its interior in S and length at most $\mathrm{dist}(s,u)$, so the result is a shorter walk that avoids u. Since nothing cancels, b_j has the leading term of $w_S(s,j) + \varepsilon^{d_{uj}} w_S(s,u)$. Its order is at least that of b_u, so the next pivot u' satisfies $\mathrm{dist}(s,u') \geq \mathrm{dist}(s,u)$, which completes the induction. □

In leading terms, part (3) is Dijkstra's algorithm [23]. The orders of the entries b_j are its tentative distances; the pivot rule chooses a remaining vertex of least tentative distance, which is then final; and $b_j \mapsto b_j + \varepsilon^{d_{uj}} b_u$ is its update $\mathrm{dist}_j \leftarrow \min(\mathrm{dist}_j, \mathrm{dist}_u + d_{uj})$, in which the leading coefficients also count shortest paths. Part (3) also accounts for what Dijkstra's algorithm leaves out: in this order of elimination, the fill-in of the Schur complements, the pivots and the back substitution change no leading term. In a fixed order, Gaussian elimination with back substitution still gives the distances, by part (1), but the fill-in is then needed; in leading terms, this is the elimination method of Carré [14]. All the quantities computed lie in the field $\mathbb{R}\langle\varepsilon\rangle$ of algebraic Puiseux series, so Proposition 4.7 applies: for every sufficiently small real $\varepsilon > 0$, Gaussian elimination in real arithmetic, with the remaining vertex of largest b_u as the next pivot, makes choices allowed in part (3), and, if $\mathrm{dist}(s,u) < \infty$,

$$\mathrm{dist}(s,u) = \lim_{\varepsilon \to 0^+} \frac{\log z_u(\varepsilon)}{\log \varepsilon}.$$

By part (2), PMP^T lies in the big cell B_-B of the Bruhat decomposition (Section 4.7) for every permutation matrix P, so no pivoting is needed for the factorisation to exist. The pivots of part (3) are chosen for another reason. Like the permutation in Theorem 4.11, they are read off from orders of magnitude, here those of the right-hand side, and they make the leading terms available without fill-in or back substitution.

For an undirected graph, that is, when $d_{ij} = d_{ji}$, the matrix M is symmetric. It is

positive definite, because it differs from I by infinitesimals, so the elimination computes $M = LDL^T$, and $M = (LD^{1/2})(LD^{1/2})^T$ is its Cholesky factorisation; the pivots in D and their square roots have leading term 1. For a directed graph, M is not symmetric, but the elimination is still the Cholesky factorisation of a Hermitian matrix. By Proposition 6.6(1), whose proof uses only field operations and so holds over $\mathbb{R}\{\{\varepsilon\}\}$, the Doolittle factorisation of M is the Cholesky factorisation of the Hermitian matrix (M, M^T) over ${}^2\mathbb{R} \otimes \mathbb{R}\{\{\varepsilon\}\}$. The involution of ${}^2\mathbb{R}$ exchanges M and M^T, that is, it reverses every arrow. So Dijkstra's algorithm, like Rutishauser's LR algorithm, can be obtained from an algorithm for Hermitian matrices over a $*$-algebra, here followed by an auto-limit. This reinterprets a classical algorithm and gives no new or faster way to find shortest paths: Dijkstra's algorithm, which works with the orders alone, is much cheaper than Gaussian elimination over $\mathbb{R}\{\{\varepsilon\}\}$ or in real arithmetic.

4.11 A Chain of Limits

The factorisations of this chapter reduce to the Hermitian eigendecomposition by a chain of limits. The singular value decomposition of a matrix W is a Hermitian eigendecomposition: over the $*$-algebra of the quiver of the singular value decomposition (Theorem A), or classically, of the matrix $\left(\begin{smallmatrix} 0 & W^* \\ W & 0 \end{smallmatrix}\right)$. The next proposition obtains the QR factorisation $W = QR$ (the factorisation, not the QR algorithm) as a limit of the singular value decomposition, with the columns of W weighted by decreasing orders of magnitude. No finite algorithm computes a singular value decomposition, so this limit is taken through the identification of $\mathbb{R}\langle\varepsilon\rangle$ with germs of semialgebraic functions, as in the proof of Proposition 4.7, rather than as the auto-limit of an algorithm. With the rows weighted instead, the QR factorisation in turn has the Bruhat decomposition as an auto-limit (Theorem 4.11), and the LU factorisation when the leading principal minors are non-zero (Proposition 1.1). Finally, Dijkstra's shortest-path algorithm is the leading-order part of Gaussian elimination (Proposition 4.18). So the chain runs from the Hermitian eigendecomposition, through the singular value decomposition and the QR factorisation, to the Bruhat and LU factorisations and to Dijkstra's algorithm.

Proposition 4.19. *Let $W \in \mathbb{R}^{m\times n}$ have linearly independent columns $w_1, \dots, w_n$, and let $W = QR$ be its QR factorisation, where Q has orthonormal columns $q_1, \dots, q_n$ and R is upper triangular with positive diagonal. Put $D_\varepsilon = \operatorname{diag}(1, \varepsilon, \dots, \varepsilon^{n-1})$, and let $WD_\varepsilon = U\Sigma V^T$ be a singular value decomposition over $\mathbb{R}\langle\varepsilon\rangle$, which exists by Theorem 4.2, in which U has orthonormal columns $u_1, \dots, u_n$, V is orthogonal, $\Sigma = \operatorname{diag}(\sigma_1, \dots, \sigma_n)$ with $\sigma_1 \geq \cdots \geq \sigma_n > 0$, and the signs are chosen so that $u_k^T w_k > 0$. Then $\sigma_k = \varepsilon^{k-1} r_{kk}\big(1 + O(\varepsilon^2)\big)$, the matrices U and $U^T W = \Sigma V^T D_\varepsilon^{-1}$ are finite, and*

$$\operatorname{st}(U) = Q, \qquad \operatorname{st}\big(\Sigma V^T D_\varepsilon^{-1}\big) = R.$$

In particular, as a real parameter ε tends to 0^+, the left singular vectors of WD_ε, with these signs, tend to the columns of Q.

Proof. For $1 \leq k \leq n$, let Λ^k denote the k-th compound matrix, whose columns are indexed by the subsets $J \subseteq \{1, \dots, n\}$ with k elements. Since Λ^k is multiplicative, $\Lambda^k(WD_\varepsilon) = \Lambda^k(U)\,\Lambda^k(\Sigma)\,\Lambda^k(V)^T$ is a singular value decomposition, so the largest singular value of $\Lambda^k(WD_\varepsilon)$ is $s_k = \sigma_1 \cdots \sigma_k$, with left singular vector $u_1 \wedge \cdots \wedge u_k$. The column indexed by J is $\varepsilon^{e_J} w_J$, where $w_J = \bigwedge_{j\in J} w_j$ and $e_J = \sum_{j\in J}(j-1)$. The least exponent, $e_k = 0 + 1 + \cdots + (k-1)$, occurs only for $J = [k] = \{1, \dots, k\}$, every other J has $e_J \geq e_k + 1$, and $\|w_{[k]}\| = r_{11} \cdots r_{kk}$, the volume spanned by $w_1, \dots, w_k$. Bounding s_k below by the norm of the column $[k]$ and above by the Frobenius norm gives $s_k^2 = \varepsilon^{2e_k}(r_{11} \cdots r_{kk})^2\big(1 + O(\varepsilon^2)\big)$, and $\sigma_k = s_k / s_{k-1}$ gives the formula for σ_k. If x is a unit left singular vector for s_k, then

$$s_k^2 = \big\|\Lambda^k(WD_\varepsilon)^T x\big\|^2 = \sum_J \varepsilon^{2e_J}(x \cdot w_J)^2 = \varepsilon^{2e_k}(x \cdot w_{[k]})^2 + O(\varepsilon^{2e_k+2}),$$

so $(x \cdot \hat{w})^2 = 1 + O(\varepsilon^2)$ for the unit vector $\hat{w} = w_{[k]}/\|w_{[k]}\| = q_1 \wedge \cdots \wedge q_k$, and $\operatorname{st}(x) = \pm\hat{w}$. Taking $x = u_1 \wedge \cdots \wedge u_k$ and using that st is a ring homomorphism, $\operatorname{st}(u_1) \wedge \cdots \wedge \operatorname{st}(u_k) = \pm q_1 \wedge \cdots \wedge q_k$, so the orthonormal vectors $\operatorname{st}(u_1), \dots, \operatorname{st}(u_k)$ span the same subspace as $q_1, \dots, q_k$. By induction on k, $\operatorname{st}(u_k)$ is a unit vector in this subspace orthogonal to $q_1, \dots, q_{k-1}$, so $\operatorname{st}(u_k) = \pm q_k$; the sign is $+$, because $u_k^T w_k > 0$ and $q_k^T w_k = r_{kk} > 0$. Hence

$\mathrm{st}(U) = Q$ and $\mathrm{st}(U^T W) = Q^T W = R$, and $U^T W = \Sigma V^T D_\varepsilon^{-1}$ because $U^T W D_\varepsilon = \Sigma V^T$. Finally, the singular values have distinct orders, so for small real $\varepsilon > 0$ they are distinct and the decomposition with these signs is unique; the identification of $\mathbb{R}\langle\varepsilon\rangle$ with germs of semialgebraic functions (Section 4.4) gives the last statement. □

The proposition is elementary; its proof uses minors, as does that of Theorem 4.11. Together with that theorem and Propositions 1.1 and 4.18, it shows that the factorisations of this chapter, and Dijkstra's algorithm, are reached from the Hermitian eigendecomposition by exact encodings and limits, one step at a time; the steps weight different things, rows or columns, and do not combine into a single limit. The relation is structural, not computational: in practice the singular value decomposition is computed by algorithms built from QR factorisations, not the other way round.

Chapter 5

Doubly-Signed Quivers and the Structure of Real $*$-Algebras

Chapter 3 stored the elements of a $*$-algebra as matrices indexed by positions c_k, where c runs over orbits, each with its self-adjoint orbit idempotent 1_c, and k is a counter, with entries in the corners between the orbit idempotents. This chapter explains where the orbits and the corners come from, that is, what (i, j) indexing means over a finite-dimensional real $*$-algebra. Its main result, Theorem 5.16, which is part (1) of Theorem A, is an analogue of Gabriel's theorem for real $*$-algebras: every finite-dimensional real $*$-algebra A is $*$-isomorphic to

$$\mathrm{End}(V, \eta) \otimes_{\mathbb{R}^{\mathcal{O}}} \mathbb{R}Q/I,$$

a matrix algebra, with prescribed sizes and signs, over the quotient of the path $*$-algebra $\mathbb{R}Q$ of a doubly-signed quiver Q by an ideal of relations I. The first factor supplies the positions. Their orbits are the orbits $\mathcal{O}$ of the vertices of Q under the transpose and the conjugation, and the orbit c has $\dim V_c$ positions (Corollary 5.18). The second factor supplies the entries. The quiver, the sizes and the signatures of the forms are invariants of A, and all continuous parameters lie in I.

Two algebras with the quiver Q have distinct roles in this thesis. The quotient $\mathbb{R}Q/\mathbb{R}Q_{\geq 2}$, in which all products of arrows vanish, is the algebra whose Hermitian matrices encode the classical decompositions (Chapter 6). The path $*$-algebra $\mathbb{R}Q$ itself, in

which no relations hold between paths, is where computations take place: elements of $\mathbb{R}Q/I$ are lifted to $\mathbb{R}Q$ by a linear section of the quotient map, the arrows are inverted between their end points and the result is completed (Chapter 7), and the results are reduced modulo I at the end (Theorem 7.8). In this way the operations of Chapter 3 need to be implemented only once, in the completed localisation of a path $*$-algebra.

The chapter has two parts. Sections 5.1 to 5.4 introduce four kinds of quivers, each with more structure than the one before: quivers; *symmetric quivers*, which carry an involution that reverses arrows; *signed quivers*, which carry in addition a sign on each vertex and arrow that the involution fixes; and *doubly-signed quivers*, which carry a second involution, preserving arrows, with a second family of signs, which record whether the scalars at a vertex are real, complex or quaternionic. Sections 5.5 to 5.8 are about real $*$-algebras. Doubly-signed quivers classify the basic real $*$-algebras whose radical squares to zero (Theorem 5.11); the $*$-simple real $*$-algebras are matrix algebras, with signs, over ten algebras, one for each type of vertex (Lemma 5.12); the path $*$-algebra of a doubly-signed quiver is defined explicitly, by paths with coefficient blocks of size at most two (Definition 5.13), the form used in computations; and the structure theorem describes every finite-dimensional real $*$-algebra.

5.1 Quivers

Quivers, path algebras and representations were introduced in Section 2.6. This section recalls what a quiver records about an algebra, and how its representation type, which indicates whether canonical forms can exist, is read off from it.

What a quiver records. A finite-dimensional algebra A has two layers of discrete data. The first is its semisimple quotient $A/\operatorname{rad}(A)$, a product of matrix algebras over division algebras (Theorem 2.8). The second is the bimodule $\operatorname{rad}(A)/\operatorname{rad}(A)^2$ over $A/\operatorname{rad}(A)$, which is a direct sum of simple bimodules with multiplicities. Together they determine $A/\operatorname{rad}(A)^2$, and over an algebraically closed field they are recorded, up to Morita equivalence, by a quiver: a vertex for each simple factor and, from i to j, as many arrows as

the multiplicity of the simple bimodule between the factors i and j. Over such a field, the basic algebras whose radical squares to zero are exactly the algebras kQ/R_Q^2 (Section 2.6), so they are classified by discrete data. The next layer is not discrete: an algebra kQ/I with $R_Q^3 \subseteq I \subseteq R_Q^2$ is determined by the subspace I/R_Q^3 of the span of the paths of length two, and such subspaces vary in continuous families. By Gabriel's theorem (Section 2.6), relations are all that is needed beyond the quiver.

5.1.1 Representation Type and the Tits Form

Whether a decomposition can have a canonical form depends on the *representation type* of the quiver involved. The Tits form gives a simple criterion, which can be understood by counting degrees of freedom. In this subsection k is algebraically closed.

Fix a dimension vector $\mathbf{d} = (d_i)_{i \in Q_0}$. The representations of Q with dimension vector $\mathbf{d}$ form the affine space

$$\operatorname{Rep}(Q, \mathbf{d}) = \bigoplus_{\alpha \in Q_1} \operatorname{Hom}\big(k^{d_{s(\alpha)}}, k^{d_{t(\alpha)}}\big), \qquad \dim \operatorname{Rep}(Q, \mathbf{d}) = \sum_{\alpha : i \to j} d_i d_j,$$

and the isomorphism classes are the orbits of the group $\operatorname{GL}(\mathbf{d}) = \prod_i \operatorname{GL}(d_i)$, of dimension $\sum_i d_i^2$, acting by change of basis. The group dimension is the number of degrees of freedom available for choosing bases, and the dimension of $\operatorname{Rep}(Q, \mathbf{d})$ is the number of degrees of freedom that a representation has.

Definition 5.1 (Tits form)**.** The *Tits form* of Q is the quadratic form

$$q_Q(\mathbf{d}) = \sum_{i \in Q_0} d_i^2 - \sum_{\alpha : i \to j} d_i d_j = \dim \operatorname{GL}(\mathbf{d}) - \dim \operatorname{Rep}(Q, \mathbf{d}).$$

Proposition 5.2. *If $q_Q(\mathbf{d}) \leq 0$ for some non-zero dimension vector $\mathbf{d}$, then Q has infinitely many isomorphism classes of representations of dimension vector $\mathbf{d}$.*

Proof. The scalar matrices $\lambda I \in \operatorname{GL}(\mathbf{d})$ act trivially, so every orbit has dimension at most $\dim \operatorname{GL}(\mathbf{d}) - 1 < \dim \operatorname{Rep}(Q, \mathbf{d})$. A finite union of such orbits cannot fill $\operatorname{Rep}(Q, \mathbf{d})$. □

In numerical terms: if $q_Q(\mathbf{d}) < 0$, the group has too few degrees of freedom to bring

an arbitrary representation into a simple normal form such as a diagonal one; the isomorphism classes then depend on at least $1-q_Q(\mathbf{d})$ continuous parameters. What distinguishes the representation types is how this number grows with the dimension vector. The precise classification is as follows. A connected quiver is called

- *of finite type* if it has only finitely many indecomposable representations up to isomorphism. By Gabriel's classification of quivers of finite type [31], this holds if and only if its underlying graph is a Dynkin diagram of type A_n, D_n, E_6, E_7 or E_8, if and only if q_Q is positive definite; the indecomposable representations then correspond to the positive roots via their dimension vectors.

- *tame* if it is not of finite type, but for each dimension vector the indecomposable representations form finitely many one-parameter families. This holds if and only if the underlying graph is an extended Dynkin diagram $\tilde{A}_n$, $\tilde{D}_n$, $\tilde{E}_6$, $\tilde{E}_7$ or $\tilde{E}_8$, if and only if q_Q is positive semidefinite but not definite [24]. The loop quiver, whose representations are classified by the Jordan normal form, is an example.

- *wild* otherwise, in which case q_Q is not positive semidefinite. Classifying the representations of a wild quiver contains the problem of classifying pairs of matrices up to simultaneous similarity, and is considered intractable [25, 54].

The *cycle rank* of a quiver is $|Q_1|-|Q_0|+c$, where c is the number of connected components of its underlying graph; it is the number of independent cycles of that graph, counting loops and multiple arrows as cycles. The Dynkin diagrams are trees, and among the extended Dynkin diagrams only $\tilde{A}_n$, a single cycle, is not a tree, so a connected quiver of cycle rank at least 2 is wild. Nearly all quivers are wild. For example, the Kronecker quiver with two arrows $u \rightrightarrows v$ has $q_Q(\mathbf{d}) = (d_u - d_v)^2 \geq 0$ and is tame (its representations are matrix pencils, classified by the Kronecker canonical form), whereas with three arrows $q_Q(1,1) = -1$ and the quiver is wild. Likewise, the quiver with one vertex and two loops has $q_Q(d) = -d^2$ and is wild. The Tits form also has a homological interpretation, which we do not use: $q_Q(\mathbf{dim}\, V) = \dim_k \operatorname{End}(V) - \dim_k \operatorname{Ext}^1(V,V)$ for every representation V [3].

5.2 Symmetric Quivers

An involution of an algebra A permutes the simple components of $A/\operatorname{rad}(A)$, which are the vertices of its quiver, and, because it reverses products, it maps an arrow from i to j to an arrow from the image of j to the image of i. A quiver that describes a $*$-algebra must record this.

Definition 5.3 (Symmetric quiver)**.** A *symmetric quiver* is a quiver Q together with a map $\dagger : Q_0 \sqcup Q_1 \to Q_0 \sqcup Q_1$ that maps vertices to vertices and arrows to arrows, satisfies $(x^\dagger)^\dagger = x$, and reverses arrows: if $\alpha : i \to j$ then $\alpha^\dagger : j^\dagger \to i^\dagger$.

Symmetric quivers were introduced by Derksen and Weyman [21]. Their definition also requires every arrow from a vertex i to $i^\dagger$ to be fixed; we impose this only for signed quivers (Definition 5.4), whose signs allow a pair of exchanged arrows from i to $i^\dagger$ to be replaced by two fixed ones. Each vertex is either fixed by the involution or exchanged with another vertex, and the same holds for arrows. An arrow $\alpha : i \to j$ with $\alpha^\dagger = \alpha$ satisfies $j = i^\dagger$; it is a loop if i is fixed.

The path algebra of a symmetric quiver is a $*$-algebra, with the k-linear involution

$$e_v^* = e_{v^\dagger}, \qquad (\alpha_\ell \cdots \alpha_1)^* = \alpha_1^\dagger \cdots \alpha_\ell^\dagger.$$

This involution preserves R_Q^2, so kQ/R_Q^2 is a $*$-algebra as well. These algebras are easy to compute with, since the paths form a basis and the involution is path reversal composed with $\dagger$. After complexification, the path $*$-algebra of every doubly-signed quiver has the same form, except that each path carries a coefficient block of size at most two and the involution carries signs (Proposition 5.14).

For example, a loop x at a fixed vertex, with $x^\dagger = x$, gives the polynomial ring $k[x]$ with $x^* = x$. Two fixed vertices u, v with an arrow $a : u \to v$ and its partner $a^\dagger : v \to u$ give the four-dimensional algebra kQ/R_Q^2 of Example 2.26(3) with $a^* = a^\dagger$, the algebra of the singular value decomposition (Section 3.4).

5.3 Signed Quivers

A symmetric quiver does not determine the involution of the algebras it describes. By the Skolem–Noether theorem (Theorem 2.9), a k-linear involution of $M_n(k)$, for k algebraically closed of characteristic different from 2, has the form $X \mapsto GX^TG^{-1}$ with $G^T = \pm G$: it is the adjoint involution of a symmetric or of a skew-symmetric form, and it is said to be of *orthogonal* or of *symplectic type* [58]. Moreover, for a suitable choice of the arrows, the involution of the algebra maps each arrow fixed by † to plus or minus itself. Signs on the fixed vertices and arrows record these two kinds of information.

Definition 5.4 (Signed quiver)**.** A *signed quiver* is a symmetric quiver $(Q, \dagger)$ together with a sign $\varepsilon(x) \in \{+1, -1\}$ for every vertex and every arrow x that is fixed by the involution, $x^\dagger = x$. It is convenient to extend ε to all of $Q_0 \sqcup Q_1$ by setting $\varepsilon(x) = 0$ when $x^\dagger \neq x$; then $\varepsilon : Q_0 \sqcup Q_1 \to \{-1, 0, +1\}$ is characterised by

$$\varepsilon(x)^2 = \begin{cases} 1 & \text{if } x^\dagger = x, \\ 0 & \text{if } x^\dagger \neq x. \end{cases}$$

We also impose the following *normalisation*: every arrow from a vertex v to $v^\dagger$ is fixed by the involution.

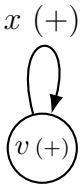
The normalisation removes a redundancy. If $\alpha : v \to v^\dagger$, then $\alpha^\dagger$ also goes from v to $v^\dagger$, and if $\alpha^\dagger \neq \alpha$ the two arrows span the same space as $\alpha + \alpha^\dagger$ and $\alpha - \alpha^\dagger$, which are fixed by the involution with signs $+1$ and -1. So a pair of exchanged arrows from v to $v^\dagger$ can always be replaced by two fixed arrows with opposite signs.

Signed quivers describe the $*$-algebras A with $\operatorname{rad}(A)^2 = 0$ and a k-linear involution over an algebraically closed field k of characteristic different from 2; over $\mathbb{R}$ they do not suffice (Section 5.4). The signs have the following meaning. A vertex v with $v^\dagger = v$ corresponds to a simple component of $A/\operatorname{rad}(A)$ that is stable under the involution, and the vertex sign $\varepsilon(v) = +1$ or -1 records whether the involution is of orthogonal or of symplectic type there. A pair of vertices $v \neq v^\dagger$ corresponds to two simple components exchanged by the involution, as in ${}^2\mathbb{R}$. A fixed arrow α goes from a vertex v to $v^\dagger$, and the

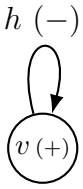
involution maps it to $\varepsilon(\alpha)\alpha$, so that α behaves like a symmetric $(+1)$ or skew-symmetric (-1) matrix. Arrows with $\alpha^\dagger \neq \alpha$ come in pairs.

Remark 5.5. The name is taken from Shmelkin [80], who records the same data in a folded form: a quiver with signs in $\{-1, 0, +1\}$ on its vertices and arrows, from which the symmetric quiver with its signs is obtained by adding a partner for each unsigned vertex, other than his *twins*, and for each unsigned arrow [80, Section 3]. Our signed quivers are the unfolded form, in which the involution is part of the data. His sign on a loop α at a fixed vertex v records whether the bilinear form $(x, y) \mapsto \langle V_\alpha x, y\rangle$ of a $*$-representation (Definition 5.6) is symmetric or skew-symmetric; this is $\varepsilon(v)\varepsilon(\alpha)$ in our notation, so the two conventions differ at loops on vertices of sign -1.

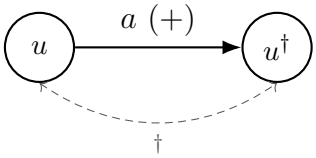
Figure 5.1 shows four signed quivers. In the diagrams, vertices are drawn as circles labelled with their name and, if fixed by the involution, their sign; dashed grey lines connect vertices or arrows exchanged by †; arrows fixed by † are labelled with their sign.



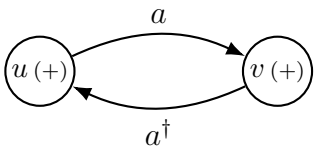
(a) Symmetric loop: symmetric matrices under orthogonal similarity.



(b) Skew loop: skew-symmetric matrices under orthogonal similarity.



(c) Exchanged vertices with a symmetric arrow: symmetric matrices under congruence.



(d) A pair of exchanged arrows: rectangular matrices under $W \mapsto RWP^T$ (the SVD).

Figure 5.1: Four signed quivers and the matrix problems they encode (Chapter 6). Signs of fixed vertices and arrows are shown in parentheses; the dashed line in (c) indicates that † exchanges u and $u^\dagger$. The problems are those of the real reading of the quivers, in which a fixed vertex of sign $+1$ carries a real inner product, as in the doubly-signed quivers of Figure 5.2.

5.3.1 $*$-Representations

For a signed quiver, the analogue of a representation is a representation together with compatible forms. Write $\tilde{\varepsilon}(v) = \varepsilon(v)$ if $v^\dagger = v$ and $\tilde{\varepsilon}(v) = 1$ otherwise, and choose signs κ_α for the arrows with $\kappa_\alpha = \varepsilon(\alpha)$ if $\alpha^\dagger = \alpha$ and

$$\kappa_\alpha \kappa_{\alpha^\dagger} = \tilde{\varepsilon}(s(\alpha))\,\tilde{\varepsilon}(t(\alpha)) \qquad \text{if } \alpha^\dagger \neq \alpha,$$

for instance $\kappa_\alpha = 1$ for one arrow of each exchanged pair. This is the first condition of (5.3) on the signs of the block model of Section 5.7, in which $g_v = \tilde{\varepsilon}(v)$.

Definition 5.6. A *$*$-representation* of a signed quiver $(Q, \dagger, \varepsilon)$ over k is a representation V of Q (Definition 2.27) together with a non-degenerate bilinear form $\langle \cdot, \cdot \rangle$ on $V = \bigoplus_v V_v$ such that

1. $\langle V_u, V_v \rangle = 0$ unless $v = u^\dagger$, and $\langle y, x \rangle = \langle x, y \rangle$ for $x \in V_u$, $y \in V_{u^\dagger}$ when $u \neq u^\dagger$;
2. for each vertex v with $v^\dagger = v$, the restriction of the form to V_v is symmetric if $\varepsilon(v) = 1$ and skew-symmetric if $\varepsilon(v) = -1$;
3. for each arrow α and all $x \in V_{s(\alpha)}$, $y \in V_{t(\alpha)^\dagger}$: $\langle V_\alpha x, y \rangle = \kappa_\alpha \langle x, V_{\alpha^\dagger} y \rangle$.

An isomorphism of $*$-representations is an isomorphism of representations that is an isometry.

Condition (3) says that $V_{\alpha^\dagger}$ is the adjoint of V_α, up to the sign κ_α. The condition on $\kappa_\alpha \kappa_{\alpha^\dagger}$ is forced: writing (3) for α and for $\alpha^\dagger$ and using the symmetry of the forms gives $\tilde{\varepsilon}(s(\alpha))\tilde{\varepsilon}(t(\alpha))\langle y, V_\alpha x \rangle = \kappa_\alpha \kappa_{\alpha^\dagger} \langle y, V_\alpha x \rangle$, so any other choice would force $V_\alpha = 0$, for example for an arrow from a vertex of sign $+1$ to one of sign -1. The choice of κ_α within an exchanged pair does not matter, since replacing $V_{\alpha^\dagger}$ by $-V_{\alpha^\dagger}$ changes it. For example, for the signed quiver of Figure 5.1(a), a $*$-representation is a vector space with a non-degenerate symmetric form and a self-adjoint endomorphism; over $\mathbb{R}$ and with a positive definite form, its classification is the spectral theorem. For Figure 5.1(d), it consists of two inner product spaces and a linear map between them together with its

adjoint; its classification is the singular value decomposition. Chapter 6 proves a general version of these statements in the language of matrices over $*$-algebras. For the doubly-signed quivers of Section 5.4 one additionally requires a real structure, compatible with the conjugation, on the complexified representation.

The count of degrees of freedom of Section 5.1.1 applies to $*$-representations as well, with $\mathrm{GL}(\mathbf{d})$ replaced by the product G of the groups at the vertices (an orthogonal or a symplectic group at each fixed vertex, according to its sign, and a general linear group for each pair of exchanged vertices) and $\mathrm{Rep}(Q, \mathbf{d})$ replaced by the space of $*$-representations with given forms. Every orbit has dimension at most $\dim G$, so there are infinitely many isomorphism classes as soon as $\dim G$ is smaller than the dimension of that space. An analogous trichotomy, governed by root systems of signed quivers, is developed in [21, 80].

5.4 Doubly-Signed Quivers

Over $\mathbb{R}$, a simple algebra is $M_n(D)$ with $D \in \{\mathbb{R}, \mathbb{C}, \mathbb{H}\}$. To describe a real algebra B by a quiver, we pass to its complexification $B_{\mathbb{C}} = B \otimes_{\mathbb{R}} \mathbb{C}$, on which complex conjugation acts by $c(b \otimes z) = b \otimes \bar{z}$. This conjugation is a conjugate-linear *automorphism*, not an anti-automorphism, so it maps an arrow from i to j to an arrow from the conjugate of i to the conjugate of j. Each simple component of B gives rise to simple components of $B_{\mathbb{C}}$ as follows: $M_n(\mathbb{R})$ gives $M_n(\mathbb{C})$, fixed by c; $M_n(\mathbb{H})$ gives $M_{2n}(\mathbb{C})$, again fixed by c; and $M_n(\mathbb{C})$ gives $M_n(\mathbb{C}) \times M_n(\mathbb{C})$, two components exchanged by c. On a component fixed by c, the Skolem–Noether theorem shows that $c(X) = G\bar{X}G^{-1}$ with $G\bar{G} = \pm I$, the sign being $+1$ for real and -1 for quaternionic type. A doubly-signed quiver records this conjugation and its sign, alongside the involution and the signs of a signed quiver.

Definition 5.7 (Doubly-signed quiver)**.** A *doubly-signed quiver* is a signed quiver Q, whose involution $x \mapsto x^{\top}$ we call the *transpose* and whose sign function we denote by $\varepsilon_{\mathrm{form}}$, together with

- a map $\overline{(\cdot)} : Q_0 \sqcup Q_1 \to Q_0 \sqcup Q_1$, the *conjugation*, that maps vertices to vertices and arrows to arrows, satisfies $\bar{\bar{x}} = x$, and preserves arrows: if $\alpha : i \to j$ then $\bar{\alpha} : \bar{i} \to \bar{j}$;

- a sign $\varepsilon_{\mathrm{div}}(v) \in \{+1, -1\}$ for every vertex v with $\bar{v} = v$, extended by $\varepsilon_{\mathrm{div}}(v) = 0$ when $\bar{v} \neq v$;

subject to the following rules:

1. the transpose and the conjugation commute; their composite is written $(\cdot)^H = \overline{(\cdot)}^{\top}$ and called the *conjugate transpose*;

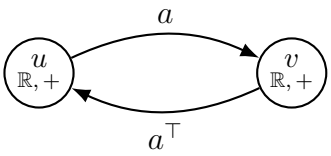
2. $\varepsilon_{\mathrm{div}}(v^{\top}) = \varepsilon_{\mathrm{div}}(v)$ for every vertex v;

3. $\varepsilon_{\mathrm{form}}(\bar{x}) = \varepsilon_{\mathrm{form}}(x)$ for every vertex or arrow x;

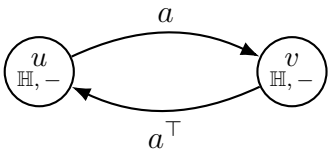
4. if both end points of an arrow $\alpha : i \to j$ are fixed by the conjugation, then $\bar{\alpha} = \alpha$ when $\varepsilon_{\mathrm{div}}(i) = \varepsilon_{\mathrm{div}}(j)$, and $\bar{\alpha} \neq \alpha$ otherwise;

5. every arrow α from a vertex v to v^H is fixed by the conjugate transpose, that is, $\bar{\alpha} = \alpha^{\top}$.

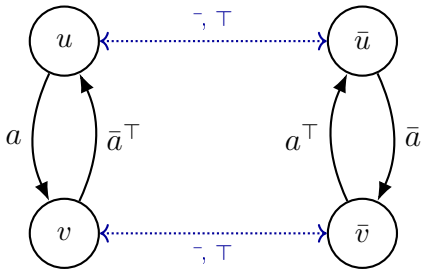
Thus a doubly-signed quiver carries two involutions: the transpose, which reverses arrows, and the conjugation, which preserves them and plays the role of complex conjugation. It carries two sign functions: $\varepsilon_{\mathrm{div}}$ on the vertices fixed by the conjugation, which distinguishes $\mathbb{R}$ from $\mathbb{H}$, and $\varepsilon_{\mathrm{form}}$ on the vertices and arrows fixed by the transpose, which distinguishes symmetric from skew-symmetric behaviour. The scalars at a vertex v are real if $\varepsilon_{\mathrm{div}}(v) = +1$, complex if $\varepsilon_{\mathrm{div}}(v) = 0$ (in which case v and $\bar{v}$ together form one vertex, with complex scalars, of the real algebra), and quaternionic if $\varepsilon_{\mathrm{div}}(v) = -1$. Rules (1) to (3) express the facts that conjugation and transposition commute, that transposition does not change the type of division algebra, and that conjugation does not change the symmetry type. Rule (4) reflects how conjugation acts on the arrows between two vertices fixed by it. Between two vertices with the same $\varepsilon_{\mathrm{div}}$ it acts as a real structure on the space of arrows, so a basis of fixed arrows can be chosen, and we always choose one; between a real and a quaternionic vertex its square is -1, so it fixes no arrow, and the arrows come in exchanged pairs. Rule (5) is a normalisation of the same kind: the conjugate transpose acts on the arrows from v to v^H as a real structure, so these arrows can be chosen fixed.



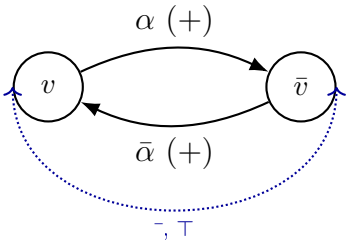
(a) Real SVD. Both vertices are fixed by the transpose and by conjugation; a and $a^\top$ are exchanged by the transpose and fixed by conjugation.



(b) Quaternionic SVD. As in (a), but both vertices are quaternionic with $\varepsilon_{\text{div}} = \varepsilon_{\text{form}} = -1$.



(c) Complex SVD. Each complex vertex appears as a pair exchanged both by conjugation and by the transpose (unitary type); the arrows form two 2-cycles exchanged by conjugation.



(d) Takagi factorisation. A unitary-type complex vertex with an arrow from v to $\bar{v} = v^\top$ that is fixed by the transpose with sign $+1$; conjugation exchanges α and $\bar{\alpha}$.

Figure 5.2: Doubly-signed quivers for four classical decompositions, drawn after complexification. Vertex labels give the division algebra ($\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$) and $\varepsilon_{\text{form}}$ for vertices fixed by the respective involutions; dotted blue lines join vertices exchanged by conjugation $^-$ and by the transpose $\top$. Chapter 6 derives the corresponding matrix problems.

Both facts are part of Theorem 5.11(3). When v is fixed by the conjugation, $v^H = v^\top$, and rule (5) follows from the normalisation of Definition 5.4 and rule (4).

The vertex types of a doubly-signed quiver correspond exactly to the ten families of classical groups over $\mathbb{R}$ (Table 5.1); this is the classification of involutions on simple real algebras [47, 58, 90]. For the doubly-signed quiver of a real $*$-algebra (Theorem 5.11), the unitary group at each vertex is the group in the table. Note the pattern: when $\varepsilon_{\text{div}} = \varepsilon_{\text{form}}$ the group is compact ($\mathrm{O}(n)$, $\mathrm{Sp}(n)$), whereas when $\varepsilon_{\text{div}} = -\varepsilon_{\text{form}}$ it is not, for $n \geq 2$ (for $n = 1$, $\mathrm{O}^*(2)$ is a circle group).

In labels we write the type of a vertex as a pair $(D, \varepsilon_{\text{form}})$, with the division algebra $D = \mathbb{R}, \mathbb{C}, \mathbb{H}$ in place of $\varepsilon_{\text{div}} = +1, 0, -1$, and with $\varepsilon_{\text{form}} = 0$ for the exchange types, in which $v^\top \notin \{v, \bar{v}\}$. The type with $v^\top = \bar{v}$ is written $(\mathbb{C}, \mathrm{u})$, for unitary, and in diagrams the signs ± 1 are abbreviated to $\pm$. For example, $(\mathbb{R}, +1)$ is the type of $\mathbb{R}$ with the trivial

Table 5.1: The vertex types of a doubly-signed quiver, the corresponding $*$-simple real $*$-algebras of minimal size, and the unitary groups U_n of the matrix $*$-algebras over them. Indefinite unitary groups such as $\mathrm{O}(p,q)$, $\mathrm{U}(p,q)$ and $\mathrm{Sp}(p,q)$ arise from the same vertex types when the matrix algebra carries an indefinite form (Remark 6.2).

Vertex data	**$*$-simple $*$-algebra (minimal size)**	**Unitary group of M_n**
$\varepsilon_{\mathrm{div}} = +1$, $\varepsilon_{\mathrm{form}} = +1$	$\mathbb{R}$, trivial involution	$\mathrm{O}(n)$
$\varepsilon_{\mathrm{div}} = +1$, $\varepsilon_{\mathrm{form}} = -1$	$M_2(\mathbb{R})$, symplectic involution	$\mathrm{Sp}(2n,\mathbb{R})$
$\varepsilon_{\mathrm{div}} = -1$, $\varepsilon_{\mathrm{form}} = -1$	$\mathbb{H}$, quaternion conjugation	$\mathrm{Sp}(n)$
$\varepsilon_{\mathrm{div}} = -1$, $\varepsilon_{\mathrm{form}} = +1$	$\mathbb{H}$, involution of orthogonal type	$\mathrm{O}^*(2n)$
$\varepsilon_{\mathrm{div}} = 0$, $\varepsilon_{\mathrm{form}} = +1$	$\mathbb{C}$, trivial involution	$\mathrm{O}(n,\mathbb{C})$
$\varepsilon_{\mathrm{div}} = 0$, $\varepsilon_{\mathrm{form}} = -1$	$M_2(\mathbb{C})$, symplectic involution	$\mathrm{Sp}(2n,\mathbb{C})$
$\varepsilon_{\mathrm{div}} = 0$, $v^\top = \bar{v}$	$\mathbb{C}$, complex conjugation	$\mathrm{U}(n)$
$\varepsilon_{\mathrm{div}} = +1$, $v^\top \neq v$	$\mathbb{R} \times \mathbb{R}^{\mathrm{op}}$, exchange involution	$\mathrm{GL}_n(\mathbb{R})$
$\varepsilon_{\mathrm{div}} = -1$, $v^\top \neq v$	$\mathbb{H} \times \mathbb{H}^{\mathrm{op}}$, exchange involution	$\mathrm{GL}_n(\mathbb{H})$
$\varepsilon_{\mathrm{div}} = 0$, $v^\top \notin \{v, \bar{v}\}$	$\mathbb{C} \times \mathbb{C}^{\mathrm{op}}$, exchange involution	$\mathrm{GL}_n(\mathbb{C})$

involution, and $(\mathbb{R}, 0)$ that of $\mathbb{R} \times \mathbb{R}^{\mathrm{op}}$.

5.5 Real $*$-Algebras with Radical Square Zero

We now turn to real $*$-algebras, beginning with those whose radical squares to zero. We show that each of them has a doubly-signed quiver, that the signs carry the information described in Section 5.4, that every doubly-signed quiver arises in this way, and that a basic such algebra is determined by its doubly-signed quiver. Two lemmas are needed first: a $*$-stable version of the Wedderburn–Malcev theorem, which has an elementary proof when the radical squares to zero, and a lemma on lifting unitaries.

Lemma 5.8. *Let A be a finite-dimensional $*$-algebra over a field of characteristic 0 with $R = \mathrm{rad}(A)$ and $R^2 = 0$. Then A has a Wedderburn–Malcev decomposition $A = S \oplus R$ with $S^* = S$.*

Proof. By Lemma 2.11, $R^* = R$, so the involution descends to an involution $x \mapsto x^\natural$ of $\bar{A} = A/R$. By Theorem 2.12 there is an algebra homomorphism $\varphi : \bar{A} \to A$ that is a section of the quotient map. Define $\psi(x) = \varphi(x^\natural)^*$. Then ψ is again a homomorphic section: $\psi(xy) = \varphi(y^\natural x^\natural)^* = (\varphi(y^\natural)\varphi(x^\natural))^* = \psi(x)\psi(y)$, and $\psi(x) \equiv (x^\natural)^\natural = x$ modulo R.

Let $D = \psi - \varphi$, which takes values in R. Since $R^2 = 0$,

$$D(xy) = \psi(x)\psi(y) - \varphi(x)\varphi(y) = D(x)\varphi(y) + \varphi(x)D(y).$$

Put $j = \frac{1}{2}(\varphi + \psi) = \varphi + \frac{1}{2}D$. Then

$$j(x)j(y) = \varphi(xy) + \tfrac{1}{2}\big(D(x)\varphi(y) + \varphi(x)D(y)\big) + \tfrac{1}{4}D(x)D(y) = \varphi(xy) + \tfrac{1}{2}D(xy) = j(xy),$$

so j is a homomorphic section, and $S = j(\bar{A})$ is a subalgebra complementary to R. Finally $j(x)^* = \frac{1}{2}(\varphi(x)^* + \varphi(x^\natural)) = j(x^\natural)$, because $\psi(x^\natural) = \varphi(x)^*$; hence $S^* = S$. □

If N is the radical of a finite-dimensional real $*$-algebra and $h \in 1 + N$ is self-adjoint, we put

$$h^{-1/2} = \sum_{j \geq 0} \binom{-1/2}{j} (h-1)^j.$$

The sum is finite because N is nilpotent, $h^{-1/2}$ is a self-adjoint polynomial in h, and $h^{-1/2}hh^{-1/2} = 1$, since this is an identity of formal power series in $h - 1$.

Lemma 5.9. *Let A be a finite-dimensional real $*$-algebra with radical N.*

1. *Every unitary of A/N is the image of a unitary of A.*
2. *If $f, f' \in A$ are self-adjoint idempotents whose images in A/N are conjugate by a unitary of A/N, then $f' = ufu^*$ for a unitary $u \in A$. In particular $fAf \cong f'Af'$ as $*$-algebras.*

Proof. (1) If $v \in A$ lifts the unitary $\bar{v}$, then $h = v^*v \in 1+N$ is self-adjoint, and $u = vh^{-1/2}$ lifts $\bar{v}$ and satisfies $u^*u = h^{-1/2}hh^{-1/2} = 1$.

(2) By (1), the unitary of A/N that conjugates the image of f to that of f' lifts to a unitary $u \in A$, and $g = ufu^*$ is a self-adjoint idempotent with $g \equiv f'$ modulo N. Then $z = f'g + (1 - f')(1 - g) \equiv 1$ is invertible (Proposition 3.2), $f'z = f'g = zg$, and $z^*z = gf'g + (1-g)(1-f')(1-g)$ commutes with g. Hence $w = z(z^*z)^{-1/2}$ is unitary, and since $(z^*z)^{-1/2}$ commutes with g, we get $wg = zg(z^*z)^{-1/2} = f'z(z^*z)^{-1/2} = f'w$, that

is, $wgw^* = f'$. So $f' = (wu)f(wu)^*$, and $x \mapsto (wu)x(wu)^*$ restricts to a $*$-isomorphism $fAf \to f'Af'$. □

Let B be a finite-dimensional real $*$-algebra with $\mathrm{rad}(B)^2 = 0$, and let $B_{\mathbb{C}} = B \otimes_{\mathbb{R}} \mathbb{C}$ be its complexification. On $B_{\mathbb{C}}$ there are two commuting structures: the $\mathbb{C}$-linear extension of the involution of B, which we continue to write as $x \mapsto x^*$ and which is an anti-automorphism, and the conjugation $c = \mathrm{id} \otimes \overline{(\cdot)}$, which is a conjugate-linear automorphism. By Lemma 5.8 applied to B, there is a $*$-stable Wedderburn–Malcev decomposition $B = S_B \oplus \mathrm{rad}(B)$, and then $B_{\mathbb{C}} = S \oplus R$ with $S = S_B \otimes \mathbb{C}$ and $R = \mathrm{rad}(B) \otimes \mathbb{C} = \mathrm{rad}(B_{\mathbb{C}})$, where S is stable under both $*$ and c. Write $S = \prod_i S_i$ with $S_i \cong M_{n_i}(\mathbb{C})$ simple, and let 1_i be the unit of S_i. Then $R = \bigoplus_{i,j} 1_j R 1_i$, and $1_j R 1_i$ is an S_j–S_i-bimodule, hence isomorphic to a direct sum of m_{ji} copies of the simple bimodule $M_{n_j \times n_i}(\mathbb{C})$.

Definition 5.10. The *quiver of* $B_{\mathbb{C}}$ has a vertex for each simple component S_i and m_{ji} arrows from i to j.

Theorem 5.11 (Doubly-signed quivers of real $*$-algebras)**.** *Let B be a finite-dimensional real $*$-algebra with $\mathrm{rad}(B)^2 = 0$, and let Q be the quiver of $B_{\mathbb{C}}$.*

1. *The involution $*$ of $B_{\mathbb{C}}$ induces an involution $(\cdot)^\top$ of Q that reverses arrows, and the conjugation c induces an involution $\overline{(\cdot)}$ of Q that preserves arrows; these commute.*

2. *For a vertex i with $i^\top = i$, the restriction of $*$ to $S_i \cong M_{n_i}(\mathbb{C})$ is $X \mapsto GX^TG^{-1}$ with $G^T = \varepsilon_{\mathrm{form}}(i)G$ and $\varepsilon_{\mathrm{form}}(i) \in \{\pm 1\}$. For a vertex with $\bar{i} = i$, the restriction of c is $X \mapsto C\bar{X}C^{-1}$ for some C with $C\bar{C} = \varepsilon_{\mathrm{div}}(i)I$ and $\varepsilon_{\mathrm{div}}(i) \in \{\pm 1\}$; the simple component of $B/\mathrm{rad}(B)$ underlying S_i is a matrix algebra over $\mathbb{R}$ if $\varepsilon_{\mathrm{div}}(i) = 1$ and over $\mathbb{H}$ if $\varepsilon_{\mathrm{div}}(i) = -1$. Vertices with $\bar{i} \neq i$ come from simple components of $B/\mathrm{rad}(B)$ that are matrix algebras over $\mathbb{C}$.*

3. *On the multiplicity space of the arrows from i to $i^\top$, the involution $*$ induces a linear involution, normalised as in Step 3 of the proof, whose ± 1-eigenspaces define arrows fixed by the transpose with signs $\varepsilon_{\mathrm{form}}(\alpha) = \pm 1$. On the multiplicity space of the arrows between two vertices i, j fixed by conjugation, c induces a conjugate-linear*

map whose square is $\varepsilon_{\mathrm{div}}(i)\varepsilon_{\mathrm{div}}(j)$. *On the multiplicity space of the arrows from* i *to* i^H, *the composite* $c \circ *$ *induces a conjugate-linear map whose square is* 1, *so these arrows can be chosen fixed by the conjugate transpose.*

4. *With these data,* Q *is a doubly-signed quiver* $Q(B)$, *which does not depend, up to isomorphism, on the choices made in its construction. It is unchanged when* B *is replaced by a matrix algebra* $M_n(B)$ *with the conjugate-transpose involution, or by a corner* fBf, *where* f *is a self-adjoint idempotent with* $BfB = B$; *for example, by a basic* $*$*-algebra of* B *(Definition 2.14).*

Conversely, every doubly-signed quiver Q *arises in this way, from the basic* $*$*-algebra* B_Q *of Definition 5.13, whose semisimple quotient is a product of algebras from Table 5.1, and* B *is determined by* $Q(B)$ *up to the replacements in (4), in the following sense: two real* $*$*-algebras with radical square zero and isomorphic doubly-signed quivers have* $*$*-isomorphic basic* $*$*-algebras. In particular, two basic real* $*$*-algebras with radical square zero and isomorphic doubly-signed quivers are* $*$*-isomorphic. If* A *is a complex algebra with a* $\mathbb{C}$*-linear involution, the same construction without the conjugation* c *yields a signed quiver.*

Proof. Throughout, $N = \mathrm{rad}(B)$ and $R = \mathrm{rad}(B_{\mathbb{C}})$. Since $R^2 = 0$, the product of an element of $B_{\mathbb{C}}$ with an element of R depends only on the image of the former in $B_{\mathbb{C}}/R$. So R is a bimodule over $B_{\mathbb{C}}/R$, and the quotient map identifies S with $B_{\mathbb{C}}/R$, compatibly with $*$ and c. The constructions below use S only through this identification, so they do not depend on the choice of S_B.

(1) The involution $*$ is an anti-automorphism of $B_{\mathbb{C}}$ that preserves S and R. So it permutes the simple components S_i, defining $i \mapsto i^\top$, and maps $R_{ji} = 1_j R 1_i$ onto $R_{i^\top j^\top}$, reversing arrows. Likewise c permutes the S_i, defining $i \mapsto \bar{i}$, and maps R_{ji} onto $R_{\bar{j}\bar{i}}$, preserving arrows. These maps of the vertices are involutions, and they commute because $*$ and c commute. The corresponding involutions of the arrows are constructed in Step 4 below.

(2) If $i^\top = i$, then $*$ restricts to a $\mathbb{C}$-linear anti-automorphism of $S_i \cong M_n(\mathbb{C})$ of order two. Composing with the matrix transpose gives an automorphism, which is inner by the

Skolem–Noether theorem (Theorem 2.9); so $X^* = GX^TG^{-1}$ for some invertible G. Since $*$ has order two, $G(G^T)^{-1}$ is central, $G^T = \lambda G$, and $G = (G^T)^T = \lambda^2 G$ forces $\lambda = \pm 1$. The case of c is the same with complex conjugation in place of the transpose: $c(X) = C\bar{X}C^{-1}$ with $C\bar{C} = \mu I$. Conjugating this identity gives $\bar{C}C = \bar{\mu}I$, while $\bar{C} = \mu C^{-1}$ gives $\bar{C}C = \mu I$; hence μ is real, and rescaling C by a positive real number gives $\mu = \pm 1$. The fixed algebra of c on S_i is $M_n(\mathbb{R})$ if $\mu = 1$ and $M_{n/2}(\mathbb{H})$ if $\mu = -1$ [58]. If $\bar{i} \neq i$, the fixed points of c on $S_i \times S_{\bar{i}}$ form a copy of $M_n(\mathbb{C})$.

Step 1: coordinates. Let F be a $*$-simple factor of S_B. By the proof of Lemma 5.12, whose existence part does not use the present theorem, $F \cong M_{r,\eta}(K)$ for some $r \geq 1$, some signs η and one of the ten algebras K of Table 5.1. By the inspection of Table 5.2 in the proof of Proposition 5.14(3), which does not use the present theorem either, K is $*$-isomorphic to the algebra of fixed points of the map c of (5.2) in $\prod_v M_{d_v}(\mathbb{C})$, with the involution of (5.2), where v runs through the complex vertices of one row of that table. Hence $F \otimes_{\mathbb{R}} \mathbb{C} \cong M_r(K \otimes_{\mathbb{R}} \mathbb{C}) \cong \prod_v M_r\big(M_{d_v}(\mathbb{C})\big)$. This matches the simple components of $F \otimes_{\mathbb{R}} \mathbb{C}$ with the vertices v of the row, compatibly with the transpose and the conjugation. It also gives coordinates $S_i \cong M_{n_i}(\mathbb{C})$, with $n_i = rd_v$ for the vertex v matched with i, in which, for $X \in S_i$,

$$X^* = G_i X^T G_i^{-1} \in S_{i^\top}, \qquad c(X) = C_i \overline{X} C_i^{-1} \in S_{\bar{i}}.$$

Here $G_i = \eta \otimes G_v$ and $C_i = I_r \otimes C_v$ are Kronecker products, and η also denotes the diagonal matrix of the signs. Since $G_{v^\top} = G_v$, $C_{\bar{v}} = C_v$ and η is real, (5.1) gives

$$G_{i^\top} = G_i, \qquad C_{\bar{i}} = C_i, \qquad G_i^T = g_i G_i, \qquad C_i \overline{C_i} = h_i I, \qquad C_{i^\top} \overline{G_i}\, C_i^T = G_{\bar{i}},$$

where $g_i = g_v$ and $h_i = h_v$. In particular, the vertex data of (2) are those of the row: $\varepsilon_{\text{form}}(i) = g_i$ if $i^\top = i$, and $\varepsilon_{\text{div}}(i) = h_i$ if $\bar{i} = i$. The numbers g_i and h_i are constant on orbits, and $g_i = 1$, so that G_i is symmetric, if $i^\top \neq i$.

Step 2: multiplicity spaces. For vertices i and j, let $M_{ji} = M_{n_j \times n_i}(\mathbb{C})$, an S_j–S_i-bimodule through these coordinates. The algebra $S_j \otimes S_i^{\text{op}} \cong M_{n_j n_i}(\mathbb{C})$ is simple, M_{ji} is its

only simple module, and the endomorphisms of M_{ji} are the scalars. Hence the evaluation map $M_{ji} \otimes U_{ji} \to R_{ji}$, $Y \otimes u \mapsto u(Y)$, is an isomorphism, where $U_{ji} = \mathrm{Hom}_{S_j\text{-}S_i}(M_{ji}, R_{ji})$ is the multiplicity space; so $m_{ji} = \dim U_{ji}$. On M_{ji} we define the reference maps

$$\phi(Y) = G_i Y^T G_j^{-1} \in M_{i^\top j^\top}, \qquad \psi(Y) = C_j \overline{Y} C_i^{-1} \in M_{\overline{j}\overline{i}}.$$

Writing elements of S by their coordinates, we have $\phi(aYb) = b^*\phi(Y)a^*$ and $\psi(aYb) = c(a)\psi(Y)c(b)$ for $a \in S_j$ and $b \in S_i$. Hence, for $u \in U_{ji}$, the maps $\tau(u) = * \circ u \circ \phi^{-1}$ and $\gamma(u) = c \circ u \circ \psi^{-1}$ are again bimodule maps. They define a linear map $\tau : U_{ji} \to U_{i^\top j^\top}$ and a conjugate-linear map $\gamma : U_{ji} \to U_{\overline{j}\overline{i}}$ with

$$u(Y)^* = \tau(u)\big(\phi(Y)\big), \qquad c\big(u(Y)\big) = \gamma(u)\big(\psi(Y)\big);$$

that is, under the evaluation isomorphisms, $*$ is $\phi \otimes \tau$ and c is $\psi \otimes \gamma$. The identities of Step 1 give

$$\phi(\phi(Y)) = g_i g_j\, Y, \qquad \psi(\psi(Y)) = h_i h_j\, Y, \qquad \psi(\phi(Y)) = \phi(\psi(Y)),$$

as in the proof of Proposition 5.14(1) for a path of length one with signs 1. Since $*$ and c are commuting maps of order two, these identities give, for all $u \in U_{ji}$,

$$\tau(\tau(u)) = g_i g_j\, u, \qquad \gamma(\gamma(u)) = h_i h_j\, u, \qquad \gamma(\tau(u)) = \tau(\gamma(u)). \tag{R}$$

In particular, τ is a linear involution of $U_{i^\top i}$. If $\overline{i} = i$ and $\overline{j} = j$, then γ is a conjugate-linear map of U_{ji} with square $\varepsilon_{\mathrm{div}}(i)\varepsilon_{\mathrm{div}}(j)$. The map $\gamma \circ \tau$, which is induced by $c \circ *$, is a conjugate-linear map of U_{iH_i} with square 1. This proves (3), apart from the choice of bases, which we make in Step 4.

Step 3: the signs are invariants. The involution τ of $U_{i^\top i}$ does not depend on the coordinates, because the reference map ϕ on $M_{i^\top i}$ does not. If $i^\top = i$, then $M_{ii} = S_i$ as a bimodule, and ϕ is the involution of S_i. If $i^\top \neq i$, then G_i is symmetric. After the coordinates of $S_{i^\top}$ are changed by $X \mapsto G_i^{-1} X G_i$, the involution maps $X \in S_i$

to $X^T \in S_{i^\top}$, and ϕ becomes the transpose of $M_{i^\top i} = M_{n_i}(\mathbb{C})$. Any two systems of coordinates in which $*$ maps $X \in S_i$ to $X^T \in S_{i^\top}$ differ by $X \mapsto PXP^{-1}$ on S_i and $X \mapsto P^{-T}XP^T$ on $S_{i^\top}$. This changes $Y \in M_{i^\top i}$ into $P^{-T}YP^{-1}$, which commutes with the transpose. Hence the eigenspaces of τ on $U_{i^\top i}$, and their dimensions, depend only on B. (If n_i is even, as at the vertices of type $(\mathbb{H}, 0)$, one could also use coordinates in which $*$ maps $X \in S_i$ to $\hat{J}X^T\hat{J}^{-1}$ for a skew-symmetric $\hat{J}$, with the reference map $Y \mapsto \hat{J}Y^T\hat{J}^{-1}$. This would change the signs of all arrows from i to $i^\top$. Step 1 excludes such coordinates, because there $G_i^T = g_i G_i$ with $g_i = 1$; this is the convention of Table 5.2.)

Step 4: arrows. We call bases $\mathcal{B}_{ji}$ of the spaces U_{ji} *adapted* if they come with involutions $\alpha \mapsto \alpha^\top$ and $\alpha \mapsto \bar{\alpha}$ of $\mathcal{B} = \bigsqcup_{i,j} \mathcal{B}_{ji}$, mapping $\mathcal{B}_{ji}$ onto $\mathcal{B}_{i^\top j^\top}$ and onto $\mathcal{B}_{\bar{j}\bar{i}}$ respectively, with the following properties. The two involutions commute and satisfy the normalisation of Definition 5.4 and rules (4) and (5) of Definition 5.7, and there are signs $\kappa_\alpha, \mu_\alpha \in \{\pm 1\}$ with

$$\tau(\alpha) = \kappa_\alpha\, \alpha^\top, \qquad \gamma(\alpha) = \mu_\alpha\, \bar{\alpha} \qquad (\alpha \in \mathcal{B}),$$

and $\mu_\alpha = 1$ whenever $\bar{\alpha} = \alpha$. The arrows of Q from i to j are then the elements of $\mathcal{B}_{ji}$, with these involutions. If $\alpha^\top = \alpha$, its sign $\varepsilon_{\mathrm{form}}(\alpha) = \kappa_\alpha$ is the eigenvalue of τ at α. This is the construction of statements (1) and (3).

Adapted bases exist. The group $\Gamma = \{1, \top, {}^-, H\} \cong (\mathbb{Z}/2)^2$ acts on pairs of vertices by $(i,j)^\top = (j^\top, i^\top)$, $\overline{(i,j)} = (\bar{i}, \bar{j})$ and $(i,j)^H = (j^H, i^H)$, where (i,j) stands for the arrows from i to j. In each orbit of Γ we choose a pair (i,j) and a basis of U_{ji} according to the stabiliser $\Gamma_{(i,j)}$ of the pair, and we transport the basis to the other pairs of the orbit by τ and γ:

(a) $\Gamma_{(i,j)} = \{1\}$. Take any basis of U_{ji}. For each of its elements α, put $\alpha^\top = \tau(\alpha)$, $\bar{\alpha} = \gamma(\alpha)$ and $\bar{\alpha}^\top = \overline{\alpha^\top} = \tau(\gamma(\alpha)) = \gamma(\tau(\alpha))$. The two involutions act on these four elements as Γ acts on the four pairs of the orbit.

(b) $\Gamma_{(i,j)} = \{1, \top\}$, that is, $j = i^\top$ and $\bar{i} \neq i$. Take a basis of eigenvectors $\alpha = \alpha^\top$ of the involution τ of $U_{i^\top i}$, and put $\bar{\alpha} = \gamma(\alpha)$. By (R), $\tau(\bar{\alpha}) = \kappa_\alpha \bar{\alpha}$, so $\bar{\alpha}^\top = \bar{\alpha}$ and

$\varepsilon_{\mathrm{form}}(\bar{\alpha}) = \varepsilon_{\mathrm{form}}(\alpha)$.

(c) $\Gamma_{(i,j)} = \{1, {}^{-}\}$, that is, $\bar{i} = i$, $\bar{j} = j$ and $j \neq i^{\top}$. If $\varepsilon_{\mathrm{div}}(i) = \varepsilon_{\mathrm{div}}(j)$, then γ is a real structure on U_{ji}; take a basis of fixed vectors $\alpha = \bar{\alpha}$. Otherwise $\gamma^2 = -1$, and γ, together with the complex scalars, makes U_{ji} a right vector space over $\mathbb{H}$. A basis $\beta_1, \ldots, \beta_k$ over $\mathbb{H}$ gives the basis $\beta_1, \gamma(\beta_1), \ldots, \beta_k, \gamma(\beta_k)$ over $\mathbb{C}$, and we put $\bar{\beta}_l = \gamma(\beta_l)$. In both cases, put $\alpha^{\top} = \tau(\alpha)$ and $\overline{\alpha^{\top}} = \bar{\alpha}^{\top}$ for the elements α of the basis.

(d) $\Gamma_{(i,j)} = \{1, H\}$, that is, $j = i^H$ and $\bar{i} \neq i$. Then $\gamma \circ \tau$ is a real structure on $U_{i^H i}$; take a basis of fixed vectors α. By (R), $\tau(\alpha) = \gamma(\gamma(\tau(\alpha))) = \gamma(\alpha)$, and we put $\alpha^{\top} = \bar{\alpha} = \tau(\alpha)$, whose transpose and conjugate are both α.

(e) $\Gamma_{(i,j)} = \Gamma$, that is, $j = i^{\top}$ and $\bar{i} = i$. Then τ is an involution and γ is a real structure of $U_{i^{\top} i}$, because $h_i h_{i^{\top}} = 1$. They commute, so γ preserves the eigenspaces of τ, and each eigenspace has a basis of fixed vectors of γ. We take these vectors, with $\alpha^{\top} = \bar{\alpha} = \alpha$.

In each case, (R) shows that τ and γ map each element of the resulting bases to plus or minus its image under the corresponding involution, with $\mu_\alpha = 1$ when $\bar{\alpha} = \alpha$. The normalisation and rules (4) and (5) hold by construction. Indeed, the arrows from i to $i^{\top}$ lie over pairs of type (b) or (e); the arrows between two vertices fixed by the conjugation, over pairs of type (c) or (e); and the arrows from i to i^H, over pairs of type (d) or (e). In type (e), $\varepsilon_{\mathrm{div}}(i^{\top}) = \varepsilon_{\mathrm{div}}(i)$ by Step 1, and $i^H = i^{\top}$.

(4) The remaining rules of Definition 5.7 follow from the fact that $*$ commutes with c. Since $*$ maps S_i anti-isomorphically onto $S_{i^{\top}}$ and commutes with c, the component S_i is c-stable if and only if $S_{i^{\top}}$ is, with the same type, so $\varepsilon_{\mathrm{div}}(i^{\top}) = \varepsilon_{\mathrm{div}}(i)$. Similarly $\varepsilon_{\mathrm{form}}(\bar{i}) = \varepsilon_{\mathrm{form}}(i)$ for the vertices. For an arrow with $\alpha^{\top} = \alpha$, also $\bar{\alpha}^{\top} = \bar{\alpha}$, and (R) gives $\kappa_\alpha \mu_\alpha \bar{\alpha} = \gamma(\tau(\alpha)) = \tau(\gamma(\alpha)) = \mu_\alpha \kappa_{\bar{\alpha}} \bar{\alpha}$, so $\varepsilon_{\mathrm{form}}(\bar{\alpha}) = \varepsilon_{\mathrm{form}}(\alpha)$, for any adapted bases. So Q is a doubly-signed quiver.

Up to isomorphism, a doubly-signed quiver is determined by its vertices, with their involutions and signs, by the numbers of arrows from i to j, and by the numbers of arrows

from i to $i^\top$ with sign $+1$ and with sign -1. Indeed, by the normalisation and rules (3) to (5), the arrows over an orbit of Γ are determined up to isomorphism by these numbers for one pair (i, j) of the orbit. In the notation of the five cases above: in case (a), each arrow from i to j lies in a free orbit of the two involutions; in case (b), the arrows from i to $i^\top$ are fixed by the transpose, and the conjugation maps them to arrows with the same signs; in case (c), they are all fixed by the conjugation or come in exchanged pairs, according to whether $\varepsilon_{\mathrm{div}}(i) = \varepsilon_{\mathrm{div}}(j)$; in case (d), they are fixed by the conjugate transpose; and in case (e), they are fixed by both involutions. For $Q(B)$, the vertex data depend only on B by (2), the numbers of arrows are the integers m_{ji}, and the numbers of arrows from i to $i^\top$ of each sign are the dimensions of the eigenspaces of τ, which depend only on B by Step 3. So $Q(B)$ does not depend, up to isomorphism, on the choices of S_B, of the coordinates and of the adapted bases.

Replacing B by $M_n(B)$ replaces S_i by $M_n(S_i)$ and R_{ji} by $M_n(R_{ji})$. With the matrices $I_n \otimes G_i$ and $I_n \otimes C_i$, which are those of Step 1 for $M_n(B)$, the reference maps are the transpose and the entrywise conjugation of $M_n(\mathbb{C})$ tensored with ϕ and ψ. The involution and the conjugation of $M_n(R_{ji})$ are the same maps tensored with $*$ and c. So the spaces U_{ji} and the maps τ and γ do not change, and neither do the vertex data; hence $Q(M_n(B)) \cong Q(B)$.

Now let f be a self-adjoint idempotent with $BfB = B$. Its image in $B/N \cong S_B$ is a self-adjoint idempotent $f_0 \in S_B$, and $f = uf_0u^*$ for a unitary $u \in B$ by Lemma 5.9(2); so we may assume that $f \in S_B$. Write $f = \sum_i f_i$ with $f_i \in S_i$. Then $f_i^* = f_{i^\top}$ and $c(f_i) = f_{\bar{i}}$, and $f_i \neq 0$ for every i because $BfB = B$. The algebra fBf has the radical fNf, whose square is zero, and the simple components $f_iS_if_i$, with the same transpose and conjugation. Moreover $f_jR_{ji}f_i \cong f_jM_{ji}f_i \otimes U_{ji}$, where $f_jM_{ji}f_i$ is the simple $f_jS_jf_j$–$f_iS_if_i$-bimodule. Since $\phi(f_jYf_i) = f_{i^\top}\phi(Y)f_{j^\top}$ and $\psi(f_jYf_i) = f_{\bar{j}}\psi(Y)f_{\bar{i}}$, the restrictions of ϕ and ψ are reference maps for fBf with the same properties, and they induce the same maps τ and γ on the same spaces U_{ji}. At the arrows from i to $i^\top$, they are the reference maps of Step 3 for fBf. If $i^\top = i$, the restriction of ϕ is the involution of $f_iS_if_i$. If $i^\top \neq i$, the coordinates of Step 3 can be chosen so that f_i and $f_{i^\top} = f_i^T$ are

both the diagonal matrix whose first rank f_i entries are 1 and whose other entries are 0; then the restriction of the transpose is the transpose of the upper left blocks. Finally, the vertex data do not change. If $i^\top = i$, the symmetric or skew-symmetric form defined by G_i restricts to a form of the same type on the image of f_i, which is non-degenerate because $f_i^* = f_i$. If $\bar{i} = i$, the real or quaternionic structure $x \mapsto C_i\bar{x}$ preserves the image of f_i, because $c(f_i) = f_i$. Since $Q(B)$ is determined up to isomorphism by these data, $Q(fBf) \cong Q(B)$.

The converse. Let Q be a doubly-signed quiver, with admissible signs. By Proposition 5.14, B_Q is a real $*$-algebra with radical M_Q, $M_Q^2 = 0$ and $S_Q \cong \prod_o K_o$, so that it is basic by Lemma 5.12(3). Its complexification is $S_\mathbb{C} \oplus M_\mathbb{C}$ with the maps (5.2). These parts of the proof of Proposition 5.14 do not use the present theorem. The block coordinates are coordinates as in Step 1, with $r = 1$ and $\eta = 1$. The copies of $M_{d_j \times d_i}(\mathbb{C})$ in $M_\mathbb{C}$ that belong to the arrows $\alpha : i \to j$ show that these arrows form a basis of U_{ji}, and (5.2) gives $\tau(\alpha) = \kappa_\alpha \alpha^\top$ and $\gamma(\alpha) = \mu_\alpha \bar{\alpha}$. Since Q satisfies the rules of Definition 5.7 and the signs are admissible, this basis is adapted, and the doubly-signed quiver that it defines is Q. Hence $Q(B_Q) = Q$.

Uniqueness. Let B be basic. Every self-adjoint idempotent of $S_B \cong B/N$ is central. So, by Lemma 5.12(3), $r = 1$ in Step 1 for every $*$-simple factor of S_B, and we may take $\eta = 1$, since $M_{1,(-1)}(K) = M_{1,(1)}(K)$. Choose adapted bases, let $Q = Q(B)$ be the resulting doubly-signed quiver, and let $\kappa_\alpha, \mu_\alpha$ be the signs. By (R),

$$\tau(\tau(\alpha)) = \kappa_\alpha \kappa_{\alpha^\top} \alpha = g_i g_j \alpha, \qquad \gamma(\gamma(\alpha)) = \mu_\alpha \mu_{\bar{\alpha}} \alpha = h_i h_j \alpha,$$

and $\gamma(\tau(\alpha)) = \kappa_\alpha \mu_{\alpha^\top} \overline{\alpha^\top}$ equals $\tau(\gamma(\alpha)) = \mu_\alpha \kappa_{\bar{\alpha}} \bar{\alpha}^\top$, for $\alpha \in \mathcal{B}_{ji}$. This gives (5.3). Since moreover $\kappa_\alpha = \varepsilon_{\mathrm{form}}(\alpha)$ when $\alpha^\top = \alpha$ and $\mu_\alpha = 1$ when $\bar{\alpha} = \alpha$, the signs are admissible. Let B_Q be built with these signs. Define $\Psi : B_\mathbb{C} \to S_\mathbb{C} \oplus M_\mathbb{C}$ on S as the coordinate isomorphism $S \to S_\mathbb{C} = \prod_i M_{d_i}(\mathbb{C})$ of Step 1 (all vertices of a row of Table 5.2 have the same d_v, G_v and C_v), and on R as the linear map with $\alpha(Y) \mapsto Y\alpha$ for $\alpha \in \mathcal{B}_{ji}$ and $Y \in M_{ji}$. Then Ψ is an isomorphism of algebras, because it is an isomorphism of bimodules on R and $R^2 = M_\mathbb{C}^2 = 0$. It commutes with $*$ and c: on S by Step 1, and on R

because, for instance, $\alpha(Y)^* = \kappa_\alpha \alpha^\top\big(\phi(Y)\big)$ is mapped to $\kappa_\alpha G_i Y^T G_j^{-1} \alpha^\top = (Y\alpha)^*$. So Ψ restricts to a $*$-isomorphism from B, the algebra of fixed points of c in $B_\mathbb{C}$, onto B_Q, the algebra of fixed points of c in $S_\mathbb{C} \oplus M_\mathbb{C}$.

If B' is another basic real $*$-algebra with radical square zero and $Q(B') \cong Q$, then in the same way $B' \cong B_Q$ for some admissible signs, after transport along the isomorphism. It remains to show that B_Q does not depend on the admissible signs. Let (κ, μ) and (κ', μ') be admissible, and put $a_\alpha = \kappa_\alpha \kappa'_\alpha$ and $b_\alpha = \mu_\alpha \mu'_\alpha$. By (5.3) and the conditions on fixed arrows,

$$a_{\alpha^\top} = a_\alpha, \qquad b_{\bar\alpha} = b_\alpha, \qquad a_\alpha b_{\alpha^\top} = b_\alpha a_{\bar\alpha},$$

and $a_\alpha = 1$ if $\alpha^\top = \alpha$, $b_\alpha = 1$ if $\bar\alpha = \alpha$. In each orbit of arrows under the two involutions, choose an arrow α. Put $z_\alpha = \sqrt{-1}$ if $\bar\alpha = \alpha^\top \neq \alpha$ and $a_\alpha b_\alpha = -1$, and $z_\alpha = 1$ otherwise; then put $z_{\alpha^\top} = a_\alpha z_\alpha$, $z_{\bar\alpha} = b_\alpha \bar z_\alpha$ and $z_{\bar\alpha^\top} = a_{\bar\alpha} b_\alpha \bar z_\alpha$. A check of the five kinds of orbits, using the identities above, shows that these values are consistent and that $z_{\beta^\top} = a_\beta z_\beta$ and $z_{\bar\beta} = b_\beta \bar z_\beta$ for every arrow β. Consider the map of $S_\mathbb{C} \oplus M_\mathbb{C}$ that fixes $S_\mathbb{C}$ and multiplies the copy of $M_{d_t \times d_s}(\mathbb{C})$ that belongs to each arrow β by z_β. By (5.2), it carries the involution and the conjugation defined by (κ, μ) to those defined by (κ', μ'). It is an algebra automorphism, because $M_\mathbb{C}^2 = 0$, so it restricts to a $*$-isomorphism between the two algebras B_Q. Hence two basic real $*$-algebras with radical square zero and isomorphic doubly-signed quivers are $*$-isomorphic.

Every B has a basic $*$-algebra. Let $f \in S_B$ be the sum of the matrix units E_{11} of the factors $M_{r,\eta}(K)$ in Step 1. Then f is self-adjoint and $BfB = B$. Moreover $fBf/fNf \cong fS_Bf$ is a product of the algebras K, so fBf is basic by Lemma 5.12(3). By (4), $Q(fBf) \cong Q(B)$. So two real $*$-algebras with radical square zero and isomorphic doubly-signed quivers have $*$-isomorphic basic $*$-algebras.

Signed quivers. Let A be a finite-dimensional complex algebra with a $\mathbb{C}$-linear involution and $\mathrm{rad}(A)^2 = 0$. The same steps apply without c. Coordinates as in Step 1 come from (2) at the vertices with $i^\top = i$. For $i^\top \neq i$, they come from coordinates of $S_{i^\top}$ in which $*$ maps $X \in S_i$ to X^T. Then only τ, the first identity of (R), and cases (a) and (b) of Step 4 without the conditions on the conjugation are needed, and they yield a signed

quiver. □

5.6 ∗-Simple Real ∗-Algebras and Matrices with Signs

A ∗-algebra is ∗-*simple* if it is non-zero and has no ∗-stable two-sided ideals other than 0 and itself; a semisimple ∗-algebra is the product of its ∗-simple factors. For a doubly-signed quiver Q, let $\mathcal{O}(Q)$ be the set of orbits of the group generated by $(\cdot)^\top$ and $\overline{(\cdot)}$ on the vertices of Q. If B is a finite-dimensional real ∗-algebra with $\operatorname{rad}(B)^2 = 0$, then by Theorem 5.11(2) and its proof the orbits of $Q(B)$ correspond to the ∗-simple factors of $B/\operatorname{rad}(B)$. Each orbit c has one of the ten types of Table 5.1, and we write K_c for the ∗-algebra of minimal size listed there. We say that c, or K_c, is of *signature type* if K_c is $\mathbb{R}$ with the trivial involution, $\mathbb{C}$ with complex conjugation or $\mathbb{H}$ with quaternion conjugation, that is, if the unitary groups U_n listed in Table 5.1 are compact for every n.

For a ∗-algebra B and signs $\eta = (\eta_1, \dots, \eta_n) \in \{\pm 1\}^n$, let $M_{n,\eta}(B)$ denote $M_n(B)$ with the involution $X \mapsto \eta X^* \eta$, where X^* is the conjugate transpose and η also denotes $\operatorname{diag}(\eta_1, \dots, \eta_n)$. Thus the (i,j) entry of the adjoint of X is $\eta_i \eta_j (x_{ji})^*$. This is the ∗-algebra of B-linear endomorphisms of B^n with the adjoint involution of the Hermitian form $h(x,y) = \sum_i x_i \eta_i y_i^*$ (Remark 6.2).

Lemma 5.12. *Let S be a finite-dimensional real ∗-algebra that is ∗-simple. Then $S \cong M_{n,\eta}(K)$ for some $n \geq 1$, some signs η and one of the ten ∗-algebras K of Table 5.1. Here K and n are determined by S, and the type of K is the type of the doubly-signed quiver of S. Moreover:*

1. *every invertible self-adjoint element of K has the form $\pm uu^*$ with $u \in K^\times$, and the sign can be taken to be $+$ if K is not of signature type; in that case one can take $\eta = (1, \dots, 1)$;*
2. *if K is of signature type and $M_{n,\eta}(K) \cong M_{n,\eta'}(K)$, then the numbers of entries $+1$ and -1 in η agree with those in η', up to exchanging the two numbers;*
3. *every self-adjoint idempotent of S is central if and only if $n = 1$.*

Proof. The radical of S is a $*$-stable ideal different from S (Lemma 2.11), so it is zero and S is semisimple. Suppose first that S is not simple, and let S_1 be a simple two-sided ideal. Then $S_1 \cap S_1^*$ is a $*$-stable ideal different from S, hence zero, and $S_1 + S_1^* = S$. So $S = S_1 \times S_1^*$ with $S_1 \cong M_n(D)$ for some $D \in \{\mathbb{R}, \mathbb{C}, \mathbb{H}\}$, and the involution identifies S_1^* with S_1^{op}. Hence $S \cong M_n(D) \times M_n(D)^{\mathrm{op}}$ with the involution that exchanges the factors. This is $M_n(K)$ with the conjugate transpose, where $K = D \times D^{\mathrm{op}}$ carries the exchange involution: the pair (X, Y^{op}) corresponds to the matrix with entries $(x_{ij}, y_{ji}^{\mathrm{op}})$. It is one of the three exchange types, with $\eta = (1, \dots, 1)$.

Suppose now that S is simple, $S \cong M_m(D)$. Its involution is the adjoint involution of a non-degenerate ε-Hermitian form h on D^m with respect to an involution θ of D [58, Ch. I, §4]. We may take θ to be the identity of $\mathbb{R}$, the identity or the conjugation of $\mathbb{C}$, or quaternion conjugation; when θ is the conjugation of $\mathbb{C}$, we may take $\varepsilon = 1$, replacing h by ih. If $\theta = \mathrm{id}$ and $\varepsilon = -1$, then h is alternating, so m is even and h has a symplectic basis. Grouping that basis into hyperbolic pairs identifies S with $M_{m/2}(K)$, where $K = M_2(D)$ carries the symplectic involution and $\eta = (1, \dots, 1)$. In all other cases h has an orthogonal basis (step 4 of the discussion of Theorem 2.23), $h \cong \langle \lambda_1, \dots, \lambda_m \rangle$, and $\langle \lambda \rangle \cong \langle u\lambda\theta(u) \rangle$ for every $u \in D^\times$. This normalises each λ_k to ± 1 if (D, θ, ε) is $(\mathbb{R}, \mathrm{id}, 1)$, $(\mathbb{C}, {}^-, 1)$ or $(\mathbb{H}, {}^-, 1)$, and to 1 if it is $(\mathbb{C}, \mathrm{id}, 1)$; then $S \cong M_{m,\eta}(K)$ with $K = (D, \theta)$ and $\eta_k = \lambda_k$. If $(D, \theta, \varepsilon) = (\mathbb{H}, {}^-, -1)$, the λ_k are non-zero pure quaternions. The elements $u\lambda\bar{u}$ with $u \in \mathbb{H}^\times$ are all the non-zero pure quaternions, so the λ_k can be normalised to j. The adjoint involution is then $X \mapsto j^{-1}X^*j$, and $S \cong M_m(K)$, where $K = \mathbb{H}$ carries the involution $x \mapsto j^{-1}\bar{x}j$, which is of orthogonal type, and $\eta = (1, \dots, 1)$. In every case K is one of the ten algebras of Table 5.1. By Theorem 5.11(4), applied to the corner $E_{11}SE_{11} \cong K$, the doubly-signed quivers of S and K agree. The ten algebras have different doubly-signed quivers, so K is determined by S, and then so is n, because $\dim S = n^2 \dim K$.

(1) Let $\lambda \in K$ be invertible and self-adjoint. If K is of signature type, its self-adjoint elements are real numbers, and $\lambda = \pm uu^*$ with $u = |\lambda|^{1/2}$. Otherwise we find u with $\lambda = uu^*$ as follows. For $\mathbb{C}$ with the trivial involution, take $u = \lambda^{1/2}$. For $\mathbb{H}$

with the involution $x \mapsto j^{-1}\bar{x}j$, whose self-adjoint elements are the combinations of 1, i and k, the element λj is a non-zero pure quaternion; so $\lambda j = uj\bar{u}$ for some u, and then $\lambda = uj\bar{u}j^{-1} = uu^*$. For $M_2(D)$ with the symplectic involution, u^* is the adjugate of u, so $u^* = \operatorname{tr}(u)I - u$; the self-adjoint elements are therefore the scalars μ, and $\mu = uu^*$ for $u = \operatorname{diag}(\mu, 1)$. For $D \times D^{\mathrm{op}}$, the self-adjoint elements are the pairs (d, d^{op}), and $(d, d^{\mathrm{op}}) = uu^*$ for $u = (d, 1)$. In particular $-1 = uu^*$ for some u when K is not of signature type. Replacing the k-th standard basis vector b_k of K^n by ub_k then changes $h(b_k, b_k) = \eta_k$ into $u\eta_k u^* = -\eta_k$. So all signs can be made $+1$ by an isometry, which induces a $*$-isomorphism of the corresponding algebras $M_{n,\eta}(K)$.

(2) Let $\psi : M_{n,\eta}(K) \to M_{n,\eta'}(K)$ be a $*$-isomorphism. If $K = \mathbb{C}$ and ψ is not $\mathbb{C}$-linear, we compose it with entrywise complex conjugation, which is a $*$-automorphism of $M_{n,\eta'}(\mathbb{C})$ because η' is real. Then ψ is linear over the centre ($\mathbb{R}$ or $\mathbb{C}$) of the central simple algebra $M_n(K)$, so $\psi(X) = gXg^{-1}$ for an invertible g by the Skolem–Noether theorem (Theorem 2.9). Comparing $\psi(\eta X^*\eta)$ with $\eta'\psi(X)^*\eta'$ shows that the matrix $\eta g^*\eta' g$ commutes with every matrix, so $g^*\eta' g = \lambda\eta$ for a central scalar λ, which is real because both sides are Hermitian. Thus the Hermitian matrices η' and $\lambda\eta$ are congruent, and the claim follows from Sylvester's law of inertia and its Hermitian and quaternionic analogues (Theorem 2.23).

(3) If $n \geq 2$, then E_{11} is a self-adjoint idempotent that is not central. If $n = 1$, then $S \cong K$, and the only self-adjoint idempotents of K are 0 and 1. Indeed, $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{H}$ have no other idempotents; the self-adjoint elements of $M_2(D)$ with the symplectic involution are the scalars, as shown in (1); and those of $D \times D^{\mathrm{op}}$ are the pairs (d, d^{op}), which are idempotent only for $d \in \{0, 1\}$. □

By part (3), a real $*$-algebra A is basic as a $*$-algebra (Definition 2.14) exactly when every $*$-simple factor of $A/\operatorname{rad}(A)$ is one of the ten algebras of Table 5.1.

5.7 Path $*$-Algebras

We now construct the path $*$-algebra $\mathbb{R}Q$ of a doubly-signed quiver Q. Its complexification is built explicitly, as the path algebra of Q with small matrices as coefficients, and $\mathbb{R}Q$ is the set of fixed points of a conjugation. This is the form in which Chapter 7 localises and completes $\mathbb{R}Q$, and in which the implementation of Chapter 8 stores it.

Let Q be a doubly-signed quiver; we work with its complexified vertices and arrows, on which the transpose and the conjugation act. Give each vertex v the block size d_v and the matrices $G_v, C_v \in \mathrm{GL}_{d_v}(\mathbb{C})$ of Table 5.2, where $J = \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right)$. Thus $d_v = 2$ if $\varepsilon_{\mathrm{div}}(v) = -1$ or $\varepsilon_{\mathrm{form}}(v) = -1$, and $d_v = 1$ otherwise; $G_v = J$ if $\varepsilon_{\mathrm{form}}(v) = -1$, $C_v = J$ if $\varepsilon_{\mathrm{div}}(v) = -1$, and otherwise they are identity matrices. Put $g_v = \varepsilon_{\mathrm{form}}(v)$ if $v^\top = v$ and $g_v = 1$ otherwise, and $h_v = \varepsilon_{\mathrm{div}}(v)$ if $\bar{v} = v$ and $h_v = 1$ otherwise. By rules (2) and (3) of Definition 5.7, g and h are constant on the orbits of the transpose and the conjugation. Moreover $G_{v^\top} = G_v$ and $C_{\bar{v}} = C_v$, and one checks from the table that

$$G_v^T = g_v G_v, \qquad C_v \overline{C_v} = h_v I, \qquad C_{v^\top} \overline{G_v}\, C_v^T = G_{\bar{v}}. \tag{5.1}$$

Table 5.2: The block model of the ten vertex types of Table 5.1: the names used in Chapters 7 and 8, the complex vertices of an orbit, and the block size, vertex form and real structure at each of them.

Name	Vertex data	Complex vertices	d_v	G_v	C_v
`R`	$\varepsilon_{\mathrm{div}} = +1$, $\varepsilon_{\mathrm{form}} = +1$	b	1	I	I
`Rsp`	$\varepsilon_{\mathrm{div}} = +1$, $\varepsilon_{\mathrm{form}} = -1$	b	2	J	I
`Hsp`	$\varepsilon_{\mathrm{div}} = -1$, $\varepsilon_{\mathrm{form}} = -1$	b	2	J	J
`Hor`	$\varepsilon_{\mathrm{div}} = -1$, $\varepsilon_{\mathrm{form}} = +1$	b	2	I	J
`Cor`	$\varepsilon_{\mathrm{div}} = 0$, $\varepsilon_{\mathrm{form}} = +1$	$b, \bar{b}$	1	I	I
`Csp`	$\varepsilon_{\mathrm{div}} = 0$, $\varepsilon_{\mathrm{form}} = -1$	$b, \bar{b}$	2	J	I
`Cu`	$\varepsilon_{\mathrm{div}} = 0$, $b^\top = \bar{b}$	$b, \bar{b}$	1	I	I
`RR`	$\varepsilon_{\mathrm{div}} = +1$, $b^\top \neq b$	$b, b^\top$	1	I	I
`HH`	$\varepsilon_{\mathrm{div}} = -1$, $b^\top \neq b$	$b, b^\top$	2	I	J
`CC`	$\varepsilon_{\mathrm{div}} = 0$, $b^\top \notin \{b, \bar{b}\}$	$b, \bar{b}, b^\top, \bar{b}^\top$	1	I	I

Let T be the complex vector space of finite sums $\sum_w X_w\, w$ over the paths w of Q, with $X_w \in M_{d_t \times d_s}(\mathbb{C})$ for $w : s \to t$, and with the product $(X\, w)(Y\, w') = (XY)\, (ww')$, which is

zero when ww' is not defined. Since $M_{d_t\times d_m}(\mathbb{C})\otimes_{M_{d_m}(\mathbb{C})} M_{d_m\times d_s}(\mathbb{C}) \cong M_{d_t\times d_s}(\mathbb{C})$, T is the tensor algebra $T_{S_\mathbb{C}}(M_\mathbb{C})$ of the bimodule $M_\mathbb{C} = \bigoplus_{\alpha:s\to t} M_{d_t\times d_s}(\mathbb{C})$ over $S_\mathbb{C} = \prod_v M_{d_v}(\mathbb{C})$, which contains one copy of the simple bimodule $M_{d_t\times d_s}(\mathbb{C})$ for each arrow $\alpha : s \to t$. Given signs $\kappa_\alpha, \mu_\alpha \in \{\pm 1\}$ for the arrows, extended multiplicatively to paths, define, for a path $w : s \to t$,

$$(X\,w)^* = \kappa(w)\, G_s X^T G_t^{-1}\, w^\top, \qquad c(X\,w) = \mu(w)\, C_t \overline{X} C_s^{-1}\, \bar{w}, \tag{5.2}$$

where $w^\top$ is the path of the transposed letters of w in reverse order and $\bar{w}$ is the path of its conjugated letters. We call the signs *admissible* if, for every arrow $\alpha : s \to t$,

$$\kappa_\alpha \kappa_{\alpha^\top} = g_s g_t, \qquad \mu_\alpha \mu_{\bar{\alpha}} = h_s h_t, \qquad \kappa_\alpha \mu_{\alpha^\top} = \mu_\alpha \kappa_{\bar{\alpha}}, \tag{5.3}$$

and moreover $\kappa_\alpha = \varepsilon_{\text{form}}(\alpha)$ whenever $\alpha^\top = \alpha$, and $\mu_\alpha = 1$ whenever $\bar{\alpha} = \alpha$.

Definition 5.13. Let Q be a doubly-signed quiver, and fix admissible signs; they exist by Proposition 5.14(2). The *path $*$-algebra* of Q is the real algebra

$$\mathbb{R}Q := T^c = \{x \in T : c(x) = x\},$$

with the involution $*$ of (5.2). It is graded by the length of paths. We write S_Q and M_Q for its parts of degree 0 and 1, $\mathbb{R}Q_{\geq \ell}$ for the ideal of elements of degree at least ℓ, and

$$B_Q := \mathbb{R}Q/\mathbb{R}Q_{\geq 2} = S_Q \oplus M_Q.$$

$\mathbb{R}Q_{\geq 1}$ is the *arrow ideal*. An ideal I of $\mathbb{R}Q$ is *admissible* if $I^* = I$ and $\mathbb{R}Q_{\geq m} \subseteq I \subseteq \mathbb{R}Q_{\geq 2}$ for some $m \geq 2$.

Proposition 5.14 (The block model)**.** *Let Q be a doubly-signed quiver.*

1. *The formulas (5.2) define a $\mathbb{C}$-linear anti-automorphism $*$ and a conjugate-linear automorphism c of T. They satisfy $x^{**} = x$, $c(c(x)) = x$ and $c(x^*) = c(x)^*$ for all $x \in T$ if and only if the signs satisfy (5.3) for every arrow.*

2. *There are admissible signs.*

3. *For admissible signs, $\mathbb{R}Q$ is a real $*$-algebra with $T = \mathbb{R}Q \oplus i\mathbb{R}Q$, so that $\mathbb{C} \otimes_{\mathbb{R}} \mathbb{R}Q \cong T$. It is the tensor algebra $T_{S_Q}(M_Q)$, with the involution that reverses tensors, and $S_Q \cong \prod_{o \in \mathcal{O}(Q)} K_o$, with the algebras K_o of Table 5.1. The algebra B_Q is a basic real $*$-algebra with radical M_Q, $M_Q^2 = 0$ and doubly-signed quiver Q.*

Proof. (1) For paths $w : s \to t$ and $w' : r \to s$, the coefficient of $(ww')^\top = w'^\top w^\top$ in $\big((X\,w)(Y\,w')\big)^*$ is

$$\kappa(w)\kappa(w')\,G_r Y^T X^T G_t^{-1} = \big(\kappa(w')\,G_r Y^T G_s^{-1}\big)\big(\kappa(w)\,G_s X^T G_t^{-1}\big),$$

which is the coefficient of $(Y\,w')^*(X\,w)^*$. So $*$ reverses products; in the same way c preserves them, because $C_s^{-1} C_s = I$. Using (5.1), $G_{v^\top} = G_v$ and $C_{\bar v} = C_v$, one computes

$$(X\,w)^{**} = \kappa(w)\kappa(w^\top)\,g_s g_t\,X\,w, \qquad c(c(X\,w)) = \mu(w)\mu(\bar w)\,h_s h_t\,X\,w,$$

and finds that $c\big((X\,w)^*\big)$ and $c(X\,w)^*$ are $\kappa(w)\mu(w^\top)$ and $\mu(w)\kappa(\bar w)$ times the same element, because the third identity of (5.1) gives $C_{s^\top}\overline{G_s} = G_{\bar s} C_s^{-T}$ and $\overline{G_t}^{-1} C_{t^\top}^{-1} = C_t^T G_{\bar t}^{-1}$. Each of the three scalar factors is the product of the corresponding factors of the letters of w, because the inner vertices of w contribute $g_v^2 = h_v^2 = 1$. Hence the three identities hold on T if and only if they hold on the arrows, which is (5.3).

(2) For each orbit of arrows under the group generated by the transpose and the conjugation, choose a representative $\alpha : s \to t$, put $\kappa_\alpha = \varepsilon_{\mathrm{form}}(\alpha)$ if $\alpha^\top = \alpha$ and $\kappa_\alpha = 1$ otherwise, and define, for $\beta \in \{\alpha, \bar\alpha\}$ and $\beta' \in \{\alpha, \alpha^\top\}$,

$$\kappa_\beta = \kappa_\alpha, \qquad \kappa_{\beta^\top} = g_s g_t\,\kappa_\alpha, \qquad \mu_{\beta'} = 1, \qquad \mu_{\bar{\beta}'} = h_s h_t.$$

These prescriptions agree where the orbit has fewer than four elements. If $\alpha^\top = \alpha$, then $t = s^\top$, so $g_s g_t = g_s^2 = 1$. If $\bar\alpha = \alpha$, then $\bar s = s$ and $\bar t = t$, and rule (4) of Definition 5.7 gives $\varepsilon_{\mathrm{div}}(s) = \varepsilon_{\mathrm{div}}(t)$, so $h_s h_t = 1$. If $\bar\alpha = \alpha^\top$, then $t = \bar s^\top$ lies in the orbit of s, so $g_s g_t = h_s h_t = 1$. Every arrow β of the orbit has $g_{s(\beta)} g_{t(\beta)} = g_s g_t$ and $h_{s(\beta)} h_{t(\beta)} = h_s h_t$,

and by construction $\kappa_{\beta^\top} = g_s g_t \kappa_\beta$, $\kappa_{\bar\beta} = \kappa_\beta$, $\mu_{\bar\beta} = h_s h_t \mu_\beta$ and $\mu_{\beta^\top} = \mu_\beta$. This gives (5.3). Finally, if $\beta^\top = \beta$ for some β in the orbit, then $\alpha^\top = \alpha$, the orbit is contained in $\{\alpha, \bar\alpha\}$, and $\kappa_\beta = \varepsilon_{\text{form}}(\alpha) = \varepsilon_{\text{form}}(\beta)$ by Definition 5.7(3); and if $\bar\beta = \beta$ for some β, then $\bar\alpha = \alpha$ and every μ_β equals 1.

(3) Since c is a conjugate-linear automorphism of order two, $T = T^c \oplus iT^c$, and T^c is a real subalgebra, stable under $*$ because $*$ commutes with c. Since c is multiplicative and preserves the path degree, the multiplication maps identify T^c with the tensor algebra of its part $M_Q = M_{\mathbb{C}}^c$ of degree 1 over its part $S_Q = S_{\mathbb{C}}^c$ of degree 0, because they do so after extending scalars to $\mathbb{C}$. The quotient $B_Q = S_Q \oplus M_Q$ is a real $*$-algebra with radical M_Q and $M_Q^2 = 0$. Its complexification is $S_{\mathbb{C}} \oplus M_{\mathbb{C}}$ with the maps (5.2), and comparison with Theorem 5.11 shows that its doubly-signed quiver is Q: the involution acts on a factor $M_{d_v}(\mathbb{C})$ with $v^\top = v$ by $X \mapsto G_v X^T G_v^{-1}$, where $G_v^T = \varepsilon_{\text{form}}(v) G_v$; the conjugation acts on a factor with $\bar v = v$ by $X \mapsto C_v \overline{X} C_v^{-1}$, where $C_v \overline{C_v} = \varepsilon_{\text{div}}(v) I$; on the copy of $M_{d_t \times d_s}(\mathbb{C})$ that belongs to an arrow α with $\alpha^\top = \alpha$, the involution is $\varepsilon_{\text{form}}(\alpha)$ times the map $X \mapsto G_s X^T G_t^{-1}$, which is the reference map of Theorem 5.11(3); and the conjugation maps the copy that belongs to α onto the copy that belongs to $\bar\alpha$, so that $c \circ *$ maps the copy that belongs to an arrow with $\bar\alpha = \alpha^\top$ onto itself. Finally, the part of S_Q at an orbit o of vertices is the algebra of fixed points of c in $\prod_{v \in o} M_{d_v}(\mathbb{C})$, with the involution of (5.2), and inspection of the ten rows of Table 5.2 identifies it with the algebra K_o of Table 5.1. For one vertex b with $C_b = I$ the fixed points form $M_{d_b}(\mathbb{R})$, with the transpose (type `R`) or the symplectic involution $X \mapsto J X^T J^{-1}$ (`Rsp`). With $C_b = J$ they form the quaternions, as computed after this proof, with quaternion conjugation (`Hsp`) or with the transpose, which is $x \mapsto j^{-1} \bar x j$ and whose fixed points have dimension 3, so that it is of orthogonal type (`Hor`). For two vertices $b, \bar b$ exchanged by c, the fixed points are the pairs $(X, \overline{X})$, a copy of $M_{d_b}(\mathbb{C})$, with the transpose (`Cor`), the symplectic involution (`Csp`) or, when $b^\top = \bar b$, the conjugate transpose (`Cu`). For two vertices $b, b^\top$ fixed by c (`RR`, `HH`), and for the four vertices of type `CC`, the fixed points form $D \times D$ with $D = \mathbb{R}$, $\mathbb{H}$ or $\mathbb{C}$, and the involution exchanges the two factors. So $S_Q \cong \prod_o K_o$, and B_Q is basic as a $*$-algebra (Lemma 5.12(3)). □

By part (3), B_Q realises the converse statement of Theorem 5.11: every doubly-signed quiver is the doubly-signed quiver of a basic real $*$-algebra with radical square zero. By the uniqueness statement of that theorem, B_Q is determined by Q up to $*$-isomorphism, whatever admissible signs are chosen. Hence so is $\mathbb{R}Q = T_{S_Q}(M_Q)$: since $M_Q^2 = 0$, a $*$-isomorphism between two such algebras B_Q induces compatible isomorphisms of their parts S_Q and M_Q, and hence of the tensor algebras. The results of Chapters 6 to 8 about $\mathbb{R}Q$ and its quotients do not use this uniqueness; they hold for any fixed choice of admissible signs.

When every K_c is a division algebra, $\mathbb{R}Q$ without its involution is the tensor algebra of a species in the sense of Dlab and Ringel [24]. In general it is Morita equivalent to one, because $T_S(M)$ is Morita equivalent to $T_{eSe}(eMe)$ for every idempotent $e \in S$ with $SeS = S$. If every vertex of Q has type $(\mathbb{R}, +1)$, then all blocks have size 1, $S_Q = \mathbb{R}^{Q_0}$, the arrows form a basis of M_Q, and $\mathbb{R}Q$ is the path algebra of Definition 2.25; for the signs of Proposition 5.14(2), its involution is that of Section 5.2, multiplied by $\varepsilon_{\text{form}}(\alpha)$ on each arrow α fixed by the transpose. In general, if $e \in S_{\mathbb{C}}$ is the sum of one diagonal matrix unit from each factor $M_{d_v}(\mathbb{C})$, then eTe is the complex path algebra of the quiver underlying Q. So T is Morita equivalent to that path algebra, which justifies the name.

For the real singular value decomposition (Figure 5.2(a)), all blocks have size 1, $G = C = I$, and the signs are 1, so that $(X\,a)^* = X\,a^\top$; this is the algebra of Section 3.4, with $b = a^\top$. For the quaternionic singular value decomposition (Figure 5.2(b)), $d_u = d_v = 2$ and all the matrices G and C equal J. The fixed points of $X \mapsto J\overline{X}J^{-1}$ in $M_2(\mathbb{C})$ are the blocks $\left(\begin{smallmatrix} p & q \\ -\bar{q} & \bar{p} \end{smallmatrix}\right)$, which represent the quaternions $p + qj$, and $X \mapsto JX^TJ^{-1}$ maps such a block to the block of the conjugate quaternion. So the involution of the block model multiplies by conjugate quaternions, as in Section 6.2.5. In the arithmetic of T, the matrices G_v and C_v are the only places where quaternions and symplectic forms enter.

5.8 The Structure Theorem: An Analogue of Gabriel's Theorem

Theorem 5.11 describes the real $*$-algebras whose radical squares to zero. We now show that every finite-dimensional real $*$-algebra is a matrix algebra, with sizes and signs at the orbits of vertices, over a quotient $\mathbb{R}Q/I$ of the path $*$-algebra of its doubly-signed quiver (Theorem 5.16). We first describe these matrix algebras. To assemble matrices over a basic $*$-algebra whose vertices have different sizes, we use the following tensor product.

Definition 5.15. Let $\mathcal{O}$ be a finite set, and let $\mathbb{R}^{\mathcal{O}}$ carry the trivial involution, with minimal idempotents 1_c for $c \in \mathcal{O}$. An $\mathbb{R}^{\mathcal{O}}$-$*$-*algebra* is a $*$-algebra B with a unital $*$-homomorphism $\mathbb{R}^{\mathcal{O}} \to B$, that is, with pairwise orthogonal self-adjoint idempotents $1_c \in B$ such that $\sum_c 1_c = 1$. We write $B_{cd} = 1_c B 1_d$.

1. The *tensor product over* $\mathbb{R}^{\mathcal{O}}$ of two $\mathbb{R}^{\mathcal{O}}$-$*$-algebras E and B is

$$E \otimes_{\mathbb{R}^{\mathcal{O}}} B := \bigoplus_{c,d \in \mathcal{O}} E_{cd} \otimes_{\mathbb{R}} B_{cd} = e\,(E \otimes_{\mathbb{R}} B)\,e, \qquad e = \sum_{c \in \mathcal{O}} 1_c \otimes 1_c.$$

   Since e is a self-adjoint idempotent of the $*$-algebra $E \otimes_{\mathbb{R}} B$ (Definition 2.15), this is a $*$-algebra with unit e. When the 1_c are central in E and in B, it is the usual tensor product over $\mathbb{R}^{\mathcal{O}}$.

2. For each $c \in \mathcal{O}$, let V_c be a finite-dimensional real vector space with a non-degenerate symmetric bilinear form η_c, and let (V, η) be their orthogonal sum. We write $\mathrm{End}(V, \eta)$ for $\mathrm{End}_{\mathbb{R}}(V)$ with the adjoint involution of η. It is an $\mathbb{R}^{\mathcal{O}}$-$*$-algebra, 1_c being the orthogonal projection onto V_c.

Choose bases (v_i) of the spaces V_c that are orthogonal for η, with $\eta(v_i, v_i) = \eta_i \in \{\pm 1\}$, and write $c(i)$ for the index of the space containing v_i. The corresponding matrix units $e_{ij} \in \mathrm{End}(V)$ satisfy $e_{ij}^* = \eta_i \eta_j e_{ji}$. Therefore $\sum_{i,j} e_{ij} \otimes x_{ij} \mapsto (x_{ij})$ identifies $\mathrm{End}(V, \eta) \otimes_{\mathbb{R}^{\mathcal{O}}} B$ with the $*$-algebra of generalised matrices $X = (x_{ij})$ with entries

$x_{ij} \in B_{c(i)c(j)}$, multiplied as matrices, with the involution

$$X \longmapsto \eta X^* \eta, \qquad \text{that is,} \qquad (x_{ij}) \longmapsto \big(\eta_i \eta_j \, (x_{ji})^*\big). \tag{5.4}$$

These are matrices over B indexed by idempotents, as in Section 3.1, with $n_c = \dim V_c$ rows and columns for each c, and with signs. Special cases are $M_n(B)$ with the conjugate transpose (all $n_c = n$ and all $\eta_i = 1$), the corner algebras $eM_N(B)e$ at diagonal idempotents e (all $\eta_i = 1$), and $M_{n,\eta}(K)$ (when $\mathcal{O}$ is a single point). In the language of Chapter 2, $\mathrm{End}(V,\eta) \otimes_{\mathbb{R}^{\mathcal{O}}} B$ is the $*$-algebra of B-linear endomorphisms, with the adjoint involution, of the Hermitian B-module $\bigoplus_i B1_{c(i)}$ with the form $\sum_i x_i \eta_i y_i^*$. The theorem below is therefore a statement of Hermitian Morita theory [57, Ch. I, §9].

Through $\mathbb{R}^{\mathcal{O}(Q)} \to S_Q$, the quotients $\mathbb{R}Q/I$ of the path $*$-algebra of a doubly-signed quiver Q are $\mathbb{R}^{\mathcal{O}(Q)}$-$*$-algebras.

Throughout the rest of this section, A is a finite-dimensional real $*$-algebra with radical N. Since N^2 is stable under the involution, A/N^2 is a $*$-algebra whose radical squares to zero, and we call $Q(A) := Q(A/N^2)$ the *doubly-signed quiver of* A. Since $(A/N^2)/(N/N^2) = A/N$, the orbits of $Q(A)$ correspond to the $*$-simple factors of A/N.

Theorem 5.16 (Structure theorem; an analogue of Gabriel's theorem)**.** *Let A be a finite-dimensional real $*$-algebra, $Q = Q(A)$ its doubly-signed quiver, and $\mathcal{O} = \mathcal{O}(Q)$.*

1. *There are non-zero finite-dimensional real spaces V_c with non-degenerate symmetric bilinear forms η_c ($c \in \mathcal{O}$), and an admissible ideal I of $\mathbb{R}Q$, such that*

$$A \;\cong\; \mathrm{End}(V,\eta) \otimes_{\mathbb{R}^{\mathcal{O}}} \mathbb{R}Q/I$$

   as $*$-algebras. Here $\mathbb{R}Q/I$ is basic as a $*$-algebra, and $\mathbb{R}Q_{\geq m} \subseteq I$ for every m with $\mathrm{rad}(A)^m = 0$. *Moreover, η_c can be taken positive definite if c is not of signature type, and of signature (p_c, q_c) with $p_c \geq q_c$ if it is; we then call the decomposition* normalised.

2. *Let $A \cong \mathrm{End}(V',\eta') \otimes_{\mathbb{R}^{\mathcal{O}'}} \mathbb{R}Q'/I'$ be another normalised decomposition, for a doubly-*

signed quiver Q' with $\mathcal{O}' = \mathcal{O}(Q')$, non-zero spaces V'_c and an admissible ideal I'. Then there is an isomorphism $Q' \cong Q$ under which $\dim V'_c = \dim V_c$ and the signatures of η'_c and η_c agree. Thus Q, the dimensions $n_c = \dim V_c$ and the signatures (p_c, q_c) are invariants of A.

3. *In the situation of (2), let $B = \mathbb{R}Q/I$ and $B' = \mathbb{R}Q'/I'$. Then B' is $*$-isomorphic to B^t, the algebra B with the involution $x \mapsto x^{*_t} := tx^*t$, for some $t = \sum_c \tau_c 1_c$ with $\tau_c \in \{\pm 1\}$ and $\tau_c = 1$ unless c is of signature type and $p_c = q_c$. Conversely, $\mathrm{End}(V,\eta) \otimes_{\mathbb{R}^{\mathcal{O}}} B^t \cong \mathrm{End}(V,\eta) \otimes_{\mathbb{R}^{\mathcal{O}}} B$ for every such t.*

Since t is self-adjoint with $t^2 = 1$, the map $x \mapsto tx^*t$ in (3) is again an involution. The condition $V'_c \neq 0$ in (2) cannot be dropped: $\mathbb{R} \cong \mathrm{End}(V,\eta) \otimes_{\mathbb{R}^2} (\mathbb{R} \times \mathbb{R})$ with $V_1 = \mathbb{R}$ and $V_2 = 0$.

Remark 5.17. By Lemma 5.12(3), A is basic as a $*$-algebra exactly when all $n_c = 1$, and then Theorem 5.16 says that $A \cong \mathbb{R}Q/I$. This is the analogue for real $*$-algebras of Gabriel's theorem [3, Theorem II.3.7]. In general, the theorem separates three kinds of data. The quiver Q, the sizes n_c and the signatures (p_c, q_c) are discrete invariants of A. The twist t in (3) is a finite ambiguity; it occurs only at orbits of split signature $p_c = q_c$, and it is genuine (Example 5.21). All continuous parameters lie in the ideal I, which is determined by $\mathbb{R}Q/I$ up to $*$-automorphisms of $\mathbb{R}Q/\mathbb{R}Q_{\geq m}$ (Proposition 5.23). When $\mathrm{rad}(A)^2 = 0$, we have $I = \mathbb{R}Q_{\geq 2}$, and $B^t \cong B$ for every t, via the map that fixes S_Q and sends $r \in M_Q$ to tr. In that case $A \cong \mathrm{End}(V,\eta) \otimes_{\mathbb{R}^{\mathcal{O}}} B_Q$ is determined by Q, the sizes and the signatures, which refines Theorem 5.11.

The structure theorem also accounts for the indexing of Section 3.1.

Corollary 5.18 (Positions)**.** *In the situation of Theorem 5.16(1), put $B = \mathbb{R}Q/I$ and $n_c = \dim V_c$, and call the pairs c_k with $c \in \mathcal{O}$ and $1 \leq k \leq n_c$ the* positions.

1. *A is $*$-isomorphic to the algebra of matrices $X = (x_{c_k,d_l})$ indexed by the positions, with entries $x_{c_k,d_l} \in 1_c B 1_d$, multiplied as matrices, with the involution $X \mapsto (\eta_{c_k}\eta_{d_l}\, x^*_{d_l,c_k})$ of (5.4).*

2. *For $n \geq 1$, $M_n(A)$ with the conjugate transpose is $*$-isomorphic to* $\mathrm{End}(V^n, \eta^n) \otimes_{\mathbb{R}^o} B$, *where (V^n, η^n) is the orthogonal sum of n copies of (V, η). Thus $M_n(A)$ has the doubly-signed quiver Q and the relations I, with $n\, n_c$ positions and the signature (np_c, nq_c) at the orbit c.*

Proof. Part (1) is the identification (5.4), applied to Theorem 5.16(1). For part (2), recall from Section 2.3.3 that $M_n(A) \cong M_n(\mathbb{R}) \otimes_{\mathbb{R}} A$, with the transpose on $M_n(\mathbb{R})$. Moreover $M_n(\mathbb{R}) \otimes_{\mathbb{R}} \mathrm{End}(V, \eta) = \mathrm{End}(\mathbb{R}^n \otimes V, I_n \otimes \eta)$, and both $M_n(\mathbb{R}) \otimes_{\mathbb{R}} \big(\mathrm{End}(V, \eta) \otimes_{\mathbb{R}^o} B\big)$ and $\mathrm{End}(\mathbb{R}^n \otimes V, I_n \otimes \eta) \otimes_{\mathbb{R}^o} B$ are the corner of $M_n(\mathbb{R}) \otimes_{\mathbb{R}} \mathrm{End}(V) \otimes_{\mathbb{R}} B$ at the idempotent $\sum_c I_n \otimes 1_c \otimes 1_c$. The form $I_n \otimes \eta$ is the orthogonal sum of n copies of η. On $\mathbb{R}^n \otimes V_c$ it is positive definite where η_c is, and it has signature (np_c, nq_c), so the decomposition is normalised, and Theorem 5.16(2) gives the invariants. □

So the positions of Section 3.1 are intrinsic to A. Their orbits are the orbits of the vertices of Q under the transpose and the conjugation, the number of positions in each orbit is an invariant, and the entries between the positions of the orbits c and d lie in the corner $1_c(\mathbb{R}Q/I)1_d$. Passing from A to matrices over A only multiplies the numbers of positions. The order in which the positions are listed is not intrinsic. It is chosen by the algorithm, and it matters (Proposition 3.9).

5.8.1 Examples

The following examples illustrate the three parts of Theorem 5.16; its proof follows them, in Section 5.8.2.

Example 5.19 (Truncated polynomial rings)**.** Let Q have one vertex of type $(\mathbb{R}, +1)$ and one loop x fixed by the transpose, with sign ± 1. Then $\mathbb{R}Q = \mathbb{R}[x]$ with $x^* = \pm x$, and the admissible ideals are the ideals (x^m) with $m \geq 2$. By Theorem 5.16, the real $*$-algebras with doubly-signed quiver Q are the algebras $M_{n,\eta}(\mathbb{R}[x]/(x^m))$ with $\eta = (1, \ldots, 1, -1, \ldots, -1)$, containing $p \geq q$ entries $+1$ and -1 respectively, and these are pairwise non-isomorphic. For $m = 2$ these are the algebras of Sections 6.2.1 and 6.2.2, with an indefinite form when $q > 0$. For $m \geq 3$ they are matrices over truncated power

series, as in Chapter 4; for instance, $\mathbb{R}[\varepsilon]/(\varepsilon^3)$ with $\varepsilon^* = \varepsilon$ is $\mathbb{R}Q/(x^3)$ for the sign $+1$.

Example 5.20 (Degenerating diagonal forms)**.** Let $G = \operatorname{diag}(s_1\varepsilon^{w_1}, \dots, s_d\varepsilon^{w_d})$ with signs $s_i = \pm 1$ and integers $w_i \geq 0$, and consider

$$\Lambda_G = \{X \in M_d(\mathbb{R}[\varepsilon]) : G^{-1}X^TG \in M_d(\mathbb{R}[\varepsilon])\},$$

where $G^{-1}X^TG$ is computed in $M_d(\mathbb{R}[\varepsilon, \varepsilon^{-1}])$, with the involution $X \mapsto G^{-1}X^TG$, the adjoint involution of the form G. The reduction $A_G = \Lambda_G/\varepsilon\Lambda_G$ is a real $*$-algebra of dimension d^2. It describes the adjoint involution "at $\varepsilon = 0$", where G itself degenerates. Let $\omega_1 < \cdots < \omega_r$ be the distinct values of the w_i. Then

$$A_G \cong \operatorname{End}(V, \eta) \otimes_{\mathbb{R}^r} \mathbb{R}Q/I.$$

Here Q is the double of the quiver $1 \to 2 \to \cdots \to r$: all vertices have type $(\mathbb{R}, +1)$, and there are arrows $a_j : j \to j+1$ and $a_j^\top : j+1 \to j$. The ideal I is generated by the 2-cycles $a_j^\top a_j$ and $a_j a_j^\top$, and (V_j, η_j) is the form $\langle s_i : w_i = \omega_j \rangle$. Indeed, Λ_G consists of the matrices whose (k, l) entry is divisible by $\varepsilon^{\max(0, w_l - w_k)}$. The elements $\hat{e}_{kl} = \varepsilon^{\max(0, w_l - w_k)} E_{kl}$ form a basis of A_G, with $\hat{e}_{kl}^* = s_k s_l \hat{e}_{lk}$, and $\hat{e}_{kl}\hat{e}_{lm} = \hat{e}_{km}$ if w_l lies between w_k and w_m, and 0 otherwise. On the other side, $\mathbb{R}Q/I$ has a basis of monotone paths, which multiply by the same rule. So only the order of the weights and the signs matter. Because I is spanned by paths, rescaling the arrows $a_j^\top$ by signs shows that $B^t \cong B$ for every t, so the signs can be normalised as in Theorem 5.16(1) without changing $\mathbb{R}Q/I$. For $r = 2$, $\mathbb{R}Q/I$ is the four-dimensional algebra of the singular value decomposition (Section 3.4). For $r \geq 3$, its radical does not square to zero, since $a_2 a_1 \neq 0$. We have verified this description by exact computation for several choices of weights and signs.

Example 5.21 (The twist in Theorem 5.16(3) is necessary)**.** Let Q have two vertices $1, 2$ of type $(\mathbb{R}, +1)$, a loop ℓ at 1 fixed by the transpose with sign $+1$, and arrows $a : 1 \to 2$ and $a^\top : 2 \to 1$. Write $b = a^\top = a^*$. The ideal $I = (\ell^2 - ba,\ ab,\ a\ell,\ \ell b)$ is admissible: it is $*$-stable, since the involution fixes $\ell^2 - ba$ and exchanges $a\ell$ with ℓb, and it contains every path of length three. The algebra $B = \mathbb{R}Q/I$ has the basis $e_1, e_2, \ell, a, b, \ell^2$.

Let $t = -e_1 + e_2$, so that the involution of B^t maps a to $a^{*_t} = -b$. In B there are $x \in e_2 \operatorname{rad}(B) e_1$ and a self-adjoint $y \in e_1 \operatorname{rad}(B) e_1$ with $x^*x = y^2 \neq 0$, namely $x = a$ and $y = \ell$. In B^t there are no x and y with $x^{*_t}x = y^2 \neq 0$: $x = \gamma a$ gives $x^{*_t}x = -\gamma^2\ell^2$, whereas $y = \alpha\ell + \beta\ell^2$ gives $y^2 = \alpha^2\ell^2$. A $*$-isomorphism $B \to B^t$ may be assumed to fix e_1 and e_2, by Lemma 5.9(2) and because e_1Be_1 and e_2Be_2 have different dimensions; it would then preserve this property. Hence $B \ncong B^t$. Nevertheless, for $V_1 = \mathbb{R}^2$ with $\eta_1 = \operatorname{diag}(1, -1)$ and $V_2 = \mathbb{R}$, the 17-dimensional algebras $\operatorname{End}(V, \eta) \otimes_{\mathbb{R}^2} B$ and $\operatorname{End}(V, \eta) \otimes_{\mathbb{R}^2} B^t$ are isomorphic, as Theorem 5.16(3) predicts; an isomorphism exchanges the two basis vectors of V_1.

5.8.2 Proof of the Structure Theorem

Proof of Theorem 5.16. Let $N = \operatorname{rad}(A)$.

Step 1: matrix units. By Theorem 2.12, $A = S \oplus N$ for a $*$-stable subalgebra $S \cong A/N$; let $S = \prod_{c \in \mathcal{O}} S_c$ be its decomposition into $*$-simple factors. By Lemma 5.12, with $K = K_c$ because the type of S_c is that of the orbit c, and since $M_{n,\eta}(K) = M_{n,-\eta}(K)$ and the signs may be permuted by conjugating with a permutation matrix, there are $*$-isomorphisms $S_c \cong M_{n_c,\eta_c}(K_c)$ with normalised signs $\eta_c = (\eta_{c,1}, \dots, \eta_{c,n_c})$: all equal to $+1$ if c is not of signature type, and otherwise p_c entries $+1$ followed by $q_c \leq p_c$ entries -1. In particular $\eta_{c,1} = 1$. Let $E^c_{kl} \in S_c$ correspond to the matrix with entry 1 at position (k, l). Then $E^c_{kl}E^c_{lm} = E^c_{km}$ and $(E^c_{kl})^* = \eta_{c,k}\eta_{c,l}E^c_{lk}$.

Step 2: a basic $*$-algebra. Put $f_c = E^c_{11}$, $f = \sum_c f_c$ and $u_{c,k} = E^c_{k1}$. We write i, j for pairs (c, k), and $c(i) = c$, $\eta_i = \eta_{c,k}$ and $u_i = u_{c,k}$ for $i = (c, k)$. Then f is a self-adjoint idempotent, $u_i = u_i f_{c(i)}$, and

$$u_i^* u_j = \delta_{ij}\, \eta_i\, f_{c(i)}, \qquad \sum_i \eta_i\, u_i u_i^* = 1. \tag{5.5}$$

Let $B = fAf$, a $*$-algebra with unit f and an $\mathbb{R}^{\mathcal{O}}$-$*$-algebra with $1_c = f_c$. We have $B = fSf \oplus fNf$, where $fSf = \prod_c f_c S_c f_c \cong \prod_c K_c$ is semisimple and fNf is a nilpotent ideal of B. Hence $\operatorname{rad}(B) = fNf$ and $B/\operatorname{rad}(B) \cong \prod_c K_c$, and B is basic as a $*$-algebra

by Lemma 5.12(3).

Step 3: A is a matrix algebra over B. Let $V_c = \mathbb{R}^{n_c}$ with the form $\mathrm{diag}(\eta_c)$, and define $\Phi : A \to \mathrm{End}(V,\eta) \otimes_{\mathbb{R}^\circ} B$, in the coordinates of (5.4), by

$$\Phi(a)_{ij} = \eta_i\, u_i^*\, a\, u_j \;\in\; f_{c(i)} A f_{c(j)} = B_{c(i)c(j)}.$$

By (5.5),

$$(\Phi(a)\Phi(b))_{ij} = \eta_i u_i^* a \Big(\sum_k \eta_k u_k u_k^*\Big) b u_j = \Phi(ab)_{ij}, \qquad \Phi(1)_{ij} = \eta_i u_i^* u_j = \delta_{ij} f_{c(i)},$$

and $\Phi(a^*)_{ij} = \eta_i (u_j^* a u_i)^* = \eta_i\eta_j (\Phi(a)_{ji})^*$, which is the involution (5.4). The map $X \mapsto \sum_{i,j} u_i\, x_{ij}\, \eta_j u_j^*$ is inverse to Φ, again by (5.5). So Φ is a $*$-isomorphism. For a single orbit and positive signs this is Proposition 3.1.

Step 4: B has doubly-signed quiver Q. Let $N_B = \mathrm{rad}(B) = fNf$. By (5.5),

$$fN^2f = \sum_i \eta_i\, (fNu_i)(u_i^* Nf) \subseteq (fNf)^2 = N_B^2,$$

since $fNu_i = fNu_if$ and $u_i^*Nf = fu_i^*Nf$; the reverse inclusion is clear. So $B/N_B^2 = \bar f(A/N^2)\bar f$, where $\bar f$ is the image of f. Moreover $(A/N^2)\bar f(A/N^2) = A/N^2$, because $1 = \sum_i \eta_i u_i f u_i^*$. Thus B/N_B^2 is a basic $*$-algebra of A/N^2, and $Q(B) = Q(A/N^2) = Q$ by Theorem 5.11(4).

Step 5: a presentation of B. Let $\bar S = B/N_B$ and $\bar M = N_B/N_B^2$. Then $\bar M$ is an $\bar S$-bimodule, and the involution of B induces involutions of $\bar S$ and of $\bar M$ with $(srs')^* = s'^* r^* s^*$. So the tensor algebra $T = T_{\bar S}(\bar M)$, with the involution that reverses tensors, is a $*$-algebra. The subalgebra $S' = fSf$ of B is $*$-stable with $B = S' \oplus N_B$; let $\sigma : \bar S \to S'$ be the inverse of the quotient map, a $*$-isomorphism with $\sigma(\bar f_c) = f_c$. Since S' is semisimple and $\mathbb{R}$ has characteristic 0, S' is separable, and every S'-bimodule is semisimple [70, Ch. 10]. So N_B has an S'-bimodule projection p with kernel N_B^2. Then $p' = \frac{1}{2}(p + *\circ p \circ *)$ is an S'-bimodule map, because $S'^* = S'$. It vanishes on N_B^2 and induces the identity on N_B/N_B^2, because $(N_B^2)^* = N_B^2$. So it is a projection with kernel N_B^2, and it commutes with the involution.

Its image W is a $*$-stable sub-bimodule with $N_B = W \oplus N_B^2$. The inverse of the quotient map $W \to \bar{M}$ is a map $\rho : \bar{M} \to W$ with $\rho(srs') = \sigma(s)\rho(r)\sigma(s')$ and $\rho(r^*) = \rho(r)^*$. By the universal property of the tensor algebra, there is an algebra homomorphism $\varphi : T \to B$ with $\varphi|_{\bar{S}} = \sigma$ and $\varphi(r_1 \otimes \cdots \otimes r_\ell) = \rho(r_1) \cdots \rho(r_\ell)$. It commutes with the involutions, because it does so on $\bar{S}$ and $\bar{M}$ and both involutions reverse products.

The homomorphism φ is surjective. From $N_B = W + N_B^2$ we obtain, by induction, $N_B = W + W^2 + \cdots + W^{m-1} + N_B^m$ for all m, and $N_B^m \subseteq fN^m f = 0$ if $N^m = 0$; so $\varphi(T) \supseteq S' + N_B = B$. Its kernel J is $*$-stable. It contains $T_{\geq m}$, because $\varphi(T_{\geq m}) \subseteq N_B^m$. It is contained in $T_{\geq 2}$, because φ maps $\bar{S} \oplus \bar{M}$ isomorphically onto $S' \oplus W$, $\varphi(T_{\geq 2}) \subseteq N_B^2$, and $B = S' \oplus W \oplus N_B^2$.

Finally, B/N_B^2 and B_Q are basic radical-square-zero $*$-algebras with the same doubly-signed quiver (Step 4). So by Theorem 5.11 there is a $*$-isomorphism $\chi : B/N_B^2 \to B_Q$. It maps the radical $\bar{M}$ onto M_Q and induces a $*$-isomorphism $\bar{S} \to B_Q/M_Q \cong S_Q$, compatibly with the bimodule structures. Hence it induces a degree-preserving $*$-isomorphism $T \to \mathbb{R}Q$. The image I of J is an admissible ideal, $I \supseteq \mathbb{R}Q_{\geq m}$ if $N^m = 0$, and $\mathbb{R}Q/I \cong B$.

Step 6: proof of (1). In degree 0, the isomorphism $B \cong \mathbb{R}Q/I$ of Step 5 restricts to a $*$-isomorphism $fSf \cong S_Q$. This maps each f_c to the unit $1_{\pi(c)}$ of a factor $K_{\pi(c)} \cong K_c$, for a type-preserving bijection π of $\mathcal{O}$. Combining this with Step 3, we get

$$A \cong \operatorname{End}(V, \eta) \otimes_{\mathbb{R}^\mathcal{O}} B \cong \operatorname{End}(V', \eta') \otimes_{\mathbb{R}^\mathcal{O}} \mathbb{R}Q/I, \qquad (V'_{\pi(c)}, \eta'_{\pi(c)}) = (V_c, \eta_c).$$

This proves (1); the remaining claims of (1) were shown in Steps 2 and 5.

Proof of (2). Consider any normalised decomposition $A \cong \operatorname{End}(V', \eta') \otimes_{\mathbb{R}^{\mathcal{O}'}} B'$ with $B' = \mathbb{R}Q'/I'$ and all $V'_c \neq 0$, and write $n'_c = \dim V'_c$. The ideal $\operatorname{End}(V', \eta') \otimes_{\mathbb{R}^{\mathcal{O}'}} \operatorname{rad}(B')$ is nilpotent. Its square is $\operatorname{End}(V', \eta') \otimes_{\mathbb{R}^{\mathcal{O}'}} \operatorname{rad}(B')^2$, and the quotient by it is $\operatorname{End}(V', \eta') \otimes_{\mathbb{R}^{\mathcal{O}'}} S_{Q'} = \prod_c M_{n'_c, \eta'_c}(K_c)$, which is semisimple. Hence $A/N \cong \prod_c M_{n'_c, \eta'_c}(K_c)$, and $A/N^2 \cong \operatorname{End}(V', \eta') \otimes_{\mathbb{R}^{\mathcal{O}'}} B_{Q'}$ because $B'/\operatorname{rad}(B')^2 = \mathbb{R}Q'/\mathbb{R}Q'_{\geq 2} = B_{Q'}$. Since every V'_c is non-zero, the idempotent $\sum_c e_{(c,1)(c,1)} \otimes 1_c$ is full, and the corner at it is $B_{Q'}$, because $\eta'_{c,1} = 1$. So Theorem 5.11(4) gives an isomorphism $\theta' : Q' \to Q(A/N^2) = Q$, under which the $*$-simple factor of A/N belonging to $\theta'(c)$ is $M_{n'_c, \eta'_c}(K_c)$. Applying this to both decompositions in

(2), and comparing the two descriptions of each factor, Lemma 5.12 shows that $n'_c = n_{\theta(c)}$ for the resulting isomorphism $\theta : Q' \to Q$. It also shows that, if c is of signature type, the signatures of η'_c and $\eta_{\theta(c)}$ agree up to order, hence agree, because both are normalised.

Proof of (3). Let $f_c \in A$ ($c \in \mathcal{O}$) and $f'_{c'} \in A$ ($c' \in \mathcal{O}'$) be the self-adjoint idempotents corresponding to $e_{(c,1)(c,1)} \otimes 1_c$ under the two isomorphisms, and put $f = \sum_c f_c$ and $f' = \sum_{c'} f'_{c'}$. Since $\eta_{c,1} = \eta'_{c',1} = 1$, the corners of the two decompositions at these idempotents are B and B', so $fAf \cong B$ and $f'Af' \cong B'$. By the proof of (2), each $*$-simple factor F of A/N contains the image of exactly one f_c and of exactly one $f'_{c'}$, with $\theta(c') = c$, and $F \cong M_{n_c,\eta_c}(K_c)$ is the $*$-algebra of the Hermitian K_c-module $(K_c^{n_c}, \eta_c)$. The images $\bar{f}_c$ and $\bar{f}'_{c'}$ are self-adjoint idempotents of F with corners isomorphic to K_c, that is, orthogonal projections onto free submodules of rank one. A Hermitian form on a free module of rank one is $\langle\lambda\rangle$ for an invertible self-adjoint $\lambda \in K_c$, so by Lemma 5.12(1) it is isometric to $\langle 1\rangle$ if c is not of signature type, and to $\langle 1\rangle$ or $\langle -1\rangle$ if it is. By Witt's cancellation theorem, a consequence of Theorem 2.23, two such projections are conjugate by a unitary of F if and only if the forms on their images are isometric. So if c is not of signature type, $\bar{f}_c$ and $\bar{f}'_{c'}$ are conjugate. If it is, the projections fall into at most two conjugacy classes, and by Sylvester's law of inertia every family of n_c pairwise orthogonal ones contains p_c members of one class and q_c of the other. Both $\bar{f}_c$ and $\bar{f}'_{c'}$ belong to a class that contributes $p_c = \max(p_c, q_c)$ members, because $\eta_{c,1} = \eta'_{c',1} = 1$ and $(p'_{c'}, q'_{c'}) = (p_c, q_c)$. Hence $\bar{f}_c$ and $\bar{f}'_{c'}$ are conjugate unless c is of signature type, $p_c = q_c$, and they lie in different classes. In that case put $\tau_c = -1$, and let f''_c correspond to $e_{(c,k)(c,k)} \otimes 1_c$ under the first isomorphism, for some k with $\eta_{c,k} = -1$. Otherwise put $\tau_c = 1$ and $f''_c = f_c$. By (5.4), the corner of $\mathrm{End}(V,\eta) \otimes_{\mathbb{R}^\sigma} B$ at the idempotent corresponding to $f'' = \sum_c f''_c$ is B with the involution multiplied by $\tau_c\tau_d$ on B_{cd}; that is, $f''Af'' \cong B^t$. The images of f'' and f' are conjugate in every factor F, hence by a unitary of A/N, and Lemma 5.9 gives $B' \cong f'Af' \cong f''Af'' \cong B^t$.

Conversely, $\mathrm{End}(V,\eta) \otimes_{\mathbb{R}^\sigma} B^t$ and $\mathrm{End}(V,t\eta) \otimes_{\mathbb{R}^\sigma} B$, where $t\eta$ multiplies η_c by τ_c, have the same underlying algebra and, by (5.4), the same involution. If $\tau_c = -1$ only where $p_c = q_c$, there is an isometry $(V,t\eta) \to (V,\eta)$ that preserves each V_c. Conjugation by it is

a $*$-isomorphism $\mathrm{End}(V, t\eta) \otimes_{\mathbb{R}^o} B \to \mathrm{End}(V, \eta) \otimes_{\mathbb{R}^o} B$. □

5.8.3 Uniqueness of the Relations

The ideal I in Theorem 5.16 is not unique. For example, let Q have one vertex of type $(\mathbb{R}, +1)$ and two loops x, y fixed by the transpose with sign $+1$, so that $\mathbb{R}Q = \mathbb{R}\langle x, y\rangle$ with $x^* = x$ and $y^* = y$. The $*$-automorphism of $\mathbb{R}Q$ that fixes x and maps y to $x+y$ maps the admissible ideal $I = (x^2 - y^2,\, xy,\, yx)$ to an admissible ideal that contains $x^2 + xy \notin I$, with an isomorphic quotient. The following proposition shows that this is the only kind of ambiguity. It uses a $*$-stable version of the conjugacy part of the Wedderburn–Malcev theorem.

Lemma 5.22. *Let B be a finite-dimensional real $*$-algebra, and let S_1, S_2 be $*$-stable subalgebras with $B = S_1 \oplus \mathrm{rad}(B) = S_2 \oplus \mathrm{rad}(B)$. Then $S_2 = uS_1u^*$ for a unitary $u \in 1 + \mathrm{rad}(B)$.*

Proof. By Theorem 2.12, $S_2 = gS_1g^{-1}$ for some $g \in 1 + \mathrm{rad}(B)$. Applying the involution gives $S_2 = g^{-*}S_1g^*$, so $h = g^*g$ satisfies $hS_1h^{-1} = S_1$. For $s \in S_1$, the element $hsh^{-1} - s$ lies in $S_1 \cap \mathrm{rad}(B) = 0$, because $h \equiv 1$ modulo $\mathrm{rad}(B)$. So h commutes with S_1, and so does the polynomial $h^{-1/2}$ in h. Then $u = gh^{-1/2}$ is unitary, and $uS_1u^* = gS_1g^{-1} = S_2$. □

Proposition 5.23. *Let I and I' be admissible ideals of $\mathbb{R}Q$ with $\mathbb{R}Q_{\geq m} \subseteq I \cap I'$. If $\mathbb{R}Q/I \cong \mathbb{R}Q/I'$ as $*$-algebras, then $I'/\mathbb{R}Q_{\geq m} = \alpha(I/\mathbb{R}Q_{\geq m})$ for a $*$-automorphism α of $\mathbb{R}Q/\mathbb{R}Q_{\geq m}$.*

Proof. Write $T = \mathbb{R}Q/\mathbb{R}Q_{\geq m}$ and $B' = \mathbb{R}Q/I'$. Let $\pi' : T \to B'$ be the quotient map, and let $\pi = \chi \circ \pi_I$, where $\pi_I : T \to \mathbb{R}Q/I$ is the quotient map and $\chi : \mathbb{R}Q/I \to B'$ is the given isomorphism. Both π and π' are surjective $*$-homomorphisms that map S_Q isomorphically onto $*$-stable complements of $\mathrm{rad}(B')$, map $T_{\geq 1}$ onto $\mathrm{rad}(B')$, and induce isomorphisms $M_Q \to \mathrm{rad}(B')/\mathrm{rad}(B')^2$. By Lemma 5.22, after replacing π by $x \mapsto u\pi(x)u^*$ for a unitary u, which does not change its kernel, we may assume $\pi(S_Q) = \pi'(S_Q)$. Then $\beta = (\pi'|_{S_Q})^{-1} \circ \pi|_{S_Q}$ is a $*$-automorphism of S_Q, and $\pi(srs') = \pi'(\beta(s))\,\pi(r)\,\pi'(\beta(s'))$ for $r \in M_Q$. Since S_Q is separable, M_Q is projective as an S_Q-bimodule, even with

the action twisted by β. So $\pi|_{M_Q}$ lifts through the surjection $\pi' : T_{\geq 1} \to \operatorname{rad}(B')$ to a map $\lambda_0 : M_Q \to T_{\geq 1}$ with $\lambda_0(srs') = \beta(s)\lambda_0(r)\beta(s')$ and $\pi' \circ \lambda_0 = \pi|_{M_Q}$. The map $\lambda = \frac{1}{2}(\lambda_0 + * \circ \lambda_0 \circ *)$ has the same two properties and commutes with the involution. By the universal property of the tensor algebra, β and λ define a $*$-endomorphism α of T; it is well defined on T because $\lambda(M_Q) \subseteq T_{\geq 1}$. Moreover $\pi' \circ \alpha = \pi$, because both sides are homomorphisms that agree on S_Q and M_Q. On the associated graded algebra, α is induced by β and by the degree-one part of λ. The latter is bijective, because π and π' induce isomorphisms $M_Q \to \operatorname{rad}(B')/\operatorname{rad}(B')^2$; hence α is bijective. Finally, $\pi' \circ \alpha = \pi$ gives $\alpha(I/\mathbb{R}Q_{\geq m}) \subseteq I'/\mathbb{R}Q_{\geq m}$, with equality because both quotients have dimension $\dim B'$. □

Chapter 6

Doubly-Signed Quivers Encode Matrix Decompositions

This chapter proves the two outcomes of Section 1.2 for the encoded matrices: parts (2) and (3) of Theorem A, which make precise the sense in which a doubly-signed quiver *encodes* a matrix decomposition, and, with Proposition 3.9, parts (1) to (3) of Theorem B, which say what the programs of Section 1.2 compute on the encoded matrices.

Let Q be a doubly-signed quiver. Its path $*$-algebra $\mathbb{R}Q$ is graded by the length of paths: the vertices have degree 0 and the arrows degree 1. Consider the Hermitian matrices of degree 1, that is, the matrices indexed by positions (Corollary 5.18) whose entries are linear combinations of arrows. Theorem 6.1 shows that unitary similarity of such matrices, over $\mathbb{R}Q$ or over any quotient of $\mathbb{R}Q$ by homogeneous relations, is the same as unitary similarity by unitary matrices of degree 0. These form a product of classical groups, one for each orbit of vertices, and the space of Hermitian matrices of degree 1 is a direct sum of classical matrix spaces, one for each orbit of arrows. The decomposition indexed by Q is, by definition, a normal form for this problem. Section 3.4 showed this for the singular value decomposition, and Section 6.2 works through a catalogue of examples.

Section 6.3 runs the generic algorithms on Hermitian matrices of degree 1, in the completed localisation Λ of Chapter 7. When every connected component of the complexified quiver is an oriented cycle or a linearly oriented path, which holds for the whole cata-

logue, Householder reduction, whenever all its steps are defined, transforms the matrix only by unitaries of degree 0, and so computes a condensed form for the classical problem (Proposition 6.4). For the quivers of the singular value decomposition and of the Takagi factorisation, two steps of the QR algorithm are one step of the QR algorithm on a Gram matrix, and a single step is a Cholesky LR step on it (Proposition 6.5). The split-complex numbers ${}^2\mathbb{R}$ are a different $*$-algebra: like the algebra of the real singular value decomposition, they are $\mathbb{R} \times \mathbb{R}$ in degree 0, but their involution exchanges the two idempotents instead of fixing them. Over ${}^2\mathbb{R}$, one step of the QR algorithm is two steps of Rutishauser's LR algorithm on an invertible real matrix, whenever these are defined (Proposition 6.6). Section 6.4 treats Hermitian matrices with a part of degree 0, which leads to perturbation theory, and Section 6.5 discusses what representation type says about generalised algorithms.

6.1 The Encoding Theorem

Throughout this chapter, Q is a doubly-signed quiver, $\mathcal{O} = \mathcal{O}(Q)$ is the set of orbits of its vertices under the transpose and the conjugation, and $A = \mathbb{R}Q/I$, where $I \subseteq \mathbb{R}Q_{\geq 2}$ is an ideal that is stable under the involution and spanned by homogeneous elements, for example $I = 0$ or $I = \mathbb{R}Q_{\geq m}$ with $m \geq 2$ (Definition 5.13). Then $A = \bigoplus_{d \geq 0} A_d$ is graded by the path degree, the involution preserves degrees, $A_0 = S_Q = \prod_{c \in \mathcal{O}} K_c$, and $A_1 = M_Q$. For $I = \mathbb{R}Q_{\geq 2}$ we obtain $A = B_Q$, whose radical squares to zero; this is the case of Section 3.4. Fix real spaces V_c with non-degenerate symmetric forms η_c, put $n_c = \dim V_c$, and let

$$E = \operatorname{End}(V, \eta) \otimes_{\mathbb{R}^{\mathcal{O}}} A$$

be the $*$-algebra of matrices indexed by the positions c_k ($c \in \mathcal{O}$, $1 \leq k \leq n_c$) with entries in A (Definition 5.15 and (5.4)). It is graded by $E_d = \operatorname{End}(V, \eta) \otimes_{\mathbb{R}^{\mathcal{O}}} A_d$, and we write $X = X_0 + X_1 + \cdots$ for the homogeneous parts of $X \in E$. Let $\operatorname{Herm}(E_1)$ be the set of Hermitian elements of degree 1, and $\mathrm{U}(E)$ the unitary group of E.

For each orbit $[\alpha]$ of arrows under the transpose and the conjugation, let $M_{[\alpha]} \subseteq M_Q$

be the real form of the sum of the copies of $M_{d_t \times d_s}(\mathbb{C})$ that belong to the arrows of $[\alpha]$ in the block model (Proposition 5.14).

Theorem 6.1 (Encoding theorem)**.** *In this situation:*

1. *If $U \in \mathrm{U}(E)$, then $U_0 \in \mathrm{U}(E_0)$. Conversely, $\mathrm{U}(E_0) \subseteq \mathrm{U}(E)$.*

2. *Let $H, H' \in \mathrm{Herm}(E_1)$. If $H' = UHU^*$ for some $U \in \mathrm{U}(E)$, then $H' = U_0 H U_0^*$. If $A = B_Q$, then $UHU^* = U_0 H U_0^*$ for all $U \in \mathrm{U}(E)$ and all $H \in \mathrm{Herm}(E_1)$. In either case, two elements of $\mathrm{Herm}(E_1)$ are unitarily similar over A if and only if they lie in the same orbit of $\mathrm{U}(E_0)$.*

3. *$\mathrm{U}(E_0) = \prod_{c \in \mathcal{O}} \mathrm{U}_c$, where U_c is the unitary group of $M_{n_c, \eta_c}(K_c)$. If η_c is positive definite, or c is not of signature type, then U_c is isomorphic to the classical group of Table 5.1; otherwise it is an indefinite form of it (Remark 6.2). The space $\mathrm{Herm}(E_1)$ is the direct sum, over the orbits $[\alpha]$ of arrows, of $\mathrm{U}(E_0)$-stable subspaces $\mathcal{H}_{[\alpha]}$. On $\mathcal{H}_{[\alpha]}$ only the factors U_c and $\mathrm{U}_{c'}$ act, where c and c' are the orbits of the source and the target of α:*

   (a) *if $c \neq c'$, then $\mathcal{H}_{[\alpha]} \cong M_{n_{c'} \times n_c}(P_{[\alpha]})$, where $P_{[\alpha]} = 1_{c'} M_{[\alpha]} 1_c$ is a $K_{c'}$–K_c-bimodule, and $(g_c, g_{c'})$ acts by $W \mapsto g_{c'} W g_c^{-1}$;*

   (b) *if $c = c'$, then $\mathcal{H}_{[\alpha]}$ is the space of matrices $W = (w_{kl})$ of size n_c with entries in $M_{[\alpha]}$ and $w_{kl} = \eta_k \eta_l \, w_{lk}^*$, and $g \in \mathrm{U}_c$ acts by $W \mapsto g W g^{-1}$.*

Proof. (1) A product of homogeneous elements is homogeneous of the sum of their degrees, and all degrees are non-negative, so $(XY)_0 = X_0 Y_0$. Moreover $(X^*)_0 = (X_0)^*$, because the involution preserves degrees. Applying this to $U^*U = UU^* = 1$ gives $U_0^* U_0 = U_0 U_0^* = 1$. The converse is clear.

(2) For $H \in E_1$ and $U \in E$, the part of degree 1 of UHU^* is $U_0 H U_0^*$, because the parts of U and U^* have non-negative degrees, and only their parts of degree 0 combine with H to degree 1. Hence $H' = (UHU^*)_1 = U_0 H U_0^*$. If $A = B_Q$, then $E_d = 0$ for $d \geq 2$, and UHU^* has no part of degree 0, so $UHU^* = (UHU^*)_1$. The last statement follows, by (1).

(3) The orbit idempotents 1_c are central in S_Q, so E_0 is the product of the algebras $\mathrm{End}(V_c,\eta_c)\otimes_{\mathbb{R}}K_c = M_{n_c,\eta_c}(K_c)$, and its unitary group is the product of theirs. In the block model, $M_Q\otimes_{\mathbb{R}}\mathbb{C} = M_{\mathbb{C}}$ is the direct sum of copies of $M_{d_t\times d_s}(\mathbb{C})$, one for each arrow, and by (5.2) the involution and the conjugation map the copy of α onto the copies of $\alpha^\top$ and $\bar{\alpha}$. So the sum of the copies over an orbit is stable under both, and it is the complexification of $M_{[\alpha]}$. Hence $M_Q = \bigoplus_{[\alpha]} M_{[\alpha]}$ is a direct sum of S_Q-sub-bimodules that are stable under the involution, E_1 is the direct sum of the spaces $\mathrm{End}(V,\eta)\otimes_{\mathbb{R}^o} M_{[\alpha]}$, each stable under the involution and under multiplication by E_0 on both sides, and $\mathrm{Herm}(E_1)$ is the direct sum of their Hermitian parts $\mathcal{H}_{[\alpha]}$, each stable under $X\mapsto UXU^*$ for $U\in \mathrm{U}(E_0)$. The arrows of $[\alpha]$ go from vertices of c to vertices of c' or, after transposition, from c' to c, so $M_{[\alpha]}\subseteq 1_{c'}M_Q1_c + 1_cM_Q1_{c'}$, and only U_c and $\mathrm{U}_{c'}$ act on $\mathcal{H}_{[\alpha]}$. If $c\neq c'$, an element X of $\mathcal{H}_{[\alpha]}$ has the block $W = 1_{c'}X1_c \in M_{n_{c'}\times n_c}(P_{[\alpha]})$, its block $1_cX1_{c'} = W^*$ is determined by W, and its other blocks vanish; every W occurs, since $W+W^*$ is Hermitian. For $U=(g_c,g_{c'})$ the block $1_{c'}(UXU^*)1_c$ is $g_{c'}Wg_c^*$, and $g_c^* = g_c^{-1}$. If $c=c'$, then $\mathcal{H}_{[\alpha]}$ consists of the Hermitian elements of $\mathrm{End}(V_c,\eta_c)\otimes_{\mathbb{R}} M_{[\alpha]}$, which by (5.4) form the stated space, and g acts by $X\mapsto gXg^* = gXg^{-1}$. □

Theorem 6.1 answers the question of what a doubly-signed quiver encodes. *Given Q and the spaces (V_c,η_c), the decomposition indexed by Q is a normal form for the Hermitian matrices of degree* 1 *under unitary similarity.* By part (2), this problem does not depend on the homogeneous relations: it is the same over B_Q, over $\mathbb{R}Q/\mathbb{R}Q_{\geq m}$ and over $\mathbb{R}Q$ itself, and it is the action of $\mathrm{U}(E_0)$ on $\mathrm{Herm}(E_1)$. By part (3), it is a direct sum of classical problems, one for each orbit of arrows, coupled through the groups at the vertices. The numbers n_c are the sizes of the matrices; for example, $n_u\neq n_v$ gives rectangular matrices in the singular value decomposition. In the language of Section 5.3.1, $\mathrm{Herm}(E_1)$ is a space of $*$-representations of Q, and unitary similarity by E_0 is their isomorphism. By part (2), every algorithm that transforms Hermitian matrices by unitary similarity over A, such as a generalised Jacobi algorithm, or the QR algorithm taken in double steps (over the quivers of the singular value decomposition a single step exchanges the two orbits, Proposition 6.5), acts on Hermitian matrices of degree 1 through the classical groups at the

vertices; Section 6.3 makes this concrete for algorithms run in the completed localisation.

Remark 6.2 (Indefinite forms). By Lemma 5.12(1), η_c may be taken positive definite at every orbit c that is not of signature type, since there every invertible self-adjoint element of K_c has the form uu^*. At an orbit of signature type, the signature of η_c is part of the data: U_c is the indefinite group $\mathrm{O}(p,q)$, $\mathrm{U}(p,q)$ or $\mathrm{Sp}(p,q)$, and the problems of Theorem 6.1(3) become their indefinite analogues. By Theorem 2.23, the signature is an invariant of the Hermitian module and belongs to the data of the representation rather than to the quiver.

6.2 A Catalogue of Decompositions and Their Classical Analogues

We now apply Theorem 6.1 to a series of doubly-signed quivers. In each case we describe the $*$-algebra $B = B_Q$, compute the Hermitian matrices of degree 1 and the action of the unitary matrices of degree 0, and identify the classical decomposition. Each of these decompositions can also be derived classically, case by case, often as the Hermitian eigendecomposition of a larger block matrix: for example, the singular value decomposition of W from the eigendecomposition of the symmetric Jordan–Wielandt matrix $\left(\begin{smallmatrix} 0 & W^T \\ W & 0 \end{smallmatrix}\right)$ [34], and the quaternionic decompositions from the representation of a quaternionic matrix by a complex matrix of twice the size [96]. The encoding gives a uniform derivation, not the only one. By Theorem 6.1(2), the same problem arises over $\mathbb{R}Q/I$ for every homogeneous I. Unless stated otherwise, every orbit has n positions and the forms η_c are positive definite, so that $E = M_n(B)$ with the conjugate transpose. We write $B = S \oplus R$ with $S = S_Q$ and $R = M_Q = \mathrm{rad}(B)$, and $\mathrm{Herm}_n(R)$ for the Hermitian matrices with entries in R; these are the Hermitian matrices of degree 1. Diagrams of the quivers are in Figures 5.1 and 5.2, and the results are summarised in Table 6.1. Following the convention of Chapter 5, in labels we write the division algebra $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$ in place of the sign $\varepsilon_{\mathrm{div}} = +1, 0, -1$. Section 6.3 describes what the generic algorithms compute for each entry (Table 6.2).

6.2.1 The Symmetric Eigendecomposition

Quiver. One vertex v of type $(\mathbb{R}, +1)$ with one loop x fixed by the transpose with sign $+1$ (Figure 5.1(a)).

Algebra. $B = \mathbb{R}[x]/(x^2)$, the dual numbers, with the trivial involution $x^* = x$.

Encoding. A matrix $H = xW \in M_n(R)$ is Hermitian if and only if $W^T = W$. The unitary group of $S = \mathbb{R}$ is $\mathrm{O}(n)$, and $U_0(xW)U_0^* = x\,(PWP^T)$ for $U_0 = P$.

Classical analogue. Orthogonal similarity of real symmetric matrices, whose normal form is the spectral theorem $W = P\Lambda P^T$ [50]. The diagonal entries of the normal form, $x\lambda_i$, are the "eigenvalues" of H over B. Section 6.4 shows that the full Hermitian matrices over this algebra, $H = X + xW$, are the first-order perturbations of symmetric matrices, recovering [39] and Theorem 4.12 modulo ε^2.

6.2.2 The Normal Form of Skew-Symmetric Matrices

Quiver. One vertex of type $(\mathbb{R}, +1)$ with one loop h fixed by the transpose with sign -1 (Figure 5.1(b)).

Algebra. The dual numbers $B = \mathbb{R}[h]/(h^2)$ with the involution $h^* = -h$.

Encoding. Since $(hW)^* = W^T h^* = -hW^T$, the matrix hW is Hermitian if and only if W is skew-symmetric. The action is again $W \mapsto PWP^T$ with $P \in \mathrm{O}(n)$.

Classical analogue. Orthogonal similarity of real skew-symmetric matrices. Its normal form is

$$W = P\Big(\bigoplus_j \begin{pmatrix} 0 & \sigma_j \\ -\sigma_j & 0 \end{pmatrix} \oplus 0\Big)P^T, \qquad \sigma_j > 0,$$

see [50, Corollary 2.5.11]. Here diagonalisation is impossible: the only Hermitian 1×1 matrix with radical entry is 0, because $(hw)^* = -hw$. Apart from the zero block, the indecomposable summands therefore have dimension two, which explains the 2×2 blocks. These are the parity blocks of Corollary 4.17 reduced modulo ε^2.

6.2.3 The Real Singular Value Decomposition

Quiver. Two vertices u, v of type $(\mathbb{R}, +1)$ and arrows $a : u \to v$ and $a^\top : v \to u$ exchanged by the transpose (Figures 5.1(d) and 5.2(a)).

Algebra. The four-dimensional algebra B of Section 3.4, with basis e_u, e_v, a, b, where $b = a^* = a^\top$.

Encoding. $\mathrm{Herm}_n(R) = \{aW + bW^T : W \in M_n(\mathbb{R})\}$, $\mathrm{U}_n(S) = \mathrm{O}(n) \times \mathrm{O}(n)$, and the action is $W \mapsto RWP^T$ (equation (3.2)). With n positions for u and m for v, $W \in M_{m\times n}(\mathbb{R})$. This is case (a) of Theorem 6.1(3), with $P_{[a]} = \mathbb{R}a$.

Classical analogue. The singular value decomposition $W = R^T \Sigma P$ [34]. The vertices contribute the two orthogonal factors, and the pair of arrows contributes the matrix being decomposed.

6.2.4 The Complex Singular Value Decomposition

Quiver. As for the real SVD, but with two complex vertices of unitary type; after complexification each becomes a pair exchanged both by conjugation and by the transpose (Figure 5.2(c)).

Algebra. $S = \mathbb{C} \times \mathbb{C}$, each factor with complex conjugation as involution, and $R = \mathbb{C}a \oplus \mathbb{C}a^*$, where a goes from u to v, $za = az$ for $z \in \mathbb{C}$, and $(za)^* = \bar{z}\, a^*$.

Encoding. $\mathrm{Herm}_n(R) = \{aW + a^*W^H : W \in M_n(\mathbb{C})\}$, $\mathrm{U}_n(S) = \mathrm{U}(n) \times \mathrm{U}(n)$, and the action is $W \mapsto RWP^H$.

Classical analogue. The complex singular value decomposition $W = R^H \Sigma P$. The only change compared with the real SVD is the vertex type, which replaces $\mathrm{O}(n)$ by $\mathrm{U}(n)$; this is exactly what the conjugation in a doubly-signed quiver records. In the diagram, the conjugate pair $(v, \bar{v})$ indicates that one must keep track of $\bar{U} = (U^H)^T$ as well as U.

6.2.5 The Quaternionic Singular Value Decomposition

Quiver. As for the real SVD, but with two quaternionic vertices with $\varepsilon_{\mathrm{div}} = \varepsilon_{\mathrm{form}} = -1$ (Figure 5.2(b)).

Algebra. $S = \mathbb{H} \times \mathbb{H}$ with quaternion conjugation, and $R = \mathbb{H}a \oplus \mathbb{H}a^*$ with $qa = aq$ for $q \in \mathbb{H}$ and $(qa)^* = a^*\bar{q}$.

Encoding. $\mathrm{Herm}_n(R) = \{aW + a^*W^* : W \in M_n(\mathbb{H})\}$, where W^* is the quaternionic conjugate transpose. The unitary group of $M_n(\mathbb{H})$ with this involution is the compact symplectic group $\mathrm{Sp}(n)$, so the action is $W \mapsto RWP^*$ with $P, R \in \mathrm{Sp}(n)$.

Classical analogue. The singular value decomposition of quaternionic matrices [96]. That quaternions carry, by default, a *symplectic* Hermitian form is visible in the quiver: after complexification, quaternion conjugation becomes the symplectic involution of $M_2(\mathbb{C})$, so $\varepsilon_{\text{form}} = -1$. In the block model it is recorded by the matrix $G_v = J$ at the vertex (Table 5.2).

6.2.6 Hermitian Eigendecompositions over $\mathbb{C}$ and $\mathbb{H}$

Quiver. One vertex of unitary type (complex) or of type $(\mathbb{H}, -1)$ (quaternionic), with a loop x. In the complex case, the transpose of x is its conjugate partner, and after complexification there are two vertices $v, \bar{v}$ with one loop each, exchanged by both involutions.

Algebra. $S = \mathbb{C}$ with conjugation (respectively $\mathbb{H}$ with quaternion conjugation), and $R = \mathbb{C}x$ (respectively $\mathbb{H}x$) with $zx = xz$ and $x^* = x$.

Encoding. $(wx)^* = x\bar{w} = \bar{w}x$, so xW is Hermitian if and only if $W^H = W$ (respectively $W^* = W$). The action is $W \mapsto PWP^H$ with $P \in \mathrm{U}(n)$ (respectively $P \in \mathrm{Sp}(n)$). In the complex case the orbit of arrows is $\{x, \bar{x}\}$, with $x^\top = \bar{x} \neq x$. Both arrows are loops at the same orbit, so this is case (b) of Theorem 6.1(3), in which the Hermitian condition is a symmetry condition on a single matrix.

Classical analogue. The spectral theorem for complex Hermitian and for quaternionic Hermitian matrices [50, 96].

6.2.7 The Takagi Factorisation

Quiver. One complex vertex of unitary type, with an arrow α from v to its conjugate $\bar{v} = v^\top$ that is fixed by the transpose with sign $+1$ (Figure 5.2(d)).

Algebra. $S = \mathbb{C}$ with complex conjugation, and $R = \mathbb{C}a$ with the *twisted* bimodule

structure $za = a\bar{z}$, and $a^* = a$. One checks that the involution is consistent: $(za)^* = a^*\bar{z} = a\bar{z} = za$.

Encoding. For $H = aW$, with entries $w_{ij}a$, we get $(H^*)_{ij} = (w_{ji}a)^* = w_{ji}a$, so H is Hermitian if and only if $W^T = W$, that is, W is complex symmetric. For $U_0 = P$ with $P \in \mathrm{U}(n)$,

$$(U_0 H U_0^*)_{ij} = \sum_{k,l} p_{ik}\, w_{kl}\, a\, \bar{p}_{jl} = \sum_{k,l} p_{ik}\, w_{kl}\, p_{jl}\, a = (PWP^T)_{ij}\, a,$$

using $a\bar{p} = pa$. So the action is *unitary congruence* $W \mapsto PWP^T$.

Classical analogue. The Takagi factorisation: every complex symmetric matrix is $W = P\Sigma P^T$ with P unitary and Σ diagonal with non-negative entries [82], [50, Corollary 4.4.4]. Comparing with the complex Hermitian eigendecomposition of Section 6.2.6, the only difference in the quiver is that the arrow joins v to its conjugate $\bar{v}$ instead of being a loop at v; algebraically, this is the difference between the untwisted rule $zx = xz$ and the twisted rule $za = a\bar{z}$. If instead $a^* = -a$ (sign -1), W is complex skew-symmetric and the normal form under unitary congruence is $\bigoplus_j \sigma_j \left(\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right) \oplus 0$ [94].

6.2.8 Congruence of Symmetric Matrices and the LDL^T Decomposition

Quiver. A pair of real vertices $u, u^\top$ exchanged by the transpose (type $(\mathbb{R}, 0)$, so that the vertex orbit is ${}^2\mathbb{R}$), with an arrow $a : u \to u^\top$ fixed by the transpose with sign $+1$ (Figure 5.1(c)).

Algebra. $S = {}^2\mathbb{R}$ with idempotents $e = (1, 0)$, $e^* = (0, 1)$, and $R = \mathbb{R}a$ with $a = e^*ae$ and $a^* = a$.

Encoding. aW is Hermitian if and only if $W^T = W$. By Section 1.3.2, $\mathrm{U}_n({}^2\mathbb{R}) = \{eP + e^*P^{-T} : P \in \mathrm{GL}_n(\mathbb{R})\}$, and a computation as in Section 3.4 gives $U_0(aW)U_0^* = a\,(P^{-T}WP^{-1})$. Writing $g = P^{-T}$, the action is congruence $W \mapsto gWg^T$ by arbitrary invertible g.

Classical analogue. Sylvester's law of inertia: W is congruent to $\mathrm{diag}(I_p, -I_q, 0)$, and

(p, q) is a complete invariant. Numerically, a congruence normal form is computed from the symmetric indefinite factorisation $\Pi W \Pi^T = LDL^T$, in which Π is a permutation chosen by symmetric pivoting, L is unit lower triangular and D is block diagonal with 1×1 and 2×2 blocks (Bunch–Kaufman pivoting) [34]; a final congruence diagonalises D. With sign -1 one obtains congruence of skew-symmetric matrices, whose normal form is a symplectic (Darboux) basis.

Completing the cycle. In this example the path algebra is acyclic. Every column x with entries in R satisfies $x^*x = 0$ already in $\mathbb{R}Q$, because $a^*a = aa$ is not a path; so the algorithms that need norms do not apply, and the generic algorithm for this quiver is the LDL^* factorisation, which needs none (Section 6.3). The stabiliser of the normal form is large (an indefinite orthogonal group), and correspondingly there are many normal-form algorithms: g may be chosen lower triangular, as in LDL^T, or composed of an orthogonal eigenvector matrix and a diagonal scaling. Adding a reverse arrow $b : u^\top \to u$, also fixed with sign $+1$, removes this ambiguity. A Hermitian matrix with radical entries is then $aW + bZ$ with W, Z symmetric, and $\mathrm{GL}_n(\mathbb{R})$ acts by $(W, Z) \mapsto (gWg^T, g^{-T}Zg^{-1})$. If we fix $Z = \varepsilon I$ with $\varepsilon > 0$, for instance a formal infinitesimal, then the admissible g are exactly those with $g^Tg = I$, and the problem becomes the orthogonal eigendecomposition of W. In Chapter 7 the reverse arrow reappears as the inverse a^{-1} in the localisation of the path algebra (Section 7.2).

6.2.9 Similarity of Real Matrices

Quiver. A pair of real vertices $v, v^\top$ exchanged by the transpose, with a loop x at v and its partner $x^\top$ at $v^\top$.

Algebra. $S = {}^2\mathbb{R}$, $R = \mathbb{R}x \oplus \mathbb{R}x^*$ with $x = exe$.

Encoding. $\mathrm{Herm}_n(R) = \{xX + x^*X^T : X \in M_n(\mathbb{R})\}$ and, as in Section 1.3.2, the action of $U_0 = eP + e^*P^{-T}$ is $X \mapsto PXP^{-1}$.

Classical analogue. Similarity of real matrices, whose normal form is the real Jordan form. As a Hermitian eigendecomposition, it is a direct sum of blocks $xJ + x^*J^T$, one for each real Jordan block J, none of which is unitarily similar to a direct sum of smaller

blocks; each can be taken tridiagonal, since every real matrix that is similar to a single real Jordan block is similar to a tridiagonal one. This is the radical version of the example of Section 1.3.2, where the same problem arose from the semisimple algebra ${}^2\mathbb{R}$ itself. The underlying quiver (a loop) is tame (Section 5.1.1): the normal form has continuous parameters, the eigenvalues, and a discrete part, the Jordan structure, which depends discontinuously on the matrix (Section 1.4).

Table 6.1: Doubly-signed quivers, the groups and matrix spaces that they encode by Theorem 6.1, and the corresponding classical decompositions. Vertex labels give the division algebra and $\varepsilon_{\text{form}}$; "u" denotes a complex vertex of unitary type, and $\rightleftarrows$ a pair of arrows exchanged by the transpose.

Quiver	**Group**	**Matrices W**	**Classical decomposition**
$(\mathbb{R},+)$, loop $(+)$	$\mathrm{O}(n)$	real symmetric	spectral theorem
$(\mathbb{R},+)$, loop $(-)$	$\mathrm{O}(n)$	real skew-symmetric	real normal form, 2×2 blocks
$(\mathbb{R},+) \rightleftarrows (\mathbb{R},+)$	$\mathrm{O}(n)\times\mathrm{O}(n)$	real rectangular	real SVD
$(\mathbb{C},\mathrm{u}) \rightleftarrows (\mathbb{C},\mathrm{u})$	$\mathrm{U}(n)\times\mathrm{U}(n)$	complex rectangular	complex SVD
$(\mathbb{H},-) \rightleftarrows (\mathbb{H},-)$	$\mathrm{Sp}(n)\times\mathrm{Sp}(n)$	quaternionic rectangular	quaternionic SVD
$(\mathbb{C},\mathrm{u})$, loop	$\mathrm{U}(n)$	complex Hermitian	Hermitian spectral theorem
$(\mathbb{H},-)$, loop $(+)$	$\mathrm{Sp}(n)$	quaternionic Hermitian	quaternionic spectral theorem
$(\mathbb{C},\mathrm{u})$, $v\to\bar{v}$ $(+)$	$\mathrm{U}(n)$	complex symmetric	Takagi factorisation
$(\mathbb{C},\mathrm{u})$, $v\to\bar{v}$ $(-)$	$\mathrm{U}(n)$	complex skew-symmetric	Youla normal form
$(\mathbb{R},0)$, $u\to u^\top$ $(+)$	$\mathrm{GL}_n(\mathbb{R})$	real symmetric	Sylvester inertia, LDL^T
$(\mathbb{R},0)$, loops $x, x^\top$	$\mathrm{GL}_n(\mathbb{R})$	real square	real Jordan form

6.2.10 From a Matrix Problem to a Quiver

Theorem 6.1 can be read backwards, to encode a new matrix problem and run the generic algorithms on it. The problem must have the form that the theorem describes: a product of classical groups acting on a direct sum of matrix spaces, each of which is moved by one or two of the groups.

1. For each group factor, choose the vertex type of Table 5.1 whose unitary group it is,

and give the corresponding orbit c as many positions n_c as the index n of the group U_n in that table, which is the size of the matrices over K_c on which the factor acts; for an indefinite group, choose a form η_c of the right signature (Remark 6.2).

2. For each matrix space that is moved by two factors, as $W \mapsto g'Wg^{-1}$, add an orbit of arrows between the two orbits; for each space that is moved by one factor, add an orbit of arrows within its orbit (Theorem 6.1(3)). Table 6.1 shows which arrows give which actions. For example, an arrow from a vertex to its transpose gives congruence, of symmetric or skew-symmetric matrices according to the sign of the arrow, and a loop at a vertex of exchange type gives similarity.

3. Encode the data as a Hermitian matrix of degree 1, indexed by the positions in a cyclic order (Section 3.1); its entries between the positions of two orbits are the combinations of arrows of Theorem 6.1(3).

4. Run the generic algorithm over the completed localisation Λ of Chapter 7, as in Listing 8.1.

5. Check that the unitary factor and the result contain no inverse letters. Then the identities that certify the result, such as $H = UTU^*$ and $U^*U = I$, hold over the original algebra (Theorem 7.8), and the coefficients of the arrows in the result are the classical answer. By Proposition 6.4, this is automatic for Householder reduction when its steps are defined and every component of the complexified quiver is an oriented cycle or a linearly oriented path.

The generic algorithms report a breakdown when a column is isotropic, as for the congruence quiver (Section 6.2.8), or when a denominator is not invertible in the corner of its column. Example 3.10 carries out steps (3) to (5) by hand for the real singular value decomposition.

6.3 Generic Algorithms on Encoded Matrices

The generic algorithms of this thesis are run in the completed localisation Λ of the complexified path $*$-algebra (Chapter 7), on matrices indexed by positions. A Hermitian matrix of degree 1 over A is lifted to Λ by sending each arrow to itself. If an algorithm transforms it only by unitary matrices of degree 0, and returns a matrix of degree 1, then by Theorem 6.1 it has computed with the classical problem encoded by Q, and by Theorem 7.8 its results reduce to A. This section gives a condition on Q under which Householder reduction behaves in this way, and it identifies one step of the QR algorithm for several quivers of the catalogue. Table 6.2 summarises the results. They describe what the generic algorithms compute; they do not claim that these algorithms are better in practice than the specialised classical ones, such as the bidiagonalisation of Golub and Kahan [33] as implemented in LAPACK, which are faster and better studied.

What this section uses from Chapter 7. The *localisation* kQ_{loc} adjoins to the path algebra an inverse α^{-1}, from e_w to e_v, of every arrow $\alpha : v \to w$ (Definition 7.1). Its elements are linear combinations of *reduced words*: products of letters α and α^{-1} with matching end points and no factor $\alpha\alpha^{-1}$ or $\alpha^{-1}\alpha$. The *degree* of a word is the number of its letters α minus the number of its letters α^{-1}, so that a path has its length as degree. The *completed localisation* Λ consists of the series $\sum_{d \geq d_0} x_d$ in which each x_d is a finite combination of reduced words of degree d (Definition 7.4); for one vertex with one loop t, it is the field of Laurent series $k((t))$. An element whose part of lowest degree is invertible is itself invertible (Lemma 7.5). When the underlying graph of every component of the quiver has at most one cycle, and that cycle has non-zero degree, every corner of Λ is a space of matrices over a field (Proposition 7.6); the degree of a cycle is the degree of the reduced word that goes once around it, up to sign. $*$-square roots are computed by the graded rule of Section 7.5, which, for a homogeneous element of degree $2d$, looks for a root of degree d. Results return to the original algebra when the quantities that certify them contain no inverse letters (Theorem 7.8).

Lemma 6.3. *Let Q be a connected quiver with n vertices and cycle rank r (Section 5.1.1),*

fix a vertex o, and for each vertex v let $\ell(v)$ be the degree of a reduced word from o to v. The following are equivalent:

1. *every reduced word of degree 0 is trivial;*
2. *either $r = 0$ and the $\ell(v)$ are pairwise distinct, or $r = 1$, the cycle of Q has non-zero degree δ, and the $\ell(v)$ are pairwise distinct modulo δ;*
3. *Q is linearly oriented of type A_n, $1 \to 2 \to \cdots \to n$, or an oriented cycle; for $n = 1$, a vertex or a loop.*

In that case, the homogeneous elements of degree 0 of Λ are the linear combinations of the vertex idempotents, with coefficient blocks.

Proof. By Proposition 7.2 and the proof of Proposition 7.6, the reduced words from v to w correspond to the elements $g \in F_r$, and the word of g has degree $\ell(w) - \ell(v) + \varphi(g)$ for a homomorphism $\varphi : F_r \to \mathbb{Z}$; the trivial word e_v corresponds to $w = v$ and $g = 1$. (Another choice of the words that define ℓ changes $\ell(v)$ by an element of $\varphi(F_r)$.) Hence (1) holds if and only if φ is injective and $\ell(w) - \ell(v) \notin \varphi(F_r)$ for all $v \neq w$. As in the proof of Proposition 7.6, φ is injective if and only if $r = 0$, or $r = 1$ and the cycle has non-zero degree δ; then $\varphi(F_r)$ is 0 or $\delta\mathbb{Z}$. This proves that (1) and (2) are equivalent.

Along an arrow $\alpha : v \to w$, $\ell(w) = \ell(v) + 1$, modulo δ if $r = 1$. Assume (2). If $r = 0$, Q is a tree on which ℓ is injective. A vertex with three neighbours would give two of them the same value of ℓ, so Q is a path, and ℓ is monotone along it, so all its arrows point in the same direction. If $r = 1$, the values $\ell(v)$ are n distinct residues modulo δ, so $n \leq |\delta|$. But $|\delta|$ is at most the length of the cycle, which is at most n. Hence the cycle passes through all the vertices, so that Q is the cycle, and all its arrows point in the same direction. Conversely, for $1 \to \cdots \to n$ we may take $\ell(v) = v - 1$, and for an oriented cycle with n vertices $\delta = n$ and ℓ takes the values $0, \ldots, n - 1$. The last statement holds because a homogeneous element of Λ is a finite linear combination of reduced words of its degree. □

Every connected component of the complexified quiver of every quiver of Table 6.1 satisfies the conditions of the lemma. These components are single vertices with one

loop, oriented 2-cycles, for the singular value decompositions and for the Takagi and Youla factorisations, and, for the congruence quiver, a single arrow. The conditions are restrictive: they fail, for example, for the quiver of Example 7.7, in which $a^{-1}b$ is a non-trivial word of degree 0.

Proposition 6.4. *Suppose that every connected component of the complexified quiver of Q satisfies the conditions of Lemma 6.3, and that every form η_c is positive definite, so that the involution of E is the conjugate transpose. Let $H \in \mathrm{Herm}(E_1)$, and run Householder tridiagonalisation on H over Λ, with a square-root rule that returns a real homogeneous root of degree 1 for a real self-adjoint homogeneous element of degree 2, such as the graded rule of Section 7.5. Suppose that every step is defined: every root lies in the corner required by Proposition 3.8, and every denominator is invertible in the corner of its column. Then every reflector lies in $\mathrm{U}(E_0)$, and $H = UTU^*$ with $U \in \mathrm{U}(E_0)$ and $T \in \mathrm{Herm}(E_1)$ tridiagonal. These identities hold over every homogeneous quotient A of $\mathbb{R}Q$, so T lies in the orbit of H under the group $\mathrm{U}(E_0)$ of Theorem 6.1.*

Proof. Suppose that the current matrix is Hermitian of degree 1, and consider a step with column x, whose entries have degree 1. Then x^*x is homogeneous of degree 2, the root γ and its negative are homogeneous of degree 1, so is $w = x - \hat{e}_1\gamma$, and $s = w^*x$ is homogeneous of degree 2. The inverse of s from f to f is homogeneous of degree -2: if y is the inverse, then comparing the parts of degree 0 in $ys = f$ and $sy = f$ shows that the part of y of degree -2 is also an inverse, and inverses are unique. Hence $P = I - ws^{-1}w^*$ is homogeneous of degree 0. It is real, because all the quantities are, so by Lemma 6.3 its entries are real linear combinations of vertex idempotents with coefficient blocks, that is, $P \in E_0$; and it is unitary by Proposition 3.8. So the next matrix PHP^* is again Hermitian of degree 1, and the first two claims follow by induction. Since U and T contain no inverse letters, Theorem 7.8 gives the identities over A. □

For the real singular value decomposition, Proposition 3.9 identifies the reflectors with those of the bidiagonalisation of Golub and Kahan. For the symmetric and skew-symmetric loops, the same computation, with the loop in place of a and b, shows that they are the classical Householder reflectors of W; for instance, for the skew loop a column hy

has $(hy)^*(hy) = -|y|^2h^2$, with the graded root $|y|\,h$. For the other quivers of the catalogue except the congruence quiver, every step was defined in the experiments of Chapter 8, with U of degree 0 and T tridiagonal (Table 8.2). Over the congruence quiver every column with radical entries is isotropic (Section 6.2.8), and Householder reduction stops at the first non-zero column.

The QR algorithm can also be identified. For an invertible matrix W over $D \in \{\mathbb{R}, \mathbb{C}, \mathbb{H}\}$, let $W = Q_W R_W$ be its QR factorisation, with R_W upper triangular with positive real diagonal. Both this factorisation and the Cholesky factorisation $G = LL^*$ of a positive definite Hermitian matrix, with L lower triangular with positive real diagonal, are unique. One step of the *Cholesky LR algorithm*, a variant of Rutishauser's LR algorithm [75], maps $G = LL^*$ to L^*L.

Proposition 6.5. *Run one step $H = QR \mapsto H' = RQ$ of the QR algorithm over Λ, with Q and R computed by the Gram–Schmidt process with graded square roots whose coefficients are principal roots, such as $\gamma = |\hat{y}|\,a$ below.*

1. *For the symmetric and skew-symmetric loops and the Hermitian loops over $\mathbb{C}$ and $\mathbb{H}$, let $H = xW$ with W invertible, where x is the loop. Then $H' = x\,R_W Q_W$: the step is the classical unshifted QR step on W.*

2. *For the singular value decomposition over D, with the positions in cyclic order, let H encode an invertible square W. Then Q is homogeneous of degree 0 and exchanges the orbits u and v, and H' encodes $W_1 = R_W Q_{W^*}$, with the roles of u and v exchanged. Moreover $W_1^*W_1 = L^*L$ and $W_1W_1^* = L'^*L'$, where $WW^* = LL^*$ and $W^*W = L'L'^*$: the step is one step of the Cholesky LR algorithm on each Gram matrix. Two steps give W_2 with $W_2^*W_2 = R_1Q_1$ and $W_2W_2^* = R_2Q_2$, where $W^*W = Q_1R_1$ and $WW^* = Q_2R_2$: one step of the unshifted QR algorithm on each Gram matrix.*

3. *For the Takagi factorisation, let H encode an invertible complex symmetric W. Then H' encodes $W_1 = R_W\overline{Q_W} = Q_W^*W\overline{Q_W}$, a unitary congruence of W, and $W_1^*W_1 = L^*L$, where $WW^* = LL^*$. Two steps give W_2 with $W_2W_2^* = R_2Q_2$, where*

$$WW^* = Q_2 R_2.$$

Proof. We prove (2) for $D = \mathbb{R}$; over $\mathbb{C}$ and $\mathbb{H}$ the transposes become conjugate transposes. The columns of H at the positions u_j and v_i are supported on the positions of v and of u respectively, so they are orthogonal, and the Gram–Schmidt process treats the columns of u and of v separately. The column of u_j is $a\,(We_j)$, with the entries $w_{ij}a$ at the positions v_i. For columns ay and ay' with real y and y' we have $(ay')^*(ay) = (y'^T y)\,ba$. So a residual $a\hat{y} \neq 0$ has the graded root $\gamma = |\hat{y}|\,a$, with $\gamma^{-1} = |\hat{y}|^{-1}a^{-1}$, and the normalised column is $(\hat{y}/|\hat{y}|)\,e_v$, because $aa^{-1} = e_v$. Hence the Gram–Schmidt process on the columns of u is the classical one on the columns of W, with the residuals multiplied by a and the normalised columns by e_v: the column of Q at the position u_j has the entries $(Q_W)_{ij}e_v$ at the positions v_i, and R has the entries $(R_W)_{kj}a$ at the positions (u_k, u_j). In the same way the columns of v give the entries $(Q_{W^T})_{ji}e_u$ of Q at the positions (u_j, v_i) and the entries $(R_{W^T})_{ki}b$ of R at the positions (v_k, v_i). Thus Q is homogeneous of degree 0, and after the step the positions u_k carry the idempotent e_v and the positions v_i carry e_u. The product RQ has the entries $(R_W Q_{W^T})_{ki}a$ at the positions (u_k, v_i), their adjoints at (v_i, u_k), and no others, which proves the first claim. From $W^T = Q_{W^T}R_{W^T}$ we get $WQ_{W^T} = R_{W^T}^T$, so

$$W_1^T W_1 = Q_{W^T}^T W^T W Q_{W^T} = R_{W^T}R_{W^T}^T, \qquad WW^T = R_{W^T}^T R_{W^T} = LL^T \text{ with } L = R_{W^T}^T.$$

Similarly $W_1 W_1^T = R_W R_W^T$ and $W^T W = R_W^T R_W$. A second step applies this to W_1, so $W_2^T W_2$ is the result of two Cholesky LR steps on $W^T W$, and $W_2 W_2^T$ of two on WW^T. Finally, let $G = LL^T$ be positive definite, $L^T L = L_1 L_1^T$, and $G = Q_G R_G$. Then $G^2 = (LL_1)(LL_1)^T$ and $G^2 = G^T G = R_G^T R_G$, so $R_G = (LL_1)^T$ by the uniqueness of the Cholesky factorisation, and

$$L_1^T L_1 = (LL_1)^{-1} G\,(LL_1) = R_G^{-T} G R_G^T = R_G G R_G^{-1} = R_G Q_G,$$

where the third equality holds because $R_G^T R_G = G^2$ commutes with G.

(1) This is the same computation, with the loop in place of a and b and without the exchange of orbits. For the skew loop h, a residual $h\hat{y}$ has $(h\hat{y})^*(h\hat{y}) = -|\hat{y}|^2 h^2$ and the

graded root $|\hat{y}|\,h$.

(3) The real elements of degrees 0 and 1 are $s(z) = ze_v + \bar{z}e_{\bar{v}}$ and $m(z) = z\alpha + \bar{z}\bar{\alpha}$ with $z \in \mathbb{C}$, and $s(z)m(w) = m(\bar{z}w)$, $m(w)s(z) = m(wz)$, $m(w)^* = m(w)$ and $m(w)^*m(w') = \bar{w}w'\,\bar{\alpha}\alpha + w\bar{w}'\,\alpha\bar{\alpha}$. Hence the Gram–Schmidt process on the columns of $H = (m(w_{ij}))$ is the classical one on the columns of W. A residual $m(\hat{y})$ has the graded root $|\hat{y}|(\alpha + \bar{\alpha})$, whose inverse is $|\hat{y}|^{-1}(\alpha^{-1} + \bar{\alpha}^{-1})$, and the normalised column is $s(\bar{\hat{y}}/|\hat{y}|)$. So $Q = (s(\overline{(Q_W)_{ij}}))$, $R = (m((R_W)_{ij}))$, and $RQ = (m((R_W\overline{Q_W})_{ij}))$. Since $W = W^T = R_W^T Q_W^T$ and $Q_W^T\overline{Q_W} = I$, we get $WW^* = R_W^T\overline{R_W} = LL^*$ with $L = R_W^T$, and $W_1^*W_1 = Q_W^T\,\overline{W}W\,\overline{Q_W} = \overline{R_W}R_W^T = L^*L$. The matrix W_1 is again symmetric, so $W_1W_1^* = \overline{W_1^*W_1} = \overline{L^*L}$. Applying the first statement to W_1 and conjugating, $W_2W_2^*$ is the result of two Cholesky LR steps on WW^*, and the last part of the proof of (2) applies. □

The last step of the proof of (2), that two steps of the Cholesky LR algorithm on a positive definite matrix are one step of the QR algorithm, is classical; see [91] and the references there. What the proposition adds is that a single step of the generic QR algorithm, for the quiver of the singular value decomposition or of the Takagi factorisation, is a Cholesky LR step on the Gram matrices. For the singular value decomposition, part (2) can also be derived without $*$-algebras. Read as a real or complex matrix, with the positions interleaved, H is the Hermitian matrix $\left(\begin{smallmatrix} 0 & W^* \\ W & 0 \end{smallmatrix}\right)$ with its rows and columns interleaved, the generic step is the classical QR step on it, and its square is $\mathrm{diag}(W^*W, WW^*)$, interleaved in the same way; and if two unshifted QR steps take a matrix G to G_2, then one QR step takes G^2 to G_2^2. Part (2) also explains the role of the double steps in the experiments of Chapter 8. A single step exchanges the two orbits: QQ^* is the identity, but Q^*Q is the identity of the swapped indexing, so the step is a unitary transformation from one indexing to the other rather than a unitary similarity over A. Only a double step returns to the original positions, where it performs one step of the unshifted QR algorithm on W^*W and on WW^*, and so converges to the singular values.

The split-complex numbers ${}^2\mathbb{R}$ give a different relation between the QR and LR al-

gorithms, over a different $*$-algebra. In degree 0, the algebra of the real singular value decomposition and ${}^2\mathbb{R}$ are both $\mathbb{R} \times \mathbb{R}$, but the involution of the former fixes the two idempotents, so that its unitary group is $\mathrm{O}(n) \times \mathrm{O}(m)$, whereas that of ${}^2\mathbb{R}$ exchanges them, so that its unitary group is $\mathrm{GL}_n(\mathbb{R})$. Over ${}^2\mathbb{R}$, the algorithms become elimination algorithms, and the QR algorithm becomes the LR algorithm on an invertible real matrix, whenever the steps are defined, rather than on a positive definite Gram matrix. We state this for ${}^2\mathbb{R}$ itself, which encodes similarity of real matrices in degree 0 (Section 1.3.2).

Proposition 6.6. *Let $X \in \mathrm{GL}_n(\mathbb{R})$, let $H = (X, X^T)$ be the corresponding Hermitian matrix over ${}^2\mathbb{R}$, and use the $*$-square roots $(c, 1)$ of the self-adjoint elements (c, c).*

1. *The Cholesky factorisation $H = R^*R$, with $R = (R_1, R_2)$ upper triangular, is defined if and only if the leading principal minors of X are non-zero, and then $X = R_2^T R_1$ is the Doolittle factorisation of X, with R_2^T unit lower triangular.*

2. *One step $H = QR \mapsto RQ$ of the QR algorithm, with the Gram–Schmidt process, is defined if and only if the leading principal minors of X^2 are non-zero. It then maps X to UXU^{-1}, where $X^2 = LU$ is the Doolittle factorisation. If the two steps $X = L_1U_1 \mapsto U_1L_1 = L_2U_2 \mapsto U_2L_2$ of the LR algorithm are defined, then $U_2L_2 = UXU^{-1}$.*

With the roots $(1, c)$ instead, R changes by a diagonal factor: (1) gives the Crout factorisation, and the result of (2) changes by a diagonal similarity. Statement (2), with this remark, and the tridiagonalisation by similarity of Section 1.3.2 carry over to the similarity quiver of Section 6.2.9, with $H = xX + x^\top X^T$. Statement (1) carries over only with roots that are not homogeneous: the pivots $c\,(x + x^\top)$ of this H have odd degree, so the graded rule finds no root, but $c\,x + e_{v^\top}$ is a root of $c\,(x + x^\top)$, and with these roots $H = R^*R$ with $R = xU + e_{v^\top}L^T$, where $X = LU$ is the Doolittle factorisation.

Proof. (1) For $R = (R_1, R_2)$ we have $R^* = (R_2^T, R_1^T)$, so $R^*R = (R_2^T R_1, R_1^T R_2)$, and $H = R^*R$ says that $X = R_2^T R_1$. The factorisation takes as diagonal entries of R the roots $(c, 1)$ of the pivots (c, c), so R_2 has unit diagonal. A pivot can be inverted if and only if $c \neq 0$, and the pivots are the ratios of consecutive leading principal minors of X.

(2) Since H is invertible, no residual of the Gram–Schmidt process vanishes, and the process is defined if and only if the Cholesky factorisation of H^*H is, with the same roots. Its factor is R, because $R^*R = H^*H$. Now $H^*H = H^2 = (X^2, (X^2)^T)$, so by (1) the step is defined if and only if the leading principal minors of X^2 are non-zero, and then $X^2 = R_2^T R_1 = LU$. The first component of $RQ = RHR^{-1}$ is $R_1 X R_1^{-1} = UXU^{-1}$. If the two LR steps are defined, then $X^2 = L_1U_1L_1U_1 = L_1L_2U_2U_1$, so $L = L_1L_2$ and $U = U_2U_1$ by the uniqueness of the Doolittle factorisation, and $U_2L_2 = U_2U_1XU_1^{-1}U_2^{-1} = UXU^{-1}$.

With the roots $(1, c)$, R_1 has unit diagonal instead, and R is multiplied by a diagonal matrix. For the similarity quiver, the real elements $p\,x^k + q\,(x^\top)^k$ with $p, q \in \mathbb{R}$ multiply and are adjoint to each other like the elements $(p, q)\,t^k$ of ${}^2\mathbb{R}((t))$, because $xx^\top = x^\top x = 0$ and $x^* = x^\top$. In the QR step and in Householder reduction, the elements whose roots are taken, v^*v for columns v of degree 1, have degree 2, and their graded roots have degree 1. So every computation of the QR step and of Householder reduction is the one over ${}^2\mathbb{R}$, with the quantities of degree k multiplied by the corresponding powers of x and $x^\top$. □

The proposition makes precise the remark of Section 1.3.2 that the generalised algorithm can break down although the classical problem has a solution: the QR step breaks down exactly at the isotropic vectors, that is, when a leading principal minor of X^2 vanishes. It can also succeed where the LR algorithm fails: for $X = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ the LR algorithm stops at once, whereas $X^2 = I$, and the QR step is defined.

6.4 Beyond Degree One: Perturbation Theory

Theorem 6.1 concerns Hermitian matrices of degree 1. For a general Hermitian matrix $H = H_0 + H_1$ over $B = B_Q$, with $H_0 \in \mathrm{Herm}_n(S)$ and $H_1 \in \mathrm{Herm}_n(R)$, the radical part U_1 of a unitary matrix also acts, and the problem becomes one of first-order perturbation theory. We illustrate this with the dual numbers, where the conclusion is the spectral theorem of [39].

Proposition 6.7. *Let $B = \mathbb{R}[\varepsilon]/(\varepsilon^2)$ with the trivial involution. Every $H = X + \varepsilon W \in \mathrm{Herm}_n(B)$ is unitarily similar to a diagonal matrix $\mathrm{diag}(\lambda_i + \varepsilon\mu_i)$ with $\lambda_i, \mu_i \in \mathbb{R}$.*

Table 6.2: What the generic algorithms compute on the Hermitian matrices of degree 1 encoded by the quivers of Table 6.1, and over ${}^2\mathbb{R}$. The statements are proved in Propositions 3.9, 6.4, 6.5 and 6.6 under their hypotheses: every step defined, W square and invertible for the QR steps, and the LR steps defined; for the real loops under Householder reduction, see the remark after Proposition 6.4. That every step is defined, the bidiagonalisation over $\mathbb{C}$ and $\mathbb{H}$, Lanczos and LDL^* were observed in the experiments of Table 8.2.

Quiver	**Householder reduction**	**QR algorithm**	**Other**
symmetric and skew loops	classical Householder tridiagonalisation of W	one step: the classical QR step on W	
Hermitian loops, $\mathbb{C}$ and $\mathbb{H}$	tridiagonalisation by unitary similarity	one step: the classical QR step on W	
SVD over $\mathbb{R}$, $\mathbb{C}$, $\mathbb{H}$	bidiagonalisation; over $\mathbb{R}$, that of Golub and Kahan	two steps: a QR step on W^*W and on WW^*; one step: a Cholesky LR step on each	Lanczos: the Golub–Kahan–Lanczos process
Takagi, Youla	tridiagonalisation by unitary congruence	Takagi, two steps: a QR step on WW^*; one step: $W \mapsto Q_W^* W \overline{Q_W}$, a Cholesky LR step on WW^*	
congruence	stops: $x^*x = 0$	stops: $x^*x = 0$	LDL^*: LDL^T with the inertia of W
similarity, and ${}^2\mathbb{R}$	tridiagonalisation by similarity	one step: $X \mapsto UXU^{-1}$ with $X^2 = LU$, which is two LR steps	Cholesky over ${}^2\mathbb{R}$: Doolittle LU

Proof. The unitary matrices are $U = P(I + \varepsilon K)$ with $P \in \mathrm{O}(n)$ and $K^T = -K$, since $(I + \varepsilon K)^T(I + \varepsilon K) = I + \varepsilon(K^T + K)$. First choose P with $P^T X P = \Lambda = \mathrm{diag}(\lambda_i)$, grouping equal eigenvalues together, and replace W by $P^T W P$. Next, conjugating by $I + \varepsilon K$ gives

$$(I - \varepsilon K)(\Lambda + \varepsilon W)(I + \varepsilon K) = \Lambda + \varepsilon\,(W + \Lambda K - K\Lambda),$$

and $(\Lambda K - K\Lambda)_{ij} = (\lambda_i - \lambda_j)k_{ij}$. Choosing $k_{ij} = -w_{ij}/(\lambda_i - \lambda_j)$ whenever $\lambda_i \neq \lambda_j$, which defines a skew-symmetric K because W is symmetric, removes all entries of W connecting distinct eigenvalues. Finally, the remaining diagonal blocks of W, one for each

eigenvalue of X, are diagonalised by an orthogonal matrix that is block diagonal and therefore commutes with Λ. $\square$

The proof shows the general pattern, which the example of Section 3.4 also exhibits. The part of degree 0, H_0, is first brought into normal form by the classical groups at the vertices. The radical part of U then removes the components of H_1 that connect distinct "eigenvalues" of H_0, by solving Sylvester equations. What remains is a problem of degree 1, governed by Theorem 6.1, for the stabiliser of the normal form of H_0. For the dual numbers this is the familiar statement that the first-order perturbations of a multiple eigenvalue are the eigenvalues of the compression of the perturbation to its eigenspace [55].

Algorithmically, the proof requires knowing which eigenvalues of H_0 coincide. Iterative algorithms such as the Jacobi method do not know this in advance. When they meet two diagonal entries with the same part of degree 0, they must divide the corresponding differences, which are zero divisors, as in the example after Example 3.4. This is the division problem. Chapter 7 solves it by computing in Λ. It can also be avoided, as in the proof above, by first computing the eigendecomposition of H_0, deciding which eigenvalues are equal, exactly or up to a tolerance, and then applying classical algorithms to the corresponding blocks of H_1.

6.5 Representation Type and the Behaviour of Generalised Algorithms

Theorem 6.1 allows a count of degrees of freedom, analogous to the Tits form of Section 5.1.1, for each decomposition. The number of continuous parameters of a normal form equals

$$\dim(\text{matrix space}) - \dim(\text{group}) + \dim(\text{stabiliser of a generic element}).$$

For the real SVD of $n \times n$ matrices the generic stabiliser is finite, and the count is $n^2 - 2 \cdot \frac{n(n-1)}{2} = n$, the number of singular values; for the Takagi factorisation it is

$n(n+1) - n^2 = n$, and for the symmetric eigendecomposition $\frac{n(n+1)}{2} - \frac{n(n-1)}{2} = n$. For the complex Hermitian eigendecomposition the stabiliser of a generic diagonal matrix is the torus $\mathrm{U}(1)^n$, so the count is $n^2 - n^2 + n = n$. In all the examples of Table 6.1, the number of parameters grows at most linearly in n.

Consider instead a vertex of type $(\mathbb{R}, +1)$ with *two* loops x and y, both fixed with sign $+1$. Then $B = \mathbb{R}[x, y]/(x, y)^2$ with $x^* = x$ and $y^* = y$, a Hermitian matrix of degree 1 is $xW_1 + yW_2$ with W_1, W_2 symmetric, and the group $\mathrm{O}(n)$ acts by simultaneous orthogonal similarity. Now

$$\dim\{(W_1, W_2)\} - \dim \mathrm{O}(n) = n(n+1) - \tfrac{n(n-1)}{2} = \tfrac{n(n+3)}{2},$$

which is a lower bound for the number of continuous parameters and grows quadratically in n. This quadratic growth is the hallmark of a wild problem, and the underlying quiver (one vertex with two loops) is indeed wild.

What does a generalised algorithm do in such a case? A Jacobi-type method for the symmetric eigenproblem, applied to $xW_1 + yW_2$, has to divide elements such as $xw_{1,ij} + yw_{2,ij}$ by elements such as $x\alpha + y\beta$. Unlike the quotients needed for the catalogue, these do not exist in the completed localisation Λ: if $\alpha, \beta \neq 0$, then $x\alpha + y\beta$ is not invertible in Λ. Indeed, it is homogeneous of degree 1, so an inverse would be homogeneous of degree -1, as in the proof of Proposition 6.4, and would therefore lie in $kQ_{\mathrm{loc}} = k[F_2]$ (Proposition 7.2). But the homomorphism from $k[F_2]$ to the ring of Laurent polynomials $k[s^{\pm 1}, t^{\pm 1}]$ that abelianises F_2 maps $x\alpha + y\beta$ to $s\alpha + t\beta$, which is not a unit, since the units of that ring are the non-zero multiples of monomials. Here the condition of Proposition 7.6 fails, since $r = 2$, and the elimination of Section 7.4 stops, as the implementation confirms. The quotients do exist in every skew field that contains the path algebra $k\langle x, y\rangle$, for example in Cohn's free field, and no such skew field is commutative (Section 7.7). Whether the Jacobi method can then be completed is a further question: its rotations also need $*$-square roots, of elements such as $1 + t^*t$, and we have not shown that these exist in the free field. If they do, the output, a "diagonalisation", has entries in the skew field that need not lie in $\widehat{kQ}$, so Theorem 7.8 gives no reduction to B, and the output lies outside

the domain of the original decomposition. We have not run such an algorithm, and we do not attempt a general theorem about wild cases. The following proposition shows that a vertex on two distinct simple oriented cycles, the situation in which a non-commutative skew field is unavoidable, always gives a wild quiver.

Proposition 6.8. *If some vertex of a connected quiver Q lies on two distinct simple oriented cycles, then Q is wild.*

Proof. Two distinct simple oriented cycles have distinct sets of arrows, so the underlying graph of Q contains two independent cycles, and its cycle rank is at least 2. By Section 5.1.1, Q is wild. □

The converse fails: the Kronecker quiver with three arrows is acyclic and wild. Thus the quivers for which a commutative coefficient field suffices in Chapter 7 include all tame quivers but also some wild ones. The completed localisation is a narrower tool. Its elimination is guaranteed to be complete only under the condition of Proposition 7.6, and without it the elimination can stop on an invertible element, even for a tame quiver: the quiver of Example 7.7 is the Kronecker quiver with two arrows.

Chapter 7

Localisation (Adjoining Inverse Arrows) and Completion of Path Algebras

Chapters 3 and 6 showed that generalised algorithms must divide by zero divisors and normalise vectors whose norms vanish. Examples are the rotation angle $2\varepsilon b/(\varepsilon(a-c))$ of the Jacobi method over the dual numbers (Section 3.2) and the Householder reflectors for the singular value decomposition (Section 3.4). This chapter constructs the algebra in which the algorithms of this thesis are run, and in which division and the $*$-square root are carried out. The encoded decompositions do not require it: for each quiver of the catalogue, the encoded problem is a classical matrix problem (Theorem 6.1 and Section 6.2), which classical algorithms solve directly, such as the bidiagonalisation of Golub and Kahan [33] for the singular value decomposition. The algebra constructed here serves to run one program, written once, over the algebras of all these quivers. Starting from a quotient kQ/I of a path algebra, it is obtained in three steps: lift to the path algebra kQ (Section 7.1), invert every arrow between its end points (Section 7.2), and complete in the path degree (Section 7.3). The result, the *completed localisation* Λ, keeps the involution and the conjugation of a doubly-signed quiver. Sections 7.4 and 7.5 show how to invert elements and take $*$-square roots in Λ, and Section 7.6 shows when results return to kQ/I. Section 7.7 looks beyond Λ, at embeddings of kQ in simple Artinian rings, in which every non-zero element between two vertices is invertible; a non-commutative

skew field is needed exactly when a vertex lies on two distinct simple oriented cycles. Throughout, k is a field. For a doubly-signed quiver, $k = \mathbb{C}$, and the constructions are applied to its complexified path $*$-algebra (Definition 5.13).

7.1 The Division Problem

Let Q be a quiver and I an ideal of kQ with $R_Q^m \subseteq I \subseteq R_Q^2$ for some m, so that kQ/I is a finite-dimensional basic algebra with quiver Q (Section 2.6). By Theorem 5.16, every finite-dimensional real $*$-algebra is a matrix algebra, indexed by positions (Section 3.1), over a quotient $\mathbb{R}Q/I$ of the path $*$-algebra of its doubly-signed quiver, so it suffices to compute in such quotients. We describe the constructions for path algebras kQ; for a path $*$-algebra they are applied to its complexification, which is a block-matrix version of $\mathbb{C}Q$ (see the remarks after Proposition 7.2). In kQ/I every arrow is nilpotent, and no non-zero element of the radical is invertible between idempotents (Section 3.2). We therefore first *lift* elements to the path algebra kQ, by a linear section of the quotient map; for instance, choose paths of length less than m whose residues form a basis of kQ/I, and send each residue to its path. In kQ, products of arrows no longer vanish: the norm x^*x of Section 3.4 becomes a non-zero element of degree 2. Arrows are still not invertible in kQ, so we make each of them invertible between its own end points, in the sense of Definition 3.5.

This chapter considers two ways to do this; Leavitt path algebras and graph C^*-algebras adjoin partial inverses of the arrows instead (Section 1.5). The first adjoins inverses of the arrows and completes the result (Sections 7.2 and 7.3). It is universal, it keeps the involution, its arithmetic is effective, and it is what the implementation of Chapter 8 uses. When every connected component of Q has at most one cycle and that cycle has non-zero degree, it already yields matrix rings over k or over fields of Laurent series (Proposition 7.6). The second embeds kQ in a simple Artinian ring $M_n(D)$, in which every non-zero element between two vertices is invertible (Section 7.7); for general quivers this requires a non-commutative skew field D, and no algorithm of this thesis uses it.

7.2 Localisation at the Arrows

Definition 7.1. Let Q be a quiver. The *localisation of kQ at the arrows* is the k-algebra kQ_{loc} obtained from kQ by adjoining, for every arrow $\alpha : v \to w$, an element α^{-1} subject to the relations

$$\alpha^{-1} = e_v\,\alpha^{-1} e_w, \qquad \alpha^{-1}\alpha = e_v, \qquad \alpha\alpha^{-1} = e_w.$$

Thus every arrow $\alpha : v \to w$ becomes invertible from e_v to e_w, with inverse α^{-1}, and nothing else is inverted. By construction, kQ_{loc} has the following universal property: every homomorphism φ from kQ to a ring R such that $\varphi(\alpha)$ is invertible from $\varphi(e_v)$ to $\varphi(e_w)$ for every arrow $\alpha : v \to w$ extends uniquely to a homomorphism $kQ_{\mathrm{loc}} \to R$. In the language of Cohn [16], kQ_{loc} is the universal localisation of kQ at the homomorphisms of projective modules $kQe_w \to kQe_v$, $y \mapsto y\alpha$, given by the arrows. In the language of Gabriel and Zisman [32], it is the algebra of the category obtained from the path category of Q by inverting all arrows, that is, of the free groupoid on Q. Each arrow is inverted only between its own end points: if two arrows α, β end at the same vertex w, then $\alpha\alpha^{-1} = \beta\beta^{-1} = e_w$.

A *word* is a product of letters α and α^{-1} ($\alpha \in Q_1$) whose end points match, or a trivial word e_v. It is *reduced* if it contains no factor $\alpha\alpha^{-1}$ or $\alpha^{-1}\alpha$, and its *degree* is the number of its letters α minus the number of its letters α^{-1}. The paths are the reduced words without letters α^{-1}, and their degree is their length. If Q is connected and has cycle rank 1, the *degree of its cycle* is the degree of a reduced word that goes once around the cycle of its underlying graph; it is defined up to sign. For an oriented cycle with n arrows it is $\pm n$, and for the Kronecker quiver $u \rightrightarrows u'$ with arrows a and b it is 0, the degree of $a^{-1}b$.

Proposition 7.2. *Let Q be a connected quiver with n vertices and cycle rank r (Section 5.1.1). Fix a vertex o and a spanning tree T of the underlying graph, and let τ_v be the reduced word along T from o to v. Let F_r be the free group on symbols ξ_α, one for each of the r arrows α not in T.*

1. *There is an isomorphism $\Phi : kQ_{\mathrm{loc}} \to M_n(k[F_r])$ with $\Phi(e_v) = E_{vv}$, $\Phi(\alpha) = E_{wv}$ for $\alpha : v \to w$ in T, and $\Phi(\alpha) = \xi_\alpha E_{wv}$ for $\alpha : v \to w$ not in T.*

2. *The reduced words form a basis of kQ_{loc}, and Φ maps a reduced word $\omega : v \to w$ to gE_{wv}, where $g \in F_r$ is obtained from ω by deleting the letters of T and replacing $\alpha^{\pm 1}$ by $\xi_\alpha^{\pm 1}$. In particular, the product of two reduced words is the reduced form of their concatenation, or zero if their end points do not match, and $kQ \to kQ_{\mathrm{loc}}$ is injective.*

Proof. (1) The proposed images of the e_v and of the arrows satisfy the relations of kQ, and each $\Phi(\alpha)$ is invertible from E_{vv} to E_{ww}, with inverse E_{vw} or $\xi_\alpha^{-1}E_{vw}$. By the universal properties of kQ and of kQ_{loc}, they define a homomorphism Φ. Since τ_v consists of letters of T, $\Phi(\tau_v) = E_{vo}$. For an arrow $\alpha : v \to w$ not in T, the element $\tau_w^{-1}\alpha\tau_v$ of $e_o kQ_{\mathrm{loc}} e_o$ is invertible there, with inverse $\tau_v^{-1}\alpha^{-1}\tau_w$. Hence $\xi_\alpha \mapsto \tau_w^{-1}\alpha\tau_v$ extends to a homomorphism from F_r to the group of units of $e_o kQ_{\mathrm{loc}} e_o$, and to an algebra homomorphism $\psi : k[F_r] \to e_o kQ_{\mathrm{loc}} e_o$. Put $\Psi(gE_{wv}) = \tau_w \psi(g) \tau_v^{-1}$. Since $\tau_v^{-1}\tau_{v'} = \delta_{vv'}e_o$, Ψ is an algebra homomorphism. For an arrow $\alpha : v \to w$ in T we have $\tau_w = \alpha\tau_v$ or $\tau_v = \alpha^{-1}\tau_w$, so $\Psi\Phi(\alpha) = \tau_w\tau_v^{-1} = \alpha$; for α not in T, $\Psi\Phi(\alpha) = \tau_w\tau_w^{-1}\alpha\tau_v\tau_v^{-1} = \alpha$. Conversely, $\Phi\psi(\xi_\alpha) = \xi_\alpha E_{oo}$, so $\Phi\Psi(gE_{wv}) = E_{wo}\, gE_{oo}\, E_{ov} = gE_{wv}$. Hence Φ and Ψ are inverse isomorphisms.

(2) Let $\omega : v \to w$ be a word. Letters of T map to matrix units and the other letters to $\xi_\alpha^{\pm 1}$ times matrix units, so $\Phi(\omega) = gE_{wv}$ with g as described. If ω is reduced, then so is g: a factor $\xi_\alpha^{\varepsilon}\xi_\alpha^{-\varepsilon}$ of g would come from a factor $\alpha^{\varepsilon}\tau\alpha^{-\varepsilon}$ of ω in which τ is a reduced word in the letters of T from a vertex to itself; such a word is trivial because T is a tree, so ω would not be reduced. Conversely, given v, w and a reduced $g \in F_r$, there is exactly one reduced word $\omega : v \to w$ with $\Phi(\omega) = gE_{wv}$: its letters outside T are prescribed by g, and between them, and at both ends, it must follow the unique reduced paths in T. Hence Φ maps the reduced words bijectively onto the basis $\{gE_{wv}\}$ of $M_n(k[F_r])$. The remaining statements follow, since paths are reduced words. □

For one vertex with one loop x, $kQ_{\mathrm{loc}} = k[x, x^{-1}]$. For the quiver of Section 3.4, with $T = \{a\}$ and $o = u$, the cycle gives $\tau_u^{-1}b\tau_v = ba$, and $kQ_{\mathrm{loc}} \cong M_2(k[g, g^{-1}])$ with $g = ba$; this is the algebra Λ_0 used there. For an oriented cycle with n vertices, $kQ_{\mathrm{loc}} \cong M_n(k[x, x^{-1}])$, and for one vertex with two loops, kQ_{loc} is the group algebra $k[F_2]$ of the

free group.

For the path $*$-algebra of a doubly-signed quiver we apply the construction to the complexification $T_{S_{\mathbb{C}}}(M_{\mathbb{C}})$ of Definition 5.13. In $T_{S_{\mathbb{C}}}(M_{\mathbb{C}})$ a path from v to w carries a single coefficient block in $M_{d_w \times d_v}(\mathbb{C})$, so that $T_{S_{\mathbb{C}}}(M_{\mathbb{C}})$ is the algebra of block matrices with entries in $M_{d_w \times d_v}(\mathbb{C}) \otimes_{\mathbb{C}} e_w \mathbb{C}Q e_v$. Replacing $\mathbb{C}Q$ by $\mathbb{C}Q_{\mathrm{loc}}$ gives its localisation: its elements are finite sums $\sum_\omega X_\omega \, \omega$ over reduced words $\omega : v \to w$ with $X_\omega \in M_{d_w \times d_v}(\mathbb{C})$, multiplied by $(X\omega)(Y\omega') = (XY)(\omega\omega')$ followed by cancellation. This is how the implementation stores elements (Chapter 8).

Lemma 7.3. *Let θ be an involution of kQ, linear or conjugate-linear, that maps every vertex idempotent to a vertex idempotent and every arrow to a non-zero multiple of an arrow. Then θ extends uniquely to an involution of kQ_{loc}, with $\theta(\alpha^{-1}) = \theta(\alpha)^{-1}$. The same holds for an automorphism of order two, linear or conjugate-linear, that maps vertices to vertices and arrows to non-zero multiples of arrows.*

Proof. Consider an anti-automorphism θ; the other case is the same without passing to opposite rings. Regard θ as a homomorphism from kQ to the opposite ring of kQ_{loc}, which is conjugate-linear if θ is; the universal property applies equally to such homomorphisms. For an arrow $\alpha : v \to w$, $\theta(\alpha) = \lambda\beta$ with $\lambda \neq 0$ and β an arrow from $\theta(e_w)$ to $\theta(e_v)$, which is invertible in kQ_{loc} with inverse $\lambda^{-1}\beta^{-1}$. In the opposite ring, $\lambda\beta$ is therefore invertible from $\theta(e_v)$ to $\theta(e_w)$. By the universal property, θ extends uniquely to an anti-homomorphism $\tilde{\theta}$ of kQ_{loc}, and $\tilde{\theta}(\alpha^{-1}) = \theta(\alpha)^{-1}$. The composite $\tilde{\theta}^2$ is a homomorphism that extends the identity of kQ, so it is the identity by the uniqueness in the universal property. □

For the complexified path $*$-algebra of a doubly-signed quiver, with coefficient blocks, the involution and the conjugation are given on paths by the formulas (5.2) of the block model (Proposition 5.14). We extend them to reduced words by $(\alpha^{-1})^\top = (\alpha^\top)^{-1}$, $\overline{\alpha^{-1}} = \bar{\alpha}^{-1}$, $\kappa(\alpha^{-1}) = \kappa(\alpha)$ and $\mu(\alpha^{-1}) = \mu(\alpha)$. The extended involution reverses products, because κ is multiplicative and cancellation removes pairs of letters $\alpha^{\pm 1}\alpha^{\mp 1}$, for which $\kappa(\alpha)\kappa(\alpha^{-1}) = 1$. It squares to the identity, because on a letter α^{-1} its square is multiplication by the same factor as on α, namely 1. The same holds for the conjugation.

7.3 Completion in the Path Degree

By Proposition 7.2(2), applied to each connected component, kQ_{loc} is graded by the degree of reduced words, since cancellation removes letters in pairs of total degree 0:

$$kQ_{\mathrm{loc}} = \bigoplus_{d\in\mathbb{Z}} (kQ_{\mathrm{loc}})_d, \qquad (kQ_{\mathrm{loc}})_d\,(kQ_{\mathrm{loc}})_{d'} \subseteq (kQ_{\mathrm{loc}})_{d+d'}.$$

Definition 7.4. The *completed localisation* Λ of kQ is the set of formal sums $x = \sum_{d\geq d_0} x_d$ with $x_d \in (kQ_{\mathrm{loc}})_d$ and $d_0 \in \mathbb{Z}$, with componentwise addition and the product $(xy)_d = \sum_{i+j=d} x_i y_j$. The *valuation* $v(x)$ is the least d with $x_d \neq 0$, with $v(0) = \infty$, and $x^\circ = x_{v(x)}$ is the *lowest-degree part* of x.

The sum defining $(xy)_d$ is finite, because $x_i = 0$ for $i < v(x)$ and $y_j = 0$ for $j < v(y)$. Hence Λ is an associative algebra that contains kQ_{loc}, $v(xy) \geq v(x) + v(y)$, and Λ is the direct sum of its corners $e_w \Lambda e_v$. Each homogeneous part x_d is a finite linear combination of reduced words, but x may involve infinitely many degrees. If an involution or a conjugation maps arrows to multiples of arrows, it preserves degrees and extends termwise to Λ.

For one vertex with one loop t, Λ is the field $k((t))$ of formal Laurent series, and for the real path $*$-algebra with $t^* = t$ it is the field $\mathbb{R}((t))$ of Section 4.6, in which sums of squares have square roots (Proposition 4.10). Thus the same construction supplies the field of Laurent series in which Theorem 4.11 computes the Bruhat decomposition, and the inverses of the present chapter; Chapter 8 uses it for both. It is not real closed; the limits of Proposition 4.7 are taken in the real closed field of algebraic Puiseux series, which contains every element of $\mathbb{R}((t))$ obtained from polynomial inputs by field operations and square roots. For the quiver of Section 3.4, $\Lambda \cong M_2(k((g)))$ with $g = ba$ (Proposition 7.6).

In computations, an element of Λ is represented by finitely many homogeneous parts together with a *precision* p: it is known modulo terms of degree at least p, as a power series is known to finitely many terms. The product of x and y is then known to precision $\min(p_x + v(y),\, p_y + v(x))$, and the operations of the next two sections are carried out to a prescribed number of degrees. This makes the arithmetic of Λ effective.

7.4 Inverses in the Completed Localisation

Lemma 7.5. *Let f and g be sums of vertex idempotents, let $x \in g\Lambda f$, and suppose that the lowest-degree part x° of x is invertible from f to g in kQ_{loc}. Then x is invertible from f to g in Λ, with inverse*

$$x^{-1} = \sum_{k\geq 0} u^k (x^\circ)^{-1}, \qquad u = f - (x^\circ)^{-1} x,$$

where $v(u) \geq 1$.

Proof. Let $d = v(x)$ and $y = (x^\circ)^{-1}$. The inverse y of the homogeneous element x° is homogeneous of degree $-d$: its part of degree $-d$ is also an inverse, as one sees by comparing the parts of degree 0 in $yx^\circ = f$ and $x^\circ y = g$, and inverses are unique. Hence $u = -y(x - x^\circ)$ lies in $f\Lambda f$ and has valuation at least $-d + (d+1) = 1$. The series $\sum_k u^k$ converges in Λ, since $v(u^k) \geq k$, and $(f-u)\sum_{k\geq 0} u^k = \sum_{k\geq 0} u^k (f-u) = f$. Since $x = x^\circ y x = x^\circ (f-u)$, we get $x^{-1}x = \sum_k u^k (f-u) = f$ and $xx^{-1} = x^\circ (f-u)\sum_k u^k y = x^\circ y = g$. □

The lemma applies, for example, to the arrows between vertices of equal block size, to $e_v + \alpha$ for a loop α at v, and to Laurent-type elements such as $2\alpha + \alpha\alpha^\top\alpha$, whose lowest-degree part 2α is a single word.

Elimination. Other elements are inverted by Gauss–Jordan elimination. With block sizes 1, an element $x \in g\Lambda f$ is a matrix (x_{wv}) with entries $x_{wv} \in e_w \Lambda e_v$, indexed by the vertices w in g and v in f; with larger block sizes, rows and columns are indexed by a vertex together with a row of its block. An entry can serve as a pivot if its lowest-degree part is a single word with a non-zero coefficient, since it is then invertible by Lemma 7.5. Among such entries the elimination chooses one of lowest degree and, among those, one whose leading coefficient has the largest modulus. It either inverts x from f to g, or it stops because no remaining entry of a column can serve as a pivot. The following proposition says when it cannot stop on an invertible element.

Proposition 7.6. *Let Q be a connected quiver with cycle rank r, and let Λ be the completed localisation of kQ, with arbitrary block sizes. The following are equivalent:*

1. *between any two vertices there is at most one reduced word of each degree;*
2. *$r = 0$, or $r = 1$ and the cycle of Q has non-zero degree.*

If they hold, there is a field K, namely k if $r = 0$ and a field of Laurent series over k if $r = 1$, such that each corner $e_w \Lambda e_v$ is isomorphic to $M_{d_w \times d_v}(K)$, compatibly with multiplication. Moreover, in exact arithmetic, the elimination just described inverts every element of Λ that is invertible between two given sums of vertex idempotents.

Proof. By Proposition 7.2, the reduced words $\omega : v \to w$ correspond bijectively to the elements $g \in F_r$, via $\Phi(\omega) = gE_{wv}$, and $\deg \omega = \deg \tau_w - \deg \tau_v + \varphi(g)$, where $\varphi : F_r \to \mathbb{Z}$ is the homomorphism with $\varphi(\xi_\alpha) = \deg(\tau_w^{-1} \alpha \tau_v)$ for $\alpha : v \to w$. So (1) says exactly that φ is injective. This is impossible for $r \geq 2$, because F_r is not abelian. For $r = 1$ it means that the generator, which corresponds to the cycle, has non-zero degree, and for $r = 0$ it is automatic.

Under (2) with $r = 1$, let ζ be the generator of F_1 of positive degree δ. A reduced word $v \to w$ corresponds to $\zeta^m E_{wv}$ for a unique $m \in \mathbb{Z}$, and its degree $\deg \tau_w - \deg \tau_v + m\delta$ increases with m. Completing in the degree therefore turns $e_w \Lambda e_v$ into the set of series $\sum_{m \geq m_0} X_m \zeta^m$ with $X_m \in M_{d_w \times d_v}(k)$, that is, into $M_{d_w \times d_v}(K)$ with $K = k((\zeta))$, compatibly with multiplication. For $r = 0$, take $K = k$. A non-zero scalar entry from v to w then corresponds to a non-zero element of K, and its lowest-degree part is a single word. So the possible pivots are exactly the non-zero entries, and the elimination is Gauss–Jordan elimination over the field K, which inverts every invertible matrix. □

For every quiver of Table 6.1, each connected component of the complexified quiver satisfies condition (2); the implementation reports the components and their cycle ranks. So does an oriented cycle with n vertices, whose completed localisation is $M_n(k((x)))$. When the condition fails, the elimination can stop on an invertible element.

Example 7.7. Let Q have two vertices u, u' and two arrows $a, b : u \to u'$. This is the complexified quiver of the doubly-signed quiver of Section 6.2.8 with a second arrow fixed

by the transpose, which encodes pairs of real symmetric matrices under simultaneous congruence. Here $r = 1$, but the cycle $a^{-1}b$ has degree 0, and $e_u \Lambda e_u = k[g, g^{-1}]$ with $g = a^{-1}b$. Put $p = e_u + g$ and

$$M = \begin{pmatrix} e_u + p^2 & p^3 + 2p \\ p & e_u + p^2 \end{pmatrix} \in M_2(k[g, g^{-1}]), \qquad \det M = (e_u + p^2)^2 - (p^3 + 2p)p = e_u.$$

Conjugating M by $\mathrm{diag}(e_u, a)$ transports it to the vertices u and u' and gives

$$x = (e_u + p^2) + (p^3 + 2p)\, a^{-1} + a\, p + a(e_u + p^2)a^{-1} \in (e_u + e_{u'})\Lambda(e_u + e_{u'}),$$

which is invertible from $e_u + e_{u'}$ to itself; its inverse is obtained in the same way from M^{-1}. But each of the four entries of x is homogeneous and has at least two words, for instance $ap = a + b$, so no entry can serve as a pivot. The implementation confirms that the elimination stops on x, and that the transported inverse of M is an inverse of x.

7.5 $*$-Square Roots in the Completed Localisation

Let Λ be the completed localisation of the complexified path $*$-algebra of a doubly-signed quiver. A word $\omega : v \to w$ carries a coefficient block in $M_{d_w \times d_v}(\mathbb{C})$, and the involution has the form $(X\omega)^* = \kappa(\omega)\, G_v X^T G_w^{-1}\, \omega^\top$, with invertible matrices G_v and signs $\kappa(\omega) = \pm 1$ (Proposition 5.14). A $*$-square root of a self-adjoint $s \in \Lambda$ is any $x \in \Lambda$ with $x^* x = s$. Roots are not unique (Lemma 3.7), and Householder reflectors work with any of them that lies in the corner of the next position and for which the denominator of Proposition 3.8 is invertible. We compute roots by three rules. Each rule checks its result word by word, below the precision of s, and reports a failure instead of returning a wrong root.

- *The graded rule.* Let s be homogeneous, with words of even length. Each word is cut in the middle, $\omega = \nu'^\top \nu$ with half-words ν, ν' of equal length, and $x = \sum_\nu Z_\nu \nu$ is built from the half-words ν, so that x has half the degree of s. The coefficients of s whose words pass through a middle vertex m form a block matrix N, and the coefficients Z

of x at m solve a congruence $Z^T G_m^{-1} Z = N$ if $m^\top = m$, a factorisation $Z'^T Z = N$ if $m^\top \neq m$, and a Hermitian congruence at vertices of type `Cu`. For real s the solution is made real. A non-homogeneous s is treated like $\sqrt{t^{2k}(a + \cdots)} = t^k \sqrt{a + \cdots}$: its lowest-degree part receives a graded root x_0, and $x = r x_0$ for a principal root r of degree 0.

- *The principal rule.* If the lowest-degree part s_0 of s has degree 0, is vertex-diagonal, and has blocks without eigenvalues in $(-\infty, 0]$, then $x = x_0 + x_1 + \cdots$ is a self-adjoint square root, where x_0 is the principal square root of s_0 at each vertex and the part x_d of degree d solves the Sylvester equation $x_0 x_d + x_d x_0 = s_d - \sum_{0<i<d} x_i x_{d-i}$, which has a unique solution because the spectra of the blocks of x_0 lie in the open right half-plane.

- *The arbitrary rule.* The graded root, multiplied by a random unitary that has terms of positive degree. It serves to test that an algorithm does not depend on the choice of root.

The refusals of the graded rule match the theory. For $s = -1_c$, where 1_c is the idempotent of an orbit c, it looks for a root whose words are trivial, that is, for an element y of the algebra K of the orbit with $y^* y = -1$. Such y exists exactly when K is not of signature type: if it is not, by Lemma 5.12(1), and if K is $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$ with its standard involution, $y^* y = |y|^2 \geq 0$. Accordingly, the graded rule refuses a root of -1 exactly at the vertex types `R`, `Hsp` and `Cu` (Table 5.2), whose unitary groups are compact. Over ${}^2\mathbb{R}$ it offers the roots $(c, 1)$ and $(1, c)$ of a self-adjoint element (c, c), and also $(\sqrt{c}, \sqrt{c})$ if $c > 0$ and $(-\sqrt{|c|}, \sqrt{|c|})$ if $c < 0$; the root $(c, 1)$ is the one used in Section 1.3.2 and in Proposition 6.6. The graded rule also refuses when N has rank greater than d_m, since Z has only d_m rows. It has two known gaps, in which a root exists but is not found: words with inverse letters, where cancellation can hide the half-words, and elements such as $(e_v + \beta)^*(e_v + \beta)$ for an arrow $\beta : w \to v$ with $w \neq v$, whose lowest-degree part does not reach every vertex that the element reaches. We have not proved that the graded rule finds a root of every element $z^* z$ with z homogeneous; in the tests of Chapter 8 it did so

for the Gram elements of every quiver of the catalogue and for 306 elements z^*z with z real and supported on a random path of a random doubly-signed quiver.

7.6 Reducing Back to the Original Algebra

After an algorithm has been executed in Λ, its results must be brought back to kQ/I.

Theorem 7.8. *Let $\widehat{kQ} \subseteq \Lambda$ consist of the elements whose words are all paths, that is, contain no letter α^{-1}.*

1. *$\widehat{kQ}$ is a subring of Λ that contains kQ and is stable under every involution and conjugation that maps arrows to multiples of arrows. An element of $\widehat{kQ}$ is invertible in $\widehat{kQ}$ if and only if its part of degree 0, which lies in the semisimple algebra S spanned by the vertices (with coefficient blocks, $S = \prod_v M_{d_v}(k)$), is invertible in S.*

2. *Let I be an ideal of kQ with $R_Q^m \subseteq I$ for some m. Then the quotient map $kQ \to kQ/I$ extends to a ring homomorphism $\rho_I : \widehat{kQ} \to kQ/I$ that vanishes on the elements of valuation at least m. If I is stable under the involution and the conjugation, then ρ_I commutes with them.*

Proof. (1) A product of two paths is a path or zero, without cancellation, so $\widehat{kQ}$ is closed under multiplication, and it is closed under the involution and the conjugation because these map paths to multiples of paths. Elements of $\widehat{kQ}$ have non-negative valuation. If the part x_0 of degree 0 of $x \in \widehat{kQ}$ is invertible in S, then $x^\circ = x_0$, and Lemma 7.5, with $f = g = 1$, gives the inverse $\sum_k u^k x_0^{-1}$, whose words are paths because those of $u = 1 - x_0^{-1}x$ are. Conversely, if $xy = yx = 1$ in $\widehat{kQ}$, then $x_0y_0 = y_0x_0 = 1$.

(2) The elements of valuation at least m form an ideal $\widehat{kQ}_{\geq m}$ of $\widehat{kQ}$, and truncation identifies $\widehat{kQ}/\widehat{kQ}_{\geq m}$ with kQ/R_Q^m, since the paths of length less than m form a basis of both. Composing with the quotient map $kQ/R_Q^m \to kQ/I$ gives ρ_I. The involution and the conjugation preserve degrees, hence $\widehat{kQ}_{\geq m}$; if they also preserve I, they commute with ρ_I. □

Corollary 7.9. *Let I be as in Theorem 7.8(2) and stable under the involution and the conjugation. Suppose that an algorithm is run in Λ on elements $y_1, \dots, y_k \in \widehat{kQ}$, which lift $a_i = \rho_I(y_i) \in kQ/I$, and that some of the quantities it computes, $z_1, \dots, z_l$, lie in $\widehat{kQ}$. Then every identity among the y_i and the z_j that is built from addition, multiplication, the involution and the conjugation, and that holds in Λ, also holds among the a_i and the $\rho_I(z_j)$ in kQ/I. The same applies entrywise to matrices indexed by positions.*

The corollary is immediate from Theorem 7.8(2). Intermediate quantities need not lie in $\widehat{kQ}$. In the Householder steps of Proposition 3.9, $s^{-1} = c'^{-1}a^{-1}b^{-1}$ has inverse letters, but the column x, the root γ and the reflector P lie in $\widehat{kQ}$, so the identities $P^*P = I$ and $Px = e_1\gamma$, which certify each step, hold in $B = kQ/R_Q^2$. Similarly, the quotient $(xb)/(xc) = b/c$ of Example 3.4, computed in $k((x))$, lies in $\widehat{kQ} = k[[x]]$, and its image b/c in the dual numbers is the intended value. A quantity that does not lie in $\widehat{kQ}$, such as $x/x^2 = x^{-1}$, has no image in kQ/I: if it is an output, the generalised algorithm has produced a result outside the original algebra, which Section 6.5 discusses for wild quivers.

For the loop quiver, Theorem 7.8 takes the following classical form.

Proposition 7.10. *Let $\mathcal{O} \subseteq k(x)$ be the ring of rational functions without a pole at $x = 0$. The map*

$$\mathcal{O} \to k[x]/(x^2), \qquad f \mapsto f(0) + f'(0)\,x,$$

is a surjective ring homomorphism whose restriction to $k[x]$ is the quotient map.

Proof. Taylor expansion at 0 embeds $\mathcal{O}$ in $k[[x]] = \widehat{kQ}$, and $\rho_{(x^2)}$ maps a power series f to $f(0) + f'(0)\,x$. □

Proposition 7.10 is Proposition 4.8 in another guise: if x is a positive infinitesimal in a real closed field, then reduction is first-order Taylor expansion. In this sense, the completed localisation replaces nilpotent infinitesimals by invertible ones: the computation is carried out in a ring in which they are invertible, and the result is truncated at the end.

7.7 Beyond the Completed Localisation: Simple Artinian Rings

When the condition of Proposition 7.6 holds and all block sizes are 1, every non-zero element of Λ between two vertices is invertible. Otherwise this can fail. In Example 7.7, the element $e_u + g$ of $e_u \Lambda e_u = k[g, g^{-1}]$ is not invertible, since the units of that ring are the non-zero multiples of powers of g; for a vertex with two loops x and y, neither is $x\alpha + y\beta$ with non-zero scalars α, β (Section 6.5). Rings in which every non-zero element between two vertices is invertible are provided by simple Artinian rings (Definition 2.7). By the Wedderburn–Artin theorem (Theorem 2.8), such a ring is $M_n(D)$ for a skew field D, and in $M_n(D)$ a non-zero element dE_{ji} is invertible from E_{ii} to E_{jj}, with inverse $d^{-1}E_{ij}$. This section shows that the path algebra of every finite quiver embeds in such a ring, and when D can be taken commutative. No algorithm of this thesis uses these embeddings; they show what lies beyond Λ.

Let Q be a quiver with vertices $1, \ldots, n$ and cycle rank r (Section 5.1.1). Choose a spanning forest T of its underlying graph, and label the r arrows not in T by symbols x_α. Let D_r be k if $r = 0$, the field of rational functions $k(x)$ if $r = 1$, and Cohn's *free field* $k\{x_\alpha : \alpha \notin T\}$ if $r \geq 2$. The free field $k\{X\}$ is the universal skew field of fractions of the free algebra $k\langle X\rangle$: an $m \times m$ matrix over $k\langle X\rangle$ is invertible over $k\{X\}$ if and only if it is *full*, that is, not a product of an $m \times s$ and an $s \times m$ matrix over $k\langle X\rangle$ with $s < m$, and every skew field generated by $k\langle X\rangle$ is a specialisation of $k\{X\}$ [16, Chapter 7]. The group algebra of the free group on X embeds in $k\{X\}$ [64], and the word problem for $k\{X\}$ is decidable [17], so its arithmetic is effective, although much more expensive than that of rational functions.

Theorem 7.11 (Embedding theorem). *1. The assignments*

$$e_v \mapsto E_{vv}, \qquad \alpha \mapsto E_{t(\alpha)s(\alpha)} \ (\alpha \in T), \qquad \alpha \mapsto x_\alpha E_{t(\alpha)s(\alpha)} \ (\alpha \notin T)$$

extend to an injective k-algebra homomorphism $\Phi : kQ \to M_n(D_r)$. Consequently,

every non-zero element of $e_j kQ e_i$ becomes invertible from E_{ii} to E_{jj} in $M_n(D_r)$.

2. *If no vertex of Q lies on two distinct simple oriented cycles, then the same assignments, with the x_α interpreted as* commuting *indeterminates, give an injective homomorphism $kQ \to M_n(k(x_\alpha : \alpha \notin T))$ into a matrix ring over a field of rational functions.*

3. *If some vertex of Q lies on two distinct simple oriented cycles, then there is no injective homomorphism from kQ to a matrix ring $M_N(F)$ over a commutative ring F, for any N.*

The homomorphisms of (1) and (2) are given by explicit formulas, and the arithmetic in their targets is effective; in this sense the embedding is constructive.

Proof. (1) For connected Q, Φ is the composite of the injective map $kQ \to kQ_{\mathrm{loc}} \cong M_n(k[F_r])$ of Proposition 7.2 with the inclusion of $k[F_r]$ in D_r: for $r = 1$ this is $k[x, x^{-1}] \subseteq k(x)$, and for $r \geq 2$ it is Lewin's embedding [64]. For a quiver with several components, apply this to each component, with distinct symbols, and place the blocks along the diagonal. A non-zero element of $e_j kQ e_i$ maps to dE_{ji} with $d \neq 0$, which is invertible from E_{ii} to E_{jj}.

(2) Suppose that no vertex lies on two distinct simple oriented cycles. Then each strongly connected component of Q is a single vertex or a single simple cycle. (Otherwise the component contains a simple cycle Z and, by the ear decomposition of strongly connected digraphs [5], a path P outside Z whose endpoints u, w lie on Z; then u lies on Z and on the simple cycle formed by P and the part of Z from w to u.) The argument below works for every choice of the spanning forest T. With commuting variables, a path p maps to a commutative monomial, which records how often each non-tree arrow is used. We show that this determines p. First, the number of times n_α that p uses each arrow α is determined: two count vectors with the same endpoints and the same non-tree counts differ by a vector supported on T whose boundary is zero, that is, by a cycle in a forest, which must vanish. Second, the counts determine the path: the arrows between different strongly connected components are used at most once each, and the ones used form a

chain in the (acyclic) condensation of Q, which determines the sequence of components visited and the entry and exit vertices; within a cyclic component, the walk from the entry vertex to the exit vertex is determined by the number of arrows it uses. Distinct paths therefore give distinct monomials, which are linearly independent in the field of rational functions.

(3) Let v lie on two distinct simple oriented cycles c_1 and c_2, regarded as elements of $e_v kQ e_v$. A closed path at v factors uniquely into paths that return to v only at their end, and c_1 and c_2 are two different such paths, so distinct words in c_1 and c_2 are distinct paths. Put $y_i = c_1 c_2^i$ for $i \geq 1$. A word in the y_i is a word in c_1 and c_2 from which the y_i can be read off, since each y_i begins with c_1, so distinct words in the y_i are distinct, hence linearly independent, paths. In particular, for every N the standard polynomial

$$s_{2N}(y_1, \ldots, y_{2N}) = \sum_{\sigma \in S_{2N}} \operatorname{sgn}(\sigma)\, y_{\sigma(1)} y_{\sigma(2)} \cdots y_{\sigma(2N)}$$

is non-zero in kQ. By the theorem of Amitsur and Levitzki [2], $s_{2N}(z_1, \ldots, z_{2N}) = 0$ for all $z_1, \ldots, z_{2N}$ in $M_N(F)$, for every commutative ring F. An injective homomorphism $kQ \to M_N(F)$ would therefore map a non-zero element to zero, which is impossible. $\square$

By (2) and (3), a commutative coefficient field suffices exactly when no vertex lies on two distinct simple oriented cycles; otherwise the quiver has cycle rank at least 2 and is wild (Proposition 6.8), and a non-commutative skew field, such as the free field, is unavoidable. When the condition of Proposition 7.6 holds and all block sizes are 1, D_r is k or $k(\zeta)$, which lies in the field K of that proposition, and the canonical map $kQ \to \Lambda \cong M_n(K)$ is Φ followed by the inclusion $M_n(D_r) \subseteq M_n(K)$. For the Kronecker quiver of Example 7.7, with $T = \{a\}$, we get $\Phi(b) = xE_{u'u}$, and the extension of Φ to kQ_{loc} maps $g = a^{-1}b$ to xE_{uu}, so the element $e_u + g$, which is not invertible in Λ, becomes the invertible element $(1 + x)E_{uu}$ of $M_2(k(x))$. Here $k(x)$ inverts an element that Λ cannot, because $g = a^{-1}b$ has degree 0, and the completion in the path degree does not affect the part of degree 0.

Involutions. The embedding Φ ignores the involution and the conjugation of a doubly-signed quiver. For the quivers of the catalogue this does not matter: by Proposition 7.6, Λ is then a finite product of algebras $M_N(K)$ over fields, one for each component of the complexified quiver, and the involution and the conjugation permute the factors. For one vertex with block size 1, the path algebra is the free algebra $k\langle X\rangle$ on the loops, and the free field carries the involution: an involution of $k\langle X\rangle$ is an isomorphism $k\langle X\rangle \to k\langle X\rangle^{\mathrm{op}}$, so, by the uniqueness of the universal skew field of fractions, it extends to an anti-automorphism of $k\{X\}$, whose square is the identity because it fixes the generating subring $k\langle X\rangle$; the same applies to the conjugation.

The same argument applies to every doubly-signed quiver, with the localisation in place of the path algebra. By Proposition 7.2, each connected component of the complexified localisation, with its blocks, is a matrix ring $M_N(\mathbb{C}[F_r])$. The group algebra $\mathbb{C}[F_r]$ is a fir, whose universal skew field of fractions U is $\mathbb{C}$, $\mathbb{C}(x)$ or the free field according as $r = 0$, $r = 1$ or $r \geq 2$ [16, 64], and $M_N(U)$ is the universal localisation of $M_N(\mathbb{C}[F_r])$ at the matrices that are full over $\mathbb{C}[F_r]$ [16, Chapter 7]. Under the isomorphisms of Proposition 7.2, the involution maps the component of a vertex v onto that of $v^\top$ by a map of the form $X \mapsto D\,\psi(X)^T D^{-1}$, where ψ is the anti-isomorphism of group algebras induced by an anti-isomorphism of the free groups, which reverses words and transposes their letters, and D is an invertible matrix whose entries are products of signs, the blocks G_v and elements of the free group. The conjugation has the same form, with an isomorphism of the free groups, complex conjugation of the coefficients and no transpose. Maps of this form send matrices that are full over $\mathbb{C}[F_r]$ to matrices that are full over $\mathbb{C}[F_{r'}]$, since they preserve factorisations through matrices of smaller size, and a matrix over the opposite of a ring is invertible exactly when its transpose is invertible over the ring. So, by the universal property of the localisations $M_N(U)$, the involution and the conjugation extend uniquely to the product of the rings $M_N(U)$, and the identities $*^2 = c^2 = 1$ and $c* = *c$ extend by uniqueness. Hence, after complexification, the localisation of every doubly-signed quiver, and with it its path algebra, embeds compatibly with the involution and the conjugation in a finite product of simple Artinian rings that they permute. Whether

the $*$-square roots that a Jacobi-type method needs exist there is open (Section 9.3).

Chapter 8

Implementation

The seven operations of Section 1.1, and the algorithms written against them, have been implemented as a Python library, `dsqlib`. This chapter describes what the library computes, how its parts correspond to the theory of the preceding chapters, and what it has been used to check.

The library computes in the path $*$-algebra of a doubly-signed quiver (Definition 5.13), extended so that every arrow is invertible and elements may be series in the path degree. In this algebra it forms inverses between vertices (Definition 3.5) and $*$-square roots (Section 3.3). Its matrix algorithms (Householder reduction, the QR algorithm, the Lanczos process, and the Cholesky, LDL^* and LU factorisations) are written once, against the seven operations. Run over the doubly-signed quivers of Chapter 6, they become classical algorithms in the cases stated in Theorem B, which covers Householder reduction, the QR algorithm and the Cholesky factorisation; the experiments of Section 8.5 show the same for the Lanczos process and the LDL^* factorisation. For example, Householder reduction over the quiver of the singular value decomposition is the bidiagonalisation of Golub and Kahan, and one step of the QR algorithm over the split-complex numbers ${}^2\mathbb{R}$ is two steps of Rutishauser's LR algorithm. No algorithm in the library is specific to a decomposition; only the translation of a classical matrix into its encoding, and back, is.

In the library, the seven operations are written `x + y`, `x - y` and `x * y` for elements, with `@` for the product of matrices; `x * y.inverse()`, where `y.inverse()` inverts y between the vertices that it touches; `x.H`, the adjoint of an element and the conjugate

transpose of a matrix; `s.sqrt()`; and `H[i, j]`, the entry of a matrix at its i-th and j-th positions.

The library consists of about 3700 lines of documented Python; its source code is printed in Appendix B, and its test suite, from which the measurements of this chapter are taken, is part of the source distribution submitted with the thesis. It uses floating-point arithmetic and depends only on NumPy and SciPy [46, 86]. Its test suite of 65 tests records every quantity that it measures, and the errors quoted in this chapter are taken from that record. The results of the thesis do not depend on the implementation; the implementation shows that the constructions are effective, and it allows the encoded decompositions to be checked numerically.

8.1 Overview

Table 8.1 lists the components of the library; each builds on those above it. Only `quiver` refers to the vertex types. The other components see only the block data that it produces (Section 8.2), and only the square-root rules use separate formulas for different block data.

Table 8.1: The components of `dsqlib` and the parts of the thesis that they implement.

Module	Contents	Theory
`quiver`	doubly-signed quivers built from the ten vertex types; the block model; the signs of the arrows	Definition 5.7, Proposition 5.14
`algebra`	the path $*$-algebra, localised at the arrows and completed in path degree; inverses between vertices	Definition 5.13, Sections 7.2–7.4
`sqrt`	$*$-square roots by three rules	Sections 3.3 and 7.5
`matrix`	matrices indexed by positions with self-adjoint idempotents	Section 3.1, Definition 5.15
`linalg`	Householder, QR, Lanczos, Cholesky, LDL^* and LU	Propositions 3.8, 3.9, 6.4, 6.5 and 6.6
`catalogue`, `problems`	the quivers of the catalogue and the matrix problems that they encode	Theorem 6.1, Table 6.1

Listing 8.1 builds the doubly-signed quiver of the complex singular value decomposition

```
import numpy as np
from dsqlib import DSQ, Algebra, problems
from dsqlib.linalg import householder

Q = DSQ.build([("u", "Cu"), ("v", "Cu")], [("a", "u", "v")])
A = Algebra(Q)
a = A.arrow("a")
A.word(["a^-1"]) * a                          # 1·e_u
z = A.real_element(A.arrow("a", 1 + 2j))  # (1+2j)·a + (1-2j)·a~
y = (z.H * z).sqrt()                          # 2.236·a + 2.236·a~

W = np.array([[3, 1], [4, 2], [0, 0.3]])
H = problems.svd_problem(W, "R")    # positions u0, v0, u1, v1, v2
T, U, _ = householder(H)            # H = U T U*
U.degree_range()                    # (0, 0)
problems.extract_svd(T, "R").real   # [[-5, 2.2], [0, 0.5], [0, 0]]
```

Listing 8.1: A session with `dsqlib`: arithmetic in the algebra of the complex SVD quiver, and Example 3.10 run by the generic Householder algorithm. The comments show the values computed.

(Section 6.2.4) and computes in its algebra. It then runs Example 3.10: it encodes the real 3×2 matrix W of that example as a Hermitian matrix H (Theorem 6.1), reduces H by the generic Householder algorithm, checks that the unitary factor has degree 0, and decodes the result, the bidiagonal matrix computed by hand in the example. The comments show the values computed; the library numbers positions from 0.

8.2 Doubly-Signed Quivers

A doubly-signed quiver is entered by giving its vertices, each with one of the ten types of Table 5.1, named as in Table 5.2, and some of its arrows. The library works with the complexified quiver, in which each vertex type expands into one, two or four complex vertices: b, its conjugate $\bar{b}$ (written `b~`) and its transpose $b^\top$ (written `b'`). It completes each arrow to its orbit under the transpose and the conjugation, asks for the sign $\varepsilon_{\text{form}}(\alpha)$ of each arrow $\alpha : v \to v^\top$, which it takes to be fixed by the transpose, takes each arrow from v to v^H to be fixed by the conjugate transpose, and checks the rules of Definition 5.7. It then sets up the block model of Proposition 5.14: the block sizes and the matrices G_v and C_v of Table 5.2, and signs κ_α and μ_α that satisfy (5.3), which it finds by a search

over each orbit of arrows; by Proposition 5.14(2), the search always succeeds. On random elements, for every quiver of the catalogue and for 120 random doubly-signed quivers covering all ten vertex types, the axioms of a $*$-algebra with a commuting conjugation hold to a relative error of $8 \cdot 10^{-17}$.

8.3 The Algebra and Its Operations

The module `algebra` implements the completed localisation Λ of Chapter 7 for the complexified path $*$-algebra of a doubly-signed quiver, with the block data of Section 8.2. An element is stored as a dictionary from reduced words to coefficient blocks, together with its precision (Section 7.3). Series inverses and square roots of exact elements are computed to a fixed number of degrees beyond the lowest one. A block in which a sum cancels to within a relative tolerance (10^{-12} by default) of its summands is set to zero, so that zeros of the algebra appear as exact zeros at any scale.

Inverses are computed as in Section 7.4: by the series of Lemma 7.5 when an element lies in a single corner and its lowest-degree part is a single word with an invertible block, and otherwise by elimination with the pivoting described there, which is complete under the condition of Proposition 7.6. When cancellation uses up the precision of an exact input, the series are recomputed with more terms. By default an element is inverted between the vertices that it touches (Section 3.2); inversion between the orbit idempotents that it touches, or in the whole algebra, is also available. Every quotient x/y computed by an algorithm is checked to satisfy $(x/y)\,y = x$, and a breakdown is reported when it does not. The $*$-square roots are computed by the three rules of Section 7.5. The test suite checks the graded rule on the Gram elements of every quiver of the catalogue and on 306 random paths, its refusals at all ten vertex types, and the principal and arbitrary rules on random elements.

Matrices are indexed by positions (Section 3.1). The *plain* indexing gives every position the idempotent 1. The *orbit* indexing gives the position c_k the orbit idempotent 1_c and lists the positions in a cyclic order, which can be chosen. Products and sums check that the algebras and the indexings agree, and a method reports the entries that do not

lie in their corners.

8.4 Algorithms Written Once

The algorithms use only the seven operations of Section 1.1, with division by inverses between vertices and indexing by positions, together with comparisons to choose signs and pivots. The square-root rule and the mode of inversion are parameters of each algorithm. A zero divisor that has to be inverted, or an isotropic vector, that is a vector $v \neq 0$ with $v^*v = 0$, stops an algorithm with an error that reports the breakdown. What the algorithms compute on the encoded matrices is stated, and mostly proved, in Section 6.3 (Table 6.2); here we describe how they are implemented.

Householder reflections. The reflectors are those of Proposition 3.8, with γ taken from the square-root rule and replaced by the root $-\gamma$ when this avoids cancellation in $x_1 - \gamma$. The denominator $s = w^*x$ is inverted in the corner of its column, and a breakdown is reported when it is not invertible there; over ${}^2\mathbb{R}$ this depends on the root (Section 3.3). With the orbit indexing, the library checks that γ lies in the corner of the position to which it is assigned, and it refuses an indexing for which this fails (Section 3.4).

The QR algorithm and other algorithms. A QR step orthogonalises the columns by the Gram–Schmidt process and normalises each residual column v_j by a $*$-square root of $v_j^*v_j$. If H is rank-deficient, the default inversion maps zero to zero, so that Q is a partial isometry and $H = QR$ still holds. The Lanczos process uses full reorthogonalisation. The Cholesky, LDL^* and LU factorisations are the textbook algorithms, with every pivot inverted between vertices and every quotient checked.

8.5 Experiments

Table 8.2 summarises what the generic algorithms compute on the quivers of the catalogue and on ${}^2\mathbb{R}$, for random inputs. Each result was compared with an independent computation (singular values, eigenvalues and QR factorisations from NumPy, LU factorisations

and LR steps from a separate implementation of Gaussian elimination, quaternionic QR factorisations from a separate Gram–Schmidt process) or checked by reconstructing the input from the computed factors. The error shown is the largest one observed over all variants tested. For the Householder reduction on the SVD quiver, the variants are real, complex and quaternionic matrices, square and rectangular shapes, both cyclic orders, and the graded root with and without a random unitary factor of degree 0. For the quivers of Table 6.1 other than the congruence quiver, on which Householder reduction stops (Section 6.2.8), and the similarity quiver, whose computation was compared with the one over ${}^2\mathbb{R}$, the unitary factors of the Householder reduction were also checked to be real, unitary and of degree 0, as Proposition 6.4 predicts, and for the SVD quivers those of the Householder reduction and of the Lanczos process were checked to have every entry in its corner. With the arbitrary rule instead, the unitary factor of the Householder reduction has terms of degrees 0 to 4, and the reduced matrix is still tridiagonal.

The last row checks Theorem 4.11 on one matrix $M = LwU$ with random triangular factors in each of the 24 Bruhat cells of $\mathrm{GL}_4(\mathbb{R})$. In every case the orders of the norms give w, the standard parts give a Bruhat decomposition $M = L'wU'$, and the orthogonal factor of $\mathrm{diag}(1, t, t^2, t^3)M$ tends to the signed permutation matrix of w, with the signs of Theorem 4.11(3). The tests also check the explicit signs of Proposition 5.14(2) on 300 random doubly-signed quivers, the conditions of Lemma 6.3 for the catalogue, and that the elimination refuses to invert $x + 2y$ for the quiver with two loops, as Section 6.5 predicts.

8.6 Limitations

All arithmetic is in floating point, and series are truncated after a fixed number of degrees; exact arithmetic over Puiseux series, in which the auto-limit of Proposition 4.7 is exact, is not provided. Quotients $\mathbb{R}Q/I$ by admissible ideals (Theorem 5.16) are available only for $I = \mathbb{R}Q_{\geq m}$, and the indefinite forms of Remark 6.2 are not implemented. Inversion is complete under the condition of Proposition 7.6 and can fail without it (Example 7.7), and the graded rule has the two gaps described in Section 7.5. The QR algorithm is unshifted

Table 8.2: The classical results reproduced by the generic algorithms of `dsqlib`, with the largest error observed. Numbers in parentheses refer to the sections in which the quivers are described.

Quiver	**Algorithm**	**Classical result**	**Error**
SVD over $\mathbb{R}$, $\mathbb{C}$, $\mathbb{H}$ (6.2.3–6.2.5)	Householder	bidiagonalisation [33]; singular values kept	$5 \cdot 10^{-15}$
	Lanczos	Golub–Kahan–Lanczos bidiagonalisation	$2 \cdot 10^{-15}$
	QR step	two steps: a QR step on W^*W and WW^*; one step: a Cholesky LR step on each (Proposition 6.5)	$9 \cdot 10^{-15}$
	double QR steps	singular values, after 20–28 double steps	$4 \cdot 10^{-15}$
symmetric, skew (6.2.1, 6.2.2)	Householder	tridiagonalisation; eigenvalues kept	$9 \cdot 10^{-16}$
Hermitian over $\mathbb{C}$, $\mathbb{H}$ (6.2.6)	Householder	tridiagonalisation; eigenvalues kept	$7 \cdot 10^{-15}$
symmetric, skew, Hermitian over $\mathbb{C}$, $\mathbb{H}$	QR step	classical QR step (Proposition 6.5)	$2 \cdot 10^{-15}$
Hermitian over $\mathbb{C}$ (6.2.6)	QR iteration	eigenvalues, after 40 steps	$2 \cdot 10^{-14}$
Takagi, Youla (6.2.7)	Householder	tridiagonalisation; Takagi or singular values kept	$5 \cdot 10^{-15}$
Takagi (6.2.7)	QR step	two steps: a QR step on WW^*; one step: $Q_W^* W \overline{Q_W}$, a Cholesky LR step (Proposition 6.5)	$1 \cdot 10^{-15}$
${}^2\mathbb{R}$ (1.3.2)	Householder	tridiagonalisation of a real matrix by similarity	$1 \cdot 10^{-13}$
	QR step	two LR steps [75], relative error (Proposition 6.6)	$6 \cdot 10^{-14}$
	Cholesky	Doolittle LU decomposition (Proposition 6.6)	$2 \cdot 10^{-15}$
similarity (6.2.9)	Householder, QR step	the computations over ${}^2\mathbb{R}$ (Proposition 6.6)	0
congruence (6.2.8)	LDL^*	LDL^T with the inertia of W	$3 \cdot 10^{-16}$
one loop, $\mathbb{R}((t))$ (4.7)	Gram–Schmidt	Bruhat decomposition, all 24 cells for $n = 4$ (Theorem 4.11)	$6 \cdot 10^{-16}$

and has no deflation, and although the experiments show errors at the level of rounding, no analysis of stability has been made (Section 9.3). Finally, the library is written in pure Python over dictionaries of small blocks; a Householder reduction for a 10×10 complex SVD, with 20 positions, takes about one second. Specialised classical routines, such as those of LAPACK that NumPy and SciPy call, compute these decompositions much faster and are better studied; the library is not meant to replace them.

Chapter 9

Conclusion and Future Work

9.1 Summary of Contributions

This thesis has overloaded the seven operations through which numerical linear algebra uses its scalars (addition, subtraction, multiplication, division, the adjoint, the $*$-square root and (i, j) indexing), so that the scalars of an algorithm may come from a $*$-algebra, and it has developed the algebraic theory that makes this work. The outcome is that decompositions are unified, as instances of unitary similarity of Hermitian matrices over the algebras of doubly-signed quivers (Theorem A), and so are several of the algorithms that compute them: one program becomes different classical algorithms over different algebras (Theorem B). Most of the thesis thus reduces to the eigendecomposition of Hermitian matrices over $*$-algebras and to the algorithms that compute it, as Chapter 1 and Section 4.11 explain; none of these reductions is the only route to its results. The encoded problems have scalars in finite-dimensional $*$-algebras, but the generalised algorithms run in the completed localisation of a path $*$-algebra, which is in general infinite-dimensional; their results return to the finite-dimensional algebra when they contain no inverted arrows. We summarise the results by the part of the headline that they serve; Table 9.1 lists the main results with their status.

The seven operations. Addition, subtraction, multiplication and the adjoint are those of the path $*$-algebra of a doubly-signed quiver, which is constructed explicitly, by paths

Table 9.1: The main results of the thesis and their status: proved, observed in the experiments of Chapter 8, or open.

Result	**Where**	**Status**
Classification of basic real $*$-algebras with radical square zero	Theorem 5.11	proved; existence also by an explicit construction (Proposition 5.14(3))
Analogue of Gabriel's theorem (Theorem A(1))	Theorem 5.16	proved
Encoding theorem and catalogue (Theorem A(2), (3))	Theorem 6.1, Section 6.2	proved
Householder reflectors with any $*$-square root	Proposition 3.8	proved
Golub–Kahan bidiagonalisation (Theorem B(1))	Proposition 3.9	proved for real W with $m \geq n$; the other cases observed
Householder reduction in degree 0 (Theorem B(1))	Proposition 6.4	proved when every step is defined; that the steps are defined is observed for most quivers of the catalogue
QR steps for the SVD and Takagi quivers (Theorem B(2))	Proposition 6.5	proved
QR and Cholesky over ${}^2\mathbb{R}$ (Theorem B(3))	Proposition 6.6	proved
Bruhat decomposition (Theorem B(4))	Theorem 4.11	proved
Dijkstra's algorithm as an auto-limit	Proposition 4.18	proved
QR factorisation as a limit of the singular value decomposition	Proposition 4.19	proved
Inverses and reduction in Λ	Proposition 7.6, Theorem 7.8	proved
Embedding in simple Artinian rings, and compatibility with the involution	Theorem 7.11, Section 7.7	proved
Lanczos as Golub–Kahan–Lanczos; LDL^* with the inertia	Table 8.2	observed
Jacobi-type methods on wild quivers	Section 6.5	quotients outside Λ proved for two loops; the rest open

with coefficient blocks of size at most two in which the vertex types enter only through two small matrices at each vertex (Definition 5.13 and Proposition 5.14), and those of its completed localisation. Indexing is by *positions*, each carrying a self-adjoint idempotent, so that the involution acts as the conjugate transpose; the positions are listed in a cyclic order through the orbits of the vertices (Section 3.1), and the structure theorem shows that they are intrinsic (Corollary 5.18). Division is defined between idempotents (Definition 3.5). In a finite-dimensional $*$-algebra, invertibility is decided by the semisimple quotient (Propositions 3.2 and 3.3), and no non-zero element of the radical is invertible between idempotents; this is why the algorithms leave the finite-dimensional algebra. They divide in the completed localisation Λ: the localisation of the path algebra at its arrows, which for a connected quiver is a matrix algebra over the group algebra of a free group (Proposition 7.2) and carries the involution and the conjugation (Lemma 7.3), completed in the path degree. Inverses in Λ are computed by series and by elimination, which is complete when every connected component has cycle rank at most one and its cycle has non-zero degree (Proposition 7.6), and which can fail otherwise, already for 2×2 matrices (Example 7.7). A $*$-square root is not unique (Lemma 3.7), but Householder reflectors work with any $*$-square root in the corner of the next position for which a certain denominator is invertible (Proposition 3.8). Every self-adjoint element of a symplectic matrix algebra over a real closed field has a $*$-square root (Proposition 3.6), and in Λ the roots are computed by the graded rule (Section 7.5). Results return to the original algebra kQ/I whenever the quantities that certify them contain no inverse letters (Theorem 7.8). Over a field with an infinitesimal, comparisons make sense too: running an algorithm once over the real closed field of algebraic Puiseux series and taking standard parts computes its limit as a parameter tends to zero (Proposition 4.7). This gives, for example, the passage from Householder QR to Gaussian elimination, which Chapter 1 also derives directly, by asymptotic analysis. Tarski's transfer principle, which is constructive, suffices for all results; Robinson's nonstandard analysis is not needed.

Unified decompositions. The decompositions unified in Theorem A are orthogonal decompositions of Hermitian modules into invariant submodules. The Krull–Schmidt the-

orem (Theorem 2.19) and its Hermitian analogue (Theorem 2.23) describe them, and the uniqueness part of the latter holds for quadratic forms exactly over quadratically closed and Euclidean fields (Proposition 2.22); this is why the thesis works over $\mathbb{R}$, real closed fields and algebraically closed fields. Doubly-signed quivers, which carry a transpose that reverses arrows and a conjugation that preserves them, with two families of signs, classify the basic finite-dimensional real $*$-algebras whose radical squares to zero (Theorem 5.11), and their vertex types correspond to the ten families of real classical groups (Table 5.1). By the analogue of Gabriel's theorem, part (1) of Theorem A, every finite-dimensional real $*$-algebra is a matrix algebra, with sizes and signs at the vertices, over a quotient of the path $*$-algebra of its doubly-signed quiver by an admissible ideal (Theorem 5.16); the quiver, the sizes and the signatures are invariants, and the quotient is unique up to a sign twist at vertices of split signature. The encoding theorem, part (2) of Theorem A (Theorem 6.1), shows that unitary similarity of Hermitian matrices of degree one, over the path $*$-algebra of a doubly-signed quiver or any quotient of it by homogeneous relations, is a classical matrix problem whose group is a product of classical groups, one for each orbit of vertices. This recovers the symmetric, Hermitian and quaternionic eigendecompositions, the real normal form of skew-symmetric matrices, the real, complex and quaternionic SVDs, the Takagi factorisation, Sylvester's law of inertia and the real Jordan form (Table 6.1). Matrices with a part of degree zero lead to first-order perturbation theory (Proposition 6.7), and power series to the spectral theorem and the singular value decomposition over formal power series rings (Theorem 4.12 and Corollary 4.13), which generalise the dual-number results of [39], and to a parity-equivariant spectral theorem (Theorem 4.15) that makes precise an observation of [44].

Unified algorithms. Over the completed localisation of the quiver of the singular value decomposition, the generic Householder tridiagonalisation is the Golub–Kahan bidiagonalisation (Proposition 3.9 and Example 3.10). On the encoded matrices, Householder reduction stays within the classical problem whenever its steps are defined and every component of the complexified quiver is an oriented cycle or a linearly oriented path, as for the whole catalogue (Proposition 6.4). For the quivers of the singular value decomposition

and of the Takagi factorisation, the QR algorithm in double steps is the QR algorithm on the Gram matrices, and a single step is a Cholesky LR step (Proposition 6.5). Over the split-complex numbers ${}^2\mathbb{R}$, a different $*$-algebra, whose involution exchanges its two idempotents, the QR algorithm is Rutishauser's LR algorithm on an invertible real matrix, taken two steps at a time, whenever the steps are defined (Proposition 6.6). Run over a field of Laurent series, which is the completed localisation of the quiver with one loop, the Gram–Schmidt process for a graded inner product computes the Bruhat decomposition of every invertible matrix (Theorem 4.11). These are parts (1) to (4) of Theorem B (Table 6.2). Beyond Theorem B, Gaussian elimination over a field of Puiseux series, with the vertices of a graph eliminated in order of their tentative distances, is Dijkstra's shortest-path algorithm in leading terms (Proposition 4.18); that shortest paths can be found by Gaussian elimination over the min-plus semiring is classical [14, 35]. A Python library implements the seven operations and these algorithms; it computes in the completed localisation for any doubly-signed quiver and reproduces the classical algorithms to within rounding errors (Table 8.2).

Beyond the completed localisation. When the quiver is wild, there is no canonical form of the familiar kind, and a generalised algorithm may need quotients that do not exist in the completed localisation (Section 6.5). The path algebra of every finite quiver embeds, constructively, in a full matrix ring over a skew field, so that every non-zero element between two vertices becomes invertible (Theorem 7.11). A field of rational functions suffices if and only if no vertex lies on two distinct simple oriented cycles; otherwise a non-commutative skew field, such as Cohn's free field, is needed, and the quiver is then wild (Proposition 6.8). After complexification, the localisation also embeds, compatibly with the involution and the conjugation, in a finite product of simple Artinian rings that they permute (Section 7.7).

Taken together, these results show that $*$-algebras, combined with the operator overloading facilities of programming languages, unify the decompositions of the catalogue and several classical algorithms, and they address the main algebraic obstructions to doing this naively. Each of these decompositions can also be computed without $*$-algebras,

by the classical algorithms for it, sometimes through a block matrix such as $\left(\begin{smallmatrix} 0 & W^* \\ W & 0 \end{smallmatrix}\right)$ for the singular value decomposition (Section 1.5); these are faster and better studied than the generic algorithms.

9.2 Finite-Dimensional versus Operator-Algebraic Settings

The thesis mentions Hilbert spaces and C^*-algebras, so it is natural to ask whether the framework is inherently finite-dimensional, and whether it could be generalised to operator algebras.

The generalised algorithms run in an algebra that is, in general, not finite-dimensional. Finite-dimensional $*$-algebras are where the proposal starts, and Theorem 5.16 describes each of them as a matrix algebra over a quotient $\mathbb{R}Q/I$ of a path $*$-algebra. The algorithms, however, run in the completed localisation Λ of Chapter 7, which is infinite-dimensional as soon as the underlying graph of Q contains a cycle (Proposition 7.2); for one vertex with one loop, it is the field $\mathbb{R}((t))$ of formal Laurent series. Finite dimension is needed to describe the algebras from which the framework starts (through the Wedderburn–Malcev decomposition, the nilpotence of the radical and the description of basic algebras by quivers; Sections 2.3.1 and 2.6), and to bring results back to them, which Theorem 7.8 can do because I contains all sufficiently long paths. What is finite throughout is the discrete data: the quiver, the sizes of the matrices and the blocks at the vertices. Accordingly, the transfer principle is applied to matrices of each fixed size separately (Section 4.3.1).

Two features, however, separate the framework from operator algebras. First, the finite-dimensional algebras from which it starts are in general not semisimple, whereas a C^*-algebra has zero Jacobson radical and its finite-dimensional examples are direct sums of full matrix algebras; the dual numbers, for example, admit no C^*-norm, since $\varepsilon^*\varepsilon = 0$ (Remark 2.5). Second, Λ deals with the radical algebraically, not analytically. For every quiver of the catalogue, Λ is, after complexification, a finite product of matrix

algebras over fields (Proposition 7.6), as a finite-dimensional complex C^*-algebra is; but where Q has a cycle these fields are fields of Laurent series, and the arrows are invertible elements of Λ of positive degree, that is, invertible infinitesimals in place of nilpotent ones (Section 7.6). Λ is complete for the path degree, not for a norm. Indeed, already for one vertex with one loop, the completed localisation of the real path $*$-algebra is $\mathbb{R}((t))$, which admits no submultiplicative norm, because by the theorem of Gelfand and Mazur every real division algebra with a submultiplicative norm is isomorphic to $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$ [95]. Graph C^*-algebras, the operator algebras built from directed graphs, invert arrows differently (Section 1.5): there an arrow is a partial isometry, and it is invertible between vertices, with its adjoint as inverse, only when it is the only arrow leaving its source.

Nevertheless, there are several plausible directions of generalisation.

- *Semisimple parts.* The vertex groups of Table 5.1 have infinite-dimensional analogues (unitary groups of real, complex and quaternionic Hilbert spaces), and Hermitian modules generalise to Hilbert C^*-modules, which carry inner products with values in a C^*-algebra [62]. Approximately finite-dimensional C^*-algebras, which are inductive limits of finite-dimensional ones [19], are a natural first case.

- *Infinitesimals.* The elements of Λ are formal series in the path degree, and Theorem 4.12 is an algebraic form of Rellich's theorem. Analytic perturbation theory extends it to holomorphic families of self-adjoint operators on Hilbert spaces, at least for isolated eigenvalues of finite multiplicity [55]. Convergent power series with operator coefficients, or Banach algebras with non-zero radical, are thus natural analytic counterparts of Λ and of the radical, rather than C^*-algebras.

- *Sequences of matrices.* The C^*-algebraic approach to numerical analysis of Hagen, Roch and Silbermann [45] studies sequences (A_n) of matrices, such as finite sections of Toeplitz operators, in the C^*-algebra of bounded matrix sequences modulo null sequences. Running a generalised algorithm uniformly along such a sequence amounts to replacing the scalars by a $*$-algebra built from these sequences, so their framework provides a possible bridge from the matrices of finite size treated here to

operators on Hilbert spaces.

We conclude that the framework is not inherently finite-dimensional. Its algorithms run in Λ, which is in general infinite-dimensional, and finite dimension is needed at the two ends, to describe the algebras from which the framework starts and to bring results back to them. As it stands, however, the framework is algebraic rather than analytic, and its discrete data are finite: Λ is complete for the path degree rather than for a norm, the radicals of the finite-dimensional algebras have no counterpart in C^*-algebras, and quivers and matrices are finite. Extensions to operator algebras appear feasible for the semisimple parts and for analytic perturbations, and we leave them to future work.

9.3 Limitations and Open Problems

- *Numerical behaviour.* We have not studied the numerical stability or conditioning of generalised algorithms. The thesis addresses the algebraic obstacles first, because they turned out to be more substantial than expected. The experiments of Section 8.5 show errors at the level of rounding for the algorithms and quivers tested, but they do not replace an analysis.

- *Relations.* Theorem 5.16 describes every finite-dimensional real $*$-algebra by its doubly-signed quiver together with an admissible ideal of relations. The encoding theorem (Theorem 6.1) applies to Hermitian matrices of degree one when the relations are homogeneous, as for $\mathbb{R}[\varepsilon]/(\varepsilon^3) = \mathbb{R}Q/(x^3)$. When they are not, the path degree is not defined on the quotient. For matrices with a part of degree zero, we only have the perturbation theory of Section 6.4 and the power series results of Chapter 4. A general encoding theorem for path $*$-algebras with relations is open.

- *Involutions and square roots.* The completed localisation carries the involution, and for every quiver of the catalogue it is a product of simple Artinian rings that the involution permutes. Beyond Λ, every doubly-signed quiver, including one with a vertex on two distinct simple oriented cycles, embeds after complexification, compatibly with the involution and the conjugation, in a finite product of simple Artinian

rings that they permute (Section 7.7). It is open whether the $*$-square roots that a Jacobi-type method needs exist there, and whether the graded rule of Section 7.5 finds a $*$-square root of every element z^*z with z homogeneous.

- *Reducing results.* Theorem 7.8 brings results back from the completed localisation whenever the quantities that certify them contain no inverse letters. Results computed in the free field, which quivers with a vertex on two distinct simple oriented cycles require, have no such reduction yet.

- *Wild quivers.* Our explanation of the behaviour of generalised algorithms on wild quivers (Section 6.5) is supported by examples, not by a general theorem: for two loops we showed that the quotients needed by a Jacobi-type method do not exist in the completed localisation, but not whether the method can be completed in a skew field, or what it returns. Arithmetic in the free field, which these cases require, is also expensive.

9.4 Future Research Directions

- Develop the library of Chapter 8 into a tool or compiler that applies these transformations automatically to existing code bases, for example in Fortran, C++ or Python with NumPy.

- Explore the connection between the representation type of a quiver and the numerical stability and convergence of the corresponding generalised algorithms.

- Extend the framework to operator algebras along the lines of Section 9.2, beginning with sequences of matrices in the sense of [45].

- Extend the visualisation of Appendix A to 3×3 matrices; we are not aware of an analogous construction.

9.5 The Role of the Visualisation Appendix

Appendix A uses the geometry of low-dimensional $*$-algebras to depict the dynamics of the QR algorithm on 2×2 real matrices. Several objects from geometry and Lie theory appear there, and we summarise their roles.

- A real symmetric matrix is *positive semidefinite* (PSD) if all its eigenvalues are non-negative. The warm-up visualisation draws a PSD 2×2 matrix M as the ellipse $\{Mv : \|v\| = 1\}$.

- $\mathrm{PSL}_2(\mathbb{R}) = \mathrm{SL}_2(\mathbb{R})/\{\pm I\}$ is the group of Möbius transformations $x \mapsto (ax+b)/(cx+d)$ of the real projective line $\mathbb{RP}^1$. A 2×2 real matrix of positive determinant, up to a non-zero scalar factor, is an element of this group, so its dynamics under the QR algorithm can be studied through its action on $\mathbb{RP}^1$.

- $\mathrm{PU}(1,1)$ is the group of complex 2×2 matrices preserving the indefinite Hermitian form $|z_1|^2 - |z_2|^2$ on $\mathbb{C}^2$, modulo scalars; it acts by Möbius transformations on the unit disc and on its boundary circle. $\mathrm{SO}^+(2,1)$ is the identity component of the group of linear maps preserving the form $X^2 + Y^2 - Z^2$; it acts on the cone $X^2 + Y^2 = Z^2$ and hence on the circle of its rays. The *accidental isomorphisms* $\mathrm{PSL}_2(\mathbb{R}) \cong \mathrm{PU}(1,1) \cong \mathrm{SO}^+(2,1)$ identify $\mathbb{RP}^1$ with a circle, so that a matrix can be drawn as a transformation of a circle.

- *Lie sphere geometry* [15] represents oriented circles, oriented lines and points of the plane as points of a quadric in $\mathbb{RP}^4$, in such a way that oriented tangency becomes orthogonality with respect to a quadratic form of signature $(3,2)$. This turns the computation of the envelope of a family of circles into linear algebra, and it is one way of computing the circles that represent a matrix; Propositions A.4 and A.5 also describe them directly, in a chart given by a Möbius transformation. The corresponding group of transformations is $\mathrm{SO}(3,2)$, which is related to $\mathrm{Sp}(4,\mathbb{R})$ by a further accidental isomorphism.

The connection with the rest of the thesis is through the projective lines over the two-

dimensional $*$-algebras $\mathbb{C}$, the dual numbers and ${}^2\mathbb{R}$, whose geometries are the Möbius, Laguerre and Minkowski planes; planar Lie sphere geometry contains all three [8, 15, 92]. The appendix works in two of them: its pictures are drawn in the Möbius plane, the projective line over $\mathbb{C}$, and the arrows $x \mapsto f(x)$ of a matrix form the graph of f, a chain in the Minkowski plane $\mathbb{RP}^1 \times \mathbb{RP}^1$, the projective line over ${}^2\mathbb{R}$. The real 2×2 matrices enter through the group of Lie sphere geometry, whose identity component $\mathrm{PSp}(4, \mathbb{R}) \cong \mathrm{SO}^+(3, 2)$ is the projective unitary group of the 2×2 matrices over $M_2(\mathbb{R})$ with its symplectic involution; the points of Lie sphere geometry are the Lagrangian planes in $\mathbb{R}^4$, a three-dimensional part of the projective line over $M_2(\mathbb{R})$, which is the four-dimensional Grassmannian of planes in $\mathbb{R}^4$.

Bibliography

[1] Gene Abrams, Pere Ara, and Mercedes Siles Molina. *Leavitt Path Algebras*, volume 2191 of *Lecture Notes in Mathematics*. Springer, London, 2017. doi: 10.1007/978-1-4471-7344-1.

[2] Shimshon A. Amitsur and Jacob Levitzki. Minimal identities for algebras. *Proceedings of the American Mathematical Society*, 1(4):449–463, 1950. doi: 10.1090/S0002-9939-1950-0036751-9.

[3] Ibrahim Assem, Daniel Simson, and Andrzej Skowroński. *Elements of the Representation Theory of Associative Algebras, Volume 1: Techniques of Representation Theory*, volume 65 of *London Mathematical Society Student Texts*. Cambridge University Press, Cambridge, 2006. doi: 10.1017/CBO9780511614309.

[4] Maurice Auslander, Idun Reiten, and Sverre O. Smalø. *Representation Theory of Artin Algebras*, volume 36 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, Cambridge, 1995. doi: 10.1017/CBO9780511623608.

[5] Jørgen Bang-Jensen and Gregory Gutin. *Digraphs: Theory, Algorithms and Applications*. Springer Monographs in Mathematics. Springer, London, second edition, 2009. doi: 10.1007/978-1-84800-998-1.

[6] Jess Banks, Jorge Garza-Vargas, and Nikhil Srivastava. Global convergence of Hessenberg shifted QR I: Exact arithmetic. *Foundations of Computational Mathematics*, 25(5):1561–1594, 2025. doi: 10.1007/s10208-024-09658-7.

[7] Saugata Basu, Richard Pollack, and Marie-Françoise Roy. *Algorithms in Real Alge-*

braic Geometry, volume 10 of *Algorithms and Computation in Mathematics*. Springer, Berlin, Heidelberg, second edition, 2006. doi: 10.1007/3-540-33099-2.

[8] Walter Benz. *Vorlesungen über Geometrie der Algebren: Geometrien von Möbius, Laguerre-Lie, Minkowski in einheitlicher und grundlagengeometrischer Behandlung*, volume 197 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 1973. doi: 10.1007/978-3-642-88670-6.

[9] Jacek Bochnak, Michel Coste, and Marie-Françoise Roy. *Real Algebraic Geometry*, volume 36 of *Ergebnisse der Mathematik und ihrer Grenzgebiete (3)*. Springer, Berlin, Heidelberg, 1998. doi: 10.1007/978-3-662-03718-8.

[10] Albrecht Böttcher and Bernd Silbermann. *Introduction to Large Truncated Toeplitz Matrices*. Universitext. Springer, New York, 1999. doi: 10.1007/978-1-4612-1426-7.

[11] Angelika Bunse-Gerstner, Ralph Byers, Volker Mehrmann, and Nancy K. Nichols. Numerical computation of an analytic singular value decomposition of a matrix valued function. *Numerische Mathematik*, 60(1):1–39, 1991. doi: 10.1007/BF01385712.

[12] Angelika Bunse-Gerstner, Ralph Byers, and Volker Mehrmann. A chart of numerical methods for structured eigenvalue problems. *SIAM Journal on Matrix Analysis and Applications*, 13(2):419–453, 1992. doi: 10.1137/0613028.

[13] Hendrik Jan Buurema. *A Geometric Proof of Convergence for the QR Method*. PhD thesis, Rijksuniversiteit te Groningen, Groningen, 1970.

[14] Bernard A. Carré. An algebraic approach to network routing problems. *IMA Journal of Applied Mathematics*, 7(3):273–294, 1971. doi: 10.1093/imamat/7.3.273.

[15] Thomas E. Cecil. *Lie Sphere Geometry: With Applications to Submanifolds*. Universitext. Springer, New York, 1992. doi: 10.1007/978-1-4757-4096-7.

[16] Paul M. Cohn. *Free Ideal Rings and Localization in General Rings*, volume 3 of *New Mathematical Monographs*. Cambridge University Press, Cambridge, 2006. doi: 10.1017/CBO9780511542794.

[17] Paul M. Cohn and Christophe Reutenauer. On the construction of the free field. *International Journal of Algebra and Computation*, 9(3–4):307–323, 1999. doi: 10.1142/S0218196799000205.

[18] Anthony J. Cox and Nicholas J. Higham. Row-wise backward stable elimination methods for the equality constrained least squares problem. *SIAM Journal on Matrix Analysis and Applications*, 21(1):313–326, 1999. doi: 10.1137/S0895479898335957.

[19] Kenneth R. Davidson. *C^*-Algebras by Example*, volume 6 of *Fields Institute Monographs*. American Mathematical Society, Providence, RI, 1996. doi: 10.1090/fim/006.

[20] James Demmel and William Kahan. Accurate singular values of bidiagonal matrices. *SIAM Journal on Scientific and Statistical Computing*, 11(5):873–912, 1990. doi: 10.1137/0911052.

[21] Harm Derksen and Jerzy Weyman. Generalized quivers associated to reductive groups. *Colloquium Mathematicum*, 94(2):151–173, 2002. doi: 10.4064/cm94-2-1.

[22] Harm Derksen and Jerzy Weyman. *An Introduction to Quiver Representations*, volume 184 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2017. doi: 10.1090/gsm/184.

[23] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

[24] Vlastimil Dlab and Claus Michael Ringel. Indecomposable representations of graphs and algebras. *Memoirs of the American Mathematical Society*, 6(173), 1976. doi: 10.1090/memo/0173.

[25] Yuriy A. Drozd. Tame and wild matrix problems. In Vlastimil Dlab and Peter Gabriel, editors, *Representation Theory II: Proceedings of the Second International Conference on Representations of Algebras (Ottawa, 1979)*, volume 832 of *Lecture Notes in Mathematics*, pages 242–258. Springer, Berlin, Heidelberg, 1980. doi: 10.1007/BFb0088467.

[26] David S. Dummit and Richard M. Foote. *Abstract Algebra.* John Wiley & Sons, Hoboken, NJ, third edition, 2004.

[27] Alan Edelman and Sungwoo Jeong. Fifty three matrix factorizations: A systematic approach. *SIAM Journal on Matrix Analysis and Applications*, 44(2):415–480, 2023. doi: 10.1137/21M1416035.

[28] Heike Faßbender, D. Steven Mackey, Niloufer Mackey, and Hongguo Xu. Hamiltonian square roots of skew-Hamiltonian matrices. *Linear Algebra and its Applications*, 287 (1–3):125–159, 1999. doi: 10.1016/S0024-3795(98)10137-4.

[29] John G. F. Francis. The QR transformation: A unitary analogue to the LR transformation—Part 1. *The Computer Journal*, 4(3):265–271, 1961. doi: 10.1093/comjnl/4.3.265.

[30] John G. F. Francis. The QR transformation—Part 2. *The Computer Journal*, 4(4): 332–345, 1962. doi: 10.1093/comjnl/4.4.332.

[31] Peter Gabriel. Unzerlegbare Darstellungen I. *Manuscripta Mathematica*, 6(1):71–103, 1972. doi: 10.1007/BF01298413.

[32] Peter Gabriel and Michel Zisman. *Calculus of Fractions and Homotopy Theory*, volume 35 of *Ergebnisse der Mathematik und ihrer Grenzgebiete.* Springer, Berlin, Heidelberg, 1967. doi: 10.1007/978-3-642-85844-4.

[33] Gene H. Golub and William Kahan. Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(2):205–224, 1965. doi: 10.1137/0702016.

[34] Gene H. Golub and Charles F. Van Loan. *Matrix Computations.* Johns Hopkins Studies in the Mathematical Sciences. Johns Hopkins University Press, Baltimore, MD, fourth edition, 2013.

[35] Michel Gondran and Michel Minoux. *Graphs, Dioids and Semirings: New Models and*

Algorithms, volume 41 of *Operations Research/Computer Science Interfaces Series*. Springer, New York, 2008. doi: 10.1007/978-0-387-75450-5.

[36] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Society for Industrial and Applied Mathematics, Philadelphia, PA, second edition, 2008. doi: 10.1137/1.9780898717761.

[37] Ran Gutin. Towards a singular value decomposition and spectral theory for all rings. arXiv preprint arXiv:2112.10247, 2021.

[38] Ran Gutin. Matrix decompositions over the double numbers. arXiv preprint arXiv:2105.08047, 2021.

[39] Ran Gutin. Generalizations of singular value decomposition to dual-numbered matrices. *Linear and Multilinear Algebra*, 70(20):5107–5114, 2022. doi: 10.1080/03081087.2021.1903830.

[40] Ran Gutin. A visualisation for conveying the dynamics of iterative eigenvalue algorithms over PSD matrices. arXiv preprint arXiv:2204.00383, 2022.

[41] Ran Gutin. An interactive visualisation for all 2×2 real matrices, with applications to conveying the dynamics of iterative eigenvalue algorithms. arXiv preprint arXiv:2211.02075, 2022.

[42] Ran Gutin. A conjecture concerning $*$-algebras that unifies some matrix decompositions. arXiv preprint arXiv:2308.04513, 2023.

[43] Ran Gutin. An analogue of the relationship between SVD and pseudoinverse over double-complex matrices. *Linear and Multilinear Algebra*, 71(11):1775–1793, 2023. doi: 10.1080/03081087.2022.2076798.

[44] Ran Gutin. Unitary canonical forms over Clifford algebras, and an observed unification of some real-matrix decompositions. *Linear and Multilinear Algebra*, 72(14): 2288–2311, 2024. doi: 10.1080/03081087.2023.2252979.

[45] Roland Hagen, Steffen Roch, and Bernd Silbermann. *C^*-Algebras and Numerical Analysis*, volume 236 of *Pure and Applied Mathematics*. Marcel Dekker, New York, 2001.

[46] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[47] Sigurdur Helgason. *Differential Geometry, Lie Groups, and Symmetric Spaces*, volume 80 of *Pure and Applied Mathematics*. Academic Press, New York, 1978.

[48] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, second edition, 2002. doi: 10.1137/1.9780898718027.

[49] Nicholas J. Higham. *Functions of Matrices: Theory and Computation*, volume 104 of *Other Titles in Applied Mathematics*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2008. doi: 10.1137/1.9780898717778.

[50] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, second edition, 2013. doi: 10.1017/CBO9781139020411.

[51] Roger A. Horn and Vladimir V. Sergeichuk. Canonical forms for complex matrix congruence and *congruence. *Linear Algebra and its Applications*, 416(2–3):1010–1032, 2006. doi: 10.1016/j.laa.2006.01.005.

[52] James E. Humphreys. *Linear Algebraic Groups*, volume 21 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, 1975.

[53] Nathan Jacobson. *Basic Algebra II*. W. H. Freeman, New York, second edition, 1989.

[54] Victor G. Kac. Infinite root systems, representations of graphs and invariant theory. *Inventiones Mathematicae*, 56(1):57–92, 1980. doi: 10.1007/BF01403155.

[55] Tosio Kato. *Perturbation Theory for Linear Operators*. Classics in Mathematics. Springer, Berlin, Heidelberg, second edition, 1995. doi: 10.1007/978-3-642-66282-9.

[56] Martin Kleinsteuber, Uwe Helmke, and Knut Hüper. Jacobi's algorithm on compact Lie algebras. *SIAM Journal on Matrix Analysis and Applications*, 26(1):42–69, 2004. doi: 10.1137/S0895479802420069.

[57] Max-Albert Knus. *Quadratic and Hermitian Forms over Rings*, volume 294 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 1991. doi: 10.1007/978-3-642-75401-2.

[58] Max-Albert Knus, Alexander Merkurjev, Markus Rost, and Jean-Pierre Tignol. *The Book of Involutions*, volume 44 of *American Mathematical Society Colloquium Publications*. American Mathematical Society, Providence, RI, 1998. doi: 10.1090/coll/044.

[59] Anders Kock. *Synthetic Differential Geometry*, volume 333 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, Cambridge, second edition, 2006. doi: 10.1017/CBO9780511550812.

[60] Tsit-Yuen Lam. *A First Course in Noncommutative Rings*, volume 131 of *Graduate Texts in Mathematics*. Springer, New York, second edition, 2001. doi: 10.1007/978-1-4419-8616-0.

[61] Tsit-Yuen Lam. *Introduction to Quadratic Forms over Fields*, volume 67 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2005. doi: 10.1090/gsm/067.

[62] E. Christopher Lance. *Hilbert C^*-Modules: A Toolkit for Operator Algebraists*, volume 210 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, Cambridge, 1995. doi: 10.1017/CBO9780511526206.

[63] Ricardo S. Leite, Nicolau C. Saldanha, and Carlos Tomei. Dynamics of the symmetric eigenvalue problem with shift strategies. *International Mathematics Research Notices*, 2013(19):4382–4412, 2013. doi: 10.1093/imrn/rns186.

[64] Jacques Lewin. Fields of fractions for group algebras of free groups. *Transactions of the American Mathematical Society*, 192:339–346, 1974. doi: 10.1090/S0002-9947-1974-0338055-4.

[65] Ottmar Loos. *Symmetric Spaces. I: General Theory*. Mathematics Lecture Note Series. W. A. Benjamin, New York, 1969.

[66] Jonathan Lorand. *Duality Involutions, Representations, and Geometry*. PhD thesis, Universität Zürich, Zürich, 2020. URL https://www.zora.uzh.ch/id/eprint/197366/.

[67] D. Steven Mackey, Niloufer Mackey, and Françoise Tisseur. Structured factorizations in scalar product spaces. *SIAM Journal on Matrix Analysis and Applications*, 27(3): 821–850, 2006. doi: 10.1137/040619363.

[68] Diane Maclagan and Bernd Sturmfels. *Introduction to Tropical Geometry*, volume 161 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2015. doi: 10.1090/gsm/161.

[69] David Marker. *Model Theory: An Introduction*, volume 217 of *Graduate Texts in Mathematics*. Springer, New York, 2002. doi: 10.1007/b98860.

[70] Richard S. Pierce. *Associative Algebras*, volume 88 of *Graduate Texts in Mathematics*. Springer, New York, 1982. doi: 10.1007/978-1-4757-0163-0.

[71] Heinz-Georg Quebbemann, Winfried Scharlau, and Manfred Schulte. Quadratic and Hermitian forms in additive and abelian categories. *Journal of Algebra*, 59(2):264–289, 1979. doi: 10.1016/0021-8693(79)90126-1.

[72] Iain Raeburn. *Graph Algebras*, volume 103 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI, 2005. doi: 10.1090/cbms/103.

[73] Paulo Ribenboim. *The Theory of Classical Valuations*. Springer Monographs in Mathematics. Springer, New York, 1999. doi: 10.1007/978-1-4612-0551-7.

[74] Abraham Robinson. *Non-standard Analysis*. Studies in Logic and the Foundations of Mathematics. North-Holland, Amsterdam, 1966.

[75] Heinz Rutishauser. Solution of eigenvalue problems with the LR-transformation. In *Further Contributions to the Solution of Simultaneous Linear Equations and the Determination of Eigenvalues*, volume 49 of *National Bureau of Standards Applied Mathematics Series*, pages 47–81. National Bureau of Standards, Washington, DC, 1958.

[76] Winfried Scharlau. *Quadratic and Hermitian Forms*, volume 270 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 1985. doi: 10.1007/978-3-642-69971-9.

[77] Ralf Schiffler. *Quiver Representations*. CMS Books in Mathematics. Springer, Cham, 2014. doi: 10.1007/978-3-319-09204-1.

[78] Aidan H. Schofield. *Representations of Rings over Skew Fields*, volume 92 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, Cambridge, 1985. doi: 10.1017/CBO9780511661914.

[79] Vladimir V. Sergeichuk. Classification problems for systems of forms and linear mappings. *Mathematics of the USSR-Izvestiya*, 31(3):481–501, 1988. doi: 10.1070/IM1988v031n03ABEH001086.

[80] Dmitri A. Shmelkin. Signed quivers, symmetric quivers and root systems. *Journal of the London Mathematical Society. Second Series*, 73(3):586–606, 2006. doi: 10.1112/S0024610706022812.

[81] Earl J. Taft. Invariant Wedderburn factors. *Illinois Journal of Mathematics*, 1(4): 565–573, 1957. doi: 10.1215/ijm/1255380679.

[82] Teiji Takagi. On an algebraic problem related to an analytic theorem of Carathéodory and Fejér and on an allied theorem of Landau. *Japanese Journal of Mathematics*, 1: 83–93, 1924. doi: 10.4099/jjm1924.1.0_83.

[83] Terence Tao. Ultrafilters, nonstandard analysis, and epsilon management. *What's New* (blog), https://terrytao.wordpress.com/2007/06/25/ultrafilters-nonstandard-analysis-and-epsilon-management/, 2007.

[84] Alfred Tarski. *A Decision Method for Elementary Algebra and Geometry*. University of California Press, Berkeley, CA, second edition, 1951.

[85] Charles F. Van Loan. A symplectic method for approximating all the eigenvalues of a Hamiltonian matrix. *Linear Algebra and its Applications*, 61:233–251, 1984. doi: 10.1016/0024-3795(84)90034-X.

[86] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17 (3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[87] Charles T. C. Wall. Classification of Hermitian forms. VI. Group rings. *Annals of Mathematics. Second Series*, 103(1):1–80, 1976. doi: 10.2307/1971019.

[88] David S. Watkins. The QR algorithm revisited. *SIAM Review*, 50(1):133–145, 2008. doi: 10.1137/060659454.

[89] Klaus Weihrauch. *Computable Analysis: An Introduction*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2000. doi: 10.1007/978-3-642-56999-9.

[90] Hermann Weyl. *The Classical Groups: Their Invariants and Representations*, volume 1 of *Princeton Mathematical Series*. Princeton University Press, Princeton, NJ, 1939.

[91] Hongguo Xu. The relation between the QR and LR algorithms. *SIAM Journal on Matrix Analysis and Applications*, 19(2):551–555, 1998. doi: 10.1137/S0895479896299937.

[92] Isaak M. Yaglom. *Complex Numbers in Geometry*. Academic Press, New York and London, 1968.

[93] Isaak M. Yaglom. *A Simple Non-Euclidean Geometry and Its Physical Basis: An Elementary Account of Galilean Geometry and the Galilean Principle of Relativity*. Heidelberg Science Library. Springer, New York, 1979. doi: 10.1007/978-1-4612-6135-3.

[94] Dante C. Youla. A normal form for a matrix under the unitary congruence group. *Canadian Journal of Mathematics*, 13:694–704, 1961. doi: 10.4153/CJM-1961-059-8.

[95] Wiesław Żelazko. *Banach Algebras*. Elsevier, Amsterdam, 1973.

[96] Fuzhen Zhang. Quaternions and matrices of quaternions. *Linear Algebra and its Applications*, 251:21–57, 1997. doi: 10.1016/0024-3795(95)00543-9.

Appendix A

Visualising the Dynamics of the QR Algorithm on 2×2 Real Matrices

Provenance. This appendix reproduces, with editorial changes, a preprint written by the author during his doctoral studies [41] (arXiv:2211.02075); its warm-up visualisation for positive semidefinite matrices first appeared in [40]. The editorial changes consist of explanations of the figures and of the geometric objects involved, corrected formulas for the Lie quadric representation of the envelope (Lemma A.3) and for the envelopes themselves (Sections A.4.4 and A.4.5), a corrected description of the QR step in Visualisation v1.5, more precise statements about rotations and about the speed of convergence (Section A.6), and notation unified with the rest of the thesis. The appendix is independent of the main results of the thesis. It is included because it is an application of the geometry of the projective lines over the low-dimensional $*$-algebras of this thesis ($\mathbb{C}$, the dual numbers and ${}^2\mathbb{R}$, whose geometries Lie sphere geometry contains; see Section 9.5) to the teaching of numerical linear algebra.

A.1 Overview

We introduce two visualisations for use in self-study or teaching, which we call v1 and v2. Both are geometrically simple.

- v1 will be familiar to readers well versed in linear algebra, and we treat it as a

warm-up. It applies only to positive semidefinite (PSD) matrices, that is, symmetric matrices with non-negative eigenvalues. A matrix is depicted as an ellipse centred at the origin of $\mathbb{R}^2$. This is an easy application of the spectral theorem.

- v2 is less obvious, because it uses Lie sphere geometry [15]. It applies to *all* 2×2 real matrices. When the determinant is non-negative, a matrix is depicted, up to a non-zero scalar factor, as an unordered pair consisting of an *oriented circle* and its *orientation-reversed inversion* in the unit circle.

Visualisation v2 has the advantage that many properties of a 2×2 matrix M can be read off from it:

- its Jordan normal form;
- which matrices it is orthogonally similar to;
- its condition number;
- whether it is upper triangular, lower triangular, diagonal, symmetric or orthogonal;
- a geometric construction for the matrix-vector product Mv;
- a geometric construction for the matrix-matrix product MK.

There are ways to extend v1 to all 2×2 real matrices, for instance by depicting the polar decomposition of M, but most of the above properties are then lost. A visualisation of the polar decomposition might still be useful for depicting the dynamics of algorithms that compute SVDs or polar decompositions; we do not pursue this here.

Our aim is to convey the dynamics of the *unshifted* QR algorithm, $M = QR \mapsto RQ$ [29, 30], without formulas. Pedagogical treatments of the QR algorithm exist [13, 88], but they are more formula-driven and less visual than ours. For 3×3 matrices we have not found anything analogous.

A.2 Visualisation v1 (Warm-Up)

This visualisation is **valid only for PSD symmetric matrices**. A 2×2 PSD matrix M is depicted by the set $\{Mv : v \in \mathbb{R}^2,\ \|v\|_2 = 1\}$, which by the spectral theorem is an ellipse whose semi-axes are the eigenvectors of M scaled by the eigenvalues. In Figure A.1, the blue ellipse is the input matrix, the green ellipse is the result of one step of Rutishauser's LR algorithm [75] in its Cholesky form, $M = LL^T \mapsto L^T L$, which keeps M symmetric and positive semidefinite (two such steps make one step of the QR algorithm, as recalled after Proposition 6.5), and the red ellipse is the result of one iteration of the QR algorithm.

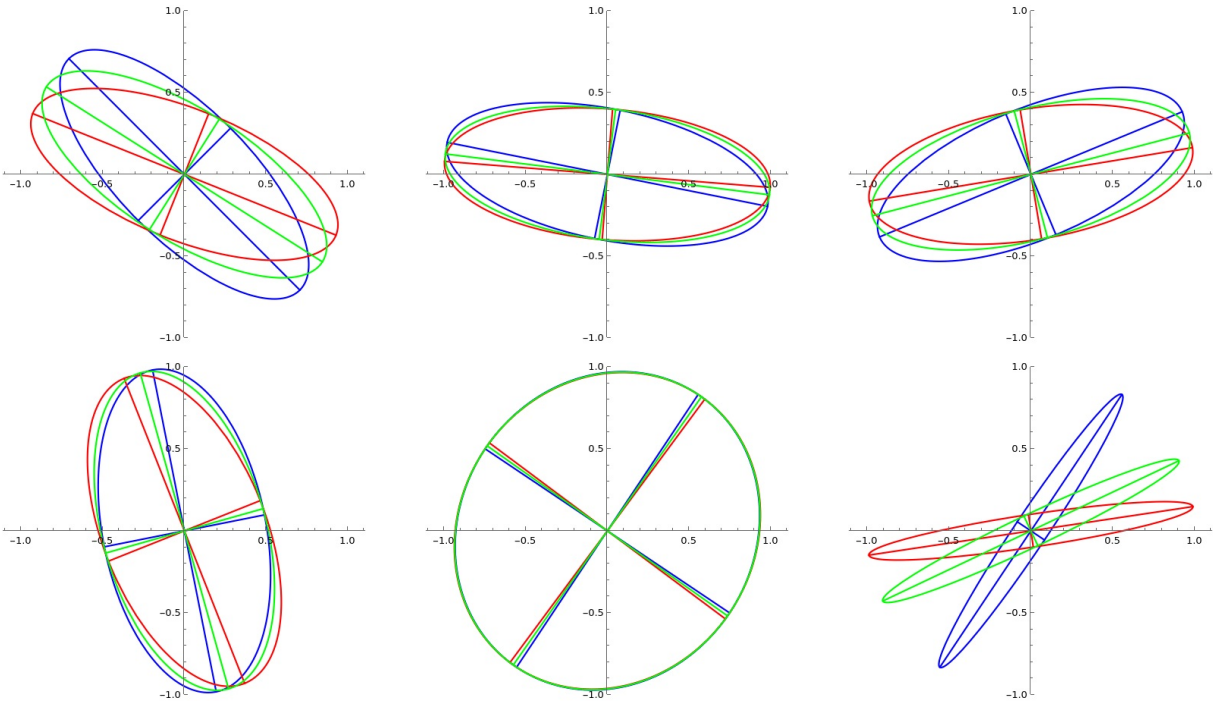


Figure A.1: Visualisation v1. Each panel shows a 2×2 PSD matrix M as the ellipse $\{Mv : \|v\| = 1\}$ (blue), together with one step of the Cholesky LR algorithm (green) and one step of the QR algorithm (red). An ellipse with its major axis along the x-axis corresponds to a diagonal matrix with decreasing diagonal entries, the limit of both algorithms.

The main observations are the following.

- Convergence happens in every case, though not necessarily quickly.
- Ellipses whose major semi-axis is parallel to the x-axis are attracting fixed points, and convergence near them is fast.

- Ellipses whose major semi-axis is parallel to the y-axis are repelling fixed points, and convergence near them is slow.
- When the eigenvalues are close to each other the ellipse is nearly a circle, and convergence is slow.
- When one eigenvalue is close to zero and the other is not, the ellipse is very flat, and convergence is fast. In the PSD case, *shifting*, which replaces M by $M - \mu I$, can be seen as flattening the ellipse.

The visualisation also illustrates why a continuous shift strategy cannot give uniform convergence, but only pointwise convergence, because unstable fixed points are unavoidable. This has been analysed using algebraic topology in [63] and is used in the recent analysis of shifted QR in [6].

A.3 Visualisation v1.5 (Warm-Up)

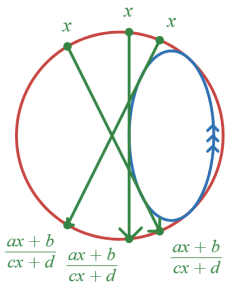


Figure A.2: Visualisation v1.5. The circle represents the real projective line $\mathbb{RP}^1$. For a matrix M, the chords $x \to f(x)$, where f is the Möbius transformation of M, envelop a curve, shown with an orientation that distinguishes f from f^{-1}.

A straightforward method for visualising all 2×2 real matrices with positive determinant uses the accidental isomorphism

$$\mathrm{PSL}_2(\mathbb{R}) \;\cong\; \mathrm{SO}^+(2,1).$$

Here $\mathrm{PSL}_2(\mathbb{R}) = \mathrm{SL}_2(\mathbb{R})/\{\pm I\}$ is the group of Möbius transformations $f(x) = \frac{ax+b}{cx+d}$ of the real projective line $\mathbb{RP}^1$, and a matrix of positive determinant determines such a transformation up to a non-zero scalar factor. $\mathrm{SO}^+(2,1)$ is the identity component of the group of linear maps of $\mathbb{R}^3$ preserving the quadratic form $X^2+Y^2-Z^2$. The isomorphism means that $\mathrm{PSL}_2(\mathbb{R})$ acts on the projective conic $\{X^2+Y^2=Z^2\} \subset \mathbb{RP}^2$, which is a circle, and this circle is identified with $\mathbb{RP}^1$.

For each point x of this circle, draw the chord from x to $f(x)$. The family of chords has an *envelope*, which can be given an orientation that distinguishes f from f^{-1}; Figure A.2 shows an example. In this picture, one step of the QR algorithm has a brief geometric description: given the envelope of f, draw the chord from ∞ to $f(\infty) = a/c$, and rotate the envelope about the centre of the circle, in the direction opposite to the arc from ∞ to $f(\infty)$, until the point $f(\infty)$ is moved to ∞. (The point ∞ is the direction of the first basis vector e_1, and $f(\infty)$ is the direction of Me_1; the rotation Q^T maps $f(\infty)$ to ∞, since $Q^T M e_1 = r_{11} e_1$.) We do not discuss this visualisation further. Visualisation v2 below is similar, but its envelopes have a simpler characterisation, at the cost of making the arcs $x \to f(x)$ round rather than straight. Overall, v2 may make it more intuitive to vary the 2×2 matrix interactively.

A.4 Visualisation v2

Our construction is based on ideas from Yaglom's books [92, 93]; here we develop them from first principles.

A.4.1 Group Viewpoints and the Boundary Circle

The accidental isomorphism

$$\mathrm{PSL}_2(\mathbb{R}) \;\cong\; \mathrm{PU}(1,1)$$

gives an action of $\mathrm{PSL}_2(\mathbb{R})$ on the unit circle $\{|z|=1\} \subset \widehat{\mathbb{C}}$. Here $\mathrm{PU}(1,1)$ is the group of complex 2×2 matrices preserving the indefinite Hermitian form $|z_1|^2 - |z_2|^2$, modulo scalars; it acts by Möbius transformations on the unit disc and on its boundary circle.

We call this circle *the boundary*, as in the Poincaré disc model of the hyperbolic plane. For x on the boundary and $f \in \mathrm{PSL}_2(\mathbb{R})$, the *canonical arrow* $x \to f(x)$ is the unique circle orthogonal to the boundary that contains x and $f(x)$.

That there is a unique circle orthogonal to the boundary through two given boundary points w and z is easy to see after a Möbius transformation that maps the boundary to the real line $\mathbb{R} \subset \mathbb{C}$. Then there is clearly a unique circle through $w, z \in \mathbb{R}$ that is symmetric under reflection in $\mathbb{R}$; it is orthogonal to $\mathbb{R}$, and it is the only such circle. Since Möbius transformations are conformal, the property is independent of the chart.

Equivalently, in the chart in which f acts as $f(x) = \frac{ax+b}{cx+d}$ on $\mathbb{R}$, we form the semicircles in the upper half-plane joining x to $f(x)$, together with their reflections in the real axis, and take their envelope. Finally we change to the chart in which the real line becomes the unit circle, via the Cayley transform $x \mapsto \frac{ix+1}{-ix+1}$. This chart is chosen because, in it, the envelope for $R_\theta M R_\theta^{-1}$ is the envelope for M rotated about the origin by the angle -2θ, where $R_\theta = \left(\begin{smallmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{smallmatrix}\right)$. In other words, *similarity by a rotation is rotation of the picture*; similarity by a reflection reflects it.

We want to compute the oriented envelope. When $\det M > 0$, it consists of two *mutually inverse* oriented circles: one is the image of the other under the inversion $z \mapsto 1/\bar{z}$ in the unit circle, with the orientation reversed. To compute these circles we use *Lie sphere geometry* [15]. Lie sphere geometry also helps to understand the simple Lie group $\mathrm{Sp}(4, \mathbb{R})$, through the accidental isomorphism $\mathrm{Sp}(4, \mathbb{R})/\{\pm I\} \cong \mathrm{SO}^\circ(3, 2)$, in which the right-hand side is the identity component of the group of transformations of Lie sphere geometry; we do not use this.

A.4.2 Oriented Cycles and the Lie Quadric

Definition A.1 (Lie quadric)**.** *Oriented cycles*, that is, oriented circles, oriented lines and points (including ∞) on the sphere $S^2 \cong \widehat{\mathbb{C}}$, are represented by the points of the quadric

$$\mathrm{L}: \quad X^2 + Y^2 + Z^2 = C^2 + S^2 \qquad \text{in } \mathbb{RP}^4,$$

with homogeneous coordinates $[X : Y : Z : C : S]$ [15]. If the left-hand side vanishes then so does the right-hand side, and $[0 : 0 : 0 : 0 : 0]$ is not a point of $\mathbb{RP}^4$; so we may normalise $X^2+Y^2+Z^2 = C^2+S^2 = 1$. Then $(X, Y, Z) \in S^2$ is the spherical centre of the cycle, and $(C, S) = (\cos r, \sin r)$ encodes its signed spherical radius r, the sign of r giving the orientation.

Let

$$Q(\xi) = X^2 + Y^2 + Z^2 - C^2 - S^2, \qquad \mathcal{B}(\xi, \eta) = XX' + YY' + ZZ' - CC' - SS'$$

be the quadratic form, of signature $(3, 2)$, and its polarisation. Two cycles $p, q \in \mathrm{L}$ are *orientedly tangent* if and only if $\mathcal{B}(p, q) = 0$.

Remark A.2. The condition $\mathcal{B}(p, q) = 0$ is a reasonable definition of oriented tangency. It says that $XX' + YY' + ZZ' = CC' + SS'$, an equality of two dot products of unit vectors, which by $u \cdot v = \cos \angle(u, v)$ means that the spherical distance between the centres equals the difference of the signed radii.

A.4.3 The Envelope as a Tangent Pair on L

Let $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in M_2(\mathbb{R})$ with $\Delta = \det M = ad - bc \neq 0$ and $\tau = \operatorname{tr} M = a + d$, and let $f(z) = \frac{az+b}{cz+d}$ be its Möbius transformation. We identify the sphere S^2 with $\widehat{\mathbb{C}}$ by the stereographic projection $z = (X + iY)/(1 - Z)$, and we draw in the disc chart of Section A.4.1, obtained from the upper half-plane by the Cayley transform; the unit circle $|z| = 1$ is then the equator $Z = 0$.

Lemma A.3 (Envelope via a Lie sphere pencil)**.** *Let $u, v, w \in \mathrm{L}$ be three canonical arrows of M, for instance those starting at the boundary points corresponding to 0, 1 and ∞, oriented consistently. The cycles orientedly tangent to all three form the intersection of* L *with the projective line*

$$\ell = \{\, x \in \mathbb{RP}^4 : \ \mathcal{B}(x, u) = \mathcal{B}(x, v) = \mathcal{B}(x, w) = 0 \,\},$$

which generically consists of exactly two points; these are the two components of the envelope. A projective representative of this pair is

$$[X : Y : Z : C : S] = \big[-(b+c) :\ d-a :\ \pm 2\sqrt{\Delta} :\ b-c :\ a+d\big] \qquad (\Delta > 0). \quad \text{(A.1)}$$

Sketch of proof. (i) Oriented tangency, $\mathcal{B} = 0$, is preserved by the action of $\mathrm{PSL}_2(\mathbb{R})$. (ii) Three independent hyperplanes in $\mathbb{RP}^4$ meet in a projective line, and a line meets a non-degenerate quadric in at most two points or is contained in it. (iii) A direct computation shows that (A.1) lies on L: $(b+c)^2+(d-a)^2+4\Delta = (a+d)^2+(b-c)^2$. (iv) That the two points are the envelope circles of Propositions A.4 and A.5 can be checked by computing the spherical centre and radius of those circles; we verified this numerically for many random matrices. For example, for $M = I$ the pair (A.1) is the boundary circle with its two orientations, and for the rotation $M = R_\theta$ it consists of the circles $|z| = |\tan(\frac{\pi}{4} \pm \frac{\theta}{2})|$, in agreement with Proposition A.5. □

A.4.4 The Two Circles as Level Sets

For a matrix with $\Delta > 0$, let f be its Möbius transformation, and work in the upper half-plane chart, in which the boundary is $\mathbb{R}$. We describe the envelope as a pair of level sets of a Möbius transformation h adapted to the fixed points of f. Since Möbius transformations map circles and lines to circles and lines, every level set $\{|h(z)| = \rho\}$ and $\{\operatorname{Arg} h(z) = \kappa\}$ is a circle or a line.

Hyperbolic case ($\Delta > 0$, $\tau^2 > 4\Delta$: two real eigenvalues of the same sign). The general picture is shown in Figure A.3.

Proposition A.4. *The envelope of the canonical arrows of a hyperbolic element f of $\mathrm{PSL}_2(\mathbb{R})$ is a pair of oriented cycles through the two fixed points of f on the boundary, obtained from one another by inversion in the boundary followed by reversal of orientation. If $p, q \in \mathbb{R}$ are the fixed points and $h(z) = \frac{z-p}{z-q}$, the envelope is $\{\operatorname{Arg} h(z) = \pm\kappa_H\}$ (modulo*

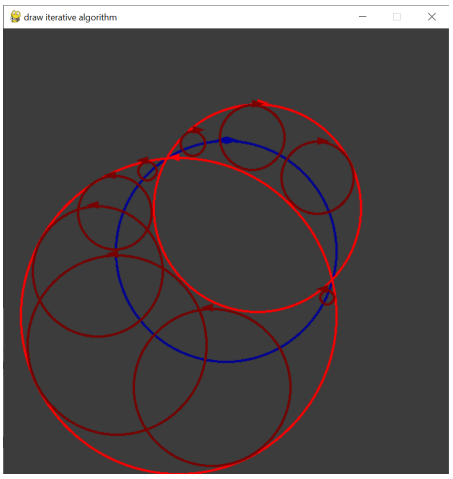
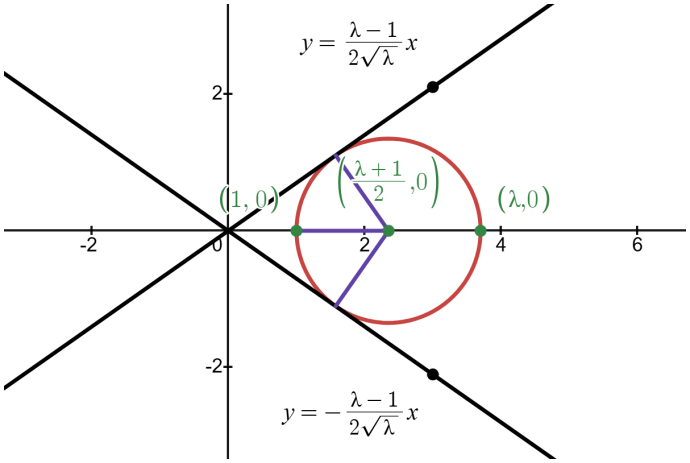


Figure A.3: The envelope for a hyperbolic matrix (two real eigenvalues of the same sign): two mutually inverse oriented circles, both passing through the two fixed points of the Möbius transformation on the boundary circle.

π), *where*

$$\tan \kappa_H = \frac{\sqrt{\tau^2 - 4\Delta}}{2\sqrt{\Delta}}.$$

Proof. There is a chart in which f is diagonal and the boundary is $\mathbb{R}$, namely the one given by h; in it $f(x) = \lambda x$, where $\lambda > 0$ is the ratio of the eigenvalues. The canonical arrow from x to λx is the circle with centre $\frac{1+\lambda}{2}x$ and radius $\frac{|\lambda-1|}{2}|x|$. A line $y = mx$ is tangent to it if and only if its distance from the centre equals the radius, that is, $m^2(1+\lambda)^2 = (\lambda-1)^2(1+m^2)$, which gives $m = \pm\frac{\lambda-1}{2\sqrt{\lambda}}$, independently of x. Hence the unoriented envelope is the pair of lines $y = \pm\frac{\lambda-1}{2\sqrt{\lambda}}x$, as in the following diagram, in which the red circle is the canonical arrow from 1 to $f(1) = \lambda$:



Writing $\lambda = \lambda_1/\lambda_2$ in terms of the eigenvalues, $\frac{\lambda-1}{2\sqrt{\lambda}} = \frac{\lambda_1-\lambda_2}{2\sqrt{\lambda_1\lambda_2}} = \frac{\sqrt{\tau^2-4\Delta}}{2\sqrt{\Delta}}$.

Next, orient the circle so that the arc from 1 to λ is clockwise, and orient the two lines

so that both are tangent to the oriented circle. Then one oriented line is obtained from the other by reflection in the real axis followed by reversal of orientation. Changing back to the chart in which the boundary is the unit circle, and then to a chart in which f is no longer diagonal, are conformal changes of chart, which preserve oriented tangency and orthogonality. This finishes the proof. □

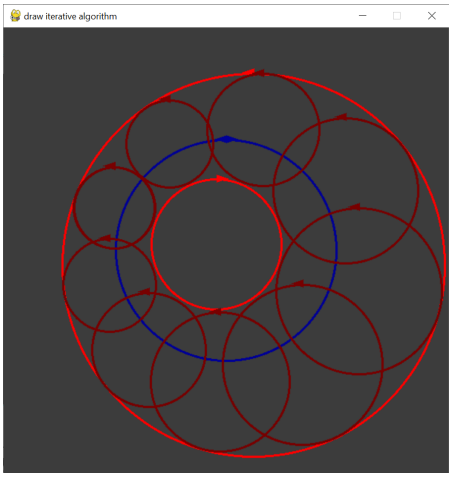


Figure A.4: The envelope for an elliptic matrix (complex conjugate eigenvalues): a circle inside the boundary circle together with its inversion, which lies outside.

Elliptic case ($\Delta > 0$, $\tau^2 < 4\Delta$: a pair of complex conjugate eigenvalues).

Proposition A.5 (Elliptic case). *Let $\Delta > 0$ and $\tau^2 < 4\Delta$, write $\cos\theta = \tau/(2\sqrt{\Delta})$ with $0 < \theta < \pi$, let p be the fixed point of f in the upper half-plane, and let $h(z) = \frac{z-p}{z-\bar{p}}$. Then the envelope consists of the two circles*

$$|h(z)| = \rho^{\pm 1}, \qquad \rho = \left|\tan\left(\frac{\pi}{4} - \frac{\theta}{2}\right)\right|.$$

Sketch of proof. The map h sends the upper half-plane to the unit disc and p to 0, and it conjugates f to the rotation $w \mapsto e^{2i\theta}w$ (up to the sign of the angle). For this rotation, the canonical arrow joining $e^{-i\theta}$ to $e^{i\theta}$ is the circle with centre $\sec\theta$ and radius $|\tan\theta|$, whose closest point to the origin is at distance $|\sec\theta - \tan\theta| = |\tan(\frac{\pi}{4} - \frac{\theta}{2})|$. By rotational symmetry the envelope inside the disc is the circle $|w| = \rho$, and the reflected arcs outside give its inversion $|w| = 1/\rho$. □

Figure A.4 shows an example. The identity matrix corresponds to $\theta \to 0$, $\rho \to 1$, where both circles approach the boundary; an orthogonal matrix with $\theta = \pi/2$ (a rotation by a right angle, up to scaling) gives $\rho = 0$, where the inner circle degenerates to the point p.

A.4.5 The Negative Determinant Case ($\Delta < 0$)

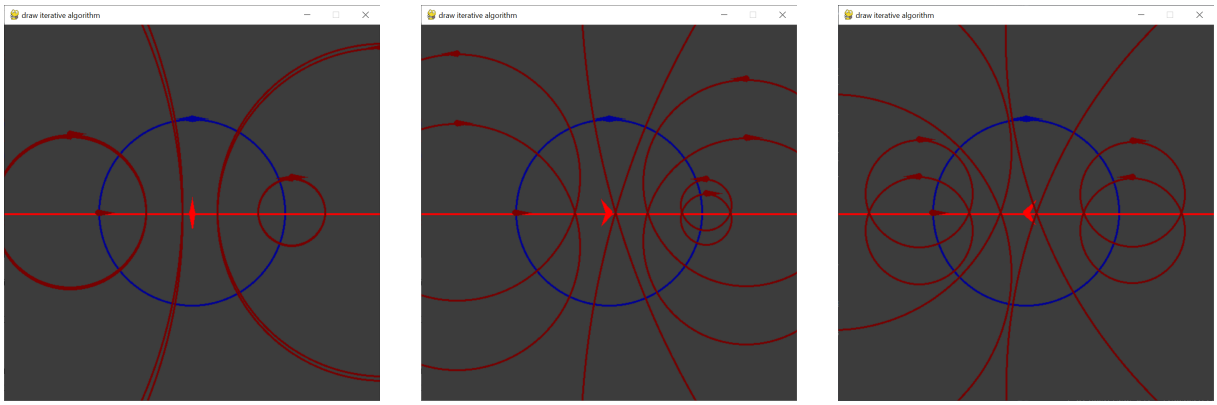


Figure A.5: Matrices with negative determinant. The Möbius transformation reverses the orientation of the boundary, and there is no two-component envelope; instead a distinguished circle orthogonal to the boundary is crossed by the canonical arrows at a fixed angle.

When $\Delta < 0$ the action on the boundary reverses orientation, and there is **no two-component envelope** (Figure A.5). After conjugation, the map takes the form $u \mapsto -\mu\, u$ with $\mu > 0$ in the upper half-plane model. The canonical arrow through x and $-\mu x$ is the circle with centre $\frac{1-\mu}{2}x$ and radius $\frac{1+\mu}{2}|x|$; it crosses the imaginary axis at height $\sqrt{\mu}\,|x|$, and at the crossing point its tangent makes an angle φ with the horizontal, independent of x, where

$$\tan\varphi = \frac{|1-\mu|}{2\sqrt{\mu}}, \qquad 0 \le \varphi < \frac{\pi}{2}. \tag{A.2}$$

Pulling back, there is a distinguished circle Γ orthogonal to the boundary (the image of the imaginary axis) such that every point of Γ lies on two canonical arrows, which cross Γ at the same angle, symmetrically.

A.4.6 Practical Recipe

1. Given $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, compute $\Delta = ad - bc$, $\tau = a + d$, and the fixed points of $f(z) = \frac{az+b}{cz+d}$, which are the roots of $cz^2 + (d-a)z - b = 0$.

2. If $\Delta > 0$, the Lie quadric pair is $[-(b+c) : d-a : \pm 2\sqrt{\Delta} : b-c : a+d]$; alternatively:

   - **Elliptic:** with the fixed point p in the upper half-plane, compute θ from $\cos\theta = \tau/(2\sqrt{\Delta})$, set $\rho = |\tan(\frac{\pi}{4} - \frac{\theta}{2})|$, and plot $\left|\frac{z-p}{z-\bar{p}}\right| = \rho^{\pm 1}$.
   - **Hyperbolic:** with the real fixed points p, q, compute κ_H from $\tan\kappa_H = \sqrt{\tau^2 - 4\Delta}/(2\sqrt{\Delta})$ and plot $\operatorname{Arg}\frac{z-p}{z-q} = \pm\kappa_H$.

   Then map the picture to the disc chart by the Cayley transform.

3. If $\Delta < 0$, conjugate to $u \mapsto -\mu u$, plot the distinguished circle Γ (the pull-back of the imaginary axis), and through sample points of Γ draw the two canonical arrows, which cross Γ symmetrically at a fixed angle determined by (A.2).

A.5 How to Read the Pictures of the QR Algorithm

The remaining figures show runs of the unshifted QR algorithm $M_k = Q_k R_k \mapsto M_{k+1} = R_k Q_k$ in visualisation v2, drawn in the disc chart. They are snapshots of an interactive program. The conventions are as follows.

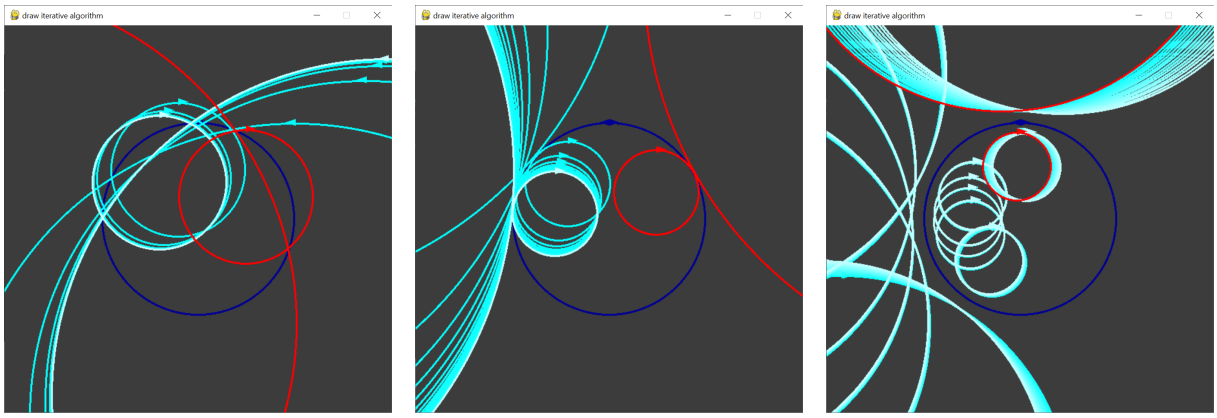
- The dark blue circle is the boundary, the unit circle. It represents the identity matrix and its scalar multiples.
- The input matrix M_0 is shown as a pair of red oriented cycles, mutually inverse in the boundary. The user controls one of them by translating it, scaling it, or reversing its orientation; the other follows, and may become a straight line. In this way the user can explore all 2×2 real matrices with non-negative determinant.

- The iterates $M_1, M_2, \ldots$ are drawn as circles in light blue, becoming lighter with each iteration until they are white.
- Each QR step is an orthogonal similarity, $M_{k+1} = Q_k^T M_k Q_k$. When $\det M > 0$ and R_k has positive diagonal entries, as below, Q_k is a rotation, and similarity by a rotation is rotation of the picture about the origin; so every iterate is a *rotated copy* of the input pair. The algorithm can only rotate.
- The points where a red cycle meets the dark blue circle correspond to the real eigenvectors of M.
- A cycle passing through the point $(-1, 0)$ corresponds to an upper triangular matrix. With R_k normalised to have positive diagonal entries, the iterates of the unshifted QR algorithm, when they converge, converge to an upper triangular matrix, so convergence is visible as the iterates rotating until they pass through $(-1, 0)$. In this picture, the whole purpose of the QR algorithm is to rotate the pair of cycles towards the point $(-1, 0)$.
- Taking the matrix inverse corresponds to reversing the orientation of the cycles.

A.6 The QR Algorithm for Matrices with Non-Negative Determinant

Consider the red cycles in Figure A.6.

- In case 1, the dark blue circle meets the red cycle at two distinct points. These correspond to two *real eigenvectors*. Since $\det M > 0$, the two eigenvalues have the same sign, so M is *diagonalisable with real eigenvalues of the same sign.*
- In case 2, the dark blue circle meets the red cycle at a single point, which corresponds to the only real eigenvector of M. Therefore M is *not diagonalisable, and its only eigenvalue is real.*



Case 1 Case 2 Case 3

Figure A.6: The unshifted QR algorithm for three matrices with positive determinant. Red: input; light blue to white: iterates; dark blue: the boundary circle. Case 1: two real eigenvalues; Case 2: a non-diagonalisable matrix; Case 3: complex eigenvalues.

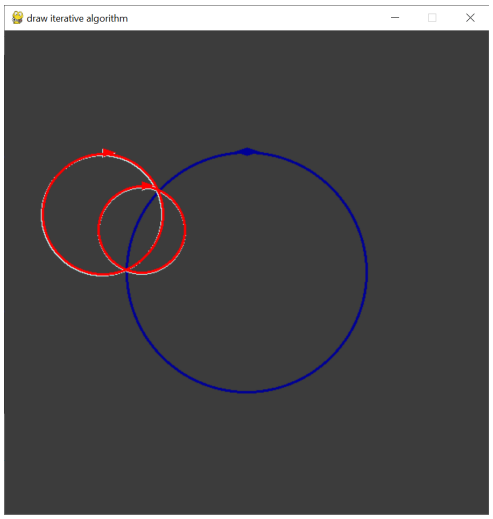
- In case 3, the red cycle does not meet the dark blue circle, so M has no real eigenvector. Its eigenvectors are complex, with a complex eigenvalue λ; since $\bar{\lambda}$ is also an eigenvalue, M is *diagonalisable over* $\mathbb{C}$ *with non-real eigenvalues.*

These are three of the real Jordan types of a 2×2 real matrix. The others are the non-zero scalar matrices, which are represented by the boundary circle itself and are fixed by the algorithm, the non-zero singular matrices, for which the two cycles coincide, and the matrices whose eigenvalues are real with opposite signs, which are treated in Section A.7.

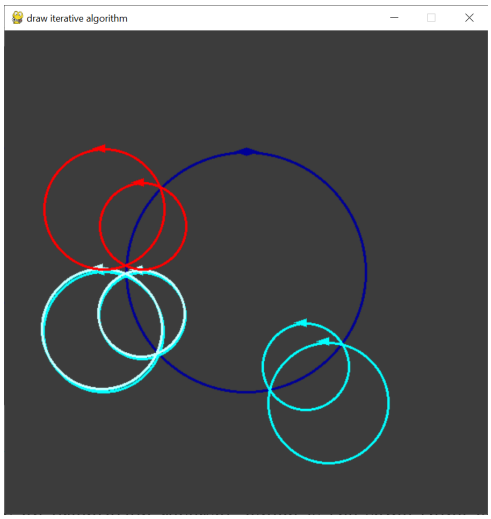
Each light-blue circle is obtained from the red one by a rotation about the origin; this reflects the fact that each iterate is orthogonally similar to the previous ones. In the first two cases the iterates converge, and in the third they do not. When the algorithm converges, the white circles pass through $(-1, 0)$, so the limit is upper triangular. Case 2 (non-diagonalisable) converges more slowly than case 1, but it still converges. Moving from case 1 to case 2 to case 3, convergence slows down until it stops.

A.6.1 Behaviour near the Upper Triangular Limit

In Figure A.7, case 5 is obtained from case 4 by reversing the orientation, which is the same as taking the matrix inverse. Case 4 is an attracting fixed point: small perturbations of it are mapped closer to it. Case 5 is a repelling fixed point: small perturbations are mapped away from it. *The matrix inverse of an attracting fixed point is a repelling fixed*



Case 4



Case 5

Figure A.7: Fixed points of the QR algorithm. Case 4 is an attracting fixed point; case 5, its inverse (orientation reversed), is a repelling fixed point.

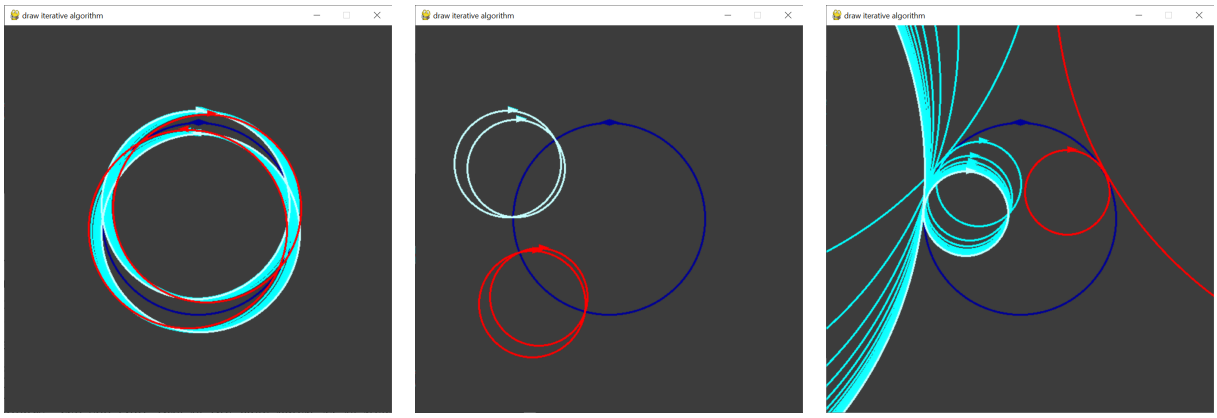
point.

Without going into detail (see [6, 63]), the existence of non-attracting fixed points is a consequence of the fact that one step $(QR = M) \mapsto RQ$ depends continuously on M. This is a topological rather than a quantitative fact. For a variant of QR to be practical, in the sense of converging uniformly rather than only pointwise, it must therefore iterate a *discontinuous* map; Wilkinson shifts, for instance, introduce the necessary discontinuity.

A.6.2 Behaviour near the Identity Matrix and near Singular Matrices

In case 6 (Figure A.8), the two red cycles are very close to the dark blue circle and barely visible, so the matrix is close to a multiple of the identity. The algorithm converges very slowly, as the large amount of light blue shows. This happens because near a repeated eigenvalue the eigenspaces are unstable: the eigenvalues of such a matrix can be estimated accurately, for instance by the Gershgorin circle theorem, but its eigenvectors cannot.

Case 7 is the opposite of case 6: the two mutually inverse cycles almost coincide with each other, as they do when one eigenvalue is much smaller in modulus than the other, $|\lambda_2| \ll |\lambda_1|$. Then convergence is **very fast**, since the subdiagonal entry is multiplied by about λ_2/λ_1 at each step. Case 8, the non-diagonalisable case, converges, but slowly.



Case 6 Case 7 Case 8

Figure A.8: Case 6: a matrix close to the identity (the red cycles nearly coincide with the boundary); convergence is very slow. Case 7: a matrix close to a singular one (the two cycles nearly coincide); convergence is very fast. Case 8: a non-diagonalisable matrix, as in case 2; convergence is slow.

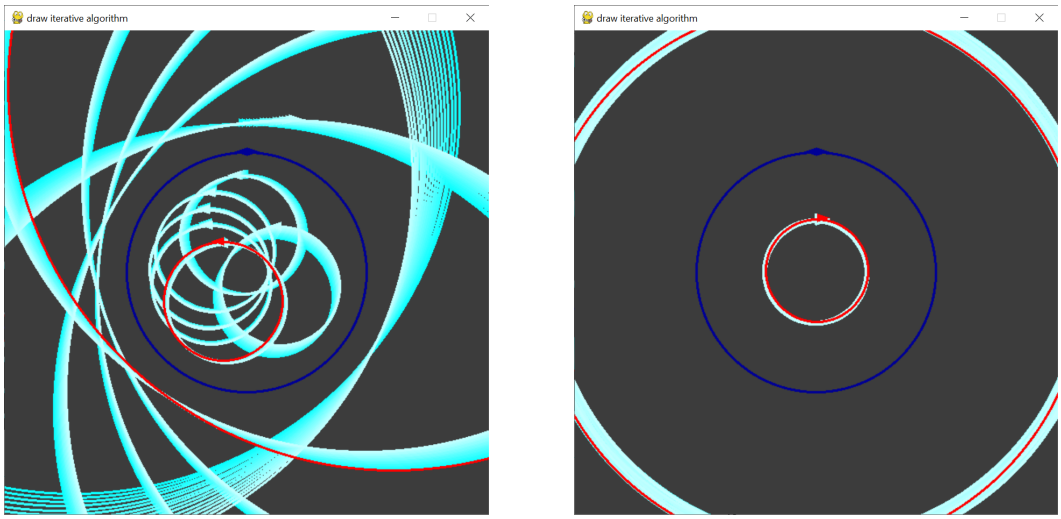
A.6.3 The Role of Shifts

If $\lambda_1/\lambda_2 \approx 1$ we are in case 6 or case 8, and otherwise, when $|\lambda_1/\lambda_2 - 1|$ is large, in case 7. In case 6, slow convergence has a fundamental cause: eigenvectors of a matrix with nearly equal eigenvalues are ill-conditioned, and those of a matrix with exactly equal eigenvalues cannot be computed from approximations of the matrix. In case 8, slow convergence occurs because the matrix lies on the boundary between having real and non-real eigenvalues; when the eigenvalues are not real, the algorithm does not converge at all. In case 7, convergence is fast because $|\lambda_2/\lambda_1|$ is small.

The purpose of *shifting* an $n \times n$ matrix is to find, heuristically, a constant μ such that $M - \mu I$ is nearly singular, which speeds up convergence. In the pictures, shifting moves the matrix away from cases 6 and 8 and towards case 7 [6].

A.6.4 The Complex Eigenvalue Case

For complex eigenvalues the iterates are nearly periodic, except when M is close to an orthogonal matrix (Figure A.9). If the eigenvalues are $re^{\pm 2\pi i m/n}$ with m/n in lowest terms, the period of the iterates is n if n is odd and $n/2$ if n is even; the halving occurs because $M^{n/2}$ is then a scalar multiple of the identity. If they are $re^{\pm 2\pi i x}$ with x irrational, the approximate periods are given by the rational approximations of x. Case 9 shows



Case 9 Case 10

Figure A.9: Matrices with complex eigenvalues. Case 9: the iterates are nearly periodic. Case 10: a matrix close to an orthogonal matrix, whose two cycles are nearly concentric with the boundary circle; it is nearly a fixed point.

the near periodicity. Case 10 is close to an orthogonal matrix: as for the rotations in Proposition A.5, its two cycles are nearly concentric with the boundary circle, and it is nearly a fixed point: a scalar multiple rQ of an orthogonal matrix has the QR factorisation $Q \cdot (rI)$, so one QR step returns rQ itself.

A.7 The QR Algorithm for Matrices with Negative Determinant

For matrices with negative determinant, the picture of Section A.4.5 is used: the user rotates one of the arrowheads, which moves the other one accordingly, and thereby specifies the pair of points where the distinguished circle Γ of the matrix meets the boundary.

Acknowledgements for this appendix. For the work in this appendix, the author would like to thank Jan Techter, Gregory Gutin and Abbas Edalat for helpful discussions.

Appendix B

Source Code of the Library

This appendix prints the source code of the package `dsqlib`, version 0.2.0, the library described in Chapter 8. The test suite, from which the measurements reported in Chapter 8 are taken, is not printed: it is part of the source distribution of the library, which is submitted with this thesis as supplementary material, together with a record of a run of the tests. The modules of the package are printed in the order of Table 8.1, preceded by `__init__.py`, which defines the public interface, and followed by `errors.py`, which defines the exceptions.

Each heading gives the path of a file relative to the root of the library's source tree. The library requires Python 3.10 or later, NumPy 1.24 or later and SciPy 1.10 or later. In the source distribution, the tests are run with pytest, with the directories `src` and `tests` on the Python path. Line numbers, shown on every fifth line, are those of the file; a line too long for the page is broken, and its continuation is marked by ↪.

The code is released under the MIT licence, which reads as follows.

```
MIT License

Copyright (c) 2026 Ran Gutin

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
```

```
    AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
    LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
    OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
    SOFTWARE.
```

B.1 The Package

B.1.1 src/dsqlib/__init__.py

```
"""dsqlib: *-algebras of doubly-signed quivers, localised at the arrows and completed in path degree.

The main objects:

* :class:`DSQ` -- a doubly-signed quiver (``DSQ.build`` from the vertex kinds of Table 5.1);
* :class:`Algebra` and :class:`Element` -- the completed localisation of its path *-algebra;
* :func:`star_sqrt` -- *-square roots by the graded, principal and arbitrary rules;
* :class:`Index` and :class:`Matrix` -- matrices indexed by self-adjoint idempotents;
* :mod:`dsqlib.linalg` -- Householder, QR, Lanczos, Cholesky, LDL and LU, written once for every DSQ;
* :mod:`dsqlib.catalogue` and :mod:`dsqlib.problems` -- the quivers of the thesis catalogue and the
  classical matrix problems they encode.

Example
-------
>>> import numpy as np
>>> from dsqlib import problems, linalg
>>> H = problems.hermitian_problem(np.array([[2.0, 1j], [-1j, 3.0]]))
>>> T, U, _ = linalg.householder(H)
"""

from . import catalogue, linalg, problems
from .algebra import INF, Algebra, Element, quaternion
from .errors import BreakdownError, DSQError, NoSquareRootError, NotInvertibleError, PrecisionError,
    ↪ QuiverError
from .matrix import Index, Matrix, cyclic_positions, orbit_index, plain_index
from .quiver import DSQ, KINDS, SIGNATURE_KINDS
from .sqrt import random_semisimple_unitary, random_unitary, sqrt_graded, sqrt_principal, sqrt_rule,
    ↪ star_sqrt

__version__ = "0.2.0"

__all__ = [
    "DSQ",
    "INF",
    "KINDS",
    "SIGNATURE_KINDS",
    "Algebra",
    "BreakdownError",
    "DSQError",
    "Element",
    "Index",
    "Matrix",
    "NoSquareRootError",
    "NotInvertibleError",
    "PrecisionError",
    "QuiverError",
    "__version__",
    "catalogue",
    "cyclic_positions",
    "linalg",
    "orbit_index",
    "plain_index",
    "problems",
    "quaternion",
    "random_semisimple_unitary",
    "random_unitary",
    "sqrt_graded",
```

```
    "sqrt_principal",
    "sqrt_rule",
    "star_sqrt",
]
```

B.1.2 src/dsqlib/quiver.py

```
"""Doubly-signed quivers (thesis, Definition 5.7), stored at the complexified level.

A doubly-signed quiver (DSQ) has vertices and arrows with two commuting involutions:

* the transpose ``T``, which reverses arrows (``a: i -> j`` gives ``a^T: j^T -> i^T``);
* the conjugation ``B``, which preserves them (``a: i -> j`` gives ``a~: i~ -> j~~~);

and signs:

* ``eps_div`` = +-1 on the vertices fixed by ``B`` (real or quaternionic), 0 elsewhere;
* ``eps_form`` = +-1 on the vertices and arrows fixed by ``T``, 0 elsewhere.

Most users build a DSQ from the ten real vertex kinds of Table 5.1 with :meth:`DSQ.build`. Each kind
expands into one, two or four complexified vertices. Each arrow given is closed under the Klein
four-group {1, T, B, TB}; its partners are named ``a'`` (transpose), ``a~~`` (conjugate) and ``a~'~~
(both), unless they coincide with ``a``.

Block model
-----------
Each complexified vertex ``v`` carries a block size ``d_v`` (1 or 2) and two ``d_v x d_v`` matrices:

* ``G_v``, the vertex form: on ``S_v = M_{d_v}(C)`` the involution is ``X -> G_v X^T G_v^-1``;
* ``C_v``, the real structure: the conjugation is ``X -> C_v conj(X) C_v^-1``.

Each arrow carries signs ``kappa``, ``mu`` = +-1 that fix the involution and the conjugation on its
bimodule: ``(Y a)* = kappa_a G_s Y^T G_t^-1 a^T`` and ``c(Y a) = mu_a C_t conj(Y) C_s^-1 a~~~. They are
found by a finite search such that ``** = 1``, ``cc = 1`` and ``c* = *c``; thesis Proposition 5.14(2)
gives an explicit solution, so the search always succeeds.
"""

from __future__ import annotations

import itertools
from collections.abc import Iterable, Mapping, Sequence
from typing import Any, NamedTuple

import numpy as np

from .errors import QuiverError

__all__ = ["DSQ", "KINDS", "SIGNATURE_KINDS", "Arrow", "Component", "Vertex", "VertexKind"]

_J2 = np.array([[0, 1], [-1, 0]], dtype=complex)


class VertexKind(NamedTuple):
    """One of the ten real vertex kinds of Table 5.1."""

    pattern: str
    """How the kind expands: 'fixed', 'cfix', 'cu', 'exch' or 'cc'."""
    eps_div: int
    eps_form: int
    algebra: str
    """The simple *-algebra of minimal size."""
    unitary_group: str
    """The unitary group of ``M_n`` of that algebra."""


KINDS: dict[str, VertexKind] = {
    "R": VertexKind("fixed", +1, +1, "R, trivial involution", "O(n)"),
    "Rsp": VertexKind("fixed", +1, -1, "M2(R), symplectic involution", "Sp(2n,R)"),
    "Hsp": VertexKind("fixed", -1, -1, "H, quaternion conjugation", "Sp(n)"),
    "Hor": VertexKind("fixed", -1, +1, "H, involution of orthogonal type", "O*(2n)"),
```

```
    "Cor": VertexKind("cfix", 0, +1, "C, trivial involution", "O(n,C)"),
    "Csp": VertexKind("cfix", 0, -1, "M2(C), symplectic involution", "Sp(2n,C)"),
    "Cu": VertexKind("cu", 0, 0, "C, complex conjugation", "U(n)"),
    "RR": VertexKind("exch", +1, 0, "R x R^op, exchange involution", "GL_n(R)"),
    "HH": VertexKind("exch", -1, 0, "H x H^op, exchange involution", "GL_n(H)"),
    "CC": VertexKind("cc", 0, 0, "C x C^op, exchange involution", "GL_n(C)"),
}
"""The ten vertex kinds of Table 5.1, by their short names."""

SIGNATURE_KINDS: tuple[str, ...] = ("R", "Hsp", "Cu")
"""The vertex kinds with a signature (compact unitary group)."""

_GROUP = ("1", "T", "B", "TB")  # the Klein four-group generated by transpose and conjugation
_SUFFIX = {"1": "", "T": "'", "B": "~", "TB": "~'"}


def _group_mul(g: str, h: str) -> str:
    """Product in the Klein group (it is abelian)."""
    s = set(g.replace("1", "")) ^ set(h.replace("1", ""))
    return {frozenset(): "1", frozenset("T"): "T", frozenset("B"): "B", frozenset("TB"): "TB"}[frozenset(s)
    ↪ ]


class Vertex:
    """A complexified vertex.

    Attributes
    ----------
    name, top, bar : str
        The vertex and its partners under the transpose and the conjugation.
    eps_div, eps_form : int
        The signs of Definition 5.7 (0 where they are not defined).
    orbit : str or None
        The base name given to :meth:`DSQ.build` (names the STAR-BAR orbit).
    kind : str
        The vertex kind (a key of :data:`KINDS`).
    block_size : int
        ``d_v``, 1 or 2 (set when the vertex joins a :class:`DSQ`).
    G, C : numpy.ndarray
        The vertex form and the real structure (set when the vertex joins a :class:`DSQ`).
    """

    def __init__(
        self,
        name: str,
        top: str,
        bar: str,
        eps_div: int,
        eps_form: int,
        orbit: str | None = None,
        kind: str | None = None,
    ) -> None:
        self.name, self.top, self.bar = name, top, bar
        self.eps_div, self.eps_form = eps_div, eps_form
        self.orbit = orbit
        self.kind: str = kind or ""
        self.block_size = 1
        self.G = self.C = self.G_inv = self.C_inv = np.eye(1, dtype=complex)
        self._g = self._h = 1  # the scalars G_{v^T} G_v^-T and C_{v~} conj(C_v)

    def __repr__(self) -> str:
        return f"Vertex({self.name!r}, kind={self.kind!r})"


class Arrow:
    """A complexified arrow ``source -> target``.

    Attributes
    ----------
    name, source, target : str
    top, bar : str
        The names of the transposed and the conjugate arrow.
    eps_form : int
```

```
        +-1 for an arrow fixed by the transpose, 0 otherwise.
    kappa, mu : int
        The signs of the involution and the conjugation on the arrow's bimodule.
    square : bool
        Whether the blocks are square (``d_source == d_target``), so that a single element ``X a`` can
        be invertible.
    """

    def __init__(self, name: str, source: str, target: str, top: str, bar: str, eps_form: int = 0) -> None:
        self.name, self.source, self.target = name, source, target
        self.top, self.bar, self.eps_form = top, bar, eps_form
        self.kappa = self.mu = 0
        self.square = True

    def __repr__(self) -> str:
        return f"Arrow({self.name!r}: {self.source} -> {self.target})"


class Component(NamedTuple):
    """A connected component of a quiver, as reported by :meth:`DSQ.localisation_structure`."""

    vertices: list[str]
    arrows: list[str]
    cycle_rank: int


class DSQ:
    """A doubly-signed quiver.

    Usually constructed with :meth:`DSQ.build`; the constructor takes complete vertex and arrow
    dictionaries, validates them (raising :class:`~dsqlib.errors.QuiverError`), sets up the block
    model and solves for the arrow signs.

    Parameters
    ----------
    vertices : mapping of str to Vertex
    arrows : mapping of str to Arrow
    """

    def __init__(self, vertices: Mapping[str, Vertex], arrows: Mapping[str, Arrow]) -> None:
        self.vertices: dict[str, Vertex] = dict(vertices)
        self.arrows: dict[str, Arrow] = dict(arrows)
        self.validate()
        self._vertex_blocks()
        self._solve_signs()
        for a in self.arrows.values():
            a.square = self.vertices[a.source].block_size == self.vertices[a.target].block_size
        self._orbit_cache: dict[str, tuple[str, ...]] | None = None

    # ------------------------------------------------------------------ construction
    @classmethod
    def build(cls, orbits: Iterable[tuple[str, str]], arrows: Iterable[Sequence[Any]] = ()) -> DSQ:
        """Build a DSQ from real vertex kinds and arrows.

        Parameters
        ----------
        orbits : iterable of (base_name, kind)
            ``kind`` is a key of :data:`KINDS`. A kind with one complexified vertex keeps the base
            name ``b``; conjugate vertices are named ``b~``, transposed ones ``b'``, and the fourth
            vertex of ``CC`` is ``b~'``.
        arrows : iterable of (name, source, target) or (name, source, target, options)
            ``source`` and ``target`` are complexified vertex names. ``options`` is a dict with

            * ``sign`` (+-1): required for an arrow fixed by the transpose, i.e. ``i -> i^T``;
            * ``top_is_bar`` (bool): declares ``a^T = a~``, as for the loop of the Hermitian
              eigenproblem over C. This holds for every arrow ``i -> i^H`` (rule (5) of
              Definition 5.7), so the option only checks that the arrow has this form.

        Returns
        -------
        DSQ

        Examples
```

```
        --------
        >>> Q = DSQ.build([("u", "Cu"), ("v", "Cu")], [("a", "u", "v")])   # the complex SVD quiver
        >>> sorted(Q.arrows)
        ['a', "a'", 'a~', "a~'"]
        """
        V: dict[str, Vertex] = {}
        for b, kind in orbits:
            if kind not in KINDS:
                raise QuiverError(f"unknown vertex kind {kind!r}; the kinds are {list(KINDS)}")
            if any(v.orbit == b for v in V.values()):
                raise QuiverError(f"orbit name {b!r} used twice")
            pattern, ed, ef = KINDS[kind][:3]
            if pattern == "fixed":
                V[b] = Vertex(b, b, b, ed, ef, b, kind)
            elif pattern == "cfix":
                V[b] = Vertex(b, b, b + "~", 0, ef, b, kind)
                V[b + "~"] = Vertex(b + "~", b + "~", b, 0, ef, b, kind)
            elif pattern == "cu":
                V[b] = Vertex(b, b + "~", b + "~", 0, 0, b, kind)
                V[b + "~"] = Vertex(b + "~", b, b, 0, 0, b, kind)
            elif pattern == "exch":
                V[b] = Vertex(b, b + "'", b, ed, 0, b, kind)
                V[b + "'"] = Vertex(b + "'", b, b + "'", ed, 0, b, kind)
            else:  # 'cc'
                names = {"1": b, "B": b + "~", "T": b + "'", "TB": b + "~'"}
                for g, nm in names.items():
                    V[nm] = Vertex(nm, names[_group_mul(g, "T")], names[_group_mul(g, "B")], 0, 0, b, kind)
        A: dict[str, Arrow] = {}
        for spec in arrows:
            name, s, t = spec[:3]
            opts = spec[3] if len(spec) > 3 else {}
            unknown = set(opts) - {"sign", "top_is_bar"}
            if unknown:
                raise QuiverError(
                    f"arrow {name}: unknown options {sorted(unknown)} (the options are 'sign', 'top_is_bar
    ↪ ')"
                )
            for v in (s, t):
                if v not in V:
                    raise QuiverError(f"arrow {name}: unknown vertex {v!r}; the vertices are {list(V)}")
            for a in cls._arrow_orbit(V, name, s, t, opts):
                if a.name in A:
                    raise QuiverError(f"arrow name {a.name!r} used twice")
                A[a.name] = a
        return cls(V, A)

    @staticmethod
    def _act(V: Mapping[str, Vertex], g: str, v: str) -> str:
        if g in ("T", "TB"):
            v = V[v].top
        if g in ("B", "TB"):
            v = V[v].bar
        return v

    @classmethod
    def _arrow_orbit(cls, V: Mapping[str, Vertex], name: str, s: str, t: str, opts: Mapping[str, Any]) ->
    ↪ list[Arrow]:
        """Close one arrow under the Klein group, respecting the normalisations of Definitions 5.4
        and 5.7."""

        def act(g: str, v: str) -> str:
            return cls._act(V, g, v)

        ends = {
            "1": (s, t),
            "T": (act("T", t), act("T", s)),
            "B": (act("B", s), act("B", t)),
            "TB": (act("TB", t), act("TB", s)),
        }
        stab = {"1"}
        sign = opts.get("sign")
        if t == V[s].top:  # arrows i -> i^T are fixed by T
            stab.add("T")
```

```
            if sign is None:
                raise QuiverError(f"arrow {name}: {s} -> {t} = {s}^T is fixed by the transpose; give its sign")
            elif sign is not None:
                raise QuiverError(f"arrow {name}: a sign is only allowed for arrows i -> i^T")
            if V[s].bar == s and V[t].bar == t and V[s].eps_div == V[t].eps_div:
                stab.add("B")  # real arrows between conjugation-fixed vertices
            if ends["TB"] == (s, t):
                stab.add("TB")  # arrows i -> i^H are fixed by the conjugate transpose (rule (5))
            elif opts.get("top_is_bar"):
                raise QuiverError(f"arrow {name}: top_is_bar needs {t} = ({s})^T~")
            for g, h in itertools.product(list(stab), repeat=2):
                stab.add(_group_mul(g, h))
            for g in stab:
                if ends[g] != (s, t):
                    raise QuiverError(f"arrow {name}: inconsistent with the vertex involutions")
            cosets: dict[frozenset[str], str] = {}
            for g in _GROUP:
                key = frozenset(_group_mul(g, h) for h in stab)
                if key not in cosets:
                    cosets[key] = name + _SUFFIX[g]

            def name_of(g: str) -> str:
                return cosets[frozenset(_group_mul(g, h) for h in stab)]

            out = []
            for key, aname in cosets.items():
                g = next(iter(key))
                ss, tt = ends[g]
                eps = int(sign) if (sign is not None and "T" in stab) else 0
                out.append(Arrow(aname, ss, tt, name_of(_group_mul("T", g)), name_of(_group_mul("B", g)), eps))
            return out

        # ------------------------------------------------------------------ checks
        def validate(self) -> None:
            """Check the axioms of Definition 5.7; raise :class:`~dsqlib.errors.QuiverError` listing every violation."""
            V, A = self.vertices, self.arrows
            err = []
            for v in V.values():
                if v.top not in V or v.bar not in V:
                    err.append(f"{v.name}: unknown partner")
                elif V[v.top].top != v.name or V[v.bar].bar != v.name:
                    err.append(f"{v.name}: maps are not involutions")
                elif V[v.top].bar != V[v.bar].top:
                    err.append(f"{v.name}: transpose and conjugation do not commute")
                if (v.bar == v.name) != (v.eps_div in (1, -1)):
                    err.append(f"{v.name}: eps_div must be +-1 exactly on conjugation-fixed vertices")
                if (v.top == v.name) != (v.eps_form in (1, -1)):
                    err.append(f"{v.name}: eps_form must be +-1 exactly on transpose-fixed vertices")
            for v in V.values():
                if v.top in V and V[v.top].eps_div != v.eps_div:
                    err.append(f"{v.name}: eps_div(v^T) != eps_div(v)")
                if v.bar in V and V[v.bar].eps_form != v.eps_form:
                    err.append(f"{v.name}: eps_form(v~) != eps_form(v)")
            for a in A.values():
                if a.source not in V or a.target not in V:
                    err.append(f"{a.name}: unknown endpoint")
                    continue
                if a.top not in A or a.bar not in A:
                    err.append(f"{a.name}: unknown partner")
                    continue
                at, ab = A[a.top], A[a.bar]
                if at.top != a.name or ab.bar != a.name:
                    err.append(f"{a.name}: maps are not involutions")
                if (at.source, at.target) != (V[a.target].top, V[a.source].top):
                    err.append(f"{a.name}: transpose must reverse the arrow")
                if (ab.source, ab.target) != (V[a.source].bar, V[a.target].bar):
                    err.append(f"{a.name}: conjugation must preserve the arrow")
                if A[at.bar].name != A[ab.top].name:
                    err.append(f"{a.name}: transpose and conjugation do not commute")
                if (a.top == a.name) != (a.eps_form in (1, -1)):
                    err.append(f"{a.name}: eps_form must be +-1 exactly on transpose-fixed arrows")
```

```
            if ab.eps_form != a.eps_form:
                err.append(f"{a.name}: eps_form(a~) != eps_form(a)")
            if a.target == V[a.source].top and a.top != a.name:
                err.append(f"{a.name}: arrows i -> i^T must be fixed by the transpose")
            conj_fixed_ends = V[a.source].bar == a.source and V[a.target].bar == a.target
            if conj_fixed_ends and (a.bar == a.name) != (V[a.source].eps_div == V[a.target].eps_div):
                err.append(f"{a.name}: between conjugation-fixed vertices, a~ = a exactly when eps_div
    ↪ agree")
            if a.target == V[V[a.source].bar].top and A[ab.top].name != a.name:
                err.append(f"{a.name}: arrows i -> i^H must be fixed by the conjugate transpose")
        if err:
            raise QuiverError("not a doubly-signed quiver:\n  " + "\n  ".join(err))

    # ------------------------------------------------------------------ vertex blocks and signs
    def _vertex_blocks(self) -> None:
        for v in self.vertices.values():
            transpose_fixed, conj_fixed = v.top == v.name, v.bar == v.name
            v.block_size = 2 if (v.eps_div == -1 or v.eps_form == -1) else 1
            eye = np.eye(v.block_size, dtype=complex)
            v.G = (_J2.copy() if v.eps_form == -1 else eye.copy()) if transpose_fixed else eye.copy()
            v.C = (_J2.copy() if v.eps_div == -1 else eye.copy()) if conj_fixed else eye.copy()
            v.G_inv, v.C_inv = np.linalg.inv(v.G), np.linalg.inv(v.C)
            v._g = v.eps_form if transpose_fixed else 1
            v._h = v.eps_div if conj_fixed else 1
            if not v.kind:
                v.kind = self._detect_kind(v)
        for v in self.vertices.values():  # the vertex data must make * and c commute
            vt, vb = self.vertices[v.top], self.vertices[v.bar]
            if not np.allclose(vt.C @ np.conj(v.G) @ v.C.T, vb.G):
                raise QuiverError(f"vertex {v.name}: C G C^T != G~ (internal error)")

    @staticmethod
    def _detect_kind(v: Vertex) -> str:
        transpose_fixed, conj_fixed = v.top == v.name, v.bar == v.name
        if transpose_fixed and conj_fixed:
            pattern = "fixed"
        elif transpose_fixed:
            pattern = "cfix"
        elif conj_fixed:
            pattern = "exch"
        elif v.top == v.bar:
            pattern = "cu"
        else:
            pattern = "cc"
        for k, kind in KINDS.items():
            p = kind.pattern
            if (
                p == pattern
                and (kind.eps_div == v.eps_div or p in ("cfix", "cu", "cc"))
                and (kind.eps_form == v.eps_form or p in ("cu", "exch", "cc"))
            ):
                return k
        raise QuiverError(f"vertex {v.name}: no vertex kind matches its data")

    def _solve_signs(self) -> None:
        """kappa, mu in {+-1} on each arrow orbit with

        (I)   kappa_a kappa_{a^T} = g_s g_t          (** = 1)
        (II)  mu_a mu_{a~}        = h_s h_t          (cc = 1)
        (III) kappa_a mu_{a^T}    = mu_a kappa_{a~}  (c* = *c)

        and the normalisations kappa_a = eps_form(a) when a^T = a, mu_a = +1 when a~ = a."""
        A = self.arrows
        done: set[str] = set()
        for a0 in A.values():
            if a0.name in done:
                continue
            orb = sorted({a0.name, a0.top, a0.bar, A[a0.top].bar})
            done |= set(orb)
            for bits in itertools.product((1, -1), repeat=2 * len(orb)):
                kap = dict(zip(orb, bits[: len(orb)], strict=True))
                mu = dict(zip(orb, bits[len(orb) :], strict=True))
                if all(self._signs_ok(A[n], kap, mu) for n in orb):
```

```
                    for n in orb:
                        A[n].kappa, A[n].mu = kap[n], mu[n]
                    break
            else:
                raise QuiverError(f"no consistent signs for the arrow orbit {orb}")

    def _signs_ok(self, a: Arrow, kap: Mapping[str, int], mu: Mapping[str, int]) -> bool:
        vs, vt = self.vertices[a.source], self.vertices[a.target]
        n = a.name
        if a.top == n and kap[n] != a.eps_form:
            return False
        if a.bar == n and mu[n] != 1:
            return False
        return (
            kap[n] * kap[a.top] == vs._g * vt._g
            and mu[n] * mu[a.bar] == vs._h * vt._h
            and kap[n] * mu[a.top] == mu[n] * kap[a.bar]
        )

    # ------------------------------------------------------------------ structure
    def star_bar_orbits(self) -> list[list[str]]:
        """The orbits of <T, B> on the vertices (they correspond to the *-simple factors of
        ``B/rad B``), each in the order of :attr:`vertices`."""
        order = list(self.vertices).index
        seen: set[str] = set()
        out = []
        for v in self.vertices:
            if v in seen:
                continue
            orb: set[str] = set()
            todo = [v]
            while todo:
                x = todo.pop()
                if x in orb:
                    continue
                orb.add(x)
                todo += [self.vertices[x].top, self.vertices[x].bar]
            seen |= orb
            out.append(sorted(orb, key=order))
        return out

    def orbit_of(self, v: str) -> tuple[str, ...]:
        """The STAR-BAR orbit of the vertex ``v``."""
        if self._orbit_cache is None:
            self._orbit_cache = {u: tuple(o) for o in self.star_bar_orbits() for u in o}
        return self._orbit_cache[v]

    def orbit_names(self) -> dict[str, tuple[str, ...]]:
        """The STAR-BAR orbits by name: the base name given to :meth:`build` (or the first vertex)."""
        out = {}
        for o in self.star_bar_orbits():
            name = self.vertices[o[0]].orbit or o[0]
            out[name] = tuple(o)
        return out

    def localisation_structure(self) -> list[Component]:
        """The connected components, with their cycle ranks. Inverting every arrow of a component with
        n vertices and cycle rank r turns its basic algebra into ``M_n(C[F_r])`` (``F_r`` free of
        rank r), before completion."""
        parent = {v: v for v in self.vertices}

        def find(x: str) -> str:
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for a in self.arrows.values():
            parent[find(a.source)] = find(a.target)
        comps: dict[str, list[str]] = {}
        for v in self.vertices:
            comps.setdefault(find(v), []).append(v)
        out = []
```

```
            for vs in comps.values():
                names = [a.name for a in self.arrows.values() if a.source in vs]
                out.append(Component(vs, names, len(names) - len(vs) + 1))
        return out

    def describe(self) -> str:
        """A multi-line summary: orbits with their kinds, arrows with their partners, components."""
        lines = ["STAR-BAR orbits (Table 5.1 kinds):"]
        for o in self.star_bar_orbits():
            v = self.vertices[o[0]]
            k = KINDS[v.kind]
            lines.append(
                f"  {{{', '.join(o)}}}: {v.kind:3s}  {k.algebra}, unitary group {k.unitary_group}, "
                f"block size {v.block_size}"
            )
        lines.append("arrows:")
        for a in self.arrows.values():
            extra = f"  eps_form {a.eps_form:+d}" if a.eps_form else ""
            lines.append(
                f"  {a.name}: {a.source} -> {a.target}   T: {a.top}   bar: {a.bar}{extra}   "
                f"blocks {self.vertices[a.target].block_size}x{self.vertices[a.source].block_size}"
            )
        lines.append("localisation (every arrow inverted), per connected component:")
        for c in self.localisation_structure():
            lines.append(f"  vertices {c.vertices}, cycle rank {c.cycle_rank}")
        return "\n".join(lines)

    def __repr__(self) -> str:
        kinds = ", ".join(f"{name}:{self.vertices[vs[0]].kind}" for name, vs in self.orbit_names().items())
        return f"DSQ(orbits=[{kinds}], arrows={list(self.arrows)})"
```

B.1.3 src/dsqlib/algebra.py

```
"""The completed localisation of the path *-algebra of a doubly-signed quiver.

Localisation
    Every arrow ``a: i -> j`` receives a formal *local* inverse ``a^-1: j -> i`` with ``a^-1 a = e_i``
    and ``a a^-1 = e_j`` (Gabriel-Zisman / Cohn localisation: only the arrow is inverted, between its
    two vertices). The result is spanned by reduced words in the arrows and their inverses, and the
    product is concatenation with cancellation; this works for every quiver, wild ones included. The
    involution extends by ``(a^-1)* = (a*)^-1`` and the conjugation by ``c(a^-1) = c(a)^-1``. When
    ``d_i != d_j`` no single element ``X a`` is invertible, but sums over several arrows can be
    (Morita equivalence), which the block model represents exactly.

Completion
    The path degree (+1 per arrow, -1 per inverse letter) is a grading. An element is a series
    ``sum_d x_d`` of homogeneous parts, bounded below and known modulo degree ``>= prec``
    (``prec = inf`` for an exact element), like a Laurent series known to some order. An element
    whose lowest-degree part is a single word is a unit, inverted by a geometric series.

Coefficients
    Everything is computed in the complexification with complex floating-point blocks: a word from
    ``s`` to ``t`` carries a ``d_t x d_s`` block. A block in which a sum cancels to within ``tol``
    (relative, default 1e-12) of the size of its summands is set to zero, so exact zeros of the
    algebra come out as zeros whatever the scale of the data. The real algebra is the fixed set of
    the conjugation; real inputs give real outputs.

Words
    A word is a reduced tuple of letters ``(arrow_name, +1 or -1)``, written as for composition of
    functions (the last letter is applied first). Functions that accept words from users take a list
    of arrow names, with ``'^-1'`` appended for an inverse letter, e.g. ``['a', "a'"]`` or ``['b^-1']``.
"""

from __future__ import annotations

import numbers
from collections import defaultdict
from collections.abc import Iterable, Mapping, Sequence
from functools import cache
from typing import TYPE_CHECKING, Any
```

```
import numpy as np

from .errors import NotInvertibleError, PrecisionError

if TYPE_CHECKING:
    from .quiver import DSQ, Arrow, Vertex

__all__ = ["INF", "Algebra", "Element", "Key", "Scalar", "Word", "quaternion"]

INF = float("inf")
"""The precision of an exact element."""

Word = tuple[tuple[str, int], ...]
"""A reduced word: a tuple of letters ``(arrow_name, +1 | -1)``; the last letter is applied first."""

Key = tuple[str, str, Word]
"""The key of a term: ``(target, source, word)``."""

_WITHIN = ("support", "orbits", "all")

Scalar = int | float | complex | np.number
"""A number that can scale an element."""
_SCALARS = (int, float, complex, np.number)


class Element:
    """An element of an :class:`Algebra`: terms ``{(target, source, word): block}``, known modulo
    degree ``>= prec``.

    Elements support ``+``, ``-``, ``*`` (the algebra product, or scaling by a number), ``/`` by a
    number and ``**`` with a non-negative integer exponent; a number added to an element stands for
    that multiple of the unit. ``x.H`` is the involution ``x*``.

    Elements are normally made by the constructors of :class:`Algebra` (``vertex``, ``word``,
    ``arrow``, ``one``, ``element``) and by arithmetic, not directly.
    """

    __slots__ = ("algebra", "prec", "terms")
    __array_ufunc__ = None  # numpy operands defer to our reflected operators

    def __init__(self, algebra: Algebra, terms: dict[Key, np.ndarray] | None = None, prec: float = INF) ->
    ↪ None:
        self.algebra = algebra
        self.terms: dict[Key, np.ndarray] = terms if terms is not None else {}
        self.prec = prec

    # ------------------------------------------------------------------ arithmetic
    def _coerce(self, other: Any) -> Element:
        if isinstance(other, Element):
            if other.algebra is not self.algebra:
                raise ValueError("elements of different algebras")
            return other
        if isinstance(other, _SCALARS):
            return self.algebra.scale(other, self.algebra.one())
        return NotImplemented

    def __add__(self, other: Any) -> Element:
        o = self._coerce(other)
        return NotImplemented if o is NotImplemented else self.algebra.add(self, o)

    __radd__ = __add__

    def __sub__(self, other: Any) -> Element:
        o = self._coerce(other)
        return NotImplemented if o is NotImplemented else self.algebra.sub(self, o)

    def __rsub__(self, other: Any) -> Element:
        o = self._coerce(other)
        return NotImplemented if o is NotImplemented else self.algebra.sub(o, self)

    def __neg__(self) -> Element:
        return self.algebra.scale(-1.0, self)
```

```
110
        def __pos__(self) -> Element:
            return self

        def __mul__(self, other: Any) -> Element:
115         if isinstance(other, Element):
                return self.algebra.mul(self, self._coerce(other))
            if isinstance(other, _SCALARS):
                return self.algebra.scale(other, self)
            return NotImplemented
120
        def __rmul__(self, other: Any) -> Element:
            if isinstance(other, _SCALARS):
                return self.algebra.scale(other, self)
            return NotImplemented
125
        def __truediv__(self, other: Any) -> Element:
            if isinstance(other, _SCALARS):
                return self.algebra.scale(1.0 / other, self)
            return NotImplemented
130
        def __pow__(self, n: int) -> Element:
            if not isinstance(n, numbers.Integral) or n < 0:
                raise ValueError("only non-negative integer powers are defined; use inverse() for inverses")
            out = self.algebra.one()
135         for _ in range(int(n)):
                out = self.algebra.mul(out, self)
            return out

        # ------------------------------------------------------------------ structure maps
140     @property
        def H(self) -> Element:
            """The involution ``x*``."""
            return self.algebra.star(self)

145     def conj(self) -> Element:
            """The conjugation ``c(x)`` (a conjugate-linear automorphism)."""
            return self.algebra.conj(self)

        def real(self) -> Element:
150         """The real part ``(x + c(x)) / 2``."""
            return self.algebra.real(self)

        def inverse(self, within: str = "support") -> Element:
            """The inverse; see :meth:`Algebra.inverse`."""
155         return self.algebra.inverse(self, within)

        def sqrt(self, rule: str = "graded", choice: str = "positive", split: str = "left", rng: Any = None) ->
        ↪  Element:
            """A *-square root ``y`` with ``y* y = x``; see :func:`dsqlib.sqrt.star_sqrt`."""
            from .sqrt import star_sqrt
160
            return star_sqrt(self, rule, choice, split, rng)

        # ------------------------------------------------------------------ degrees and precision
        @property
165     def valuation(self) -> float:
            """The lowest degree present (or ``prec``, whichever is smaller); ``inf`` for an exact zero."""
            return self.algebra.valuation(self)

        @property
170     def degrees(self) -> list[int]:
            """The degrees of the words present, in increasing order."""
            return self.algebra.degrees(self)

        @property
175     def is_zero(self) -> bool:
            """True if no term is present (the element may still be inexact: ``O(deg p)``)."""
            return not self.terms

        @property
180     def is_exact(self) -> bool:
            """True if the element is known exactly (``prec = inf``)."""
```

```
            return self.prec == INF

        def part(self, d: int) -> Element:
            """The homogeneous part of degree ``d`` (``0(deg prec)`` if the element is not known at
            degree ``d``)."""
            return self.algebra.part(self, d)

        def truncate(self, prec: float) -> Element:
            """The element known modulo degree ``>= prec`` only."""
            return self.algebra.truncate(self, prec)

        # ------------------------------------------------------------------ sizes and tests
        def norm(self) -> float:
            """The sum of the absolute values of all coefficients."""
            return self.algebra.norm(self)

        def dist(self, other: Element) -> float:
            """The norm of ``self - other`` without any rounding to zero (see :meth:`Algebra.dist`)."""
            return self.algebra.dist(self, other)

        def is_real(self, tol: float = 1e-9) -> bool:
            """Whether ``c(x) = x`` to relative tolerance ``tol``."""
            return self.algebra.is_real(self, tol)

        def is_selfadjoint(self, tol: float = 1e-9) -> bool:
            """Whether ``x* = x`` to relative tolerance ``tol``."""
            return self.algebra.is_selfadjoint(self, tol)

        @property
        def support(self) -> list[tuple[str, str]]:
            """The corners ``(target, source)`` in which the element has terms."""
            return self.algebra.support(self)

        def corner(self, target: str, source: str) -> Element:
            """The part of the element in the corner ``e_target A e_source``."""
            return self.algebra.corner(self, target, source)

        def coefficient(
            self, letters: Sequence[str] = (), target: str | None = None, source: str | None = None
        ) -> np.ndarray:
            """The block of the given word.

            Parameters
            ----------
            letters : sequence of str
                The word as arrow names (``'^-1'`` appended for inverses); empty for a vertex.
            target, source : str, optional
                The end vertices: required for the empty word (and then equal); for a non-empty word they
                default to its ends, and must agree with them if given.

            Returns
            -------
            numpy.ndarray
                The ``d_target x d_source`` block (zeros if the word is absent).

            Raises
            ------
            ValueError
                If the word is invalid, does not have the given ends, or has a degree at which the element
                is not known (``>= prec``).
            """
            return self.algebra.coefficient(self, letters, target, source)

        # ------------------------------------------------------------------ display
        def format(self, digits: int = 4) -> str:
            """The element as a sum of ``coefficient·word`` terms."""
            return self.algebra.format(self, digits)

        def __repr__(self) -> str:
            return self.algebra.format(self)


    class Algebra:
```

```
    """The completed localisation of the path *-algebra of a doubly-signed quiver.

    Parameters
    ----------
    quiver : DSQ
    tol : float, optional
        Relative cancellation tolerance: a block in which a sum cancels to within ``tol`` of the size
        of its summands is set to zero.
    relprec : int, optional
        The number of degrees (relative to the lowest one) to which series inverses and square roots
        of exact elements are computed.

    Notes
    -----
    Besides the element constructors, the class offers every operation in functional form
    (``add``, ``mul``, ``star``, ...); the operators of :class:`Element` call these.

    Examples
    --------
    >>> from dsqlib import Algebra, catalogue
    >>> A = Algebra(catalogue.svd("R"), relprec=6)
    >>> a = A.arrow("a")
    >>> A.word(["a^-1"]) * a  # a^-1 is a local inverse
    1·e_u
    >>> (1 + 0.5 * a).inverse()  # a a = 0, so the series stops
    1·e_u + 1·e_v + -0.5·a
    >>> (1 + 0.5 * (a + A.arrow("a'"))).inverse().prec  # an infinite series, known to degree relprec
    6
    """

    def __init__(self, quiver: DSQ, tol: float = 1e-12, relprec: int = 16) -> None:
        self.quiver = quiver
        self.tol = tol
        self.relprec = relprec
        self.vertices: dict[str, Vertex] = quiver.vertices
        self.arrows: dict[str, Arrow] = quiver.arrows

    def __repr__(self) -> str:
        return f"Algebra({self.quiver!r}, tol={self.tol:g}, relprec={self.relprec})"

    def block_size(self, v: str) -> int:
        """``d_v``, the block size of the vertex ``v``."""
        return self.vertices[v].block_size

    # ------------------------------------------------------------------ words
    @staticmethod
    @cache
    def degree(w: Word) -> int:
        """The path degree of a word (+1 per arrow, -1 per inverse letter)."""
        return sum(e for _, e in w)

    @staticmethod
    def concat(w1: Word, w2: Word) -> Word:
        """The reduced product ``w1 w2`` (``w2`` applied first)."""
        i, j = len(w1), 0
        while i > 0 and j < len(w2) and w1[i - 1][0] == w2[j][0] and w1[i - 1][1] == -w2[j][1]:
            i -= 1
            j += 1
        return w1[:i] + w2[j:]

    @staticmethod
    def word_inverse(w: Word) -> Word:
        """The inverse word ``w^-1``."""
        return tuple((n, -e) for n, e in reversed(w))

    def word_transpose(self, w: Word) -> Word:
        """The transposed word ``w^T``."""
        return tuple((self.arrows[n].top, e) for n, e in reversed(w))

    def word_conjugate(self, w: Word) -> Word:
        """The conjugate word ``w~``."""
        return tuple((self.arrows[n].bar, e) for n, e in w)
```

```
    def word_kappa(self, w: Word) -> int:
        """The product of the involution signs ``kappa`` of the letters of ``w``."""
        k = 1
        for n, _ in w:
            k *= self.arrows[n].kappa
        return k

    def word_mu(self, w: Word) -> int:
        """The product of the conjugation signs ``mu`` of the letters of ``w``."""
        m = 1
        for n, _ in w:
            m *= self.arrows[n].mu
        return m

    def word_ends(self, w: Word) -> tuple[str, str]:
        """``(target, source)`` of a non-empty word."""
        n, e = w[-1]
        a = self.arrows[n]
        src = a.source if e > 0 else a.target
        n, e = w[0]
        a = self.arrows[n]
        tgt = a.target if e > 0 else a.source
        return tgt, src

    @staticmethod
    def word_str(w: Word, vertex: str | None = None) -> str:
        """A readable form of a word (``e_v`` for the empty word at ``v``)."""
        if not w:
            return f"e_{vertex}"
        return "·".join(n if e > 0 else n + "⁻¹" for n, e in w)

    def parse_word(self, letters: Iterable[str], check: bool = True) -> Word:
        """The reduced word of a list of arrow names (``'^-1'`` appended for an inverse letter).

        With ``check`` (the default) the arrows must exist and consecutive letters must compose
        (the last letter is applied first); a ``ValueError`` says which do not.
        """
        w = tuple((L[:-3], -1) if L.endswith("^-1") else (L, 1) for L in letters)
        if check:
            for n, _ in w:
                if n not in self.arrows:
                    raise ValueError(f"unknown arrow {n!r}; the arrows are {list(self.arrows)}")

            def src(letter: tuple[str, int]) -> str:
                a = self.arrows[letter[0]]
                return a.source if letter[1] > 0 else a.target

            def tgt(letter: tuple[str, int]) -> str:
                a = self.arrows[letter[0]]
                return a.target if letter[1] > 0 else a.source

            for i in range(len(w) - 1):
                if tgt(w[i + 1]) != src(w[i]):
                    raise ValueError(
                        f"the letters {w[i + 1][0]} and {w[i][0]} do not compose "
                        f"({tgt(w[i + 1])} != {src(w[i])}); the last letter is applied first"
                    )
        red: Word = ()
        for L in w:
            red = self.concat(red, (L,))
        return red

    # ------------------------------------------------------------------ construction
    def element(self, terms: Mapping[Key, np.ndarray], prec: float = INF) -> Element:
        """An element from ``{(target, source, word): block}``, dropping words of degree ``>= prec``
        and zero blocks."""
        out = {}
        for k, X in terms.items():
            if self.degree(k[2]) >= prec:
                continue
            if np.any(X != 0):
                out[k] = X
        return Element(self, out, prec)
```

```
    def zero(self, prec: float = INF) -> Element:
        """Zero; with a finite ``prec``, the element ``0(deg prec)``."""
        return Element(self, {}, prec)

    def idempotent(self, v: str) -> Element:
        """The vertex idempotent ``e_v``."""
        if v not in self.vertices:
            raise ValueError(f"unknown vertex {v!r}; the vertices are {list(self.vertices)}")
        return Element(self, {(v, v, ()): np.eye(self.block_size(v), dtype=complex)})

    def one(self) -> Element:
        """The unit ``1 = sum of e_v``."""
        return Element(self, {(v, v, ()): np.eye(x.block_size, dtype=complex) for v, x in self.vertices.
    ↪ items()})

    def _block(self, X: Any, rows: int, cols: int) -> np.ndarray:
        X = np.atleast_2d(np.asarray(X, dtype=complex))
        if X.shape == (1, 1) and (rows, cols) != (1, 1):
            if rows != cols:
                raise ValueError("a scalar block needs equal block sizes")
            X = X[0, 0] * np.eye(rows, dtype=complex)
        if X.shape != (rows, cols):
            raise ValueError(f"block must be {rows}x{cols}")
        return X

    def vertex(self, v: str, X: Any = 1.0) -> Element:
        """``X e_v`` with ``X`` a ``d_v x d_v`` block (a number means a multiple of the identity)."""
        if v not in self.vertices:
            raise ValueError(f"unknown vertex {v!r}; the vertices are {list(self.vertices)}")
        d = self.block_size(v)
        return self.element({(v, v, ()): self._block(X, d, d)})

    def word(self, letters: Iterable[str], X: Any = 1.0) -> Element:
        """``X w`` for the word ``w`` given as a list of arrow names (``'^-1'`` for a local inverse;
        the last letter is applied first). ``X`` is a ``d_target x d_source`` block or a number."""
        red = self.parse_word(letters)
        if not red:
            raise ValueError("the word reduces to a vertex; use vertex()")
        t, s = self.word_ends(red)
        return self.element({(t, s, red): self._block(X, self.block_size(t), self.block_size(s))})

    def arrow(self, name: str, X: Any = 1.0) -> Element:
        """``X a`` for the arrow ``a`` (or ``X a^-1`` for ``name = 'a^-1'``)."""
        return self.word([name], X)

    def real_element(self, x: Element) -> Element:
        """The real element whose coefficients at the words of ``x`` are those of ``x``.

        A term ``X w`` with ``c(w) != w`` contributes ``X w + c(X w)``; a term with ``c(w) = w`` must
        have a block of real type and is kept (projected onto the real blocks). So for a complex
        vertex ``z e_v`` gives ``z e_v + conj(z) e_v~``, and for the complex SVD quiver ``z a`` gives
        ``z a + conj(z) a~``.
        """
        moved = Element(self, {k: X for k, X in x.terms.items() if self._conj_key(k) != k}, x.prec)
        fixed = Element(self, {k: X for k, X in x.terms.items() if self._conj_key(k) == k}, x.prec)
        cm = self.conj(moved)
        if set(cm.terms) & set(moved.terms):
            raise ValueError("real_element: x contains a term and its conjugate; give only one of them")
        return self.add(self.add(moved, cm), self.real(fixed))

    def _conj_key(self, k: Key) -> Key:
        t, s, w = k
        return (self.vertices[t].bar, self.vertices[s].bar, self.word_conjugate(w))

    # ------------------------------------------------------------------ valuation and precision
    def valuation(self, x: Element) -> float:
        """The lowest degree present (or ``prec``, whichever is smaller); ``inf`` for an exact zero."""
        d = min((self.degree(k[2]) for k in x.terms), default=INF)
        return min(d, x.prec)

    def truncate(self, x: Element, prec: float) -> Element:
        """``x`` known modulo degree ``>= prec`` only."""
```

```
        return self.element(dict(x.terms), min(prec, x.prec))

    def part(self, x: Element, d: int) -> Element:
        """The homogeneous part of degree ``d`` (``0(deg prec)`` if ``x`` is not known at degree ``d``)."""
        if d >= x.prec:
            return Element(self, {}, x.prec)
        return Element(self, {k: X for k, X in x.terms.items() if self.degree(k[2]) == d}, INF)

    def degrees(self, x: Element) -> list[int]:
        """The degrees of the words present, in increasing order."""
        return sorted({self.degree(k[2]) for k in x.terms})

    # ------------------------------------------------------------------ ring operations
    def add(self, x: Element, y: Element) -> Element:
        """``x + y``. A block in which the two summands cancel to within ``tol`` of their size is set to
        zero (relative cancellation, so the result does not depend on the scale of the data)."""
        prec = min(x.prec, y.prec)
        deg, tol = self.degree, self.tol
        t = {k: X for k, X in x.terms.items() if deg(k[2]) < prec}
        for k, Y in y.terms.items():
            if deg(k[2]) >= prec:
                continue
            X = t.get(k)
            if X is None:
                t[k] = Y
            else:
                S = X + Y
                if np.abs(S).max() <= tol * (np.abs(X).max() + np.abs(Y).max()):
                    del t[k]
                else:
                    t[k] = S
        return Element(self, t, prec)

    def sub(self, x: Element, y: Element) -> Element:
        """``x - y``."""
        return self.add(x, self.scale(-1.0, y))

    def scale(self, c: Scalar, x: Element) -> Element:
        """``c x`` for a number ``c``."""
        if c == 0:
            return Element(self, {}, x.prec)
        return Element(self, {k: c * X for k, X in x.terms.items()}, x.prec)

    def mul(self, x: Element, y: Element) -> Element:
        """The product ``x y``, with precision ``min(prec x + val y, prec y + val x)``."""
        prec = min(x.prec + self.valuation(y), y.prec + self.valuation(x))
        by_target = defaultdict(list)
        for (t2, s2, w2), B in y.terms.items():
            by_target[t2].append((s2, w2, self.degree(w2), B))
        out: dict[Key, np.ndarray] = {}
        mag: dict[Key, float] = {}
        for (t1, s1, w1), A in x.terms.items():
            d1, nA = self.degree(w1), np.abs(A).max() * A.shape[1]
            for s2, w2, d2, B in by_target.get(s1, ()):
                if d1 + d2 >= prec:
                    continue
                k = (t1, s2, self.concat(w1, w2))
                P = A @ B
                m = nA * np.abs(B).max()
                if k in out:
                    out[k] = out[k] + P
                    mag[k] += m
                else:
                    out[k] = P
                    mag[k] = m
        tol = self.tol  # drop the blocks where the contributions cancel
        return Element(self, {k: P for k, P in out.items() if np.abs(P).max() > tol * mag[k]}, prec)

    def star(self, x: Element) -> Element:
        """The involution: ``(X w)* = kappa(w) G_s X^T G_t^-1 w^T`` for ``w: s -> t``."""
        out = {}
        for (t, s, w), X in x.terms.items():
            Vs, Vt = self.vertices[s], self.vertices[t]
```

```
                out[(Vs.top, Vt.top, self.word_transpose(w))] = self.word_kappa(w) * (Vs.G @ X.T @ Vt.G_inv)
            return Element(self, out, x.prec)

        def conj(self, x: Element) -> Element:
            """The conjugation (conjugate-linear): ``c(X w) = mu(w) C_t conj(X) C_s^-1 w~``."""
            out = {}
            for (t, s, w), X in x.terms.items():
                Vs, Vt = self.vertices[s], self.vertices[t]
                out[(Vt.bar, Vs.bar, self.word_conjugate(w))] = self.word_mu(w) * (Vt.C @ np.conj(X) @ Vs.C_inv
        ↪ )
            return Element(self, out, x.prec)

        def real(self, x: Element) -> Element:
            """The projection ``(x + c(x)) / 2`` onto the real algebra."""
            return self.scale(0.5, self.add(x, self.conj(x)))

        def norm(self, x: Element) -> float:
            """The sum of the absolute values of all coefficients."""
            return float(sum(np.abs(X).sum() for X in x.terms.values()))

        def dist(self, x: Element, y: Element) -> float:
            """The sum of the absolute values of the coefficients of ``x - y`` over the words known in
            both, without any rounding to zero."""
            prec = min(x.prec, y.prec)
            t = {k: X for k, X in x.terms.items() if self.degree(k[2]) < prec}
            for k, Y in y.terms.items():
                if self.degree(k[2]) >= prec:
                    continue
                t[k] = t[k] - Y if k in t else -Y
            return float(sum(np.abs(X).sum() for X in t.values()))

        def is_real(self, x: Element, tol: float = 1e-9) -> bool:
            """Whether ``c(x) = x`` to relative tolerance ``tol``."""
            return self.dist(self.conj(x), x) <= tol * self.norm(x)

        def is_selfadjoint(self, x: Element, tol: float = 1e-9) -> bool:
            """Whether ``x* = x`` to relative tolerance ``tol``."""
            return self.dist(self.star(x), x) <= tol * self.norm(x)

        # ------------------------------------------------------------------ corners and coefficients
        def corner(self, x: Element, target: str, source: str) -> Element:
            """The part of ``x`` in ``e_target A e_source``."""
            return Element(self, {k: X for k, X in x.terms.items() if k[0] == target and k[1] == source}, x.
        ↪ prec)

        def support(self, x: Element) -> list[tuple[str, str]]:
            """The corners ``(target, source)`` in which ``x`` has terms."""
            return sorted({(k[0], k[1]) for k in x.terms})

        def corner_project(self, x: Element, rows: Iterable[str], cols: Iterable[str]) -> Element:
            """The part of ``x`` with target in ``rows`` and source in ``cols``."""
            rows, cols = set(rows), set(cols)
            return Element(self, {k: X for k, X in x.terms.items() if k[0] in rows and k[1] in cols}, x.prec)

        def coefficient(
            self, x: Element, letters: Sequence[str] = (), target: str | None = None, source: str | None = None
        ) -> np.ndarray:
            """The block of a word in ``x``; see :meth:`Element.coefficient`."""
            w = self.parse_word(letters)
            if w:
                t, s = self.word_ends(w)
                if (target or t, source or s) != (t, s):
                    raise ValueError(
                        f"the word {self.word_str(w)} goes from {s} to {t}, not from {source or s} to {target
        ↪ or t}"
                    )
                target, source = t, s
            elif target is None or source is None or target != source:
                raise ValueError("the empty word needs equal target and source (a vertex)")
            for v in (target, source):
                if v not in self.vertices:
                    raise ValueError(f"unknown vertex {v!r}; the vertices are {list(self.vertices)}")
            if self.degree(w) >= x.prec:
```

```
            raise ValueError(f"degree {self.degree(w)} is not known: the element is known below degree {x.
    ↪ prec}")
        X = x.terms.get((target, source, w))
        if X is not None:
            return X
        return np.zeros((self.block_size(target), self.block_size(source)), dtype=complex)

    # ------------------------------------------------------------------ inverses
    def inverse(self, x: Element, within: str = "support") -> Element:
        """The inverse of ``x``.

        Parameters
        ----------
        x : Element
        within : {'support', 'orbits', 'all'}
            Between which idempotents the inverse is taken:

            * ``'support'``: the vertices ``x`` touches. If ``x`` maps ``e_C`` to ``e_R`` (``C``, ``R``
              the sets of source and target vertices of its terms), the ``y`` in ``e_C A e_R`` with
              ``y x = e_C`` and ``x y = e_R`` (for a single corner this is Definition 3.5). Zero maps to
              zero.
            * ``'orbits'``: the STAR-BAR orbit idempotents ``x`` touches (real and self-adjoint), so a
              zero divisor of an orbit corner (such as ``(c, 0)`` in the split-complex numbers) has no
              inverse. Zero maps to zero.
            * ``'all'``: the two-sided inverse in the whole algebra.

        Returns
        -------
        Element

        Raises
        ------
        NotInvertibleError
            If there is no such inverse, or ``x`` is only known to vanish below some degree.
        PrecisionError
            If the elimination runs out of precision (a subclass of ``NotInvertibleError``).

        Notes
        -----
        A bijection between single vertices whose corners have unit leading terms is inverted corner
        by corner, by geometric series. Otherwise Gauss-Jordan elimination runs on the scalar entries,
        with pivots of lowest degree first and then of largest coefficient. This is complete when
        between any two vertices there is at most one reduced word of each degree (true for every quiver
        of the catalogue); otherwise it can miss units whose elimination needs a pivot made of several
        words of the same degree.
        """
        if within not in _WITHIN:
            raise ValueError(f"within must be one of {_WITHIN}, not {within!r}")
        if within == "all":
            verts = list(self.vertices)
            return self._gauss_jordan(x, verts, verts)
        if not x.terms:
            if x.prec == INF:
                return self.zero()
            raise NotInvertibleError(f"O(deg {x.prec}) is not known well enough to invert")
        sup = self.support(x)
        if within == "support":
            return self.inverse_between(x, {t for t, _ in sup}, {s for _, s in sup})
        rows = [v for t, _ in sup for v in self.quiver.orbit_of(t)]
        cols = [v for _, s in sup for v in self.quiver.orbit_of(s)]
        return self.inverse_between(x, rows, cols)

    def inverse_between(self, x: Element, rows: Iterable[str], cols: Iterable[str]) -> Element:
        """The ``y`` in ``e_cols A e_rows`` with ``y x = e_cols`` and ``x y = e_rows``, for ``x`` in
        ``e_rows A e_cols`` (``rows``, ``cols``: vertex sets).

        Raises
        ------
        NotInvertibleError
            If there is none.
        """
        order = list(self.vertices).index
        rows, cols = sorted(set(rows), key=order), sorted(set(cols), key=order)
```

```
        sup = self.support(x)
        if not sup:
            raise NotInvertibleError("zero is not invertible")
        if {t for t, _ in sup} == set(rows) and {s for _, s in sup} == set(cols) and len(sup) == len(rows) == len(cols):
            leads = [self._leading_unit(self.corner(x, t, s)) for t, s in sup]  # a bijection between
            if all(lu is not None for lu in leads):  # vertices: invert corner by corner
                out = self.zero()
                for (t, s), lu in zip(sup, leads, strict=True):
                    assert lu is not None
                    out = self.add(out, self._series_inverse(self.corner(x, t, s), lu))
                return out
        return self._gauss_jordan(x, rows, cols)

    def corner_inverse(self, z: Element) -> Element:
        """The inverse between vertices (Definition 3.5) of ``z`` in a single corner ``e_t A e_s``: the
        element of ``e_s A e_t`` with ``z^-1 z = e_s`` and ``z z^-1 = e_t``."""
        sup = self.support(z)
        if not sup:
            raise NotInvertibleError("zero is not invertible")
        if len(sup) != 1:
            raise ValueError(f"corner_inverse: the element is not in a single corner (support {sup})")
        lu = self._leading_unit(z)
        if lu is not None:
            return self._series_inverse(z, lu)
        ((t, s),) = sup
        return self._gauss_jordan(z, [t], [s])

    def _leading_unit(self, z: Element) -> tuple[Key, np.ndarray] | None:
        """For ``z`` in one corner: (key, block) of its lowest-degree part if that part is a unit (a
        single word with an invertible square block), else None."""
        if not z.terms:
            return None
        d0 = min(self.degree(k[2]) for k in z.terms)
        lead = [(k, X) for k, X in z.terms.items() if self.degree(k[2]) == d0]
        if len(lead) != 1:
            return None
        key, X = lead[0]
        if X.shape[0] != X.shape[1] or np.linalg.cond(X) > 1e12:
            return None
        return key, X

    def _series_inverse(self, z: Element, lead: tuple[Key, np.ndarray]) -> Element:
        """The geometric series for the inverse of ``z = z0 + r``, ``z0 = X w`` the unit leading part."""
        (t, s, w), X = lead
        z0inv = Element(self, {(s, t, self.word_inverse(w)): np.linalg.inv(X)}, INF)
        d0 = self.degree(w)
        r = self.sub(z, Element(self, {(t, s, w): X}, INF))
        if not r.terms and r.prec == INF:
            return z0inv
        prec = -d0 + ((z.prec - d0) if z.prec < INF else self.relprec)
        u = self.scale(-1.0, self.mul(z0inv, r))
        total, term = z0inv, z0inv
        while True:
            term = self.truncate(self.mul(u, term), prec)
            if not term.terms:
                break
            total = self.add(total, term)
        return self.truncate(total, prec)

    # Gauss-Jordan elimination on scalar entries. A vertex v of block size d_v contributes the d_v
    # indices (v, 0), ..., (v, d_v - 1); entry ((t, i), (s, j)) of x is the scalar series
    # sum_w X_w[i, j] w over the words w: s -> t. A scalar series is stored as (dict word -> complex,
    # prec). An entry can be a pivot when its lowest-degree part is a single word (a unit); the pivot
    # of lowest degree is taken, and among those the largest coefficient.
    def _sval(self, a: tuple[dict, float]) -> float:
        return min(min((self.degree(w) for w in a[0]), default=INF), a[1])

    def _strunc(self, a: tuple[dict, float], prec: float) -> tuple[dict, float]:
        return ({w: c for w, c in a[0].items() if self.degree(w) < prec}, min(a[1], prec))

    def _sadd(self, a: tuple[dict, float], b: tuple[dict, float], sign: float = 1.0) -> tuple[dict, float]:
        prec = min(a[1], b[1])
```

```
        t = {w: c for w, c in a[0].items() if self.degree(w) < prec}
        for w, c in b[0].items():
            if self.degree(w) >= prec:
                continue
            if w in t:
                S = t[w] + sign * c
                if abs(S) <= self.tol * (abs(t[w]) + abs(c)):
                    del t[w]
                else:
                    t[w] = S
            else:
                t[w] = sign * c
        return (t, prec)

    def _smul(self, a: tuple[dict, float], b: tuple[dict, float]) -> tuple[dict, float]:
        prec = min(a[1] + self._sval(b), b[1] + self._sval(a))
        t: dict = {}
        mag: dict = {}
        for w1, c1 in a[0].items():
            d1 = self.degree(w1)
            for w2, c2 in b[0].items():
                if d1 + self.degree(w2) < prec:
                    w = self.concat(w1, w2)
                    t[w] = t.get(w, 0) + c1 * c2
                    mag[w] = mag.get(w, 0) + abs(c1 * c2)
        return ({w: c for w, c in t.items() if abs(c) > self.tol * mag[w]}, prec)

    def _sleading(self, a: tuple[dict, float]) -> tuple[Word, complex] | None:
        if not a[0]:
            return None
        d0 = min(self.degree(w) for w in a[0])
        lead = [(w, c) for w, c in a[0].items() if self.degree(w) == d0]
        return lead[0] if len(lead) == 1 else None

    def _sinv(self, a: tuple[dict, float], relprec: float) -> tuple[dict, float]:
        lead = self._sleading(a)
        assert lead is not None
        w0, c0 = lead
        d0 = self.degree(w0)
        inv0 = ({self.word_inverse(w0): 1.0 / c0}, INF)
        r = ({w: c for w, c in a[0].items() if w != w0}, a[1])
        if not r[0] and r[1] == INF:
            return inv0
        prec = -d0 + ((a[1] - d0) if a[1] < INF else relprec)
        u = self._smul(inv0, r)
        u = ({w: -c for w, c in u[0].items()}, u[1])
        total = term = inv0
        while True:
            term = self._strunc(self._smul(u, term), prec)
            if not term[0]:
                break
            total = self._sadd(total, term)
        return self._strunc(total, prec)

    def _scalar_index(self, verts: Sequence[str]) -> list[tuple[str, int]]:
        return [(v, i) for v in verts for i in range(self.block_size(v))]

    def _gauss_jordan(self, x: Element, rverts: Sequence[str], cverts: Sequence[str]) -> Element:
        """Inverse of ``x`` from ``e_rverts`` to ``e_cverts``. For an exact ``x`` the series are
        recomputed with more precision (up to 8 relprec) when cancellations during the elimination
        leave ``x x^-1 = 1`` known to less than degree relprec; the result carries its true precision."""
        rp = self.relprec
        out = None
        for attempt in range(4):
            try:
                out = self._gj(x, rverts, cverts, rp)
            except PrecisionError:
                if x.prec < INF or attempt == 3:
                    raise
                rp *= 2
                continue
            if x.prec < INF or out.prec == INF or out.prec + self.valuation(x) >= self.relprec or attempt
        ↪ == 3:
```

```
                return out
            rp *= 2
        assert out is not None
        return out

    def _gj(self, x: Element, rverts: Sequence[str], cverts: Sequence[str], relprec: float) -> Element:
        rows, cols = self._scalar_index(rverts), self._scalar_index(cverts)
        if len(rows) != len(cols):
            raise NotInvertibleError(
                f"no inverse between vertices: {list(cverts)} and {list(rverts)} have different total "
                f"block sizes ({len(cols)} and {len(rows)})"
            )
        n = len(rows)
        ri = {r: k for k, r in enumerate(rows)}
        ci = {c: k for k, c in enumerate(cols)}
        L: list[list[tuple[dict, float]]] = [[({}, x.prec) for _ in range(n)] for _ in range(n)]
        for (t, s, w), X in x.terms.items():
            for i in range(X.shape[0]):
                for j in range(X.shape[1]):
                    if X[i, j] != 0:
                        if (t, i) not in ri or (s, j) not in ci:
                            raise ValueError("the element has terms outside the given corners")
                        L[ri[(t, i)]][ci[(s, j)]][0][w] = X[i, j]
        R: list[list[tuple[dict, float]]] = [
            [({(): 1.0}, INF) if i == j else ({}, INF) for j in range(n)] for i in range(n)
        ]
        used, pivrow = [False] * n, {}
        for k in range(n):
            best = None
            for i in range(n):
                if used[i]:
                    continue
                lu = self._sleading(L[i][k])
                if lu is None:
                    continue
                key = (self._sval(L[i][k]), -abs(lu[1]))
                if best is None or key < best[0]:
                    best = (key, i)
            if best is None:
                msg = (
                    f"no pivot for index {cols[k]}: no remaining entry of that column has a single word as
↪ "
                    "its lowest-degree part"
                )
                if any(not L[i][k][0] and L[i][k][1] < INF for i in range(n) if not used[i]):
                    raise PrecisionError(msg + " (some entries are only known to be O(deg ...): precision
↪ lost)")
                raise NotInvertibleError(msg)
            p = best[1]
            zi = self._sinv(L[p][k], relprec)
            L[p] = [self._smul(zi, v) for v in L[p]]
            R[p] = [self._smul(zi, v) for v in R[p]]
            for i in range(n):
                f = L[i][k]
                if i == p or (not f[0] and f[1] == INF):
                    continue
                L[i] = [self._sadd(L[i][j], self._smul(f, L[p][j]), -1.0) for j in range(n)]
                R[i] = [self._sadd(R[i][j], self._smul(f, R[p][j]), -1.0) for j in range(n)]
            used[p] = True
            pivrow[k] = p
        out: dict[Key, np.ndarray] = {}
        prec = INF
        for k in range(n):  # row k of the inverse maps index rows[j] to cols[k]
            s, i = cols[k]
            for j in range(n):
                t, jj = rows[j]
                ser, pr = R[pivrow[k]][j]
                prec = min(prec, pr)
                for w, c in ser.items():
                    term = (s, t, w)
                    if term not in out:
                        out[term] = np.zeros((self.block_size(s), self.block_size(t)), dtype=complex)
                    out[term][i, jj] += c
```

```
        return self.element(out, prec)

    # ------------------------------------------------------------------ square roots
    def sqrt(
        self, x: Element, rule: str = "graded", choice: str = "positive", split: str = "left", rng: Any =
     ↪ None
    ) -> Element:
        """A *-square root of ``x`` (some ``y`` with ``y* y = x``); see :func:`dsqlib.sqrt.star_sqrt`.

        With the default arguments, ``algebra.sqrt`` can be passed directly as the ``sqrt`` argument of
        the functions in :mod:`dsqlib.linalg`.
        """
        from .sqrt import star_sqrt

        return star_sqrt(x, rule, choice, split, rng)

    # ------------------------------------------------------------------ back to the path algebra
    def is_path_element(self, x: Element) -> bool:
        """Whether ``x`` has no inverse letters."""
        return all(e > 0 for k in x.terms for _, e in k[2])

    def reduce(self, x: Element, m: int = 2) -> Element:
        """The image of ``x`` in ``RQ / RQ_{>=m}`` (``m = 2`` gives the radical-square-zero algebra
        ``B_Q``). ``x`` must lie in the completed path algebra (no inverse letters) and be known up to
        degree ``m``."""
        bad = {k: X for k, X in x.terms.items() if any(e < 0 for _, e in k[2])}
        if bad:
            raise ValueError("the element has poles (terms with local inverses): " + self.format(Element(
     ↪ self, bad)))
        if x.prec < m:
            raise ValueError(f"the element is only known below degree {x.prec}; cannot reduce modulo degree
     ↪  {m}")
        return self.element({k: X for k, X in x.terms.items() if len(k[2]) < m}, INF)

    # ------------------------------------------------------------------ display
    def format(self, x: Element, digits: int = 4) -> str:
        """``x`` as a sum of ``coefficient·word`` terms, by degree."""
        if not x.terms:
            return "0" if x.prec == INF else f"O(deg {x.prec})"
        parts = []
        for (_, s, w), X in sorted(x.terms.items(), key=lambda kv: (self.degree(kv[0][2]), str(kv[0]))):
            if X.size == 1:
                c = X[0, 0]
                cs = (
                    f"{c.real:.{digits}g}"
                    if abs(c.imag) <= 1e-14 * abs(c)
                    else f"({c.real:.{digits}g}{c.imag:+.{digits}g}j)"
                )
            else:
                cs = np.array2string(np.round(X, digits), separator=",").replace("\n", "")
            parts.append(f"{cs}·{self.word_str(w, s)}")
        tail = "" if x.prec == INF else f" + O(deg {x.prec})"
        return " + ".join(parts) + tail


def quaternion(a: float, b: float = 0.0, c: float = 0.0, d: float = 0.0) -> np.ndarray:
    """The 2x2 complex block of the quaternion ``a + bi + cj + dk`` (the real form of a block at a
    vertex with real structure ``C = J``)."""
    p, q = complex(a, b), complex(c, d)
    return np.array([[p, q], [-np.conj(q), np.conj(p)]], dtype=complex)
```

B.1.4 src/dsqlib/sqrt.py

```
"""*-square roots: given ``x = x*``, some ``y`` with ``y* y = x``.

If ``y`` and ``y'`` are both *-square roots of ``x`` and ``y`` is invertible between the vertices it
touches, then ``y' = U y`` with ``U = y' y^+`` unitary there; so a *-square root is a choice, and the
rules below make that choice consistently. Every rule checks its result (``y* y = x`` word by word,
below the precision of both sides, and ``c(y) = y`` when ``x`` is real) and raises
:class:`~dsqlib.errors.NoSquareRootError` when it cannot produce one.

Rules
-----
```'graded'`` (the default)
    For homogeneous ``x`` whose words all have even length. Each word is cut in the middle,
    ``w = u'^T u``, and ``y = sum Z_u u`` is built from the half-words ``u``; for paths ``y`` is
    homogeneous of half the degree. The words of ``x`` fall into independent groups, one for each
    pair ``{m, m^T}`` of middle vertices:

    * ``m = m^T``: a congruence ``Z^T G_m^-1 Z = N`` through the ``d_m``-dimensional middle;
    * ``m != m^T``: a split ``Z'^T Z = N`` between the half-words at ``m`` and at ``m^T``.

    For real ``x`` the conjugation decides how the solution is made real: a Hermitian congruence when
    ``m~ = m^T != m`` (vertex kind Cu); explicit formulas in the real middle algebras R, M2(R), H and the
    exchanged pairs when ``m~ = m``; otherwise the solution at ``m`` is transported to ``m~``. Where
    the signature of a vertex of kind R, Hsp or Cu forbids a root, or ``x`` has rank greater than
    ``d_m`` at a middle vertex, there is no graded root and the rule says so.

    ``choice='positive'`` takes principal square roots of the blocks; ``choice='random'`` then
    multiplies by a random unitary of the semisimple part. For an exchanged pair ``(s, p)`` (``s`` the
    vertex listed first), ``split='left'`` puts ``x`` on ``s`` and 1 on ``p``, ```'right'`` the reverse
    and ```'balanced'`` equal square roots on both: over the split-complex numbers these are the roots
    ``(c, 1)``, ``(1, c)`` and ``(sqrt c, sqrt c)`` of ``(c, c)``.

    For ``x`` that is not homogeneous, the lowest-degree part ``x0`` gets the graded root ``y0`` and
    ``y = r y0`` with ``r`` the principal square root of ``(y0^+)* x y0^+``, as
    ``sqrt(t^2k (c + ...)) = t^k sqrt(c + ...)``.

```'principal'``
    For ``x`` with an invertible, vertex-diagonal lowest part of degree 0 in the semisimple part: the
    self-adjoint ``y`` with ``y^2 = x`` whose degree-0 part is the principal square root, found degree by
    degree from Sylvester equations.

```'arbitrary'``
    ``U y`` for the graded (or principal) root ``y`` and a random unitary ``U`` of the orbit corner where
    ``y`` ends: a Cayley transform with terms of positive degree, times a random unitary of the
    semisimple part. Not homogeneous; matrix entries stay in their orbit corners.

Known gaps of the graded rule (a root exists but no rule finds it): words with inverse letters, where
cancellation can hide the half-words; and an ``x`` whose lowest-degree part does not reach every vertex
that ``x`` reaches (e.g. ``(e_v + a)*(e_v + a)``, which has the root ``e_v + a``).
"""

from __future__ import annotations

import itertools
from collections.abc import Callable, Sequence
from typing import Any

import numpy as np
from scipy.linalg import solve_sylvester, sqrtm

from .algebra import INF, Algebra, Element, Key
from .errors import NoSquareRootError, NotInvertibleError

__all__ = [
    "CHOICES",
    "RULES",
    "SPLITS",
    "random_semisimple_unitary",
    "random_unitary",
    "sqrt_graded",
    "sqrt_principal",
    "sqrt_rule",
```
```

```
    "star_sqrt",
]

RULES = ("graded", "principal", "arbitrary")
CHOICES = ("positive", "random")
SPLITS = ("left", "right", "balanced")


def _psqrt(N: np.ndarray) -> np.ndarray:
    return np.asarray(sqrtm(np.asarray(N, dtype=complex)), dtype=complex)


def _rank(N: np.ndarray, rtol: float = 1e-9) -> int:
    if N.size == 0:
        return 0
    s = np.linalg.svd(N, compute_uv=False)
    return int(np.sum(s > rtol * s[0])) if s[0] > 0 else 0


def _close(A: np.ndarray, B: np.ndarray, rtol: float = 1e-9) -> bool:
    return bool(np.abs(A - B).max(initial=0) <= rtol * max(np.abs(A).max(initial=0), np.abs(B).max(initial
    ↪ =0)))


# ------------------------------------------------------------------------ block solvers
def _bilinear_candidates(M: np.ndarray, N: np.ndarray) -> list[np.ndarray]:
    """Solutions Z of Z^T M Z = N for square blocks (M invertible, N of the same symmetry as M), the
    principal one first."""
    d = M.shape[0]
    if d == 1:
        z = np.sqrt(complex(N[0, 0] / M[0, 0]))
        return [np.array([[z]]), np.array([[-z]])]
    if _close(M, M.T) and _close(N, N.T):
        return [np.linalg.solve(_psqrt(M), _psqrt(N))]
    if _close(M, -M.T) and _close(N, -N.T):  # 2x2 skew:  Z^T M Z = det(Z) M
        r = N[0, 1] / M[0, 1]
        return [np.sqrt(complex(r)) * np.eye(2, dtype=complex), np.diag([r, 1.0]).astype(complex)]
    return []


def _quaternion_symmetric_sqrt(N: np.ndarray) -> np.ndarray:
    """A quaternion block Z = [[a, c], [-c~, a~]] with Z^T Z = N, for N = [[p, i b], [i b, p~]] (b real);
    it always exists for N != 0 (the involution of orthogonal type on H has no signature)."""
    p, beta = N[0, 0], (N[0, 1] / 1j).real
    s = np.hypot(beta, abs(p))
    rho, sig = np.sqrt(max(beta + s, 0.0)), np.sqrt(max(s - beta, 0.0))
    ph = np.exp(0.5j * np.angle(p)) if abs(p) > 0 else 1.0
    P, Q = rho * ph, sig * ph  # P Q = p, |P|^2 - |Q|^2 = 2 beta
    a, c = (P + Q) / 2, np.conj((P - Q) / 2j)
    return np.array([[a, c], [-np.conj(c), np.conj(a)]], dtype=complex)


def _real_square(M: np.ndarray, N: np.ndarray, C: np.ndarray) -> np.ndarray:
    """Z^T M Z = N with Z of real type for the structure C (C = 1: real matrices, C = J: quaternions),
    square blocks of size 1 or 2."""
    d = M.shape[0]
    if not np.any(N):
        return np.zeros((d, d), dtype=complex)
    if d == 1:
        r = N[0, 0] / M[0, 0]
        if abs(r.imag) <= 1e-9 * (1 + abs(r)) and r.real >= 0:
            return np.array([[np.sqrt(r.real)]], dtype=complex)
        raise NoSquareRootError(f"no real solution of z^2 = {r:.4g} (signature)")
    quaternionic = not _close(C, np.eye(2))
    if _close(M, -M.T):  # Rsp or Hsp middle: det(Z) = r
        r = N[0, 1] / M[0, 1]
        if abs(r.imag) > 1e-9 * (1 + abs(r)):
            raise NoSquareRootError("block congruence: N is not a real multiple of M")
        r = r.real
        if r > 0:
            return np.sqrt(r) * np.eye(2, dtype=complex)
        if quaternionic:
            raise NoSquareRootError(f"no quaternion of norm {r:.4g} (signature)")
```

```
        return np.diag([r, 1.0]).astype(complex)
    if quaternionic:  # Hor middle: G = M = 1
        if not _close(M, np.eye(2)):
            raise NoSquareRootError("block congruence: unexpected form at a quaternionic vertex")
        if not (_close(N, N.T) and _close(N, np.array([[N[0, 0], N[0, 1]], [N[0, 1], np.conj(N[0, 0])]]))):
            raise NoSquareRootError("block congruence: N is not a symmetric quaternion")
        return _quaternion_symmetric_sqrt(N)
    Z = np.linalg.solve(_psqrt(M), _psqrt(N))
    if np.abs(Z.imag).max() > 1e-9 * (1 + np.abs(Z).max()):
        raise NoSquareRootError("no real solution of the block congruence")
    return Z.real.astype(complex)


def _pivots(N: np.ndarray, k: int, rows: bool | None = None) -> tuple[list[int], list[int]]:
    """Index sets (I, J) of size k maximising |det N[I, J]| (I = J for the symmetric problems)."""
    R, Cn = N.shape
    best = -1.0
    arg: tuple[list[int], list[int]] = ([], [])
    for J in itertools.combinations(range(Cn), k):
        Is = [J] if rows is None else itertools.combinations(range(R), k)
        for I in Is:
            v = abs(np.linalg.det(N[np.ix_(I, J)]))
            if v > best:
                best, arg = v, (list(I), list(J))
    return arg


def _sym_factor(M: np.ndarray, N: np.ndarray, d: int) -> np.ndarray:
    """Some complex Z (d x D) with Z^T M Z = N, N of rank <= d."""
    D = N.shape[0]
    r = _rank(N)
    if r == 0:
        return np.zeros((d, D), dtype=complex)
    if r > d:
        raise NoSquareRootError(f"x has rank {r} at a middle vertex of size {d}: no graded *-square root")
    if d == D and r == d:
        Zs = _bilinear_candidates(M, N)
        if Zs and _close(Zs[0].T @ M @ Zs[0], N):
            return Zs[0]
        raise NoSquareRootError("the block congruence has no solution")
    k = r
    Mk = M[:k, :k] if k < d else M
    if k < d and not _close(M, np.diag(np.diag(M))):
        raise NoSquareRootError("x is degenerate at a middle vertex with a skew form")
    I, _ = _pivots(N, k)
    ZJ = _bilinear_candidates(Mk, N[np.ix_(I, I)])
    if not ZJ:
        raise NoSquareRootError("the block congruence has no solution")
    Zk = np.linalg.solve(Mk, np.linalg.solve(ZJ[0].T, N[I, :]))
    Z = np.vstack([Zk, np.zeros((d - k, D), dtype=complex)])
    if not _close(Z.T @ M @ Z, N):
        raise NoSquareRootError("graded rule: x does not factor through the middle vertex")
    return Z


def _split_factor(N: np.ndarray, d: int, split: str) -> tuple[np.ndarray, np.ndarray]:
    """Some (Z', Z), each with d rows, with Z'^T Z = N (rank <= d)."""
    R, Cn = N.shape
    r = _rank(N)
    if r == 0:
        return np.zeros((d, R), dtype=complex), np.zeros((d, Cn), dtype=complex)
    if r > d:
        raise NoSquareRootError(f"x has rank {r} at a middle vertex of size {d}: no graded *-square root")
    if R == Cn == r:
        I = J = list(range(r))
    else:
        I, J = _pivots(N, r, rows=True)
    P = N[np.ix_(I, J)]
    if split == "left":
        Zk, Zpk = N[I, :], np.linalg.solve(P.T, N[:, J].T)
    elif split == "right":
        Zk, Zpk = np.linalg.solve(P, N[I, :]), N[:, J].T
    elif split == "balanced":
```

```
        if r == 1:
            S = np.sqrt(abs(P[0, 0])) * np.ones((1, 1))
            Zk, Zpk = N[I, :] / S[0, 0], (N[:, J] * S[0, 0] / P[0, 0]).T
        else:
            S = _psqrt(P)
            Zk, Zpk = np.linalg.solve(S, N[I, :]), np.linalg.solve(S.T, N[:, J].T)
    else:
        raise ValueError(f"split must be one of {SPLITS}, not {split!r}")

    def pad(Z: np.ndarray) -> np.ndarray:
        return np.vstack([Z, np.zeros((d - r, Z.shape[1]), dtype=complex)])

    Zp, Z = pad(Zpk), pad(Zk)
    if not _close(Zp.T @ Z, N):
        raise NoSquareRootError("graded rule: x does not factor through the middle vertices")
    return Zp, Z


def _phase_fix(v: np.ndarray) -> np.ndarray:
    k = int(np.argmax(np.abs(v)))
    return v * (np.conj(v[k]) / abs(v[k])) if abs(v[k]) > 0 else v


def _hermitian_factor(H: np.ndarray, N: np.ndarray, d: int) -> np.ndarray:
    """Some Z (d x D) with Z^H H Z = N (H invertible; H and N both Hermitian or both anti-Hermitian).
    For H = +-1 and a square block this is the principal square root of +-N."""
    if not _close(H, H.conj().T):
        H, N = -1j * H, -1j * N
    if not (_close(H, H.conj().T) and _close(N, N.conj().T)):
        raise NoSquareRootError("graded rule: x is not Hermitian at a middle vertex")
    N = (N + N.conj().T) / 2
    D = N.shape[0]
    if _rank(N) > d:
        raise NoSquareRootError(f"x has rank {_rank(N)} at a middle vertex of size {d}: no graded *-square
    ↪ root")
    for h in (1.0, -1.0):
        if d == D and _close(H, h * np.eye(d)):
            ev = np.linalg.eigvalsh(h * N)
            if ev.min() < -1e-12 * abs(ev).max():
                raise NoSquareRootError(
                    "graded rule: x is not of the sign of the form at a Cu-type middle vertex (signature)"
                )
            return _psqrt(h * N)
    eh, Wh = np.linalg.eigh(H)
    en, Wn = np.linalg.eigh(N)
    tol = 1e-10 * abs(en).max()
    Z = np.zeros((d, D), dtype=complex)
    free = list(range(d))
    for j in np.argsort(-abs(en)):
        if abs(en[j]) <= tol:
            continue
        match = [i for i in free if np.sign(eh[i]) == np.sign(en[j])]
        if not match:
            raise NoSquareRootError("graded rule: x does not fit the signature of the middle vertex")
        i = match[0]
        free.remove(i)
        Z += np.sqrt(en[j] / eh[i]) * np.outer(Wh[:, i], _phase_fix(Wn[:, j]).conj())
    if not _close(Z.conj().T @ H @ Z, N):
        raise NoSquareRootError("graded rule: x has too large a rank at a middle vertex")
    return Z


# ---------------------------------------------------------------------- the graded rule
_HalfKey = tuple[str, Any]  # (source vertex, half-word)


def _order(alg: Algebra) -> Callable[[_HalfKey], tuple[int, int, str]]:
    pos = {v: i for i, v in enumerate(alg.vertices)}
    return lambda k: (pos[k[0]], len(k[1]), str(k[1]))


def _layout(alg: Algebra, keys: Sequence[_HalfKey]) -> tuple[dict[_HalfKey, tuple[int, int]], int]:
    """Column offsets for a list of (source, half-word) keys."""
```

```
    off, out = 0, {}
    for k in keys:
        out[k] = (off, off + alg.block_size(k[0]))
        off += alg.block_size(k[0])
    return out, off


def _matrix(
    alg: Algebra, blocks: dict, rows: Sequence[_HalfKey], cols: Sequence[_HalfKey], transform: Callable
) -> np.ndarray:
    ro, R = _layout(alg, rows)
    co, C = _layout(alg, cols)
    N = np.zeros((R, C), dtype=complex)
    for (r, c), X in blocks.items():
        a, b = ro[r]
        e, f = co[c]
        N[a:b, e:f] = transform(r, c, X)
    return N


def _terms(alg: Algebra, m: str, keys: Sequence[_HalfKey], Z: np.ndarray) -> Element:
    lay, _ = _layout(alg, keys)
    t: dict[Key, np.ndarray] = {}
    for s, u in keys:
        a, b = lay[(s, u)]
        if np.abs(Z[:, a:b]).max(initial=0) > 0:
            t[(m, s, u)] = Z[:, a:b]
    return Element(alg, t)


def _blockrow(alg: Algebra, y: Element, v: str, keys: Sequence[_HalfKey]) -> np.ndarray:
    lay, D = _layout(alg, keys)
    Z = np.zeros((alg.block_size(v), D), dtype=complex)
    for (t, s, u), X in y.terms.items():
        if t == v:
            a, b = lay[(s, u)]
            Z[:, a:b] = X
    return Z


def _realify(alg: Algebra, y0: Element, split: str) -> Element:
    """A real *-square root of y0* y0 from a complex one: y = r y1 with y1 the real part of B y0 (B = 1,
    then fixed other choices) and r a degree-0 *-square root of K = g* g, where g y1 = y0."""
    mids = sorted({t for (t, _, _) in y0.terms}, key=list(alg.vertices).index)
    rng = np.random.default_rng(12345)
    for attempt in range(6):
        if attempt == 0:
            Bel = alg.one()
        else:
            Bel = alg.zero()
            for v in mids:
                d = alg.block_size(v)
                Bel = alg.add(Bel, alg.vertex(v, rng.normal(size=(d, d)) + 1j * rng.normal(size=(d, d))))
        y1 = alg.real(alg.mul(Bel, y0))
        keys = sorted({(s, u) for (_, s, u) in list(y0.terms) + list(y1.terms)}, key=_order(alg))
        G, ok = alg.zero(), True
        for v in mids:
            Z0, Z1 = _blockrow(alg, y0, v, keys), _blockrow(alg, y1, v, keys)
            g = Z0 @ np.linalg.pinv(Z1)
            if not _close(g @ Z1, Z0):
                ok = False
                break
            G = alg.add(G, Element(alg, {(v, v, ()): g}))
        if not ok:
            continue
        K = alg.real(alg.mul(alg.star(G), G))
        r = _graded(alg, K, split, base=True)
        return alg.mul(r, y1)
    raise NoSquareRootError("graded rule: could not find a real *-square root")


def _graded(alg: Algebra, x: Element, split: str, base: bool = False) -> Element:
    if not x.terms:
```

```
        return alg.zero()
    degs = alg.degrees(x)
    if len(degs) != 1:
        raise NoSquareRootError(f"graded rule: x is not homogeneous (degrees {degs})")
    V = alg.vertices
    real = alg.is_real(x)
    groups: dict[str, dict] = {}
    for (t, s, w), X in x.terms.items():
        if len(w) % 2:
            raise NoSquareRootError(f"graded rule: the word {alg.word_str(w, s)} has odd length")
        h = len(w) // 2
        u, up = w[h:], alg.word_transpose(w[:h])
        m = alg.word_ends(u)[0] if u else s
        groups.setdefault(m, {})[((V[t].top, up), (s, u))] = X
    key = _order(alg)

    def norm(r: _HalfKey, c: _HalfKey, X: np.ndarray) -> np.ndarray:  # N[u', u] = kappa(u') G_s'^-1 X
        return alg.word_kappa(r[1]) * V[r[0]].G_inv @ X

    y, done = alg.zero(), set()
    for m in sorted(groups, key=list(V).index):
        if m in done:
            continue
        blocks, mT, mb, d = groups[m], V[m].top, V[m].bar, V[m].block_size
        cols = sorted({c for (_, c) in blocks}, key=key)
        rows = sorted({r for (r, _) in blocks}, key=key)
        if m == mT:  # Z^T G_m^-1 Z = N
            idx = sorted(set(cols) | set(rows), key=key)
            N, M = _matrix(alg, blocks, idx, idx, norm), V[m].G_inv
            if real and mb == m:
                single = len(idx) == 1 and V[idx[0][0]].block_size == d and alg.word_conjugate(idx[0][1])
    ↪ == idx[0][1]
                if single:
                    yg = _terms(alg, m, idx, _real_square(M, N, V[m].C))
                elif base:
                    raise NoSquareRootError("graded rule: internal error (base case with several half-words
    ↪ )")
                else:
                    y0 = _terms(alg, m, idx, _sym_factor(M, N, d))
                    yg = y0 if alg.is_real(y0) else _realify(alg, y0, split)
            else:
                yg = _terms(alg, m, idx, _sym_factor(M, N, d))
                if real:
                    yg = alg.add(yg, alg.conj(yg))
                    done.add(mb)
            done.add(m)
        else:
            N = _matrix(alg, blocks, rows, cols, norm)  # Z'^T Z = N  (G = 1 off the T-fixed vertices)
            if real and mb == mT:  # Hermitian: Z' = c(Z)
                conj_cols = [(V[s].bar, alg.word_conjugate(u)) for (s, u) in cols]
                if set(conj_cols) != set(rows):
                    raise NoSquareRootError("graded rule: the half-words at m^T are not the conjugates of
    ↪ those at m")
                Nc = _matrix(alg, blocks, conj_cols, cols, norm)
                lay, _ = _layout(alg, cols)
                Ns = Nc.copy()
                for s, u in cols:  # N'[v, u] = mu(v) C_s(v)^T N[v~, u]
                    a, b = lay[(s, u)]
                    Ns[a:b, :] = alg.word_mu(u) * V[s].C.T @ Nc[a:b, :]
                Hm = V[m].C.T @ V[mT].G_inv
                Z = _hermitian_factor(Hm, Ns, d)
                yg = _terms(alg, m, cols, Z)
                yg = alg.add(yg, alg.conj(yg))
            else:
                Zp, Z = _split_factor(N, d, split)
                yg = alg.add(_terms(alg, m, cols, Z), _terms(alg, mT, rows, Zp))
                if real and mb == m:
                    if not alg.is_real(yg):
                        if base:
                            found = None
                            for sp in SPLITS:
                                Zp, Z = _split_factor(N, d, sp)
                                cand = alg.add(_terms(alg, m, cols, Z), _terms(alg, mT, rows, Zp))
```

```
                                if alg.is_real(cand):
                                    found = cand
                                    break
                            if found is None:
                                raise NoSquareRootError("graded rule: no real split at an exchanged pair")
                            yg = found
                        else:
                            yg = _realify(alg, yg, split)
                elif real:
                    yg = alg.add(yg, alg.conj(yg))
                    done |= {mb, V[mb].top}
            done |= {m, mT}
        y = alg.add(y, yg)
    return y


def sqrt_graded(
    x: Element, choice: str = "positive", split: str = "left", rng: np.random.Generator | None = None
) -> Element:
    """The graded *-square root of ``x`` (see the module documentation).

    For ``x`` that is not homogeneous, ``y = r y0`` where ``y0`` is the graded root of the lowest-degree
    part ``x0`` and ``r`` the principal square root of ``(y0^+)* x y0^+``, whose degree-0 part is the
    identity of the vertices where ``y0`` ends.

    Raises
    ------
    NoSquareRootError
        If the rule cannot produce a root.
    """
    if choice not in CHOICES:
        raise ValueError(f"choice must be one of {CHOICES}, not {choice!r}")
    if split not in SPLITS:
        raise ValueError(f"split must be one of {SPLITS}, not {split!r}")
    alg = x.algebra
    if not x.terms:  # x = 0(deg p): y = 0(deg floor(p/2)) is consistent
        y = alg.zero(prec=(x.prec // 2) if x.prec < INF else INF)
    elif len(alg.degrees(x)) == 1:
        y = _graded(alg, x, split)
        if x.prec < INF and y.terms:  # y is known modulo degree prec(x) - val(y)
            y = alg.truncate(y, x.prec - alg.valuation(y))
    else:
        y0 = _graded(alg, alg.part(x, int(alg.valuation(x))), split)
        C = {s for (_, s, _) in y0.terms}
        if any(t not in C or s not in C for (t, s, _) in x.terms):
            raise NoSquareRootError(
                "graded rule: x reaches vertices where its lowest-degree part vanishes; the graded and "
                "principal rules do not cover this case (e.g. (e_v + a)*(e_v + a) has the root e_v + a)"
            )
        try:
            y0i = alg.inverse(y0)
        except NotInvertibleError as e:
            raise NoSquareRootError(f"graded rule: the root of the lowest-degree part is not invertible ({e
    ↪ })") from e
        m = alg.mul(alg.mul(alg.star(y0i), x), y0i)
        y = alg.mul(sqrt_principal(m), y0)
    if choice == "random":
        y = alg.mul(random_semisimple_unitary(alg, rng or np.random.default_rng()), y)
    return _check(alg, x, y, "graded rule")


# ------------------------------------------------------------------------ the principal rule
def sqrt_principal(x: Element) -> Element:
    """The principal *-square root: ``y = y*`` with ``y^2 = x``, for ``x`` with an invertible,
    vertex-diagonal degree-0 lowest part in the semisimple part.

    Raises
    ------
    NoSquareRootError
        If ``x`` is not of that form, or the result fails the check.
    """
    alg = x.algebra
    if not x.terms:
```

```
        return alg.zero()
    if alg.valuation(x) < 0 or 0 not in alg.degrees(x):
        raise NoSquareRootError("principal rule: needs a lowest part of degree 0 (use rule='graded')")
    x0 = alg.part(x, 0)
    if any(w for (_, _, w) in x0.terms):
        raise NoSquareRootError("principal rule: the degree-0 part must lie in the semisimple part S")
    if any(t != s for (t, s, _) in x0.terms):
        raise NoSquareRootError("principal rule: the degree-0 part must be vertex-diagonal")
    Y0: dict[str, np.ndarray] = {}
    y = alg.zero()
    for (_t, s, _w), X in x0.terms.items():
        if np.linalg.cond(X) > 1e12:
            raise NoSquareRootError(f"principal rule: the degree-0 part of x is not invertible at vertex {s}")
        Y0[s] = _psqrt(X)
        y = alg.add(y, Element(alg, {(s, s, ()): Y0[s]}))
    missing = ({k[0] for k in x.terms} | {k[1] for k in x.terms}) - set(Y0)
    if missing:
        raise NoSquareRootError(
            f"principal rule: x reaches the vertices {sorted(missing)}, where its degree-0 part vanishes"
        )
    prec = min(x.prec, alg.relprec)
    parts = {0: y}
    for d in range(1, int(prec)):
        rhs = alg.part(x, d)
        for i in range(1, d):
            rhs = alg.sub(rhs, alg.mul(parts[i], parts[d - i]))
        yd = alg.zero()
        for (t, s, w), Rb in rhs.terms.items():
            yd = alg.add(yd, Element(alg, {(t, s, w): solve_sylvester(Y0[t], Y0[s], Rb)}))
        parts[d] = yd
        y = alg.add(y, yd)
    y = alg.truncate(y, prec)
    xx = alg.truncate(x, prec)
    if not alg.is_selfadjoint(y):
        raise NoSquareRootError("principal rule: the result is not self-adjoint (is the degree-0 part positive?)")
    return _check(alg, xx, y, "principal rule")


# ---------------------------------------------------------------------- unitaries and the arbitrary rule
def random_semisimple_unitary(alg: Algebra, rng: np.random.Generator, signs: bool = True) -> Element:
    """A random real unitary of the semisimple part: for each STAR-BAR orbit ``O``, the Cayley
    transform ``(e_O - K)(e_O + K)^-1`` of a random skew-adjoint ``K`` in ``e_O S e_O``, times a random
    sign (if ``signs``)."""
    U = alg.zero()
    for orb in alg.quiver.star_bar_orbits():
        R, eo = alg.zero(), alg.zero()
        for v in orb:
            d = alg.block_size(v)
            R = alg.add(R, alg.vertex(v, rng.normal(size=(d, d)) + 1j * rng.normal(size=(d, d))))
            eo = alg.add(eo, alg.idempotent(v))
        R = alg.real(R)
        K = alg.sub(R, alg.star(R))
        Uc = alg.mul(alg.sub(eo, K), alg.inverse(alg.add(eo, K)))
        if signs and rng.random() < 0.5:
            Uc = alg.scale(-1.0, Uc)
        U = alg.add(U, Uc)
    return U


def _paths(alg: Algebra, max_len: int) -> list[Key]:
    out: list[Key] = [(v, v, ()) for v in alg.vertices]
    frontier = list(out)
    for _ in range(max_len):
        new = []
        for t, s, w in frontier:
            for n, a in alg.arrows.items():
                if a.source == t:
                    new.append((a.target, s, ((n, 1), *w)))
        out += new
        frontier = new
    return out
```

```
575
    def random_unitary(
        alg: Algebra,
        rng: np.random.Generator,
580     vertices: Sequence[str] | set[str] | None = None,
        max_len: int = 2,
        scale: float = 0.5,
    ) -> Element:
        """A random real unitary of the corner ``e_T A e_T``, ``T = vertices`` (default: all; ``T`` should
585     be a union of STAR-BAR orbits).

        It is the Cayley transform ``(e_T - K)(e_T + K)^-1`` of ``K = R - R*``, where ``R`` is a random real
        combination of the idempotents ``e_v`` (``v`` in ``T``) and of the paths of length ``<= max_len``
        between vertices of ``T``. It has terms of positive degree: a unitary of the completion.
590     """
        T = set(alg.vertices) if vertices is None else set(vertices)
        R = alg.zero()
        for t, s, w in _paths(alg, max_len):
            if t in T and s in T:
595             dt, ds = alg.block_size(t), alg.block_size(s)
                X = scale * (rng.normal(size=(dt, ds)) + 1j * rng.normal(size=(dt, ds)))
                R = alg.add(R, Element(alg, {(t, s, w): X}))
        R = alg.real(R)
        K = alg.sub(R, alg.star(R))
600     eT = alg.zero()
        for v in T:
            eT = alg.add(eT, alg.idempotent(v))
        return alg.mul(alg.sub(eT, K), alg.inverse_between(alg.add(eT, K), T, T))

605
    def _magnitudes(alg: Algebra, x: Element, y: Element) -> dict[Key, float]:
        """For each word of the product x y, the sum over its contributions of |X| |Y| (block maxima times the
        inner dimension): the scale on which rounding errors in that coefficient live."""
        by_target: dict[str, list] = {}
610     for (t2, s2, w2), B in y.terms.items():
            by_target.setdefault(t2, []).append((s2, w2, np.abs(B).max()))
        prec = min(x.prec + alg.valuation(y), y.prec + alg.valuation(x))
        mag: dict[Key, float] = {}
        for (t1, s1, w1), A in x.terms.items():
615         a = np.abs(A).max() * A.shape[1]
            for s2, w2, b in by_target.get(s1, ()):
                w = alg.concat(w1, w2)
                if alg.degree(w) < prec:
                    k = (t1, s2, w)
620                 mag[k] = mag.get(k, 0.0) + a * b
        return mag


    def _check(alg: Algebra, x: Element, y: Element, what: str) -> Element:
625     """y* y = x word by word, below the precision of both, and c(y) = y when x is real. At each word the
        error may be 1e-8 of the larger of the coefficient of x and the size of the contributions to that
        coefficient of y* y (rounding lives on that scale), plus 1e-13 of the largest coefficient of x; so a
        word of x that y* y misses is always caught."""
        ys = alg.star(y)
630     yy = alg.mul(ys, y)
        mag = _magnitudes(alg, ys, y)
        prec = min(x.prec, yy.prec)
        big = max((np.abs(X).max() for X in x.terms.values()), default=0.0)
        for k in set(x.terms) | set(yy.terms):
635         if alg.degree(k[2]) >= prec:
                continue
            X, Y = x.terms.get(k), yy.terms.get(k)
            a = np.abs(X).max() if X is not None else 0.0
            e = np.abs((Y if Y is not None else 0) - (X if X is not None else 0)).max()
640         if e > 1e-8 * max(a, mag.get(k, 0.0)) + 1e-13 * big:
                raise NoSquareRootError(
                    f"{what}: y* y differs from x at the word {alg.word_str(k[2], k[1])} by {e:.2e} "
                    f"(coefficient of x there: {a:.2e})"
                )
645     if alg.is_real(x) and not alg.is_real(y):
            raise NoSquareRootError(f"{what}: could not find a real *-square root")
        return y
```

```
def star_sqrt(
    x: Element,
    rule: str = "graded",
    choice: str = "positive",
    split: str = "left",
    rng: np.random.Generator | None = None,
) -> Element:
    """A *-square root of ``x``: some ``y`` with ``y* y = x``.

    Parameters
    ----------
    x : Element
        A self-adjoint element.
    rule : {'graded', 'principal', 'arbitrary'}
    choice : {'positive', 'random'}
        For the graded rule: principal square roots of the blocks, or those times a random unitary
        of the semisimple part.
    split : {'left', 'right', 'balanced'}
        For the graded rule: where ``x`` goes at an exchanged pair of vertices.
    rng : numpy.random.Generator, optional
        For the random choices.

    Returns
    -------
    Element

    Raises
    ------
    NoSquareRootError
        If the rule cannot produce a root.

    Examples
    --------
    >>> from dsqlib import Algebra, catalogue
    >>> A = Algebra(catalogue.svd("C"))
    >>> z = A.real_element(A.arrow("a", 1 + 2j))      # (1+2i) a + (1-2i) a~
    >>> star_sqrt(z.H * z)
    2.236·a + 2.236·a~
    """
    if rule == "graded":
        return sqrt_graded(x, choice, split, rng)
    if rule == "principal":
        return sqrt_principal(x)
    if rule == "arbitrary":
        alg = x.algebra
        rng = rng or np.random.default_rng()
        try:
            y = sqrt_graded(x, "positive", split)
        except NoSquareRootError:
            y = sqrt_principal(x)
        if y.terms:  # a random unitary of the orbit corner where y ends
            T = {v for (t, _, _) in y.terms for v in alg.quiver.orbit_of(t)}
            y = alg.mul(alg.mul(random_unitary(alg, rng, T), random_semisimple_unitary(alg, rng)), y)
        return _check(alg, alg.truncate(x, y.prec), y, "arbitrary rule")
    raise ValueError(f"rule must be one of {RULES}, not {rule!r}")


def sqrt_rule(
    rule: str = "graded", choice: str = "positive", split: str = "left", seed: int | None = None
) -> Callable[[Element], Element]:
    """A one-argument *-square root ``x -> y`` for the ``sqrt`` argument of :mod:`dsqlib.linalg`.

    Examples
    --------
    >>> sq = sqrt_rule("graded", "random", seed=1)   # reproducible random choices
    """
    if rule not in RULES:
        raise ValueError(f"rule must be one of {RULES}, not {rule!r}")
    if choice not in CHOICES:
        raise ValueError(f"choice must be one of {CHOICES}, not {choice!r}")
    if split not in SPLITS:
```

```
        raise ValueError(f"split must be one of {SPLITS}, not {split!r}")
    rng = np.random.default_rng(seed)

    def sqrt(x: Element) -> Element:
        return star_sqrt(x, rule, choice, split, rng)

    sqrt.__doc__ = f"*-square root by the {rule!r} rule (choice={choice!r}, split={split!r})"
    return sqrt
```

B.1.5 src/dsqlib/matrix.py

```
"""Matrices over the completed localisation, indexed by self-adjoint idempotents.

A matrix index is a list of positions. Position ``i`` stands for a real self-adjoint idempotent
``E_i`` of the algebra, and entry ``(i, j)`` of a matrix ``H`` lives in the corner ``E_i A E_j``. This
generalises the usual indexing, where every ``E_i = 1``. Two indexings are provided:

:func:`plain_index`
    every ``E_i = 1``: ``n x n`` matrices over the algebra in the usual sense;
:func:`orbit_index`
    one counter per STAR-BAR orbit ``O``, standing for ``e_O`` = the sum of ``e_v`` over the vertices
    of ``O`` (real and self-adjoint, because ``O`` is closed under the transpose and the conjugation).
    The positions visit the orbits in a cyclic order and come back to the first, skipping an orbit
    whose counter is exhausted: for ``(n_u, n_v) = (3, 5)`` they are ``u0 v0 u1 v1 u2 v2 v3 v4``.

A :class:`Matrix` holds its entries together with a row index and a column index.
"""

from __future__ import annotations

import numbers
from collections.abc import Callable, Iterator, Mapping, Sequence
from typing import Any, overload

import numpy as np

from .algebra import INF, Algebra, Element

__all__ = ["Index", "Matrix", "cyclic_positions", "orbit_index", "plain_index"]

Position = tuple[str, int]


def cyclic_positions(dims: Mapping[str, int], order: Sequence[str]) -> list[Position]:
    """The positions ``(orbit, k)``, cycling through the orbits in the given order and skipping an
    orbit whose counter has reached ``dims[orbit]``.

    Examples
    --------
    >>> cyclic_positions({"u": 2, "v": 3}, ["u", "v"])
    [('u', 0), ('v', 0), ('u', 1), ('v', 1), ('v', 2)]
    """
    idx, k = [], 0
    while any(k < dims[o] for o in order):
        for o in order:
            if k < dims[o]:
                idx.append((o, k))
        k += 1
    return idx


class Index:
    """A list of positions ``(label, k)``; each label names a set of vertices whose idempotents add up
    to the idempotent ``E`` of the positions with that label.

    Parameters
    ----------
    algebra : Algebra
    positions : sequence of (label, k)
    groups : mapping of label to a tuple of vertices
```

```
    """

    def __init__(self, algebra: Algebra, positions: Sequence[Position], groups: Mapping[str, Sequence[str
    ↪ ]]) -> None:
        self.algebra = algebra
        self.positions: list[Position] = [tuple(p) for p in positions]  # type: ignore[misc]
        self.groups: dict[str, tuple[str, ...]] = {k: tuple(v) for k, v in groups.items()}
        for label, _ in self.positions:
            if label not in self.groups:
                raise ValueError(f"position label {label!r} has no vertex group")
        self._E: dict[str, Element] = {}
        for label, vs in self.groups.items():
            e = algebra.zero()
            for v in vs:
                e = algebra.add(e, algebra.idempotent(v))
            self._E[label] = e

    def __len__(self) -> int:
        return len(self.positions)

    @property
    def n(self) -> int:
        """The number of positions."""
        return len(self.positions)

    @overload
    def __getitem__(self, i: int) -> Position: ...
    @overload
    def __getitem__(self, i: slice) -> Index: ...
    def __getitem__(self, i: int | slice) -> Position | Index:
        if isinstance(i, slice):
            return Index(self.algebra, self.positions[i], self.groups)
        return self.positions[i]

    def __iter__(self) -> Iterator[Position]:
        return iter(self.positions)

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Index):
            return NotImplemented
        return (
            self.algebra is other.algebra
            and self.positions == other.positions
            and all(self.groups[p[0]] == other.groups[p[0]] for p in self.positions)
        )

    def __hash__(self) -> int:
        return hash(tuple(self.positions))

    def __repr__(self) -> str:
        return "Index(" + ", ".join(str(k) if lab == "1" else f"{lab}{k}" for lab, k in self.positions) + "
    ↪ )"

    def idempotent(self, i: int) -> Element:
        """``E_i``, the idempotent of position ``i``."""
        return self._E[self.positions[i][0]]

    def label(self, i: int) -> str:
        """The label of position ``i``."""
        return self.positions[i][0]

    def vertices(self, i: int) -> tuple[str, ...]:
        """The vertices whose idempotents add up to ``E_i``."""
        return self.groups[self.positions[i][0]]

    def position(self, label: str, k: int) -> int:
        """The position number of ``(label, k)``."""
        return self.positions.index((label, k))

    def subset(self, positions: Sequence[int]) -> Index:
        """The index made of the given positions, in that order."""
        return Index(self.algebra, [self.positions[i] for i in positions], self.groups)

```

```
        def relabel(self, mapping: Mapping[str, str]) -> Index:
            """The same positions with the labels renamed by ``mapping`` (for example after a QR step on
            the SVD indexing, which maps each orbit to the other end of the arrows)."""
            return Index(self.algebra, [(mapping.get(lab, lab), k) for lab, k in self.positions], self.groups)

    def plain_index(algebra: Algebra, n: int) -> Index:
        """``n`` positions with every ``E_i = 1`` (the label ``'1'`` stands for all vertices)."""
        return Index(algebra, [("1", k) for k in range(n)], {"1": tuple(algebra.vertices)})

    def orbit_index(algebra: Algebra, dims: Mapping[str, int], order: Sequence[str] | None = None) -> Index:
        """One counter per STAR-BAR orbit, cycling through the orbits in ``order``.

        Parameters
        ----------
        algebra : Algebra
        dims : mapping of orbit name to the number of its positions
            Orbits are named by the base names given to :meth:`DSQ.build`.
        order : sequence of orbit names, optional
            The cyclic order (default: the order of the quiver).

        Examples
        --------
        >>> from dsqlib import Algebra, catalogue
        >>> A = Algebra(catalogue.svd("R"))
        >>> orbit_index(A, {"u": 3, "v": 5}, ["u", "v"])
        Index(u0, v0, u1, v1, u2, v2, v3, v4)
        """
        names = algebra.quiver.orbit_names()
        for o in dims:
            if o not in names:
                raise ValueError(f"unknown orbit {o!r}; the orbits are {list(names)}")
        order = list(order) if order else [o for o in names if o in dims]
        if sorted(order) != sorted(dims):
            raise ValueError(f"the cyclic order {order} must list each orbit of dims {list(dims)} once")
        return Index(algebra, cyclic_positions(dims, order), {o: names[o] for o in order})


    def _exact_zero(x: Element) -> bool:
        return not x.terms and x.prec == INF


    class Matrix:
        """A matrix over an :class:`~dsqlib.algebra.Algebra`, with a row and a column :class:`Index`.

        Entry ``(i, j)`` lives in the corner ``E_i A E_j`` (``E_i`` from the row index, ``E_j`` from the
        column index); :meth:`corner_violation` measures how far the entries are from that.

        Supports ``H[i, j]`` (an :class:`~dsqlib.algebra.Element`), slicing ``H[r0:r1, c0:c1]``, ``A @ B``,
        ``A + B``, ``A - B``, ``-A``, ``c * A`` for a number ``c``, and ``A.H`` (the adjoint: transpose with
        the involution applied entrywise).

        Parameters
        ----------
        entries : list of lists of Element
        rows : Index
        cols : Index, optional
            Defaults to ``rows``.
        """

        __slots__ = ("_e", "algebra", "cols", "rows")
        __array_ufunc__ = None

        def __init__(self, entries: Sequence[Sequence[Element]], rows: Index, cols: Index | None = None) ->
        ↪ None:
            cols = rows if cols is None else cols
            if rows.algebra is not cols.algebra:
                raise ValueError("the row and column indices belong to different algebras")
            self.algebra: Algebra = rows.algebra
            self.rows, self.cols = rows, cols
            self._e: list[list[Element]] = [list(r) for r in entries]
            if len(self._e) != len(rows) or any(len(r) != len(cols) for r in self._e):
```

```
            raise ValueError(f"entries are not {len(rows)} x {len(cols)}")
        for r in self._e:
            for x in r:
                if not isinstance(x, Element) or x.algebra is not self.algebra:
                    raise TypeError("Matrix entries must be Elements of the indices' algebra")

    # ------------------------------------------------------------------ construction
    @classmethod
    def zeros(cls, rows: Index, cols: Index | None = None) -> Matrix:
        """The zero matrix."""
        cols = rows if cols is None else cols
        z = rows.algebra.zero()
        return cls([[z for _ in range(len(cols))] for _ in range(len(rows))], rows, cols)

    @classmethod
    def identity(cls, index: Index) -> Matrix:
        """``diag(E_i)``, the identity of the matrices with this index."""
        M = cls.zeros(index)
        for i in range(len(index)):
            M._e[i][i] = index.idempotent(i)
        return M

    @classmethod
    def from_function(cls, rows: Index, f: Callable[[int, int], Element | None], cols: Index | None = None)
     ↪  -> Matrix:
        """The matrix with entries ``f(i, j)`` (``None`` for zero)."""
        cols = rows if cols is None else cols
        z = rows.algebra.zero()
        return cls([[(f(i, j) or z) for j in range(len(cols))] for i in range(len(rows))], rows, cols)

    @classmethod
    def unit_vector(cls, index: Index, i: int) -> Matrix:
        """The column vector with ``E_i`` in position ``i`` (its column index is position ``i``)."""
        i = range(len(index))[i]  # normalises negative positions, raises IndexError when out of range
        z = index.algebra.zero()
        return cls([[index.idempotent(i) if k == i else z] for k in range(len(index))], index, index.subset
     ↪ ([i]))

    @classmethod
    def from_columns(cls, columns: Sequence[Sequence[Element]], rows: Index, cols: Index | None = None) ->
     ↪ Matrix:
        """The matrix with the given columns (each a list of entries). ``cols`` defaults to the first
        ``len(columns)`` positions of ``rows``."""
        cols = rows[: len(columns)] if cols is None else cols
        return cls([[c[i] for c in columns] for i in range(len(rows))], rows, cols)

    # ------------------------------------------------------------------ access
    @property
    def shape(self) -> tuple[int, int]:
        """``(number of rows, number of columns)``."""
        return (len(self.rows), len(self.cols))

    def __getitem__(self, key: tuple[Any, Any]) -> Any:
        if not (isinstance(key, tuple) and len(key) == 2):
            raise TypeError("index a Matrix with H[i, j] or H[r0:r1, c0:c1]")
        i, j = key
        if isinstance(i, numbers.Integral) and isinstance(j, numbers.Integral):
            return self._e[i][j]
        ri = range(len(self.rows))[i] if isinstance(i, slice) else [range(len(self.rows))[i]]
        cj = range(len(self.cols))[j] if isinstance(j, slice) else [range(len(self.cols))[j]]
        return Matrix([[self._e[a][b] for b in cj] for a in ri], self.rows.subset(list(ri)), self.cols.
     ↪ subset(list(cj)))

    def __setitem__(self, key: tuple[int, int], value: Element) -> None:
        if not (
            isinstance(key, tuple)
            and len(key) == 2
            and isinstance(key[0], numbers.Integral)
            and isinstance(key[1], numbers.Integral)
        ):
            raise TypeError("assign single entries: H[i, j] = x")
        if not isinstance(value, Element) or value.algebra is not self.algebra:
            raise TypeError("a Matrix entry must be an Element of the matrix's algebra")
```

```
        i, j = key
        self._e[i][j] = value

    def row(self, i: int) -> list[Element]:
        """Row ``i`` as a list of entries."""
        return list(self._e[i])

    def column(self, j: int) -> list[Element]:
        """Column ``j`` as a list of entries."""
        return [r[j] for r in self._e]

    def tolist(self) -> list[list[Element]]:
        """The entries as a list of lists (a copy of the lists; the entries are shared)."""
        return [list(r) for r in self._e]

    def copy(self) -> Matrix:
        """A copy (the entries, which are immutable in practice, are shared)."""
        return Matrix(self._e, self.rows, self.cols)

    def with_index(self, rows: Index, cols: Index | None = None) -> Matrix:
        """The same entries with another row (and column) index."""
        return Matrix(self._e, rows, rows if cols is None else cols)

    # ------------------------------------------------------------------ arithmetic
    def __matmul__(self, other: Matrix) -> Matrix:
        if not isinstance(other, Matrix):
            return NotImplemented
        if self.shape[1] != other.shape[0]:
            raise ValueError(f"shapes {self.shape} and {other.shape} do not compose")
        if self.algebra is not other.algebra:
            raise ValueError("matrices over different algebras")
        if self.cols != other.rows:
            raise ValueError(
                f"the column index {self.cols} of the left factor is not the row index {other.rows} of the "
                "right one (relabel with with_index if that is intended)"
            )
        alg = self.algebra
        out = []
        for i in range(self.shape[0]):
            row = []
            for j in range(other.shape[1]):
                acc = alg.zero()
                for k in range(self.shape[1]):
                    a, b = self._e[i][k], other._e[k][j]
                    if _exact_zero(a) or _exact_zero(b):
                        continue
                    acc = alg.add(acc, alg.mul(a, b))
                row.append(acc)
            out.append(row)
        return Matrix(out, self.rows, other.cols)

    def _check_same_shape(self, other: Matrix) -> None:
        if self.shape != other.shape:
            raise ValueError(f"shapes {self.shape} and {other.shape} differ")
        if self.algebra is not other.algebra:
            raise ValueError("matrices over different algebras")
        if self.rows != other.rows or self.cols != other.cols:
            raise ValueError("the matrices have different indices (relabel with with_index if that is intended)")

    def __add__(self, other: Matrix) -> Matrix:
        if not isinstance(other, Matrix):
            return NotImplemented
        self._check_same_shape(other)
        add = self.algebra.add
        return Matrix(
            [[add(a, b) for a, b in zip(ra, rb, strict=True)] for ra, rb in zip(self._e, other._e, strict=True)],
            self.rows,
            self.cols,
        )
```

```
        def __sub__(self, other: Matrix) -> Matrix:
            if not isinstance(other, Matrix):
                return NotImplemented
            self._check_same_shape(other)
            sub = self.algebra.sub
            return Matrix(
                [[sub(a, b) for a, b in zip(ra, rb, strict=True)] for ra, rb in zip(self._e, other._e, strict=
        ↪ True)],
                self.rows,
                self.cols,
            )

        def __neg__(self) -> Matrix:
            return (-1.0) * self

        def __mul__(self, c: Any) -> Matrix:
            if not isinstance(c, (int, float, complex, np.number)):
                return NotImplemented
            scale = self.algebra.scale
            return Matrix([[scale(c, a) for a in r] for r in self._e], self.rows, self.cols)

        __rmul__ = __mul__

        @property
        def H(self) -> Matrix:
            """The adjoint: ``(H*)_ij = (H_ji)*``."""
            star = self.algebra.star
            m, n = self.shape
            return Matrix([[star(self._e[j][i]) for j in range(m)] for i in range(n)], self.cols, self.rows)

        def conj(self) -> Matrix:
            """The entrywise conjugation."""
            conj = self.algebra.conj
            return Matrix([[conj(a) for a in r] for r in self._e], self.rows, self.cols)

        def real(self) -> Matrix:
            """The entrywise real part ``(x + c(x)) / 2``."""
            real = self.algebra.real
            return Matrix([[real(a) for a in r] for r in self._e], self.rows, self.cols)

        # ------------------------------------------------------------------ checks
        def corner_violation(self, rows: Index | None = None, cols: Index | None = None) -> float:
            """The total norm of the parts of the entries outside their corners ``E_i A E_j`` (``rows`` and
            ``cols`` override the matrix's own indices)."""
            rows, cols = rows or self.rows, cols or self.cols
            alg, bad = self.algebra, 0.0
            for i, r in enumerate(self._e):
                for j, x in enumerate(r):
                    inside = alg.corner_project(x, rows.vertices(i), cols.vertices(j))
                    bad += alg.dist(x, inside)
            return bad

        def unitary_defect(self, rows: Index | None = None, cols: Index | None = None) -> float:
            """``|U* U - diag(E_cols)| + |U U* - diag(E_rows)|``: zero exactly when ``U`` is a unitary from
            the corners of the column index to those of the row index."""
            rows, cols = rows or self.rows, cols or self.cols
            alg = self.algebra
            P1, P2 = self.H @ self, self @ self.H
            bad = 0.0
            for i in range(P1.shape[0]):
                for j in range(P1.shape[1]):
                    bad += alg.dist(P1._e[i][j], cols.idempotent(i) if i == j else alg.zero())
            for i in range(P2.shape[0]):
                for j in range(P2.shape[1]):
                    bad += alg.dist(P2._e[i][j], rows.idempotent(i) if i == j else alg.zero())
            return bad

        def off_band(self, band: int = 0) -> float:
            """The total norm of the entries with ``|i - j| > band``."""
            norm = self.algebra.norm
            return sum(norm(x) for i, r in enumerate(self._e) for j, x in enumerate(r) if abs(i - j) > band)

        def is_real(self) -> bool:
```

```
        """Whether every entry is real."""
        return all(self.algebra.is_real(x) for r in self._e for x in r)

    def is_selfadjoint(self, tol: float = 1e-9) -> bool:
        """Whether ``H* = H`` entrywise, to relative tolerance ``tol``."""
        alg = self.algebra
        m, n = self.shape
        return m == n and all(
            alg.dist(alg.star(self._e[j][i]), self._e[i][j]) <= tol * alg.norm(self._e[i][j])
            for i in range(m)
            for j in range(n)
        )

    def degree_range(self) -> tuple[int, int] | tuple[None, None]:
        """``(lowest, highest)`` degree among the entries (``(None, None)`` for the zero matrix)."""
        degs = [d for r in self._e for x in r for d in self.algebra.degrees(x)]
        return (min(degs), max(degs)) if degs else (None, None)

    def coefficients(
        self, letters: Sequence[str] = (), target: str | None = None, source: str | None = None
    ) -> np.ndarray:
        """The coefficient blocks of one word in every entry.

        Returns an array of shape ``(m, n)`` when the blocks are 1 x 1, else ``(m, n, d_t, d_s)``;
        see :meth:`Element.coefficient` for the arguments.
        """
        blocks = [[x.coefficient(letters, target, source) for x in r] for r in self._e]
        arr = np.array(blocks, dtype=complex)
        return arr[:, :, 0, 0] if arr.shape[2:] == (1, 1) else arr

    # ------------------------------------------------------------------ display
    def format(self, digits: int = 3, width: int = 40) -> str:
        """A grid of the formatted entries, each cut to ``width`` characters."""
        cells = [[self.algebra.format(x, digits) for x in r] for r in self._e]
        cells = [[c if len(c) <= width else c[: width - 1] + "..." for c in r] for r in cells]
        w = max((len(c) for r in cells for c in r), default=1)
        return "\n".join("[ " + "  ".join(c.ljust(w) for c in r) + " ]" for r in cells)

    def __repr__(self) -> str:
        m, n = self.shape
        return f"Matrix {m}x{n}, rows {self.rows}, cols {self.cols}\n{self.format()}"
```

B.1.6 src/dsqlib/linalg.py

```
"""Matrix algorithms over the *-algebra of any doubly-signed quiver, written once.

Every function takes a :class:`~dsqlib.matrix.Matrix` and, where it needs them:

``sqrt``
    a *-square root ``x -> y`` with ``y* y = x``: a callable (for example ``algebra.sqrt`` or
    :func:`~dsqlib.sqrt.sqrt_rule`), a rule name (``'graded'``, ``'principal'``, ``'arbitrary'``), or
    ``None`` for the graded rule with principal choices;
``inverse``
    how pivots are inverted: ``'support'`` (the default: between the vertices an element touches),
    ``'orbits'`` (between STAR-BAR orbit idempotents, so that a zero divisor of an orbit corner is a
    breakdown), ``'all'``, or a callable. Every quotient ``x / y`` is checked: ``(x / y) y`` (or
    ``y (y^-1 x)``) must equal ``x``, which fails when ``y`` is invertible only on part of the corner of
    ``x``; that is reported as a breakdown. Householder reflectors do not use ``inverse``: their
    denominator ``w* x`` must be invertible in the corner ``f A f`` of its column (thesis, Proposition
    'householder-any-root'), and it is inverted there.

Nothing here depends on the quiver. :class:`~dsqlib.errors.BreakdownError` is raised when a pivot
has no inverse or does not divide exactly, at an isotropic vector (``v != 0`` with ``v* v = 0``) in
Gram-Schmidt, Householder and Lanczos, when the denominator of a Householder reflector is not invertible
in the corner of its column, at a zero pivot in :func:`cholesky` and :func:`ldl`, and at a zero pivot
with nonzero entries below it in :func:`lu`. With the default ``inverse='support'`` (which maps zero to
zero) a residual column that is exactly zero, i.e. a rank-deficient ``H``, gives a zero column of ``Q``
in Gram-Schmidt: ``Q`` is then a partial isometry, ``H = Q R`` still holds, and for a self-adjoint ``H``
the QR step ``Q* H Q`` keeps the nonzero spectrum and replaces the kernel by zeros.
```

```
    Examples
    --------
    >>> import numpy as np
    >>> from dsqlib import problems
    >>> from dsqlib.linalg import householder
    >>> A = np.array([[2.0, 1.0, 0.5], [1.0, 3.0, 1.0], [0.5, 1.0, 4.0]])
    >>> T, U, _ = householder(problems.symmetric_problem(A))
    >>> T.off_band(1) == 0  # tridiagonal
    True
    >>> B = problems.extract_loop(T, ["x"]).real
    >>> bool(np.allclose(np.linalg.eigvalsh(B), np.linalg.eigvalsh(A)))
    True
    """

    from __future__ import annotations

    import functools
    from collections.abc import Callable, Sequence
    from typing import NamedTuple

    from .algebra import INF, Algebra, Element
    from .errors import BreakdownError, NotInvertibleError
    from .matrix import Matrix
    from .sqrt import sqrt_rule

    __all__ = [
        "HouseholderResult",
        "LDLResult",
        "LUResult",
        "QRResult",
        "QRStepResult",
        "cholesky",
        "householder",
        "householder_qr",
        "lanczos",
        "ldl",
        "lu",
        "qr",
        "qr_step",
    ]

    SqrtArg = Callable[[Element], Element] | str | None
    InverseArg = Callable[[Element], Element] | str | None


    class HouseholderResult(NamedTuple):
        """``H = U T U*`` with ``T`` tridiagonal."""

        T: Matrix
        U: Matrix
        residuals: list[float]
        """The size of what is left below the subdiagonal of column ``k`` after step ``k``."""


    class QRResult(NamedTuple):
        """``H = Q R`` with ``Q`` unitary and ``R`` upper triangular."""

        Q: Matrix
        R: Matrix


    class QRStepResult(NamedTuple):
        """One QR step: ``H = Q R`` and ``H_next = R Q = Q* H Q``."""

        H_next: Matrix
        Q: Matrix
        R: Matrix


    class LDLResult(NamedTuple):
        """``H = L D L*`` with ``L`` unit lower triangular and ``D`` diagonal."""

        L: Matrix
```

```
    D: Matrix


class LUResult(NamedTuple):
    """``H = L U`` with ``L`` unit lower triangular and ``U`` upper triangular."""

    L: Matrix
    U: Matrix


# --------------------------------------------------------------------- helpers
def _sqrt_fn(sqrt: SqrtArg) -> Callable[[Element], Element]:
    if sqrt is None:
        return sqrt_rule()
    if isinstance(sqrt, str):
        return sqrt_rule(sqrt)
    return sqrt


def _inverse_fn(alg: Algebra, inverse: InverseArg, what: str) -> Callable[[Element], Element]:
    base: Callable[[Element], Element]
    if inverse is None or isinstance(inverse, str):
        within = inverse or "support"
        if within not in ("support", "orbits", "all"):
            raise ValueError(f"inverse must be 'support', 'orbits', 'all' or a callable, not {within!r}")
        base = functools.partial(alg.inverse, within=within)
    else:
        base = inverse

    def guarded(x: Element) -> Element:
        try:
            return base(x)
        except NotInvertibleError as e:
            raise BreakdownError(f"{what}: a pivot has no inverse ({e})") from e

    return guarded


def _exact_zero(x: Element) -> bool:
    return not x.terms and x.prec == INF


def _dot(alg: Algebra, x: Sequence[Element], y: Sequence[Element]) -> Element:
    """``sum_i x_i* y_i``."""
    acc = alg.zero()
    for a, b in zip(x, y, strict=True):
        if _exact_zero(a) or _exact_zero(b):
            continue
        acc = alg.add(acc, alg.mul(alg.star(a), b))
    return acc


def _square(H: Matrix, what: str) -> None:
    if H.shape[0] != H.shape[1]:
        raise ValueError(f"{what} needs a square matrix, not {H.shape[0]} x {H.shape[1]}")


def _check_sign(sign: str | None) -> None:
    if sign not in ("auto", None):
        raise ValueError(f"sign must be 'auto' or None, not {sign!r}")


def _in_corner(x: Element, rows: Sequence[str], cols: Sequence[str]) -> bool:
    return all(k[0] in rows and k[1] in cols for k in x.terms)


_QUOTIENT_RTOL = 1e-9


def _checked(alg: Algebra, x: Element, q: Element, y: Element, what: str, left: bool = False) -> Element:
    """Return the quotient ``q`` (``x y^-1``, or ``y^-1 x`` if ``left``) after checking that it is exact:
    ``q y = x`` (``y q = x``). This fails when ``y`` is invertible only on part of the corner of ``x``,
    as for a pivot ``(c, 0)`` of the split-complex numbers dividing an entry ``(p, q)`` with ``q != 0``."""
```

```
    back = alg.mul(y, q) if left else alg.mul(q, y)
    if alg.dist(back, x) > _QUOTIENT_RTOL * (alg.norm(x) + alg.norm(back)):
        raise BreakdownError(f"{what}: a pivot is invertible only on part of the corner of the entry it divides")
    return q


def _corner_inverse(alg: Algebra, s: Element, verts: Sequence[str], what: str) -> Element:
    """The inverse of ``s`` from ``f`` to ``f``, ``f`` the sum of the idempotents of ``verts`` (the corner of a column), as a Householder reflector requires."""
    try:
        return alg.inverse_between(s, verts, verts)
    except NotInvertibleError as e:
        raise BreakdownError(
            f"{what}: the denominator w* x is not invertible in the corner of its column ({e}); this root of "
            "x* x gives no reflector"
        ) from e


# ---------------------------------------------------------------------- Householder
def householder(
    H: Matrix, sqrt: SqrtArg = None, inverse: InverseArg = None, sign: str | None = "auto"
) -> HouseholderResult:
    """Tridiagonalise a self-adjoint ``H`` by Householder reflectors: ``H = U T U*``.

    The reflector for the column part ``x`` is ``P = I - w (w* x)^+ w*`` with ``w = x - e_1 gamma`` and
    ``gamma = +-sqrt(x* x)``; ``P`` is unitary for *any* *-square root ``gamma``, so no phase condition is needed.

    Parameters
    ----------
    H : Matrix
        Self-adjoint.
    sqrt, inverse
        See the module documentation.
    sign : {'auto', None}
        ``'auto'`` uses ``-gamma`` (also a *-square root) when that avoids cancellation in
        ``x_0 - gamma``, as the classical algorithm does; ``None`` uses ``gamma`` as returned (the same
        in exact arithmetic, but a nearly reduced column then loses digits in floating point).

    Returns
    -------
    HouseholderResult
        ``(T, U, residuals)``.
    """
    _square(H, "householder")
    _check_sign(sign)
    alg, n = H.algebra, H.shape[0]
    sq = _sqrt_fn(sqrt)
    _inverse_fn(alg, inverse, "householder")  # validates the argument; the reflector uses its own inverse
    rows = H.rows
    T = H.tolist()
    U = Matrix.identity(rows)
    resid = []
    for k in range(n - 2):
        x = [T[i][k] for i in range(k + 1, n)]
        g = _dot(alg, x, x)
        if not g.terms and any(xi.terms for xi in x):
            raise BreakdownError(f"householder: column {k} is isotropic (x != 0, x* x = 0)")
        gamma = sq(g)
        if not _in_corner(gamma, rows.vertices(k + 1), H.cols.vertices(k)):
            raise ValueError(
                f"householder: the *-square root of x* x for column {k} does not lie in the corner of position "
                f"({k + 1}, {k}); Householder reflectors do not suit this indexing"
            )
        if sign == "auto" and alg.norm(alg.add(x[0], gamma)) > alg.norm(alg.sub(x[0], gamma)):
            gamma = alg.scale(-1.0, gamma)
        w = [alg.sub(x[0], gamma), *x[1:]]
        if all(not wi.terms for wi in w):  # x = e_1 gamma already (to the known precision)
            resid.append(sum(alg.norm(T[i][k]) for i in range(k + 2, n)))
            continue
```

```
            si = _corner_inverse(alg, _dot(alg, w, x), H.cols.vertices(k), f"householder: column {k}")
            P = Matrix.identity(rows).tolist()
            for i in range(k + 1, n):
                for j in range(k + 1, n):
                    P[i][j] = alg.sub(P[i][j], alg.mul(alg.mul(w[i - k - 1], si), alg.star(w[j - k - 1])))
            Pm = Matrix(P, rows)
            Ps = Pm.H
            T = (Pm @ Matrix(T, rows) @ Ps).tolist()
            U = U @ Ps
            resid.append(sum(alg.norm(T[i][k]) for i in range(k + 2, n)))
        return HouseholderResult(Matrix(T, rows), U, resid)


    def householder_qr(H: Matrix, sqrt: SqrtArg = None, inverse: InverseArg = None, sign: str | None = "auto")
        ↪ -> QRResult:
        """``H = Q R`` by Householder reflectors (the reflectors of :func:`householder`).

        The reflector for column ``k`` puts the *-square root ``gamma`` of ``x* x`` at position ``(k, k)``,
        so it needs ``gamma`` in the corner ``E_k A E_k``: true on the plain indexing and on one-orbit
        problems, but not on the SVD orbit indexing (where the root of a degree-2 Gram element maps one
        orbit to the other); there it raises ``ValueError`` and Gram-Schmidt (:func:`qr`) is the method.

        Returns
        -------
        QRResult
            The full factorisation: ``Q`` (``m x m``) unitary, ``Q* Q = diag(E_i)``, and ``R`` (``m x n``)
            upper triangular.

        Raises
        ------
        BreakdownError
            At an isotropic column or a pivot with no inverse.
        """
        _check_sign(sign)
        alg, (m, n) = H.algebra, H.shape
        sq = _sqrt_fn(sqrt)
        _inverse_fn(alg, inverse, "householder_qr")  # validates the argument; the reflector uses its own
        ↪ inverse
        R = H.tolist()
        Q = Matrix.identity(H.rows).tolist()
        for k in range(min(m - 1, n)):
            x = [R[i][k] for i in range(k, m)]
            g = _dot(alg, x, x)
            if not g.terms:
                if any(xi.terms for xi in x):
                    raise BreakdownError(f"householder_qr: column {k} is isotropic (x != 0, x* x = 0)")
                continue
            gamma = sq(g)
            if not _in_corner(gamma, H.rows.vertices(k), H.cols.vertices(k)):
                raise ValueError(
                    f"householder_qr: the *-square root of x* x for column {k} does not lie in the corner of "
                    f"position ({k}, {k}); use qr(H, method='gram-schmidt') on this indexing"
                )
            if sign == "auto" and alg.norm(alg.add(x[0], gamma)) > alg.norm(alg.sub(x[0], gamma)):
                gamma = alg.scale(-1.0, gamma)
            w = [alg.sub(x[0], gamma), *x[1:]]
            if all(not wi.terms for wi in w):
                continue
            si = _corner_inverse(alg, _dot(alg, w, x), H.cols.vertices(k), f"householder_qr: column {k}")
            for j in range(n):  # rows k..m-1 of R:  R <- P R
                c = _dot(alg, w, [R[k + i][j] for i in range(m - k)])
                if _exact_zero(c):
                    continue
                f = alg.mul(si, c)
                for i in range(m - k):
                    R[k + i][j] = alg.sub(R[k + i][j], alg.mul(w[i], f))
            sis = alg.star(si)  # Q <- Q P*,  P* = I - w si* w*
            for r in range(m):
                c = alg.zero()
                for i in range(m - k):
                    if not _exact_zero(Q[r][k + i]) and not _exact_zero(w[i]):
                        c = alg.add(c, alg.mul(Q[r][k + i], w[i]))
                if _exact_zero(c):
```

```
                continue
            f = alg.mul(c, sis)
            for i in range(m - k):
                Q[r][k + i] = alg.sub(Q[r][k + i], alg.mul(f, alg.star(w[i])))
    return QRResult(Matrix(Q, H.rows), Matrix(R, H.rows, H.cols))


def _gram_schmidt(H: Matrix, sq: Callable[[Element], Element], inv: Callable[[Element], Element]) -> Matrix:
    alg, (m, n) = H.algebra, H.shape
    cols: list[list[Element]] = []
    for j in range(n):
        v = H.column(j)
        for q in cols:
            c = _dot(alg, q, v)
            v = [alg.sub(v[i], alg.mul(q[i], c)) for i in range(m)]
        g = _dot(alg, v, v)
        if not g.terms and any(x.terms for x in v):
            raise BreakdownError(f"Gram-Schmidt: column {j} is isotropic (v != 0, v* v = 0)")
        r = sq(g)
        yi = inv(r)  # a zero residual (H rank-deficient) gives a zero column of Q
        cols.append([_checked(alg, v[i], alg.mul(v[i], yi), r, "Gram-Schmidt") for i in range(m)])
    return Matrix.from_columns(cols, H.rows, H.cols)


def qr(H: Matrix, sqrt: SqrtArg = None, inverse: InverseArg = None, method: str = "gram-schmidt") -> QRResult:
    """``H = Q R`` with ``R`` upper triangular.

    Parameters
    ----------
    H : Matrix
        ``m x n``.
    sqrt, inverse
        See the module documentation.
    method : {'gram-schmidt', 'householder'}
        Gram-Schmidt, with ``R_jj`` = the *-square root of the Gram element of the residual column and
        ``R = Q* H``: the thin factorisation, ``Q`` (``m x n``) with ``Q* Q = diag(E_j)`` and ``R``
        (``n x n``); it works on every indexing. Or Householder reflectors (:func:`householder_qr`): the
        full factorisation, ``Q`` (``m x m``) unitary and ``R`` (``m x n``), on indexings where the
        square roots stay in their corners.

    Returns
    -------
    QRResult

    Raises
    ------
    BreakdownError
        See the module documentation.
    """
    if method == "householder":
        return householder_qr(H, sqrt, inverse)
    if method != "gram-schmidt":
        raise ValueError(f"method must be 'householder' or 'gram-schmidt', not {method!r}")
    Q = _gram_schmidt(H, _sqrt_fn(sqrt), _inverse_fn(H.algebra, inverse, "qr"))
    return QRResult(Q, Q.H @ H)


def qr_step(H: Matrix, sqrt: SqrtArg = None, inverse: InverseArg = None, method: str = "gram-schmidt") -> QRStepResult:
    """One step of the QR algorithm: ``H = Q R`` and ``H_next = Q* H Q`` (``= R Q``).

    Parameters
    ----------
    H : Matrix
        Square.
    sqrt, inverse
        See the module documentation.
    method : {'gram-schmidt', 'householder'}
        How the factorisation is computed (see :func:`qr`).

    Returns
```

```
    -------
    QRStepResult
        ``(H_next, Q, R)``.

    Notes
    -----
    On an indexing where the square roots change corners (the SVD orbit indexing, where the root of
    a degree-2 Gram element has degree 1), one step maps each orbit to the other end of the arrows;
    use :meth:`Index.relabel` to describe the result, or take double steps.
    """
    _square(H, "qr_step")
    if method == "gram-schmidt":
        Q = _gram_schmidt(H, _sqrt_fn(sqrt), _inverse_fn(H.algebra, inverse, "qr_step"))
        R = Q.H @ H
        return QRStepResult(R @ Q, Q, R)
    Q, R = qr(H, sqrt, inverse, method)
    return QRStepResult(Q.H @ H @ Q, Q, R)


# ---------------------------------------------------------------------- Lanczos
def lanczos(
    H: Matrix,
    q1: Matrix | Sequence[Element],
    steps: int | None = None,
    sqrt: SqrtArg = None,
    inverse: InverseArg = None,
) -> Matrix:
    """Lanczos vectors ``q_1, ..., q_steps`` of a self-adjoint ``H``, with full reorthogonalisation (twice)
     ↪ .

    Parameters
    ----------
    H : Matrix
        Self-adjoint.
    q1 : Matrix or sequence of Element
        The starting vector, with ``q1* q1`` an idempotent: an ``n x 1`` matrix such as
        ``Matrix.unit_vector(H.rows, 0)``, or a list of ``n`` entries.
    steps : int, optional
        The number of vectors, at least 1 (default ``n``).

    Returns
    -------
    Matrix
        The ``n x k`` matrix of Lanczos vectors (``k <= steps``: it stops early when the residual
        vanishes, i.e. at an invariant subspace). Its column index is the first ``k`` positions of
        ``H.rows``, which describes the corners of the vectors when ``q1`` lies in the corner of
        position 0; otherwise relabel it with :meth:`Matrix.with_index`.
    """
    _square(H, "lanczos")
    alg, n = H.algebra, H.shape[0]
    steps = n if steps is None else steps
    if steps < 1:
        raise ValueError(f"steps must be at least 1, not {steps}")
    if isinstance(q1, Matrix) and q1.shape != (n, 1):
        raise ValueError(f"q1 must be an {n} x 1 matrix, not {q1.shape[0]} x {q1.shape[1]}")
    sq, inv = _sqrt_fn(sqrt), _inverse_fn(alg, inverse, "lanczos")
    q = q1.column(0) if isinstance(q1, Matrix) else list(q1)
    if len(q) != n:
        raise ValueError(f"q1 must have {n} entries, not {len(q)}")
    Q = [q]
    Ht = H.tolist()
    for _ in range(steps - 1):
        r = []
        for i in range(n):
            acc = alg.zero()
            for k in range(n):
                if _exact_zero(Ht[i][k]) or _exact_zero(Q[-1][k]):
                    continue
                acc = alg.add(acc, alg.mul(Ht[i][k], Q[-1][k]))
            r.append(acc)
        for _ in range(2):
            for qv in Q:
                c = _dot(alg, qv, r)
```

```
                r = [alg.sub(r[i], alg.mul(qv[i], c)) for i in range(n)]
            if not any(x.terms for x in r):
                break
            g = _dot(alg, r, r)
            if not g.terms:
                raise BreakdownError("lanczos: isotropic residual (r != 0, r* r = 0)")
            beta = sq(g)
            bi = inv(beta)
            Q.append([_checked(alg, r[i], alg.mul(r[i], bi), beta, "lanczos") for i in range(n)])
        return Matrix.from_columns(Q, H.rows)


    # ------------------------------------------------------------------- triangular factorisations
    def cholesky(H: Matrix, sqrt: SqrtArg = None, inverse: InverseArg = None) -> Matrix:
        """``H = R* R`` with ``R`` upper triangular and ``R_jj`` the *-square root of the pivot.

        Raises
        ------
        BreakdownError
            At a zero or non-invertible pivot.
        NoSquareRootError
            When a pivot has no *-square root by the rule used (e.g. a negative pivot at a real vertex).
        """
        _square(H, "cholesky")
        alg, n = H.algebra, H.shape[0]
        sq, inv = _sqrt_fn(sqrt), _inverse_fn(alg, inverse, "cholesky")
        R = Matrix.zeros(H.rows).tolist()
        for j in range(n):
            s = H[j, j]
            for k in range(j):
                s = alg.sub(s, alg.mul(alg.star(R[k][j]), R[k][j]))
            if not s.terms:
                raise BreakdownError(f"cholesky: zero pivot at {j}")
            R[j][j] = sq(s)
            ris = alg.star(inv(R[j][j]))
            for col in range(j + 1, n):
                s = H[j, col]
                for k in range(j):
                    s = alg.sub(s, alg.mul(alg.star(R[k][j]), R[k][col]))
                R[j][col] = _checked(alg, s, alg.mul(ris, s), alg.star(R[j][j]), "cholesky", left=True)
        return Matrix(R, H.rows)


    def ldl(H: Matrix, inverse: InverseArg = None) -> LDLResult:
        """``H = L D L*`` with ``L`` unit lower triangular (diagonal ``E_i``) and ``D`` diagonal; no square
        roots.

        Raises
        ------
        BreakdownError
            At a zero or non-invertible pivot.
        """
        _square(H, "ldl")
        alg, n = H.algebra, H.shape[0]
        inv = _inverse_fn(alg, inverse, "ldl")
        L = Matrix.identity(H.rows).tolist()
        D = [alg.zero() for _ in range(n)]
        for j in range(n):
            s = H[j, j]
            for k in range(j):
                s = alg.sub(s, alg.mul(alg.mul(L[j][k], D[k]), alg.star(L[j][k])))
            if not s.terms:
                raise BreakdownError(f"ldl: zero pivot at {j}")
            D[j] = s
            Di = inv(s)
            for i in range(j + 1, n):
                s = H[i, j]
                for k in range(j):
                    s = alg.sub(s, alg.mul(alg.mul(L[i][k], D[k]), alg.star(L[j][k])))
                L[i][j] = _checked(alg, s, alg.mul(s, Di), D[j], "ldl")
        Dm = Matrix.zeros(H.rows)
        for j in range(n):
            Dm[j, j] = D[j]
```

```
    return LDLResult(Matrix(L, H.rows), Dm)


def lu(H: Matrix, inverse: InverseArg = None) -> LUResult:
    """``H = L U`` by Gaussian elimination without pivoting: ``L`` unit lower triangular (diagonal
    ``E_i``), ``U`` upper triangular.

    Raises
    ------
    BreakdownError
        At a zero pivot with nonzero entries below it, or a pivot with no inverse.
    """
    _square(H, "lu")
    alg, n = H.algebra, H.shape[0]
    inv = _inverse_fn(alg, inverse, "lu")
    U = H.tolist()
    L = Matrix.identity(H.rows).tolist()
    for k in range(n):
        if not U[k][k].terms:
            if any(U[i][k].terms for i in range(k + 1, n)):
                raise BreakdownError(f"lu: zero pivot at {k}")
            continue
        pi = inv(U[k][k])
        for i in range(k + 1, n):
            if not U[i][k].terms:
                continue
            m = _checked(alg, U[i][k], alg.mul(U[i][k], pi), U[k][k], "lu")
            L[i][k] = m
            for j in range(k, n):
                U[i][j] = alg.sub(U[i][j], alg.mul(m, U[k][j]))
    return LUResult(Matrix(L, H.rows), Matrix(U, H.rows))
```

B.1.7 src/dsqlib/catalogue.py

```
"""The doubly-signed quivers of the thesis catalogue (Chapter 6) and the double of ``A_r``
(Example 5.20)."""

from __future__ import annotations

from collections.abc import Callable

from .quiver import DSQ

__all__ = [
    "CATALOGUE",
    "double_A",
    "hermitian",
    "ldl",
    "similarity",
    "skew_loop",
    "split_complex",
    "svd",
    "symmetric_loop",
    "takagi",
    "youla",
]


def symmetric_loop() -> DSQ:
    """Symmetric eigendecomposition: one (R, +1) vertex ``v`` with a loop ``x``, ``x* = x``.

    Completed, its algebra is the field of real Laurent series ``R((t))`` with ``t = x``."""
    return DSQ.build([("v", "R")], [("x", "v", "v", {"sign": +1})])


def skew_loop() -> DSQ:
    """Normal form of skew-symmetric matrices: one (R, +1) vertex ``v`` with a loop ``h``, ``h* = -h``."""
    return DSQ.build([("v", "R")], [("h", "v", "v", {"sign": -1})])
```

```
def svd(field: str = "R") -> DSQ:
    """Singular value decomposition over ``field`` = 'R', 'C' or 'H': vertices ``u``, ``v`` and an arrow
    ``a: u -> v`` (with its transpose ``a'``)."""
    kinds = {"R": "R", "C": "Cu", "H": "Hsp"}
    if field not in kinds:
        raise ValueError(f"field must be 'R', 'C' or 'H', not {field!r}")
    return DSQ.build([("u", kinds[field]), ("v", kinds[field])], [("a", "u", "v")])


def hermitian(field: str = "C") -> DSQ:
    """Hermitian eigendecomposition over C (a loop ``x`` at ``v`` with ``x^T = x~``) or over H (a loop
    with ``x* = x``)."""
    if field == "C":
        return DSQ.build([("v", "Cu")], [("x", "v", "v", {"top_is_bar": True})])
    if field == "H":
        return DSQ.build([("v", "Hsp")], [("x", "v", "v", {"sign": +1})])
    raise ValueError(f"field must be 'C' or 'H', not {field!r}")


def takagi() -> DSQ:
    """Takagi factorisation: a unitary-type complex vertex ``v`` and an arrow ``al: v -> v~ = v^T`` with
    sign +1."""
    return DSQ.build([("v", "Cu")], [("al", "v", "v~", {"sign": +1})])


def youla() -> DSQ:
    """Youla normal form of complex skew-symmetric matrices: ``al: v -> v~ = v^T`` with sign -1."""
    return DSQ.build([("v", "Cu")], [("al", "v", "v~", {"sign": -1})])


def ldl() -> DSQ:
    """Congruence of real symmetric matrices (``LDL^T``): an exchanged real pair ``u, u'`` and
    ``a: u -> u'`` with sign +1."""
    return DSQ.build([("u", "RR")], [("a", "u", "u'", {"sign": +1})])


def similarity() -> DSQ:
    """Similarity of real matrices: an exchanged real pair ``v, v'`` with a loop ``x`` at ``v`` (and
    ``x'`` at ``v'``)."""
    return DSQ.build([("v", "RR")], [("x", "v", "v")])


def split_complex() -> DSQ:
    """The split-complex numbers ``2R = R x R`` with ``(a, b)* = (b, a)``: one exchanged real pair,
    no arrows."""
    return DSQ.build([("u", "RR")], [])


def double_A(r: int) -> DSQ:
    """The double of ``1 -> 2 -> ... -> r`` with all vertices of kind (R, +1) (Example 5.20)."""
    return DSQ.build([(str(j), "R") for j in range(1, r + 1)], [(f"a{j}", str(j), str(j + 1)) for j in
     ↪ range(1, r)])


CATALOGUE: dict[str, Callable[[], DSQ]] = {
    "symmetric eigendecomposition": symmetric_loop,
    "skew-symmetric normal form": skew_loop,
    "real SVD": lambda: svd("R"),
    "complex SVD": lambda: svd("C"),
    "quaternionic SVD": lambda: svd("H"),
    "Hermitian eigendecomposition over C": lambda: hermitian("C"),
    "Hermitian eigendecomposition over H": lambda: hermitian("H"),
    "Takagi factorisation": takagi,
    "Youla normal form": youla,
    "LDL^T (congruence)": ldl,
    "similarity of real matrices": similarity,
    "split-complex numbers": split_complex,
}
"""The catalogue quivers by problem name (each value builds the quiver)."""
```

B.1.8 src/dsqlib/problems.py

```
"""Classical matrix problems as matrices over the *-algebras of the catalogue.

Each ``*_problem`` function returns a self-adjoint :class:`~dsqlib.matrix.Matrix` ``H`` that encodes
the classical data (its algebra is ``H.algebra`` and its index ``H.rows``); each ``extract_*``
function reads the classical matrix back from a (transformed) ``H``. Quaternions are 2 x 2 complex
blocks ``[[p, q], [-conj(q), conj(p)]]`` (see :func:`~dsqlib.algebra.quaternion`); a quaternion matrix
is an array of shape ``(m, n, 2, 2)``.

=========================  ========================================  ==========================
problem                    classical data                            read back with
=========================  ========================================  ==========================
:func:`svd_problem`        ``W`` over R, C or H (orbit indexing)     :func:`extract_svd`
:func:`svd_problem_plain`  ``W`` over R, C or H (plain indexing)     :func:`extract_svd_plain`
:func:`hermitian_problem`  Hermitian ``A`` over C or H               :func:`extract_loop`
:func:`symmetric_problem`  real symmetric ``A``                      :func:`extract_loop`
:func:`skew_problem`       real skew-symmetric ``A``                 :func:`extract_loop`
:func:`takagi_problem`     complex symmetric ``A``                   :func:`extract_loop`
:func:`youla_problem`      complex skew-symmetric ``A``              :func:`extract_loop`
:func:`split_problem`      real ``X`` over the split-complex numbers :func:`extract_split`
:func:`similarity_problem` real ``X`` on the similarity quiver       :func:`extract_loop`
:func:`congruence_problem` real symmetric ``S`` (congruence, LDL)    ``D[i, i].coefficient``
:func:`series_problem`     real ``A`` over ``R((t))``                :func:`standard_part`
=========================  ========================================  ==========================
"""

from __future__ import annotations

from collections.abc import Sequence

import numpy as np

from . import catalogue
from .algebra import Algebra, Element, quaternion
from .matrix import Matrix, orbit_index, plain_index
from .quiver import DSQ

__all__ = [
    "complex_representation",
    "congruence_problem",
    "extract_loop",
    "extract_split",
    "extract_svd",
    "extract_svd_plain",
    "hermitian_problem",
    "quaternion",
    "random_quaternion_matrix",
    "series_problem",
    "similarity_problem",
    "skew_problem",
    "split_problem",
    "standard_part",
    "svd_problem",
    "svd_problem_plain",
    "symmetric_problem",
    "t_power",
    "takagi_problem",
    "youla_problem",
]


def _block(z: object, field: str) -> np.ndarray:
    return np.asarray(z, dtype=complex).reshape(2, 2) if field == "H" else np.array([[z]], dtype=complex)


def _scalar(X: np.ndarray, field: str) -> np.ndarray | complex:
    return X if field == "H" else X[0, 0]


def _check_field(field: str, allowed: str = "RCH") -> None:
    if field not in allowed:
        raise ValueError(f"field must be one of {list(allowed)}, not {field!r}")
```

```
def _check_square(A: np.ndarray) -> None:
    if A.ndim < 2 or A.shape[0] != A.shape[1]:
        raise ValueError(f"expected a square matrix, not shape {A.shape}")


def complex_representation(W: np.ndarray) -> np.ndarray:
    """The complex ``2m x 2n`` representation of a quaternion matrix given as ``(m, n, 2, 2)`` blocks."""
    m, n = W.shape[:2]
    return W.transpose(0, 2, 1, 3).reshape(2 * m, 2 * n)


def random_quaternion_matrix(rng: np.random.Generator, m: int, n: int) -> np.ndarray:
    """A random ``m x n`` quaternion matrix (normal coefficients), as ``(m, n, 2, 2)`` blocks."""
    W = np.zeros((m, n, 2, 2), dtype=complex)
    for i in range(m):
        for j in range(n):
            W[i, j] = quaternion(*rng.normal(size=4))
    return W


# ------------------------------------------------------------- SVD over R, C, H
def svd_problem(W: np.ndarray, field: str = "R", order: Sequence[str] = ("u", "v"), relprec: int = 16) ->
    ↪ Matrix:
    """The SVD of ``W`` (``m x n``; quaternions as ``(m, n, 2, 2)`` blocks) as a self-adjoint matrix on
    the orbit-cyclic indexing of the SVD quiver ``u -> v``: ``H[(v, i), (u, j)]`` is the real element
    with coefficient ``W_ij`` at the arrow ``a``, and ``H = H*``.

    Parameters
    ----------
    W : numpy.ndarray
    field : {'R', 'C', 'H'}
    order : sequence of str
        The cyclic order of the two orbits; ``('u', 'v')`` gives positions ``u0, v0, u1, v1, ...``.
    relprec : int
        Passed to the :class:`~dsqlib.algebra.Algebra`.
    """
    _check_field(field)
    alg = Algebra(catalogue.svd(field), relprec=relprec)
    m, n = W.shape[:2]
    index = orbit_index(alg, {"u": n, "v": m}, order)
    H = Matrix.zeros(index)
    for I, (oi, ki) in enumerate(index):
        for J, (oj, kj) in enumerate(index):
            if oi == "v" and oj == "u":
                H[I, J] = alg.real_element(alg.arrow("a", _block(W[ki, kj], field)))
                H[J, I] = alg.star(H[I, J])
    return H


def extract_svd(H: Matrix, field: str = "R") -> np.ndarray:
    """The classical matrix of an SVD problem: the coefficients of ``a`` in the ``(v, u)`` entries."""
    _check_field(field)
    pos = H.rows.positions
    m = sum(1 for o, _ in pos if o == "v")
    n = sum(1 for o, _ in pos if o == "u")
    W = np.zeros((m, n, 2, 2), dtype=complex) if field == "H" else np.zeros((m, n), dtype=complex)
    for I, (oi, ki) in enumerate(pos):
        for J, (oj, kj) in enumerate(pos):
            if oi == "v" and oj == "u":
                W[ki, kj] = _scalar(H[I, J].coefficient(["a"], "v", "u"), field)
    return W


def svd_problem_plain(W: np.ndarray, field: str = "R", relprec: int = 16) -> Matrix:
    """The SVD problem on the plain indexing (every ``E_i = 1``; ``N x N`` with ``N = max(m, n)``,
    padded by zeros): ``H_ij = W_ij a + conj(W_ji) a'`` (and conjugates), i.e. ``H = W (x) a + W^H (x) a
    ↪ *``."""
    _check_field(field)
    alg = Algebra(catalogue.svd(field), relprec=relprec)
    m, n = W.shape[:2]
    index = plain_index(alg, max(m, n))
```

```
    H = Matrix.zeros(index)
    for i in range(m):
        for j in range(n):
            x = alg.real_element(alg.arrow("a", _block(W[i, j], field)))
            H[i, j] = alg.add(H[i, j], x)
            H[j, i] = alg.add(H[j, i], alg.star(x))
    return H


def extract_svd_plain(H: Matrix, m: int, n: int, field: str = "R") -> np.ndarray:
    """The classical ``m x n`` matrix of :func:`svd_problem_plain`."""
    _check_field(field)
    W = np.zeros((m, n, 2, 2), dtype=complex) if field == "H" else np.zeros((m, n), dtype=complex)
    for i in range(m):
        for j in range(n):
            W[i, j] = _scalar(H[i, j].coefficient(["a"], "v", "u"), field)
    return W


# ------------------------------------------------------------------ one-orbit problems
def _one_orbit(quiver: DSQ, A: np.ndarray, word: Sequence[str], field: str, relprec: int) -> Matrix:
    _check_square(A)
    alg = Algebra(quiver, relprec=relprec)
    n = A.shape[0]
    index = orbit_index(alg, {"v": n})
    H = Matrix.zeros(index)
    for i in range(n):
        for j in range(i, n):
            x = alg.real_element(alg.word(word, _block(A[i, j], field)))
            if i == j:
                x = alg.scale(0.5, alg.add(x, alg.star(x)))
            H[i, j] = x
            H[j, i] = alg.star(x)
    return H


def hermitian_problem(A: np.ndarray, field: str = "C", relprec: int = 16) -> Matrix:
    """A Hermitian matrix over C (or H, as ``(n, n, 2, 2)`` blocks): ``H_ij`` has coefficient ``A_ij`` at
    the loop ``x``."""
    _check_field(field, "CH")
    return _one_orbit(catalogue.hermitian(field), A, ["x"], field, relprec)


def symmetric_problem(A: np.ndarray, relprec: int = 16) -> Matrix:
    """A real symmetric matrix: ``H_ij = A_ij x`` with the loop ``x* = x``."""
    return _one_orbit(catalogue.symmetric_loop(), A, ["x"], "R", relprec)


def skew_problem(A: np.ndarray, relprec: int = 16) -> Matrix:
    """A real skew-symmetric matrix: ``H_ij = A_ij h`` with the loop ``h* = -h``."""
    return _one_orbit(catalogue.skew_loop(), A, ["h"], "R", relprec)


def takagi_problem(A: np.ndarray, relprec: int = 16) -> Matrix:
    """A complex symmetric matrix: ``H_ij`` has coefficient ``A_ij`` at the arrow ``al: v -> v~``."""
    return _one_orbit(catalogue.takagi(), A, ["al"], "C", relprec)


def youla_problem(A: np.ndarray, relprec: int = 16) -> Matrix:
    """A complex skew-symmetric matrix: ``H_ij`` has coefficient ``A_ij`` at ``al: v -> v~`` (sign -1)."""
    return _one_orbit(catalogue.youla(), A, ["al"], "C", relprec)


def extract_loop(
    H: Matrix, word: Sequence[str], target: str | None = None, source: str | None = None, field: str = "C"
) -> np.ndarray:
    """The classical matrix of a one-orbit problem: the coefficients of ``word`` in every entry
    (``['x']`` for the Hermitian and symmetric problems, ``['h']`` for the skew one, ``['al']`` for Takagi
    and Youla). ``target`` and ``source`` default to the ends of the word."""
    _check_field(field)
    n = H.shape[0]
    A = np.zeros((n, n, 2, 2), dtype=complex) if field == "H" else np.zeros((n, n), dtype=complex)
    for i in range(n):
```

```
        for j in range(n):
            A[i, j] = _scalar(H[i, j].coefficient(word, target, source), field)
    return A


# ------------------------------------------------------------- split-complex numbers and congruence
def split_problem(X: np.ndarray, relprec: int = 16) -> Matrix:
    """A real matrix over the split-complex numbers ``2R``: ``H_ij = X_ij e_u + X_ji e_u'``, so that
    ``H = (X, X^T)`` is self-adjoint."""
    _check_square(X)
    alg = Algebra(catalogue.split_complex(), relprec=relprec)
    index = orbit_index(alg, {"u": X.shape[0]})
    return Matrix.from_function(index, lambda i, j: alg.add(alg.vertex("u", X[i, j]), alg.vertex("u'", X[j,
     ↪  i])))


def extract_split(H: Matrix) -> np.ndarray:
    """The first component ``X`` of a matrix ``(X, Y)`` over the split-complex numbers."""
    n = H.shape[0]
    return np.array([[H[i, j].coefficient((), "u", "u")[0, 0] for j in range(n)] for i in range(n)])


def similarity_problem(X: np.ndarray, relprec: int = 16) -> Matrix:
    """Similarity of a real square ``X`` on the similarity quiver: ``H_ij = X_ij x + X_ji x'``, with the
    loop ``x`` at ``v`` and its transpose ``x'`` at ``v'``, so that ``H`` is self-adjoint. Read ``X`` back
    with ``extract_loop(H, ['x'], 'v', 'v', 'R')``."""
    _check_square(X)
    alg = Algebra(catalogue.similarity(), relprec=relprec)
    index = orbit_index(alg, {"v": X.shape[0]})
    return Matrix.from_function(index, lambda i, j: alg.add(alg.arrow("x", X[i, j]), alg.arrow("x'", X[j, i
     ↪ ])))


def congruence_problem(S: np.ndarray, relprec: int = 16) -> Matrix:
    """Congruence of a real symmetric ``S``: ``H_ij = S_ij a`` with ``a: u -> u'`` (the LDL^T quiver)."""
    _check_square(S)
    alg = Algebra(catalogue.ldl(), relprec=relprec)
    index = orbit_index(alg, {"u": S.shape[0]})
    return Matrix.from_function(index, lambda i, j: alg.arrow("a", S[i, j]))


# ------------------------------------------------------------- real Laurent series R((t))
def series_problem(A: np.ndarray, relprec: int = 24) -> Matrix:
    """A real matrix over the field of real Laurent series ``R((t))``, realised as the completed algebra
    of the one-loop quiver :func:`~dsqlib.catalogue.symmetric_loop` (``t`` = the loop ``x``, with its
    local inverse ``x^-1 = t^-1``); plain indexing."""
    _check_square(A)
    alg = Algebra(catalogue.symmetric_loop(), relprec=relprec)
    index = plain_index(alg, A.shape[0])
    return Matrix.from_function(index, lambda i, j: alg.vertex("v", float(A[i, j])) if A[i, j] != 0 else
     ↪ None)


def t_power(alg: Algebra, k: int) -> Element:
    """``t^k`` (``k`` in Z) in ``R((t))`` = the completed one-loop algebra."""
    k = int(k)
    if k == 0:
        return alg.idempotent("v")
    return alg.word(["x"] * k) if k > 0 else alg.word(["x^-1"] * (-k))


def standard_part(H: Matrix) -> np.ndarray:
    """The ``t^0`` coefficients of a matrix over ``R((t))``, as a float array (NaN where an entry has a
    pole or is not known at degree 0)."""
    m, n = H.shape
    S = np.zeros((m, n))
    for i in range(m):
        for j in range(n):
            x = H[i, j]
            if (x.terms and x.valuation < 0) or x.prec <= 0:
                S[i, j] = np.nan
            else:
                S[i, j] = x.coefficient((), "v", "v")[0, 0].real
```

```
    return S
```

B.1.9 src/dsqlib/errors.py

```
"""Exceptions raised by dsqlib.

Every exception derives from :class:`DSQError` and also from the built-in exception it refines, so
code that catches ``ValueError`` or ``ZeroDivisionError`` keeps working.
"""

from __future__ import annotations

__all__ = [
    "BreakdownError",
    "DSQError",
    "NoSquareRootError",
    "NotInvertibleError",
    "PrecisionError",
    "QuiverError",
]


class DSQError(Exception):
    """Base class of all dsqlib exceptions."""


class QuiverError(DSQError, ValueError):
    """The data passed to :meth:`DSQ.build` or :class:`DSQ` do not define a doubly-signed quiver.

    (Invalid arguments elsewhere raise the usual ``ValueError``, ``KeyError`` or ``TypeError``.)"""


class NotInvertibleError(DSQError, ZeroDivisionError):
    """No inverse was found: the element is zero, is known only to vanish below some degree, or is
    not a unit between the requested idempotents."""


class PrecisionError(NotInvertibleError):
    """Gauss-Jordan elimination ran out of precision: every remaining entry of a pivot column is
    only known to be 0(deg p)."""


class NoSquareRootError(DSQError, ValueError):
    """A *-square-root rule cannot produce some y with y* y = x (for example because of the
    signature of a vertex, or because x has too large a rank at a middle vertex)."""


class BreakdownError(DSQError, ZeroDivisionError):
    """A matrix algorithm met a zero or non-invertible pivot, or an isotropic vector
    (v != 0 with v* v = 0)."""
```